

Σχεδιασμός Εφαρμογών Διαδικτύου με Εργαλεία Ανοικτού Λογισμικού

PHP – MySQL

Εγχειρίδιο

Έκδοση 2.0 – Μάρτιος 2020

*Κατσάλης Νίκος
Καθηγητής ΠΕ 86 - Πληροφορικής
Μ.Δ.Ε. Επιστήμης Υπολογιστών*

Περιεχόμενα

1	Εισαγωγή.....	6
1.1	Σκοπός Εγχειριδίου	6
1.2	Οργάνωση Εγχειριδίου.....	7
1.3	Προαπαιτούμενες γνώσεις.....	7
2	Παγκόσμιος Ιστός και Προγραμματισμός	8
2.1	Παγκόσμιος Ιστός.....	8
2.1.1	Μοντέλο επικοινωνίας και βασικές έννοιες.....	8
2.1.2	HTML, XHTML, CSS	10
2.1.3	Πρότυπα	14
2.2	Προγραμματισμός στον Παγκόσμιο Ιστό	15
2.2.1	Προγραμματισμός από την πλευρά του πελάτη.....	15
2.2.2	Προγραμματισμός από την πλευρά του Εξυπηρετητή.....	15
3	Τι χρειάζομαι για την PHP-MySQL	17
4	MySQL.....	19
4.1	Περιβάλλοντα Επικοινωνίας	19
4.2	Διαχείριση Β.Δ. MySQL από τη Γραμμή Εντολών	19
4.2.1	Σύνδεση με τον MySQL Εξυπηρετητή και εκτέλεση απλών εντολών	20
4.2.2	Δημιουργία χρηστών και ανάθεση δικαιωμάτων.....	21
4.2.3	Εντολές Β.Δ.	23
4.3	Διαχείριση Β.Δ. MySQL με τον Query Browser	37
4.3.1	Σύνδεση με τον MySQL Εξυπηρετητή	38
4.3.2	Προβολή, Επιλογή και Δημιουργία Β.Δ.	40
4.3.3	Προβολή, Δημιουργία και Επεξεργασία Πινάκων Β.Δ.	42
4.3.4	Ερωτήματα Εισαγωγής, Ενημέρωσης και Επιλογής στοιχείων Πινάκων Β.Δ.	45
4.4	MySQL και PHP	47
5	Η γλώσσα PHP.....	48
5.1	Τι είναι η PHP	48
5.2	Πως λειτουργεί η PHP.....	49
5.2.1	Βασικός Τρόπος Λειτουργίας	49
5.2.2	Δομή συνολικού συστήματος	50
5.3	Κανόνες σύνταξης – Εντολές	51
5.3.1	Βγαίνοντας από την HTML και μπαίνοντας στον κώδικα PHP	51
5.3.2	Διαχωρισμός εντολών και σχόλια.....	52
5.4	Τύποι Δεδομένων	53
5.5	Μεταβλητές και Σταθερές.....	54
5.5.1	Μεταβλητές.....	54
5.5.2	Σταθερές.....	58
5.6	Τελεστές και Εκφράσεις.....	58
5.6.1	Τελεστές.....	58
5.6.2	Εκφράσεις	62
5.7	Δομές Ελέγχου	62
5.7.1	If – If-else - If-elseif-else	62
5.7.2	switch	62
5.7.3	while.....	63
5.7.4	do..while.....	63
5.7.5	for	64
5.7.6	foreach.....	64
5.7.7	break και continue.....	64
5.8	require(), require_once() και include_once(), include().....	65

5.9	Συναρτήσεις.....	65
5.9.1	Συναρτήσεις οριζόμενες από τον χρήστη	65
5.9.2	Ενσωματωμένες Συναρτήσεις.....	67
5.10	Κλάσεις και Αντικείμενα.....	67
5.11	Παραπομπή Συναρτήσεων.....	69
5.11.1	Συχνά Χρησιμοποιούμενες Ενσωματωμένες Συναρτήσεις.....	69
5.11.2	Συναρτήσεις της επέκτασης MySQL improved.....	73
5.12	Υπερκαθολικές (Superglobal) μεταβλητές	75
5.12.1	Αποστολή δεδομένων από τον πελάτη/χρήστη.....	76
5.12.2	Πληροφορίες Εξυπηρετητή, Πελάτη και Περιβάλλοντος.....	79
5.13	Συνάρτηση header για προχωρημένες λειτουργίες.....	79
5.13.1	Αποστολή Κωδικών Κατάστασης.....	80
5.13.2	Ανακατεύθυνση σε URL.....	80
5.13.3	Αποστολή Αρχείου και Προτροπή του χρήστη για αποθήκευσή	81
5.14	HTTP αναγνώριση με τη PHP	81
5.15	Χειρισμός Αρχείων στην PHP	82
5.16	Μεθοδολογία Σύνδεσης PHP-MySQL	83
5.16.1	Σύνδεση με τον MySQL εξυπηρετητή και επιλογή Β.Δ.....	83
5.16.2	Εκτέλεση Ερωτημάτων	84
5.16.3	Έλεγχος και εκτύπωση αποτελεσμάτων ερωτημάτων	87
5.16.4	Κλείσιμο της σύνδεσης	89
5.16.5	Ολοκληρωμένο παράδειγμα των βημάτων	89
5.17	Μεταφορά Αρχείου από τον Πελάτη στον Εξυπηρετητή.....	91
5.18	Σύνοδοι (Sessions)για διατήρηση στοιχείων επικοινωνίας.....	94
5.18.1	Συναρτήσεις για τις Συνόδους	94
5.18.2	Διαδικασία χρήσης των συνόδων	97
5.19	Παραδείγματα Εφαρμογών.....	101
5.19.1	Παράδειγμα 1	101
5.19.2	Παράδειγμα 2	102
5.19.3	Παράδειγμα 3	105
6	ΠΑΡΑΡΤΗΜΑ Α – ΡΥΘΜΙΣΕΙΣ WEB ΕΞΥΠΗΡΕΤΗΤΗ, PHP ΚΑΙ MySQL	109
6.1	Αρχείο http.conf	109
6.2	Αρχείο .htaccess	111
6.3	Αρχείο php.ini	112
6.4	Αρχείο my.ini ή my.cnf.....	115
7	ΠΑΡΑΡΤΗΜΑ Β – Επίδειξη Ολοκληρωμένης Εφαρμογής.....	118
7.1	Υλοποίηση Βάση Δεδομένων	118
7.2	Υλοποίηση Ιστοσελίδων	119
7.2.1	Αρχείο index.html	121
7.2.2	Αρχείο functions.php	122
7.2.3	Αρχείο layout.css	134
7.2.4	Αρχείο menu.css	135
7.2.5	Αρχείο format.css.....	136
7.2.6	Εικόνες Εφαρμογής.....	138
8	Αναφορές	141

Εικόνες

Εικόνα 1: Μοντέλο επικοινωνίας Παγκόσμιου Ιστού	10
Εικόνα 2: Βήματα επικοινωνίας/λειτουργιών στον προγραμματισμό από την πλευρά του εξυπηρετητή	16
Εικόνα 3: Σχέσεις πινάκων B.Δ. newsdb	32
Εικόνα 4: Διάλογος σύνδεσης στον MySQL Εξυπηρετητή με τον Query Browser	38
Εικόνα 5: Γενικές Ρυθμίσεις Εφαρμογής.....	39
Εικόνα 6: Ρυθμίσεις Συνδέσεων (Δημιουργία, Διαγραφή, Αλλαγή στοιχείων) και Ιστορικό.....	39
Εικόνα 7: Ρυθμίσεις που αφορούν τους Επεξεργαστές	40
Εικόνα 8: Βασική Οθόνη Εργασίας στο Query Browser.....	40
Εικόνα 9: Επιλογές Εφαρμογής από το μενού Εργαλεία.....	41
Εικόνα 10: Εισαγωγή ονόματος της νέας B.Δ. (νέου Σχήματος)	42
Εικόνα 11: Επεξεργαστής Πινάκων - Καρτέλα δημιουργίας και διαμόρφωσης πεδίων (στηλών) πίνακα.....	43
Εικόνα 12: Επιβεβαίωση εκτέλεσης ερωτήματος, που δημιουργήθηκε από τις επιλογές μας, για την δημιουργία του πίνακα.....	43
Εικόνα 13: Δημιουργία του πίνακα subjects και προβολή της υποκαρτέλας ορισμού των Ξένων Κλειδιών.....	44
Εικόνα 14: Μήνυμα επιβεβαίωσης εκτέλεσης ερωτήματος δημιουργίας πίνακα subjects.....	44
Εικόνα 15: Δημιουργία του πίνακα messages και προβολή της υποκαρτέλας ορισμού των Ξένων Κλειδιών.....	45
Εικόνα 16: Μήνυμα επιβεβαίωσης εκτέλεσης ερωτήματος δημιουργίας πίνακα messages	45
Εικόνα 17: Εισαγωγή εγγραφής χρησιμοποιώντας την προβολή εγγραφών στο αριστερό κάτω τμήμα.....	46
Εικόνα 18: Η δομή του συστήματος της PHP	50
Εικόνα 19: Αρχική Σελίδα	138
Εικόνα 20: Σελίδα εγγραφής χρήστη	138
Εικόνα 21: Σελίδα Αναζήτησης-Προβολής Μηνυμάτων (δεν χρειάζεται πιστοποίηση)	139
Εικόνα 22: Σελίδα Εισαγωγής Θέματος (δεν έχει γίνει σύνδεση και βγάζει μήνυμα)	139
Εικόνα 23: Αρχική Σελίδα (αμέσως αφού κάναμε σύνδεση στο λογαριασμό μας)	140
Εικόνα 24: Σελίδα Εισαγωγής Θέματος (είμαστε συνδεδεμένοι με το λογαριασμό μας)	140
Εικόνα 25: Σελίδα Εισαγωγής Μηνύματος (είμαστε συνδεδεμένοι με το λογαριασμό μας)	140

Πίνακες

Πίνακας 1: Τα πρότυπα του W3C που αφορούν τον Παγκόσμιο Ιστό.....**Σφάλμα! Δεν έχει οριστεί σελιδοδείκτης.**

Πίνακας 2: ΣΔΒΔ με τα οποία μπορεί και επικοινωνεί η PHP	49
Πίνακας 3: Αριθμητικοί τελεστές στην PHP	59
Πίνακας 4: Δυναδικοί τελεστές στην PHP	59
Πίνακας 5: Τελεστές σύγκρισης στην PHP	59
Πίνακας 6: Τελεστές Αύξησης-Μείωσης στην PHP	60
Πίνακας 7: Λογικοί τελεστές στην PHP	60
Πίνακας 8: Προτεραιότητα Τελεστών στην PHP	61
Πίνακας 9: Συχνά χρησιμοποιούμενες συναρτήσεις της PHP	72
Πίνακας 10: Συναρτήσεις του MySQL Module της PHP	75

1 Εισαγωγή

Εδώ και πολλά χρόνια, η χρήση του Παγκόσμιου Ιστού (WWW) έχει περάσει από την παρουσίαση στατικού περιεχομένου, μέσω στατικά δημιουργημένων ιστοσελίδων σε γλώσσα HTML, στην παρουσίαση δυναμικού περιεχομένου με βάση:

Τις ενέργειες που κάνει ο χρήστης,

1. Τα δεδομένα που αποστέλλει ο χρήστης
2. Τα στοιχεία που υπάρχουν σε Βάσεις Δεδομένων και διάφορες εφαρμογές (file και e-mail εξυπηρετητές κλπ)
3. Το ποιος είναι ο ίδιος ο χρήστης (οι προτιμήσεις, οι συνήθειές του κ.λ.π.)
4. κ.λ.π.

Με τον τρόπο αυτό δημιουργήθηκε η δυνατότητα για την παροχή νέων υπηρεσιών από τον Παγκόσμιο Ιστό, τον αποκαλούμε Web 2.0, οι οποίες συνδέονται με δεδομένα πραγματικού χρόνου (real-time) τα οποία βρίσκονται αποθηκευμένα (συνήθως σε Β.Δ.) και μπορεί ο χρήστης να διαχειριστεί. Ενδεικτικές τέτοιες υπηρεσίες περιλαμβάνουν το e-banking, e-commerce, e-government, blogs, θεματικές λίστες συζητήσεων κλπ, και έχουν δώσει μία νέα διάσταση πρόσβασης σε on-line δεδομένα, διαδραστικότητας και ζωντανού περιεχομένου στο Διαδίκτυο.

Η δυνατότητα για τη παρουσίαση δυναμικού περιεχομένου είχε ξεκινήσει αρκετά χρόνια πριν από την πλευρά του εξυπηρετητή με τη χρήση **CGI (Common Gateway Interface)**, αλλά τα χρόνια που έχουν ακολουθήσει είχαμε μια δραματική εξέλιξη σε νέες τεχνολογίες και γλώσσες που επιτρέπουν τη δημιουργία δυναμικού περιεχομένου στην πλευρά του εξυπηρετητή όπως **JSP, ASP, PHP** κλπ. Επίσης με την χρήση γλωσσών προγραμματισμού που εκτελούνται στην πλευρά του πελάτη (**VBScript, Javascript** κλπ) έχουμε τη δυνατότητα για σημαντικά καλύτερα αποτελέσματα, στον έλεγχο και τη διαδραστικότητα της επικοινωνίας.

Σήμερα υπάρχουν έτοιμες λύσεις που χρησιμοποιούν τις παραπάνω τεχνολογίες και παρέχουν ολοκληρωμένα περιβάλλοντα διαχείρισης στατικού και δυναμικού περιεχομένου, χρηστών, επιπέδων ασφαλείας κλπ (πχ Συστήματα Διαχείρισης Περιεχομένου – Content Management Systems CMS).

Το σημαντικό είναι ότι πολλές από αυτές τις τεχνολογίες, γλώσσες αλλά και ολοκληρωμένα περιβάλλοντα είναι **ΕΛ/ΛΑΚ (Ελεύθερο Λογισμικό/Λογισμικό Ανοικτού Κώδικα)**, παρέχοντας νέες δυνατότητες στην εξέλιξή τους και τη χρήση τους στο διαδίκτυο.

1.1 Σκοπός Εγχειριδίου

Σκοπός του εγχειριδίου αυτού είναι να σας παρουσιάσει τη χρήση της γλώσσας **PHP (Hypertext Preprocessor)** σε συνεργασία με **Βάσεις Δεδομένων MySQL**, για την παραγωγή ιστοσελίδων δυναμικού περιεχομένου στην πλευρά του εξυπηρετητή πριν την αποστολή τους στον πελάτη που έκανε την αίτηση.

Οπότε ο αναγνώστης πρέπει να γνωρίζει ότι δεν έχει στα χέρια του, ούτε ένα πλήρη οδηγό της PHP, η οποία είναι μια πλήρη γλώσσα με πολλές δυνατότητες χρήσης σε διάφορα περιβάλλοντα, όπως θα δούμε στο αντίστοιχο κεφάλαιο, αλλά ούτε ένα πλήρη οδηγό της MySQL (εγκατάσταση, παραμετροποίηση, διαχείριση, σύνολο εντολών εξυπηρετητή και σύνολο εντολών SQL). Αλλά έχει στα χέρια του ένα εγχειρίδιο που τον εισάγει στη χρήση της PHP για τη δημιουργία δυναμικών ιστοσελίδων και στην MySQL για την δημιουργία και διαχείριση απλών Β.Δ. και επικεντρώνεται στη συνεργασία της PHP με τη MySQL, παρουσιάζοντας τη λογική του προγραμματισμού από την πλευρά του εξυπηρετητή και δίνοντας ένα σύνολο απλών παραδειγμάτων για την κατανόησή του.

1.2 Οργάνωση Εγχειριδίου

Στο πρώτο κεφάλαιο του εγχειριδίου παρουσιάζονται βασικές έννοιες του Παγκόσμιου Ιστού, το μοντέλο επικοινωνίας, τα διάφορα πρότυπα και τα μοντέλα προγραμματισμού. Στο δεύτερο κεφάλαιο παρουσιάζονται το απαραίτητο λογισμικό που θα χρειαστεί ο αναγνώστης για να μπορέσει να δοκιμάσει τα διάφορα παραδείγματα που του δίνονται και να γράψει τις δικές του εφαρμογές σε PHP – MySQL. Στο τρίτο κεφάλαιο παρουσιάζεται η MySQL, το βασικό περιβάλλον επικοινωνίας, οι βασικές εντολές (SQL) δημιουργίας Βάσεων Δεδομένων και διαχείρισής τους και η χρήση γραφικού περιβάλλοντος για την ευκολότερη επικοινωνία με τη MySQL. Στο τέταρτο κεφάλαιο παρουσιάζονται τα βασικά στοιχεία της γλώσσας PHP (συντακτικό, μεταβλητές, δομές κλπ) και ένα μικρό μέρος από τις έτοιμες συναρτήσεις που υπάρχουν στις βιβλιοθήκες της και κυρίως αυτές που αφορούν την σύνδεσή της με βάσεις Δεδομένων MySQL. Επίσης στο ίδιο κεφάλαιο παρουσιάζεται ένας οδηγός σύνδεσης με MySQL μέσα από τον κώδικα PHP και δίνονται παραδείγματα εφαρμογών στις οποίες εκτελούνται ερωτήματα σε Β.Δ. ενός MySQL εξυπηρετητή στηριζόμενα σε δεδομένα που παρέχει ο χρήστης-πελάτης και δημιουργούνται δυναμικές ιστοσελίδες από τα αποτελέσματα των ερωτημάτων.

Τέλος το εγχειρίδιο ολοκληρώνεται με δύο παραρτήματα. Στο πρώτο παρουσιάζονται κάποιες βασικές ρυθμίσεις του Apache Web Server, της PHP και της MySQL και στο δεύτερο μια ολοκληρωμένη εφαρμογή PHP-MySQL στην οποία περιγράφονται οι απαιτήσεις και παρέχονται οι εντολές δημιουργίας της Βάσης Δεδομένων στον MySQL και όλες οι ιστοσελίδες που αποτελούν μία απλή λύση της.

1.3 Προαπαιτούμενες γνώσεις

Το εγχειρίδιο θεωρεί ότι ο αναγνώστης έχει βασικές γνώσεις όσον αφορά τον προγραμματισμό (αλγορίθμους, δομές δεδομένων κλπ) καθώς και αρκετά καλή γνώση της γλώσσας HTML. Επίσης, θα ήταν επιθυμητό, να έχει κάποιες γνώσεις σχεδίασης Βάσεων Δεδομένων και της γλώσσας SQL καθώς και χρήσης CSS για την μορφοποίηση των ιστοσελίδων.

2 Παγκόσμιος Ιστός και Προγραμματισμός

2.1 Παγκόσμιος Ιστός

Ο Παγκόσμιος ιστός (World Wide Web – WWW) είναι μία από τις σημαντικότερες υπηρεσίες στο Διαδίκτυο, η οποία έπαιξε σημαντικό ρόλο και στη διάδοση της χρήσης του, αφού αποτελεί τη βασική υπηρεσία πρόσβασης σε πληροφορίες. Η υπηρεσία αυτή στηρίζεται σε δύο βασικές ιδέες:

1. Την δυνατότητα πρόσβασης, μέσω κοινού πρωτοκόλλου (HTTP), σε πληροφορία η οποία βρίσκεται κατανεμημένη σε διάφορους Η/Υ στο Διαδίκτυο και οι οποίοι μπορεί να είναι οποιασδήποτε αρχιτεκτονικής και να τρέχουν οποιοδήποτε λειτουργικό σύστημα.
2. Την οργάνωση αυτής της πληροφορίας με τη μορφή Υπερκειμένου (Hypertext) στην οποία η πληροφορία δεν είναι μόνο κείμενο αλλά περιέχει και άλλες (οπτικές και ηχητικές) μορφές παρουσίασης και δεν έχει γραμμική δομή, όπως ένα βιβλίο, αλλά μέσω συνδέσμων (links) μπορούμε να μεταβαίνουμε από την προβολή του τρέχον υπερκειμένου σε κάποιο άλλο που μπορεί να βρίσκεται και σε διαφορετικό Η/Υ από το υπερκείμενο που περιείχε το σύνδεσμο.

2.1.1 Μοντέλο επικοινωνίας και βασικές έννοιες

Το μοντέλο επικοινωνίας στον Παγκόσμιο ιστό ακολουθεί το μοντέλο **Πελάτη-Εξυπηρετητή (Client-Server Model)**.

Πελάτης είναι συνήθως ένα πρόγραμμα περιήγησης ιστοσελίδων (**Φυλλομετρητής-Web Browser**) που εκτελείται στον Η/Υ του χρήστη που θέλει να κάνει τη πρόσβαση στην πληροφορία (φυσικά μπορεί να είναι και οποιοδήποτε πρόγραμμα, με ή χωρίς τη μεσολάβηση χρήστη, που χρησιμοποιεί το κοινό πρωτόκολλο. Παραδείγματα πελατών είναι ο **MS Internet Explorer**, ο **Navigator**, ο **Mozilla Firefox**, ο **Opera** κλπ.

Εξυπηρετητής είναι ένα πρόγραμμα εξυπηρέτησης ιστοσελίδων (**Web Server**) σε κάποιον (συνήθως άλλον από αυτόν του πελάτη) Η/Υ στον οποίο βρίσκεται η πληροφορία. Παραδείγματα εξυπηρετητών είναι ο **MS IIS**, ο **Apache**, ο **Netscape Enterprise Server** κλπ.

Το κοινό πρωτόκολλο που χρησιμοποιείται για την επικοινωνία μεταξύ του πελάτη και του εξυπηρετητή είναι το **HTTP (HyperText Transfer Protocol)**.

Η κάθε πληροφορία που είναι διαθέσιμη αποτελεί έναν **πόρο (Resource)**. Όταν λέμε **πόρος (resource)** αναφερόμαστε σε οποιοδήποτε είδος πληροφορίας (συνήθως με τη μορφή ενός αρχείου), όπως μία ιστοσελίδα, μία εικόνα, ένα video, ένα αρχείο ήχου κλπ. Αυτός ο πόρος συνήθως βρίσκεται σε κάποιο μέσο αποθήκευσης του Η/Υ στον οποίο τρέχει ο εξυπηρετητής, αλλά μπορεί να βρίσκεται και οπουδήποτε αλλού.

Ο τρόπος (η μορφή της σύνταξης) που καθορίζει ο πελάτης την πληροφορία που ζητάει, δηλαδή τον πόρο που ζητάει, είναι με την χρήση του **URL (Uniform Resource Locator - Ομοιόμορφος Εντοπισμός Πόρου)**. Στην πραγματικότητα το URL δεν χρησιμοποιείται μόνο στον Παγκόσμιο Ιστό με το πρωτόκολλο HTTP αλλά είναι μια τυποποιημένη μορφή σύνταξης που καθορίζει μοναδικά έναν πόρο καθώς και το πρωτόκολλο με το οποίο θα γίνει η πρόσβαση σε αυτόν. Η γενική μορφή είναι:

scheme://host.domain [:port]/path/ filename

Δηλαδή περιλαμβάνει, σε σειρά από αριστερά προς τα δεξιά, τέσσερα (4) μέρη:

1. Το πρωτόκολλο-σχήμα (***scheme***) πρόσβασης στον πόρο (π.χ. HTTP, HTTPS, FTP κ.λ.π.)

2. Το όνομα του Η/Υ (*host.domain*) ή τη Διεύθυνση IP του Η/Υ στον οποίο βρίσκεται ο πόρος.
3. Το αριθμό θύρας TCP/UDP (*port*) στην οποία «ακούει» ο εξυπηρετητής (π.χ. ο Web Server όταν το πρωτόκολλο είναι το HTTP) και το οποίο είναι προαιρετικό και εφόσον δεν παρέχεται έχει την προκαθορισμένη τιμή 80 (TCP) για το πρωτόκολλο HTTP καθώς και άλλες προκαθορισμένες τιμές για τα υπόλοιπα πρωτόκολλα.
4. Την ακριβή τοποθεσία του πόρου (*path/ filename*) στον εξυπηρετητή. Συνήθως είναι ένα μονοπάτι φακέλων με τελευταίο το όνομα του πόρου αλλά αυτό δεν είναι απαραίτητο στις δυναμικές σελίδες που μπορεί απλώς να είναι ένα αλφαριθμητικό που διευκολύνει στο να καθοριστεί μοναδικά ο πόρος.

Η μορφή URL θα γίνει πιο κατανοητή αν αναλύσουμε το επόμενο παράδειγμα URL:

<http://www.pc.com/hardware/parts/cpu.html>

Από αριστερά προς τα δεξιά έχουμε τα εξής τμήματα:

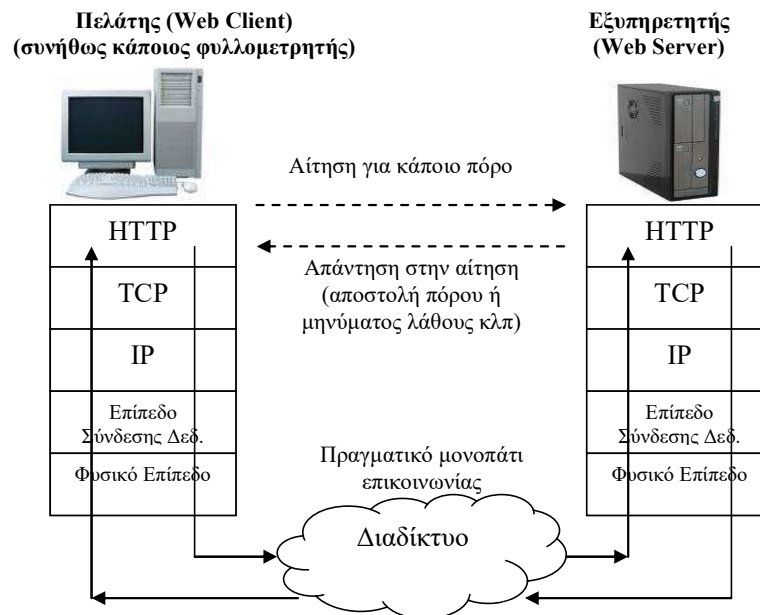
Τμήμα	Χρήση
http:	Το πρωτόκολλο πρόσβασης στο πόρο
//www.pc.com	Το domain name του Η/Υ στον οποίο βρίσκεται ο πόρος. Θα μπορούσαμε να γράψουμε και την IP διεύθυνση του Η/Υ αν την γνωρίζαμε. πχ //193.68.146.53
	Η Θύρα δεν καθορίζεται οπότε επειδή έχουμε HTTP εννοείται η θύρα 80 του TCP.
hardware/parts/cpu.html	Η ακριβή τοποθεσία (μονοπάτι φακέλων και όνομα αρχείου) του πόρου στον συγκεκριμένο εξυπηρετητή.

Στο προηγούμενο παράδειγμα όπως αναφέραμε δεν έχουμε καθορίσει τη θύρα που «ακούει» ο εξυπηρετητής και θα χρησιμοποιηθεί για την επικοινωνία μαζί του, όμως αν θέλαμε να χρησιμοποιήσουμε τη θύρα 1010 θα έπρεπε να γράψουμε ακριβώς μετά το domain name (ή την IP) του Η/Υ και πριν την ακριβή τοποθεσία, άνω κάτω τελεία και τον αριθμό της θύρας, δηλαδή:

<http://www.pc.com:1010/hardware/parts/cpu.html>

Πολλές φορές αντί να χρησιμοποιήσουμε τον όρο URL αναφερόμαστε με τον όρο **URI (Uniform Resource Identifier)** το οποίο, όμως, είναι ένας πιο γενικός όρος που μπορεί να παίρνει διαφορετικές μορφές και να αναφέρεται είτε σε τοποθεσία (location – URL) είτε σε όνομα (name – URN) κάποιου πόρου.

Στο επόμενο σχήμα φαίνεται το μοντέλο επικοινωνίας **Πελάτη–Εξυπηρετητή** (με προβολή των πρωτοκόλλων που μεσολαβούν. Στην περίπτωση χρήσης του πρωτοκόλλου HTTPS μεσολαβεί, πριν το TCP, το SSL ή TLS που αναλαμβάνει την κρυπτογράφηση/αποκρυπτογράφηση της επικοινωνίας).



Εικόνα 1: Μοντέλο επικοινωνίας Παγκόσμιου Ιστού

2.1.2 HTML, XHTML, CSS

Τα υπερκείμενα είναι γραμμένα στην γλώσσα **HTML** τα αρχικά της οποίας προέρχονται από τις λέξεις **Hypertext Markup Language** (**Γλώσσα Σήμανσης Υπερ-Κειμένου**).

Η γλώσσα HTML δεν είναι μία γλώσσα προγραμματισμού, είναι μια γλώσσα σήμανσης (**markup language**) με την οποία περιγράφουμε το περιεχόμενο-δομή ενός εγγράφου και στην πραγματικότητα είναι μια εφαρμογή (ένα υποσύνολό) της SGML. Η βασική ιδέα δημιουργίας της είναι ότι τα περισσότερα έγγραφα έχουν κοινά στοιχεία (elements) όπως επικεφαλίδες, παραγράφους, πίνακες, λίστες, φόρμες κ.λ.π.. Ένα αρχείο HTML είναι ένα απλό αρχείο κειμένου το οποίο μπορούμε να γράψουμε με οποιονδήποτε απλό συντάκτη κειμένου (αν και υπάρχουν ειδικοί συντάκτες για HTML). Τα αρχεία κειμένου HTML αποτελούνται από το κείμενο του εγγράφου μέσα στο οποίο υπάρχουν **Ετικέτες (tags)** της HTML.

Η χρήση των ετικετών είναι για να καθορίζει το είδος του κειμένου που θέλουμε. Συνήθως αποτελείται από μια ετικέτα που «ανοίγει» (το όνομά της μέσα σε < >), ακολουθείται από το κείμενο που επηρεάζει (δηλαδή καθορίζει το είδος και τη μορφή του) και την ίδια ετικέτα που «κλείνει» (/ και το όνομά της μέσα σε < >). Οπότε οι περισσότερες ετικέτες της HTML δείχνουν ως εξής:

<όνομαTag> κείμενο σελίδας που επηρεάζεται **</όνομαTag>**

Μάλιστα στην ετικέτα που ανοίγει πριν το «>» μπορούμε να έχουμε μια σειρά από ζευγάρια **ιδιότητα=τιμή** της συγκεκριμένης ετικέτας και τα οποία καθορίζουν περισσότερα για τη μορφή και το είδος του συγκεκριμένου κειμένου που περικλείει η ετικέτα.

Π.χ. **<h1> Αυτό είναι μια επικεφαλίδα </h1>**

Φυσικά παρέχονται ετικέτες και για τη ενσωμάτωση στο κείμενο εικόνων, video, ήχων και άλλων πόρων.

Δυστυχώς με την HTML δεν μπορούμε να προγραμματίσουμε, δηλαδή το έγγραφο πρέπει να δημιουργηθεί σε HTML και να το έχει ο Web Εξυπηρετητής για να το στέλνει σε όποιον πελάτη του το ζητάει. Από τη στιγμή που δημιουργηθεί το έγγραφο παραμένει ως έχει μέχρι να το αλλάξουμε εμείς. Αυτό έχει ως αποτέλεσμα **να μπορούμε να παρέχουμε μόνο στατικό**

περιεχόμενο στους πελάτες, κάτι που είναι σημαντικό αλλά περιορίζει πολύ τις δυνατότητες του παγκόσμιου ιστού (όσον αφορά τις εφαρμογές στις οποίες μπορεί να χρησιμοποιηθεί). Γι' αυτό από νωρίς υπήρχε η απαίτηση να παρέχεται **δυναμικό περιεχόμενο** στον πελάτη και η απαίτηση αυτή οδήγησε:

1. Στην ενσωμάτωση στο υπερκείμενο, μέσω κατάλληλων ετικετών HTML, κώδικα γλωσσών προγραμματισμού (scripting γλώσσες) που εκτελείται από τον πελάτη και αναφερόμαστε σε αυτό με τον όρο **προγραμματισμός από την πλευρά του πελάτη**.
2. Στην δημιουργία των ιστοσελίδων την ώρα της αίτησης, λαμβάνοντας υπόψη ένα σύνολο πληροφοριών που περιλαμβάνει η αίτηση καθώς και πληροφορία που τη στιγμή της αίτησης είναι διαθέσιμη σε Β.Δ., συστήματα αρχείων και εφαρμογές και αναφερόμαστε σε αυτό με τον όρο **προγραμματισμός από την πλευρά του εξυπηρετητή** (με τον οποίο ασχολούμαστε στο συγκεκριμένο εγχειρίδιο).

Και οι δύο μορφές δυναμικού περιεχομένου, που μπορούν να συνδυαστούν, θα αναλυθούν στις παρακάτω υποενότητες.

Ένα άλλο σημαντικό πρόβλημα της HTML είναι ότι παρέχει ένα σύνολο σταθερών προκαθορισμένων ετικετών (παρά τις πολλές επεκτάσεις που έχουν γίνει) με τις οποίες δεν μπορείς να περιγράψεις τη δομή και το περιεχόμενο πολύπλοκων εγγράφων ή άλλων δεδομένων (πχ δεν μπορείς να περιγράψεις μια μουσική παρτιτούρα, τη δομή και τα περιεχόμενα μίας αποθήκης, ενός πίνακα μιας Β.Δ., μία μαθηματική εξίσωση κλπ). Έτσι δημιουργήθηκε η **XML (eXtensible Markup Language – Επεκτάσιμη Γλώσσα Σήμανσης)** η οποία παρέχει εξελιγμένες δυνατότητες δόμησης και περιγραφής περιεχομένου. Με την XML παρέχεται ένα ευέλικτο συντακτικό, με αυστηρούς κανόνες αλλά χωρίς τους περιορισμούς χρήσης συγκεκριμένου συνόλου ετικετών (οι ετικέτες, εκτός από κάποιες με καθορισμένο ρόλο, μπορεί να οριστούν από τον χρήστη) με το οποίο μπορεί να περιγράψεις οποιαδήποτε δομημένη πληροφορία και όχι μόνο κείμενα. Φυσικά το πιο σημαντικό είναι ότι αυτή η πληροφορία μπορεί να επεξεργαστεί αυτόματα (φιλτραριστεί, ταξινομηθεί, αλλαχθεί, αναζητηθεί κλπ) και να παραχθούν ένα σύνολο αποτελέσματα από αυτή χωρίς όρια. Τέτοια αποτελέσματα μπορεί να είναι μια απλή ιστοσελίδα σε HTML, ή μια εικόνα, ή μια Β.Δ. (η σχεδιάσή της και το γέμισμά της με στοιχεία), ή μία διεπαφή εφαρμογής ή οτιδήποτε άλλο (που μπορεί να είναι και συνδυασμός πολλών μαζί). Με την XML λοιπόν λύνεται αποτελεσματικά το σημαντικό πρόβλημα της αυξανόμενης, τόσο σε ποσότητα όσο και σε είδος-πολυπλοκότητα, πληροφορίας που καλούμαστε να διαχειριστούμε στην Διαδίκτυο.

Ένα ακόμα πρόβλημα με την HTML είναι η χαλαρότητα όσον αφορά τη γραφή. Αυτό συμβαίνει για διάφορους λόγους όπως το ότι ορισμένες ετικέτες μπορεί είτε να τις κλείνουμε είτε όχι (πχ την `<br>` ή την `<p>`), να χρησιμοποιούμε για τις τιμές των ιδιοτήτων είτε μονά, είτε διπλά, είτε χωρίς καθόλου εισαγωγικά (quotes) χωρίς πρόβλημα, να γράφουμε τις ετικέτες και τις ιδιότητες άλλοτε με μικρά και άλλοτε με κεφαλαία κλπ. Αυτά και άλλα χαρακτηριστικά «χαλαρότητας» οδηγούν σε κακώς μορφοποιημένα έγγραφα. Για να λυθεί το πρόβλημα αυτό προτάθηκε η **XHTML - eXtensible HTML** (που μπορεί να θεωρηθεί σαν συνδυασμός, της αυστηρότητας, της XML και του συντακτικού της HTML) που στην πραγματικότητα είναι ορισμένη ως μια εφαρμογή της XML, είναι αυστηρότερη και καθαρότερη από την HTML. Με την XHTML τα έγγραφα είναι πια καλώς-μορφοποιημένα (well-formed) με αποτέλεσμα να είναι εύκολη η αυτόματη επεξεργασία τους.

Φυσικά η HTML συνέχισε να βελτιώνεται, για να διορθωθούν τα όποια προβλήματα, με αποτέλεσμα η έκδοση της HTML 4.01 να είναι σχεδόν ίδια με την XHTML (όποιος φυλλομετρητής υποστηρίζει HTML 4.01 υποστηρίζει και XHTML).

Για να είμαστε συμβατοί με την XHTML στην συγγραφή των ιστοσελίδων μας θα ακολουθούμε τους παρακάτω κανόνες:

- 1) Στην αρχή του εγγράφου θα μπαίνει η ετικέτα `<!DOCTYPE>`, η οποία δεν χρειάζεται κλείσιμο, και η οποία λέει στον browser ποια HTML ή XHTML τυποποίηση χρησιμοποιεί το έγγραφό σου. Η HTML 4.01 καθορίζει τρεις τύπους (DTD - Document Type Definition) εγγράφου: Strict, Transitional και Frameset. Παράδειγμα χρήσης της ετικέτας για transitional DTD είναι:
`<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">`
 και για Strict DTD είναι:
`<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">`
- 2) Όλα τα στοιχεία (elements) του εγγράφου πρέπει να είναι σωστά εμφωλευμένα (nested), δηλαδή μία ετικέτα που ανοίγει πρώτη πρέπει να κλείνει τελευταία.
- 3) Όλα τα στοιχεία (elements) του εγγράφου πρέπει να κλείνουν, δηλαδή δεν μπορώ να γράψω την ετικέτα `<br>` χωρίς να την κλείσω. Μάλιστα για ευκολία αντί να γράφω `<br></br>` μπορώ να κλείσω μια ετικέτα αν στο άνοιγμά της πριν το `<>` βάλω ένα κενό και τον χαρακτήρα `</>`, δηλαδή η ετικέτα `<br />` ανοίγει και κλείνει μαζί. Προσοχή πρέπει να δοθεί ώστε να μπαίνει το κενό.
- 4) Όλες τα στοιχεία (elements) του εγγράφου (ετικέτες, ιδιότητες, τιμές ιδιοτήτων) πρέπει να είναι γραμμένα με μικρά γράμματα. Επίσης οι τιμές ιδιοτήτων πρέπει να είναι σε εισαγωγικά.
- 5) Πρέπει να υπάρχει μόνο ένα βασικό στοιχείο (root element) μέσα στο οποίο είναι εμφωλευμένα όλα τα άλλα. Το στοιχείο αυτό είναι η ετικέτα `<html> </html>` μέσα στο οποίο είναι όλα τα άλλα στοιχεία του εγγράφου. Σε αυτή την ετικέτα καλό είναι να καθορίζουμε και την ιδιότητα `xmlns` βάζοντας την τιμή `xmlns="http://www.w3.org/1999/xhtml"`, δηλαδή θα γράφουμε `<html xmlns="http://www.w3.org/1999/xhtml">` αντί για σκέτο `<html>`. Αυτό είναι αναγκαίο για να είναι οι ιστοσελίδες μας XHTML συμβατές αν και γενικά δεν υπάρχει πρόβλημα γιατί η συγκεκριμένη τιμή στην ιδιότητα είναι η προκαθορισμένη και προστίθεται αυτόματα. Πάντως προτείνεται να την χρησιμοποιούμε.

Φυσικά τα παραπάνω αποτελούν βασικούς κανόνες και υπάρχουν και κάποια άλλα στοιχεία που πρέπει να προσέχουμε (πχ η ιδιότητα `alt` στην ετικέτα `img` είναι υποχρεωτική, για τον μοναδικό καθορισμό των αντικειμένων στο έγγραφο χρησιμοποιείται η ιδιότητα `id` κλπ).

Όμως και με την HTML 4.01 συνέχισαν να υπάρχουν πολλά προβλήματα όπως η αδυναμία στο να μπορούμε καθορίσουμε το ακριβές είδος ορισμένων τμημάτων του εγγράφου μας (π.χ. αν κάτι είναι μία ενότητα, ένα άρθρο ή επικεφαλίδα-υποσέλιδο κ.λ.π.), το οποίο δημιουργεί σημαντικό πρόβλημα στη μετάβαση στον σημασιολογικό ιστό, να μπορούμε να εισάγουμε γραφικά στην ιστοσελίδα μας, να μπορούμε να αποθηκεύουμε δεδομένα στον πελάτη (φυλλομετρητή). Τη λύση ήρθε να δώσει η HTML 5 η οποία αποτελεί ένα σημαντικό βήμα προς τον σημασιολογικό ιστό αλλά και στη δυνατότητα να προγραμματίζουμε πλήρεις εφαρμογές που θα εκτελούνται από τον φυλλομετρητή παρέχοντας σημαντικές δυνατότητες δημιουργίας εφαρμογών που θα εκτελούνται χωρίς προβλήματα σε διαφορετικές συσκευές (H/Y, tablets, smartphones, γενικότερα έξυπνες συσκευές κ.λ.π.). Με την HTML 5:

1. Απλοποιήθηκε αρκετά η δομή της σελίδας η οποία έχει τη μορφή:

```
<!DOCTYPE html>
```

```
<html lang="el">
```

```
<head>
```

Στοιχεία της επικεφαλίδας όπως `<title>`, `<meta>`, `<link>`,....

```
</head>
```

```
<body>
```

Περιεχόμενο ιστοσελίδας

</body>

</html>

2. Προστέθηκαν ένα σύνολο νέων ετικετών για να ορίζουμε καλύτερα τη δομή του εγγράφου μας και να μην χρησιμοποιούμε τα div και span που δεν ενημερώνουν για το τι είναι μία περιοχή. Φυσικά τα div και span συνεχίζουν να χρησιμοποιούνται αλλά όπου μπορούμε να καθορίσουμε πιο συγκεκριμένα τη σημασία χρησιμοποιούμε τις νέες ετικέτες όπως <header>, <footer>, <main>, <nav>, <section>, <article>, <aside>, <figure> κ.λ.π..
3. Προστέθηκαν νέα είδη (type) για τα πεδία των φορμών για ημερομηνίες, αριθμούς, email κ.λ.π. καθώς και πολλές νέες ιδιότητες των πεδίων αυτών για έλεγχο, υποχρέωση, εστίαση κ.λ.π. που βοηθούν στη δημιουργία πολύ καλών διεπαφών χρήσης (UI).
4. Προστέθηκαν οι ετικέτες video και audio με αποτέλεσμα οι browsers να μπορούν να παίζουν video και audio χωρίς την ανάγκη χρήσης προσθέτων (plugins) τα οποία δημιουργούσαν σημαντικά προβλήματα και εξάρτησης (Λ.Σ., εταιρείες κ.λ.π.) αλλά και ασφάλειας των ιστοσελίδων-εφαρμογών.
5. Προστέθηκε η ετικέτα canvas με την οποία (με χρήση javascript) μπορούμε να δημιουργήσουμε εφαρμογές που χρησιμοποιούν και γραφικά και θα εκτελούνται από τον browser σε οποιαδήποτε πλατφόρμα (desktop, mobile κλπ). Με το στοιχείο αυτό δεν έχουμε ανάγκη εξωτερικές εφαρμογές όπως flash, java applets κ.λ.π.. Ο canvas είναι μία περιοχή η οποία χρησιμοποιεί το Canvas 2D Context API (<http://www.w3.org/TR/2dcontext2>) και α) μπορούμε να σχεδιάσουμε και μετασχηματίσουμε γραμμές, σχήματα, εικόνες και κείμενο, β) μπορούμε να χειριστούμε συμβάντα που προκαλούνται όπως αυτά από τον χρήστη (π.χ. η κίνηση και το πάτημα ποντικιού και πλήκτρων) ή και από το παράθυρο, το έγγραφο κ.λ.π..
6. Προστέθηκε η δυνατότητα αποθήκευσης στον Φυλλομετρητή. Είναι η νέα δυνατότητα Web Storage API (φυσικά με χρήση javascript) και μας δίνει τη δυνατότητα να δημιουργήσουμε πλήρεις εφαρμογές αφού τώρα μπορούμε να αποθηκεύουμε περισσότερα δεδομένα με μεγαλύτερη ασφάλεια χωρίς να επιβαρύνεται η απόδοση του ιστοτόπου (όπως με τη χρήση cookies).
7. κ.λ.π

Επίσης σημαντικό πρόβλημα δημιουργούσε η χρήση της HTML τόσο για το περιεχόμενο όσο και για την εμφάνιση του εγγράφου. Η εμφάνιση (χρώμα, γραμματοσειρές, διάταξη κλπ) στη HTML καθορίζεται χρησιμοποιώντας τις κατάλληλες ιδιότητες των διαφόρων ετικετών (αλλά και κάποιες ετικέτες), που όμως καθώς υπήρχε η απαίτηση για πιο προχωρημένα χαρακτηριστικά εμφάνισης οδήγησε στις επόμενες εκδόσεις της HTML να έχουν ένα μεγάλο αριθμό νέων ιδιοτήτων (αλλά και ετικετών). Με τον τρόπο αυτό όμως οδηγηθήκαμε σε πολύπλοκα έγγραφα που ήταν δύσκολο να γραφούν αλλά και να συντηρηθούν. Η λύση δόθηκε με το διαχωρισμό της δομής από την εμφάνιση και την υιοθέτηση μιας νέας γλώσσας φύλλων στυλ, της γλώσσας **Διαδοχικών Φύλλων Στυλ (Cascading Style Sheets – CSS)**, η οποία χρησιμοποιείται για την περιγραφή της εμφάνισης ενός εγγράφου που είναι γραμμένο σε κάποια γλώσσα σήμανσης (HTML, XHTML) (στην πραγματικότητα μπορεί να χρησιμοποιηθεί σε κάθε είδος XML εγγράφου). Ο διαχωρισμός αυτός του περιεχομένου από την εμφάνιση βελτίωσε τον τρόπο συγγραφής (ευκολία, ταχύτητα κλπ) και έδωσε και ένα σύνολο νέων χαρακτηριστικών. Συγκεκριμένα επέτρεψε να έχουμε περισσότερη ευελιξία αλλά και έλεγχο στον καθορισμό της εμφάνισης, να μοιράζονται πολλά έγγραφα τα ίδια χαρακτηριστικά εμφάνισης, να παρέχουμε διαφορετικά χαρακτηριστικά εμφάνισης σε ένα έγγραφο όταν προβάλλεται και άλλα όταν εκτυπώνεται κλπ. Επίσης επειδή ο πιο ενδεδειγμένος τρόπος χρήσης των CSS είναι να συνδέουμε το έγγραφο HTML – XHTML με ένα αρχείο CSS που καθορίζει την εμφάνιση, μπορεί ο χρήστης που θα το προβάλλει να το συνδέσει με το δικό του αρχείο καθορίζοντας αυτός την εμφάνιση που προτιμά για το έγγραφο. Με τα CSS καθορίζεις

κανόνες στυλ που θα εφαρμοστούν και χρησιμοποιούν ένα σχήμα προτεραιότητας για να καθορίσουν πως θα εφαρμοστούν οι κανόνες αυτοί όταν κάποιο στοιχείο του εγγράφου ταιριάζει με περισσότερους από έναν. Στο σύγχρονο λοιπόν πρότυπο η HTML χρησιμοποιείται για να περιγράψει μόνο τη δομή του εγγράφου και τα CSS για την εμφάνιση (παρουσίαση, μορφοποίηση) του εγγράφου. Άρα με τη HTML **ορίζουμε τι είναι** κάθε τμήμα του εγγράφου (δομή-σημασία) **και όχι πως φαίνεται**. Σήμερα βρισκόμαστε στην έκδοση CSS3 που παρέχει πολύ σημαντικά στοιχεία μορφοποίησης. Φυσικά και τα CSS είναι ένα πρότυπο του W3C που μπορούμε να το βρούμε στο <https://www.w3.org/Style/>.

Προτείνεται στον αναγνώστη να γράφει τις ιστοσελίδες του (στατικές ή δυναμικές) χρησιμοποιώντας HTML 5 για το περιεχόμενο και CSS για την εμφάνιση. Φυσικά προτείνεται να χρησιμοποιείται και η Javascript για να έχουμε την απαραίτητη λειτουργικότητα στον φυλλομετρητή για ένα ολοκληρωμένο αποτέλεσμα. Οι προδιαγραφές για την HTML και τα CSS είναι του **World Wide Web Consortium (W3C)** (βλέπε επόμενη ενότητα – Πρότυπα). Στο εγχειρίδιο αυτό, όπου στη συνέχεια αναφέρουμε HTML θα εννοούμε HTML 5.

2.1.3 Πρότυπα

Ενώ ο τόπος γέννησης του Παγκόσμιου Ιστού είναι το εργαστήριο σωματιδιακής φυσικής (CERN) της Ελβετίας το 1989, από το 1994 έχει ιδρυθεί, στο εργαστήριο Επιστήμης Υπολογιστών του Massachusetts Institute of Technology (MIT/LCS), το **World Wide Web Consortium (W3C)** (<https://www.w3.org/>) το οποίο αποτελεί και τον κύριο διεθνή Οργανισμό Τυποποίησης για τον Παγκόσμιο Ιστό. Είναι οργανωμένο ως μια κοινοπραξία, όπου τα μέλη του (<https://www.w3.org/Consortium/Member/List>) συνεργάζονται με σκοπό την ανάπτυξη προτύπων (Standards) για τον Παγκόσμιο Ιστό. Φυσικά η κοινοπραξία αυτή διατηρεί και άλλες δραστηριότητες όπως η εκπαίδευση, η διατήρηση ενός ανοικτού φόρουμ συζήτησης για το μέλλον του Παγκόσμιου Ιστού, αλλά και η ανάπτυξη λογισμικού.

Ο κύριος όμως σκοπός δημιουργίας του είναι η δημιουργία των προτύπων για τη διασφάλιση της συμβατότητας, των προϊόντων των κατασκευαστών, για τον Παγκόσμιο Ιστό. Πριν την δημιουργία του W3C υπήρχε μια πληθώρα μη συμβατών εκδόσεων της HTML που υποστήριζαν στα προϊόντα τους (πχ φυλλομετρητές) οι κατασκευαστές που είχε σαν αποτέλεσμα οι ιστοσελίδες να αλλάζουν εμφάνιση ή και να μην εμφανίζονται σωστά όταν χρησιμοποιούσαμε διαφορετικούς φυλλομετρητές.

Φυσικά ο οργανισμός δεν υποχρεώνει κάποιον να ακολουθήσει τις προτάσεις (recommendations) του αλλά αφήνει την επιλογή αυτή στους κατασκευαστές. Αυτό μπορεί να έχει, ακόμα και σήμερα, το αποτέλεσμα διαφορετικά προϊόντα να έχουν διαφορετική συμπεριφορά. Αυτό που όμως έχει τεράστια σημασία για εμάς είναι **να αναπτύσσουμε τους δικούς μας ιστοτόπους (web sites) ακολουθώντας τα πρότυπα** αυτά γιατί έτσι θα διασφαλίσουμε ότι οι ιστοσελίδες μας θα προβάλλονται όπως σχεδιάστηκαν από όλο και περισσότερους φυλλομετρητές και σήμερα και στο μέλλον και δεν θα βρεθούμε στη δυσάρεστη θέση να κάνουμε μελλοντικά αλλαγές. Πολλοί ιστοτόποι που έχουν δημιουργηθεί ακολουθώντας τα πρότυπα του W3C περιλαμβάνουν εικονίδια με τα οποία δηλώνεται η συμβατότητά τους με το αντίστοιχο πρότυπο. Οι δημιουργοί ιστοχώρων μπορούν να ελέγξουν τις ιστοσελίδες τους για συμβατότητα με HTML 5, μέσω του ελεγκτή εγκυρότητας που παρέχεται στον ιστοτόπο του W3C είτε με upload του αρχείου (https://validator.w3.org/#validate_by_upload), είτε γράφοντας το URI (https://validator.w3.org/#validate_by_uri) είτε με απευθείας εισαγωγή του κειμένου (https://validator.w3.org/#validate_by_input), και να ενημερωθεί γιατί (λάθη και προειδοποιήσεις) η ιστοσελίδα του δεν ακολουθεί τα πρότυπα. Ομοίως μπορεί να ελέγξουν και τα CSS που δημιουργούν μέσω του ελεγκτή εγκυρότητας <https://jigsaw.w3.org/css-validator/>.

Τα πρότυπα του W3C που αφορούν τον Παγκόσμιο Ιστό μπορούμε να τα δούμε στην ιστοσελίδα <https://www.w3.org/TR/>.

2.2 Προγραμματισμός στον Παγκόσμιο Ιστό

2.2.1 Προγραμματισμός από την πλευρά του πελάτη

Στον προγραμματισμό από την πλευρά του πελάτη, η ιστοσελίδα HTML εμπλουτίζεται, και με χρήση κατάλληλων ετικετών ή/και ιδιοτήτων ετικετών, με κατάλληλο κώδικα γραμμένο σε κάποια γλώσσα προγραμματισμού, ο οποίος εκτελείται στον υπολογιστή του πελάτη από το πρόγραμμα περιήγησης ιστοσελίδων (Φυλλομετρητής-Web Browser) που χρησιμοποιεί όταν φορτώνεται η ιστοσελίδα για να προβληθεί καθώς και όταν γίνονται διάφορα συμβάντα. Ο κώδικας αυτός έχει συνήθως στόχο:

1. Την παροχή δυναμικών εφέ στην ιστοσελίδα.
2. Τον χειρισμό συμβάντων.
3. Την εκτέλεση ενεργειών-υπολογισμών.
4. Την αύξηση και τον έλεγχο της διαδραστικότητας μεταξύ του χρήστη και της ιστοσελίδας

Οι γλώσσες προγραμματισμού που χρησιμοποιούνται στον προγραμματισμό από την πλευρά του πελάτη είναι γλώσσες συγγραφής σεναρίων (scripting languages) και με τη βοήθεια εντολών, συναρτήσεων, αντικειμένων, ιδιοτήτων των αντικειμένων και συμβάντων στα αντικείμενα αυτά, παρέχουν:

1. Τη δυνατότητα για τον χειρισμό των αντικειμένων του προγράμματος περιήγησης ιστοσελίδων.
2. Τον χειρισμό των αντικειμένων της ιστοσελίδας (π.χ. φόρμες, κουμπιά, πίνακες, άλλων ετικετών κ.λ.π.)
3. Την επεξεργασία των δεδομένων που εμφανίζει η ιστοσελίδα και τη διεξαγωγή υπολογισμών και μετασχηματισμών σε αυτά τα δεδομένα.

Οι πιο γνωστές από αυτές τις γλώσσες είναι η **Javascript** και η **VBScript**. Αυτές οι γλώσσες μαζί με τη χρήση **Διαδοχικών Φύλλων Στυλ (Cascading Style Sheets - CSS)** και HTML μας δίνουν την λεγόμενη **DHTML (Dynamic HTML)**.

2.2.2 Προγραμματισμός από την πλευρά του Εξυπηρετητή

Ο προγραμματισμός από την πλευρά του πελάτη παρουσιάζει μία σειρά περιορισμούς που δεν επιτρέπουν την χρήση του σε εφαρμογές συγκεκριμένων τομέων που απαιτούν πρόσβαση σε συγκεκριμένους πόρους (όπως Βάσεις Δεδομένων και αρχεία που βρίσκονται στον Web εξυπηρετητή ή σε κάποιον άλλο εξυπηρετητή, εφαρμογές που τρέχουν σε συγκεκριμένους εξυπηρετητές με περιορισμένη πρόσβαση κλπ). Επίσης επειδή ο κώδικας εκτελείται στον υπολογιστή του πελάτη (από το συγκεκριμένο πρόγραμμα περιήγησης που διαθέτει), περιορίζεται και από ένα σύνολο άλλων παραγόντων όπως το είδος (εταιρεία αλλά και έκδοση) του προγράμματος περιήγησης (φυλλομετρητή) και των προτιμήσεων που έχει θέσει σε αυτό ο χρήστης (ασφάλεια, λήψη περιεχομένου κλπ). Για παράδειγμα η VBScript δεν υποστηρίζεται από όλα τα προγράμματα περιήγησης και η Javascript μπορεί να έχει απενεργοποιηθεί στις ρυθμίσεις.

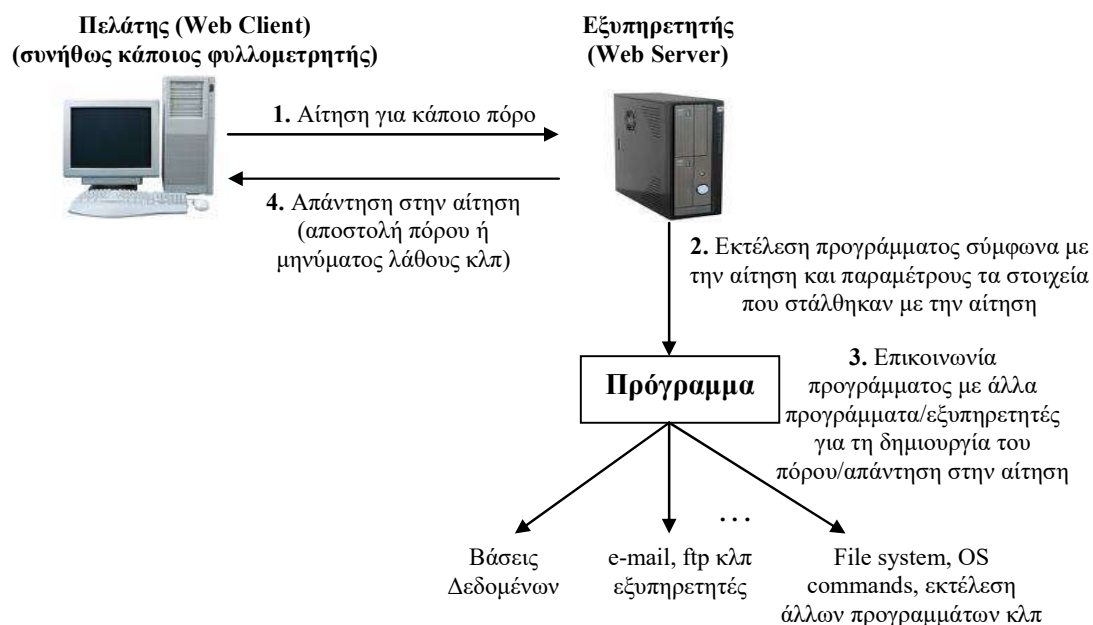
Με όλους αυτούς τους περιορισμούς εφαρμογές τομέων όπως το e-commerce, e-banking, e-government, καθώς και γενικότερης πρόσβασης σε υπό περιορισμό-έλεγχο Βάσεις Δεδομένων (πχ

Ιατρικές, Νομικές Β.Δ., Ηλεκτρονικές Βιβλιοθήκες, Β.Δ. Ειδήσεων κλπ) δεν μπορούν να υλοποιηθούν με τον προγραμματισμό από την πλευρά του πελάτη.

Λύση σε όλα αυτά δίνει ο προγραμματισμός από την πλευρά του εξυπηρετητή στον οποίο συμβαίνουν τα εξής βήματα:

1. Ο πελάτης ζητάει κάποιον πόρο (με βάση το URL αλλά και άλλες ίσως πληροφορίες που υπάρχουν στο μήνυμα αίτησης του πελάτη) από τον εξυπηρετητή.
2. Ο εξυπηρετητής με βάση τον πόρο που ζητήθηκε (και των άλλων πληροφοριών που υπάρχουν στο μήνυμα αίτησης του πελάτη) **εκτελεί κάποιο πρόγραμμα**.
3. **Το πρόγραμμα που εκτελεί ο εξυπηρετητής** μπορεί να χρησιμοποιήσει ένα σύνολο άλλων πόρων (που βρίσκονται είτε στον Η/Υ του εξυπηρετητή είτε σε άλλους Η/Υ), όπως Βάσεις Δεδομένων, Συστήματα Αρχείων, επικοινωνία με άλλα προγράμματα (e-mail, ftp εξυπηρετητές κλπ).
4. **Το πρόγραμμα αυτό με την εκτέλεσή του δημιουργεί ως αποτέλεσμα έναν πόρο** (συνήθως μία ιστοσελίδα HTML, αλλά μπορεί να είναι οποιοσδήποτε άλλος συνηθισμένος πόρος) τον οποίο στέλνει ως απάντηση ο εξυπηρετητής στον πελάτη.

Με αυτόν τον τρόπο το αποτέλεσμα που επιστρέφεται στον πελάτη είναι δυναμικό, αφού εξαρτάται από το τι ζήτησε, τις πληροφορίες που έστειλε στο μήνυμα αίτησης, στοιχεία που υπάρχουν αποθηκευμένα εκείνη τη χρονική στιγμή σε Β.Δ., αρχεία και άλλα προγράμματα με τα οποία επικοινωνεί ο εξυπηρετητής καθώς και τα ίδια τα στοιχεία του χρήστη. Μάλιστα το αποτέλεσμα που θα δημιουργηθεί μπορεί να περιλαμβάνει εκτός από HTML και κώδικα γλωσσών συγγραφής σεναρίων (scripting languages) που θα εκτελεστεί από τον πελάτη, με αποτέλεσμα να έχουμε ένα συνδυασμό προγραμματισμού από την πλευρά του εξυπηρετητή με προγραμματισμό από την πλευρά του πελάτη. Στο επόμενο σχήμα φαίνονται τα 4 διάκριτά βήματα που γίνονται στον προγραμματισμό από την πλευρά του εξυπηρετητή.



Εικόνα 2: Βήματα επικοινωνίας/λειτουργιών στον προγραμματισμό από την πλευρά του εξυπηρετητή

Οι πιο γνωστές γλώσσες/τεχνολογίες που παρέχουν τη δυνατότητα για προγραμματισμό από την πλευρά του εξυπηρετητή είναι η **PHP**, **ASP**, **JSP** και φυσικά η χρήση **CGI** (με οποιοδήποτε εκτελέσιμο γραμμένο σε οποιαδήποτε γλώσσα προγραμματισμού πχ C, Perl κλπ)

Ο σκοπός του εγχειριδίου αυτού είναι ουσιαστικά να σας εισάγει στον προγραμματισμό από την πλευρά του εξυπηρετητή, δηλαδή στη διαδικασία να γράφετε το **πρόγραμμα** αυτό που εκτελεί ο εξυπηρετητής (βήμα 2), το οποίο επικοινωνεί και χρησιμοποιεί ένα σύνολο άλλων πόρων (βήμα 3) και το οποίο παράγει ως αποτέλεσμα (δυναμικά) τον πόρο που επιστρέφεται στον πελάτη (βήμα 4). Στην περίπτωση μας το πρόγραμμα αυτό θα είναι η PHP και θα γίνεται επικοινωνία με Β.Δ. που βρίσκονται στο RDBMS του MySQL/MariaDB.

3 Τι χρειαζόμαστε για την PHP-MySQL

Για να μπορέσετε να χρησιμοποιήσετε PHP με την οποία να έχετε πρόσβαση σε MySQL Βάσεις Δεδομένων, σύμφωνα με το μοντέλο του προγραμματισμού από την πλευρά του εξυπηρετητή χρειάζονται:

1. Ένας εξυπηρετητής Παγκόσμιου Ιστού (Web Server) ο οποίος να υποστηρίζει PHP.
2. Εγκατάσταση της PHP στον συγκεκριμένο εξυπηρετητή με κάποιον τρόπο (πχ ως module ή ως CGI εκτελέσιμο).
3. Εγκατάσταση του MySQL εξυπηρετητή (στο ίδιο ή διαφορετικό H/Y).

Ευτυχώς για τη διευκόλυνσή μας υπάρχουν ολοκληρωμένα προ-ρυθμισμένα πακέτα (pre-configured packages) που εγκαθιστούν όλα τα παραπάνω, και κάνουν τις απαραίτητες ρυθμίσεις, για τα περισσότερα λειτουργικά συστήματα. Στο εγχειρίδιο αυτό προτείνουμε 2 ολοκληρωμένα πακέτα, τόσο για την ευκολία εγκατάστασης και χρήσης αλλά και γιατί ότι εγκαθίσταται είναι **ΕΛ/ΛΑΚ**.

Το πρώτο είναι το **XAMPP** το οποίο είναι ένα πακέτο που δημιουργήθηκε από την μη κερδοσκοπική ομάδα εργασίας **Apache Friends**, της οποίας στόχος είναι η προώθηση της χρήσης του Apache Web Server. Το πακέτο αυτό εγκαθιστά στον υπολογιστή μας:

1. Τον Apache Web Server (ένας από τους δημοφιλέστερους και καλύτερους web server) με υποστήριξη PHP (με τις απαραίτητες ρυθμίσεις)
2. Τον MySQL Server
3. Τον Filezilla FTP Server

Επίσης το πακέτο περιλαμβάνει και ένα σωρό βοηθητικές εφαρμογές και εργαλεία για τη διαχείριση των εξυπηρετητών (πχ MySQLAdmin, PHPMyAdmin, Webalizer κλπ), καθώς και πολλές έτοιμες εφαρμογές επίδειξης της PHP (Demo PHP Applications) με τις οποίες έχετε άμεσα πρόσβαση σε έτοιμα παραδείγματα με τον κώδικά τους. Το πακέτο παρέχεται για αρκετά Λ.Σ. (Linux, Windows, Solaris, MAC OS κλπ) από την ιστοσελίδα <https://www.apachefriends.org/index.html>.

Το **Wamp (WampServer)** είναι άλλο ένα πολύ καλό πακέτο που μπορεί να χρησιμοποιηθεί και το οποίο περιλαμβάνει:

1. Τον Apache Web Server με υποστήριξη PHP
2. Τον MySQL Server

Και αυτό το πακέτο περιλαμβάνει βοηθητικές εφαρμογές και εργαλεία για τη διαχείριση των εξυπηρετητών (πχ SQLiteManager, PHPMYAdmin). Το πακέτο παρέχεται μόνο για το Λ.Σ. των Windows από την ιστοσελίδα <http://www.wampserver.com/en/>

Φυσικά αν ήδη έχετε εγκατεστημένα κάποια από αυτά, μπορείτε απλώς να εγκαταστήσετε (ή να κάνετε τις απαραίτητες ρυθμίσεις για να ενεργοποιήσετε) τα υπόλοιπα. Έτσι αν έχετε ήδη εγκατεστημένο τον Apache Web Εξυπηρετητή ή κάποιον άλλον Web Εξυπηρετητή μπορείτε να βρείτε οδηγίες για την εγκατάσταση/ενεργοποίηση της PHP στο <http://www.php.net> και οδηγίες για την εγκατάσταση του MySQL στο <http://www.mysql.com/>.

Για τη MySQL προτείνουμε να χρησιμοποιήσετε:

1. Είτε το **CLI (Command Line Interface)** όπου συνδέεστε και δίνετε τις εντολές σας από τη γραμμή εντολών (πχ από ένα παράθυρο γαμμής εντολών των Windows)
2. Είτε το γραφικό περιβάλλον (μέσω Web) **PHPMYAdmin** που με τη χρήση ενός φυλλομετρητή συνδέεστε και με γραφικό τρόπο διαχειρίζεστε τις Β.Δ. σας. Μάλιστα ο **PHPMYAdmin** υπάρχει και στα 2 ολοκληρωμένα προ-ρυθμισμένα πακέτα (xampp και winamp) που είδαμε παραπάνω και είναι και αυτός γραμμένος σε PHP.
3. Είτε το γραφικό περιβάλλον **MySQL Workbench** το οποίο είναι εφαρμογή με την οποία συνδέστε και διαχειρίζεστε τις Β.Δ. σας γραφικά (ολοκληρωμένο γραφικό IDE), και την οποία μπορείτε να κατεβάσετε από την ιστοσελίδα της MySQL και συγκεκριμένα από τον σύνδεσμο (<https://dev.mysql.com/downloads/workbench/>).

Εμείς θα χρησιμοποιήσουμε στα παραδείγματά μας στο κεφάλαιο που θα δούμε τη MySQL το CLI και τον **PHPMYAdmin**.

Επίσης καλό θα ήταν αντί να γράφετε PHP σε έναν απλό κειμενογράφο να χρησιμοποιήσετε έναν ειδικό για PHP κειμενογράφο ο οποίος σας βοηθάει να αποφύγετε πολλά λάθη κατά τη συγγραφή του κώδικα. Τέτοιοι κειμενογράφοι υπάρχουν διαθέσιμοι δωρεάν από διάφορες εταιρείες/οργανισμούς/ομάδες χρηστών και συνήθως εμφανίζουν με διαφορετικό χρώμα το κάθε τι στον κώδικα (άλλο χρώμα τα σχόλια, άλλο οι εντολές, άλλο τα αλφαριθμητικά κλπ) και σε βοηθάνε να αποφύγεις πολύ απλά αλλά δύσκολο να εντοπισθούν λάθη. Ένας αρκετά εύχρηστος και δωρεάν είναι ο **NotePad++** που μπορείτε να προμηθευτείτε από την ιστοσελίδα <https://notepad-plus-plus.org/>. Φυσικά για πιο ολοκληρωμένη δουλειά, με διαχείριση του project σας και περισσότερες δυνατότητες προτείνεται να χρησιμοποιήσετε κάποιο ολοκληρωμένο περιβάλλον ανάπτυξης (IDE) όπως το **Netbeans** το οποίο μπορείτε να προμηθευτείτε από την ιστοσελίδα <https://netbeans.org/>.

4 MySQL

Ο MySQL εξυπηρετητής είναι ένας πολύ γρήγορος, πολλών νημάτων (multi-threaded) , πολλών χρηστών (multi-user), σταθερός και ασφαλής εξυπηρετητής (Σύστημα) Διαχείρισης Βάσεων Δεδομένων SQL (**Data Base Management System**). Είναι σχεδιασμένος για χρήση σε μεγάλα συστήματα που εκτελούν κρίσιμες και πολύ βαριές λειτουργίες σε πολύ μεγάλες Βάσεις Δεδομένων. Επίσης είναι σχεδιασμένος για να μπορεί να χρησιμοποιηθεί με εύκολο τρόπο είτε ενσωματώνοντας τον σε γενικού σκοπού λογισμικό, είτε επικοινωνώντας πολύ εύκολα με αυτόν, επιτρέποντας έτσι την χρήση του σε ένα τεράστιο εύρος εφαρμογών διαφόρων μεγεθών.

Ο MySQL εξυπηρετητής διατίθεται με διπλή άδεια χρήσης:

1. Μπορεί να χρησιμοποιηθεί ως ΕΛ/ΛΑΚ με τους όρους της GNU General Public License (<http://www.fsf.org/licenses/>).
2. Μπορεί επίσης να αγοραστούν εμπορικές άδειες χρήσης (<http://www.mysql.com/company/legal/licensing/>) ώστε να χρησιμοποιηθεί για εμπορικούς λόγους (να παραχθεί προϊόν που μπορεί να μεταπωλείται και δεν είναι ΕΛ/ΛΑΚ).

Η εγκατάσταση του MySQL εξυπηρετητή μπορεί να γίνει με πολύ εύκολο τρόπο σε διάφορα Λ.Σ..

4.1 Περιβάλλοντα Επικοινωνίας

Το βασικό περιβάλλον επικοινωνίας με τον MySQL εξυπηρετητή γίνεται μέσω **γραμμής εντολών (CLI - Command Line Interface)** όπου ο χρήστης, εφόσον έχει συνδεθεί στον MySQL εξυπηρετητή με τα κατάλληλα στοιχεία (όνομα χρήστη και κωδικό), πληκτρολογεί την εντολή και αυτή εκτελείται από τον εξυπηρετητή και λαμβάνει πίσω το αποτέλεσμα.

Επίσης παρέχονται ένα σύνολο από εργαλεία, τα οποία σου επιτρέπουν να κάνεις τις περισσότερες ενέργειες χρησιμοποιώντας γραφικό περιβάλλον. Όπως αναφέραμε και στο προηγούμενο κεφάλαιο εμείς θα χρησιμοποιήσουμε εδώ το CLI και το **PHPMyAdmin**. Στην επόμενη ενότητα θα δούμε σύντομα τις εντολές SQL με τις οποίες κάνουμε τις βασικές ενέργειες διαχείρισης Β.Δ. από τη γραμμή εντολών και στην αμέσως επόμενη θα δούμε τις ενέργειες αυτές μέσω παραδειγμάτων από τον **PHPMyAdmin**.

Ο MySQL εξυπηρετητής πριν εκκινήσει διαβάσει μια σειρά από επιλογές από ένα αρχείο επιλογών στο οποίο μπορούμε να ορίσουμε διάφορες παραμέτρους. Το αρχείο αυτό στα Windows είναι το **my.ini** (βρίσκεται στον φάκελο των Windows, μπορούμε να το δούμε με **echo %WINDIR%**, φυσικά μπορεί με την επιλογή εκκίνησης **--defaults-file** να έχει επιλεγεί κάποιο άλλο) ενώ στο Unix είναι το αρχείο **my.cnf** (συνήθως στο /etc/my.cnf ή /etc/mysql/my.cnf κλπ). Επίσης μπορεί να δημιουργηθούν τέτοια αρχεία και σε άλλους φακέλους (πχ στην περιοχή του χρήστη στο unix) αλλά να δοθούν και επιλογές μέσω της γραμμής εντολών εκκίνησης του εξυπηρετητή. Περισσότερες πληροφορίες για τις επιλογές του my.ini/my.cnf παρατίθενται στο **Παράρτημα – Ρυθμίσεις Web Εξυπηρετητή, PHP και MySQL**.

4.2 Διαχείριση Β.Δ. MySQL από τη Γραμμή Εντολών

Οι Βασικές ενέργειες που κάνει ο χρήστης με τον MySQL είναι η εκτέλεση ενός συνόλου εντολών/queries (ερωτημάτων) με τη γλώσσα SQL (καθώς και άλλες εντολές) που έχουν ως στόχο:

1. Τη δημιουργία και Διαγραφή Βάσεων Δεδομένων.
2. Τη δημιουργία, Διαγραφή και Αλλαγή πινάκων σε μία Β.Δ..
3. Την Επιλογή, Εισαγωγή, Ενημέρωση και Διαγραφή εγγραφών σε πίνακα/πίνακες μιας Β.Δ..

4. Την δημιουργία Χρηστών και την ανάθεση/Αφαίρεση δικαιωμάτων για συγκεκριμένες Β.Δ. και πίνακες Β.Δ..

Στις παρακάτω υποενότητες παρουσιάζονται με συντομία οι εντολές της SQL με τις οποίες γίνονται οι παραπάνω ενέργειες καθώς και οι εντολές σύνδεσης με τον MySQL. Είναι σημαντικό να μπορεί ο χρήστης να γράφει ένα σύνολο βασικών εντολών/ερωτημάτων σε SQL γιατί με αυτόν τον τρόπο θα επικοινωνεί και θα διαχειρίζεται τις Β.Δ. του μέσα από τη PHP. Για περισσότερες πληροφορίες προτείνεται στον αναγνώστη να μελετήσει το Εγχειρίδιο Αναφοράς (Reference Manual) που μπορεί να βρει στην ιστοσελίδα της MySQL στα εγχειρίδια (<http://dev.mysql.com/doc/>).

4.2.1 Σύνδεση με τον MySQL Εξυπηρετητή και εκτέλεση απλών εντολών

Για να συνδεθούμε στον MySQL εξυπηρετητή χρησιμοποιούμε την εντολή **mysql** την οποία εκτελούμε στη γραμμή εντολών (πρέπει να βρίσκεται στο path). Παίρνει μία μεγάλη σειρά επιλογών, από τις οποίες οι πιο συνηθισμένες είναι:

Επιλογή	Χρήση
--user=user_name ή -u user_name	Συνδέσου με όνομα χρήστη user_name
--password=[password] ή -p[password]	Συνδέσου με κωδικό password (μεταξύ επιλογής και password δεν υπάρχουν κενά)
--database=db_name ή -D db_name	Συνδέσου για να χρησιμοποιήσεις τη Β.Δ. db_name
--host=host_name ή -h host_name	Συνδέσου στο server που τρέχει στον H/Y host_name (μπορούμε να βάλουμε και IP)
--port=port_num ή -P port_num	Συνδέσου στο server που «ακούει» στη θύρα port_num
--prompt=format_str	Εμφάνισε ως γραμμή εντολών το format_str

Αν δεν δοθεί κάποια επιλογή χρησιμοποιούνται γι' αυτήν οι προεπιλεγμένες ρυθμίσεις (χρήστη, θύρας κλπ).

Παραδείγματα:

Για να συνδεθούμε στον MySQL Εξυπηρετητή στο τρέχον μηχάνημα με τις προεπιλεγμένες ρυθμίσεις (χρήστη, θύρα κλπ). Φυσικά θα μας ζητηθεί κωδικός (αν υπάρχει) για τον συγκεκριμένο χρήστη.

cli> mysql

Για να συνδεθούμε στον MySQL Εξυπηρετητή που τρέχει στον H/Y **pc1.company.gr** με όνομα χρήστη **user1** και κωδικό **pass1**

cli>mysql -u user1 -ppass1 -h pc1.company.gr

Αφού συνδεθούμε μας εμφανίζεται το prompt (το προεπιλεγμένο είναι mysql>) όπου περιμένει ο εξυπηρετητής να δώσουμε την εντολή που θα εκτελέσει.

Παράδειγμα επιτυχημένης σύνδεσης (ανοίγουμε ένα command prompt και πληκτρολογούμε):

cli>mysql -u root -p

Enter password: *****

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 221

Server version: 5.0.45-community-nt MySQL Community Edition (GPL)

Type 'help;' or 'h' for help. Type 'c' to clear the buffer.

mysql>

Με **help** μας εμφανίζεται μία λίστα των διαθέσιμων εντολών και με **exit** ή **quit** αποσυνδεόμαστε.

Οι εντολές που θα δίνουμε **θα πρέπει να τερματίζονται (εκτός από ορισμένες) με ελληνικό ερωτηματικό «;»**. Αν σε κάποια εντολή, που απαιτεί ερωτηματικό, δεν δώσουμε και πατήσουμε <enter>, τότε η γραμμή εντολών θα αλλάξει γραμμή και θα βγάλει ένα υπο-prompt «->» για να συνεχίσουμε να γράφουμε την ίδια εντολή και θα γίνεται το ίδιο μέχρι να δώσουμε ελληνικό ερωτηματικό και <enter>. Με αυτό τον τρόπο μπορώ να γράψω μία εντολή (που είναι αρκετά μακροσκελής) σε πολλές γραμμές για να είναι και πιο ευανάγνωστη. Επίσης πατώντας τα πάνω-κάτω βελάκια του πληκτρολογίου μετακινούμε στις προηγούμενες εντολές που έχω πληκτρολογήσει, ενώ πατώντας το δεξιό βελάκι συμπληρώνονται ένας-ένας οι χαρακτήρες της τελευταίας εντολής που έγραψα. Οι εντολές μπορεί να δίνονται **και με κεφαλαία και με μικρά** (δεν υπάρχει διαχωρισμός) και **μπορεί να μην γράφονται ολόκληρες**, δηλαδή αρκεί να γράφουμε αρκετούς από τους πρώτους χαρακτήρες της εντολής ώστε να καθορίζεται μονοσήμαντα ποια εντολή είναι.

Όταν γράφω μια εντολή που αναφέρεται σε κάποια ενέργεια σε Β.Δ. (δημιουργία, επιλογή, ενημέρωση κλπ) παίρνω πάντα απάντηση από τον εξυπηρετητή, για την επιτυχία ή μη καθώς και πόσο χρόνο πήρε για την ολοκλήρωσή της.

Παρακάτω βλέπουμε μερικά παραδείγματα εντολών, οι οποίες δεν αφορούν Β.Δ., που μπορούμε να δώσουμε στον εξυπηρετητή.

Εντολή εκτύπωσης έκδοσης, ημερομηνίας και τρέχουσας ώρας και το αποτέλεσμα:

mysql> select version(), current_date(), now();

version()	current_date()	now()
5.0.45-community-nt	2009-08-22	2009-08-22 14:35:16

1 row in set (0.00 sec)

mysql>

Εντολή χρήσης του εξυπηρετητή ως υπολογιστική μηχανή και αποτέλεσμα:

mysql> select 5/4, tan(17)*4+12.5;

5/4	tan(17)*4+12.5
1.2500	26.475662581899

1 row in set (0.03 sec)

mysql>

4.2.2 Δημιουργία χρηστών και ανάθεση δικαιωμάτων

Οι πληροφορίες που αφορούν τους λογαριασμούς των χρηστών διατηρούνται στη Β.Δ. **mysql** του MySQL εξυπηρετητή. Το όνομα του κάθε χρήστη έχει τη μορφή **username@hostname**. Αν καθοριστεί το hostname σε εντολές διαχείρισης χρήστη τότε η εντολή αυτή αναφέρεται στον συγκεκριμένο χρήστη με όνομα το username που συνδέεται από το hostname. Αν δεν καθοριστεί το

hostname τότε υπονοείται για hostname το % (οτιδήποτε) και η εντολή αναφέρεται σε οποιονδήποτε χρήστη με όνομα username που θα συνδεθεί από οποιοδήποτε hostname.

Για να δημιουργήσουμε έναν χρήστη με ένα συγκεκριμένο password στον MySQL εξυπηρετητή χρησιμοποιούμε την εντολή CREATE USER η οποία έχει την ακόλουθη σύνταξη:

```
CREATE USER user [IDENTIFIED BY [PASSWORD] 'password']  
[, user [IDENTIFIED BY [PASSWORD] 'password']] ...
```

Οπότε για να δημιουργήσω ένα χρήστη με όνομα **user1** και κωδικό **user1pswd** γράφουμε:

```
mysql> create user user1 identified by 'user1pswd';
```

Για να διαγράψω έναν χρήστη χρησιμοποιώ την εντολή DROP user η οποία έχει την ακόλουθη σύνταξη:

```
DROP USER user [, user] ...
```

Οπότε για να διαγράψω το χρήστη με όνομα user1 γράφουμε:

```
mysql> drop user user1;
```

Για να δώσω δικαιώματα σε έναν χρήστη για κάποια Β.Δ. χρησιμοποιώ την εντολή GRANT Η οποία έχει την ακόλουθη σύνταξη.

```
GRANT  
  priv_type [(column_list)]  
  [, priv_type [(column_list)]] ...  
ON [object_type] priv_level  
TO user [IDENTIFIED BY [PASSWORD] 'password']  
  [, user [IDENTIFIED BY [PASSWORD] 'password']] ...  
[REQUIRE {NONE | ssl_option [[AND] ssl_option] ...}]  
[WITH with_option [with_option] ...]
```

object_type:

```
TABLE  
| FUNCTION  
| PROCEDURE
```

priv_level:

```
*  
| *. *  
| db_name. *  
| db_name.tbl_name  
| tbl_name  
| db_name.routine_name
```

with_option:

```
GRANT OPTION  
| MAX_QUERIES_PER_HOUR count  
| MAX_UPDATES_PER_HOUR count  
| MAX_CONNECTIONS_PER_HOUR count  
| MAX_USER_CONNECTIONS count
```

ssl_option:

```
SSL  
| X509  
| CIPHER 'cipher'
```



```
| ISSUER 'issuer'  
| SUBJECT 'subject'
```

Το *priv_type* μπορεί να είναι ένα από ένα σύνολο δικαιωμάτων (ALL, ALTER, ALTER ROUTINE, CREATE, DELETE, SELECT, INSERT κλπ). Μπορώ να δώσω δικαιώματα σε επίπεδο καθολικό (global – όλες οι Β.Δ.), σε επίπεδο μιας Β.Δ. (όλοι οι πίνακες της συγκεκριμένης Β.Δ.), σε επίπεδο πίνακα (όλες οι στήλες του συγκεκριμένου πίνακα), σε επίπεδο στηλών (οι συγκεκριμένες στήλες του συγκεκριμένου πίνακα κλπ).

Οπότε αν θα ήθελα να δώσω δικαιώματα στον χρήστη **user1@localhost** για όλες τις ενέργειες (εκτός της ενέργειας του να δίνει ο ίδιος δικαιώματα) για όλους τους πίνακες της Β.Δ. **testdb** θα μπορούσα να γράψω:

```
mysql> grant all on testdb.* to user1@localhost;
```

Αν θα ήθελα να δώσω πλήρη δικαιώματα στον χρήστη **user1** για όλες τις ενέργειες σε όλες τις Β.Δ. (global privileges) θα έγραφα:

```
mysql> grant all on *.* to user1;
```

Αν θα ήθελα να δώσω δικαιώματα στον χρήστη **user1** για να επιλέγει και να εισάγει στη Β.Δ. **testdb** θα έγραφα:

```
mysql> grant select, insert on testdb.* to user1;
```

Για να αφαιρέσω δικαιώματα από κάποιον χρήστη χρησιμοποιώ την εντολή REVOKE της οποίας η σύνταξη έχει ως εξής:

REVOKE

```
priv_type [(column_list)]  
[, priv_type [(column_list)]] ...  
ON [object_type] priv_level  
FROM user [, user] ...
```

REVOKE ALL PRIVILEGES, GRANT OPTION

```
FROM user [, user] ...
```

Με τη δεύτερη σύνταξη παίρνεις όλα τα δικαιώματα από ένα χρήστη. Αν ήθελα να αφαιρέσω όλα τα δικαιώματα από το χρήστη **user1** θα έγραφα:

```
mysql> revoke all privileges, grant option from user1;
```

Με την πρώτη μπορείς να αφαιρείς επιλεκτικά αυτά που θέλεις. Αν θα ήθελα να αφαιρέσω τα δικαιώματα που έδωσα στον χρήστη **user1** για να επιλέγει και να εισάγει στη Β.Δ. **testdb** θα έγραφα:

```
mysql> revoke select, insert on testdb.* from user1;
```

Φυσικά υπάρχουν διάφοροι περιορισμοί και επιπλέον πληροφορίες για τις εντολές αυτές. Επίσης υπάρχουν και εντολές για μετονομασία χρήστη (RENAME) καθώς και απόδοσης/αλλαγής password (SET PASSWORD) κλπ.

4.2.3 Εντολές Β.Δ.

Ακολουθούν μια σειρά υποενότητες που παρουσιάζουν τις βασικότερες εντολές που χρησιμοποιούμε για διαχείριση Β.Δ..

4.2.3.1 Εντολές για Β.Δ. (Προβολή, Επιλογή, Δημιουργία, Διαγραφή)

Όταν έχουμε συνδεθεί σε ένα MySQL εξυπηρετητή δεν έχει επιλεγεί ακόμα κάποια Β.Δ. με την οποία θα εργαστούμε. Για να δούμε ποιες Β.Δ. υπάρχουν στον εξυπηρετητή αρκεί να γράψουμε την εντολή **show databases**. Πχ προβολή υπαρχόντων Β.Δ.

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| δematiki |
| cdc|col |
| labodb |
| mysql |
| newsdb |
| ordersdb |
| phpmysqladmin |
| politesdb |
| test |
+-----+
10 rows in set (0.25 sec)
```

```
mysql>
```

Για να δημιουργήσω μια Β.Δ. χρησιμοποιώ την εντολή **create database <όνομα ΒΔ>**. Πχ

```
mysql> create database newsdb;
```

Η πλήρης σύνταξη της εντολής είναι:

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
[create_specification] ...
```

όπου

create_specification:

```
[DEFAULT] CHARACTER SET [=] charset_name
| [DEFAULT] COLLATE [=] collation_name
```

Πχ Με την επόμενη εντολή δημιουργώ τη Β.Δ. με το συγκεκριμένο σύνολο χαρακτήρων και collation:

```
mysql> create database newsdb DEFAULT CHARACTER SET utf8 COLLATE utf8_unicode_ci;
```

Από εδώ και πέρα ότι πίνακα δημιουργήσω στη συγκεκριμένη Β.Δ., τα πεδία του που είναι δεδομένα χαρακτήρων, θα έχουν το συγκεκριμένο **encoding (character set)** και το συγκεκριμένο **collation**. Το **encoding (character set)** καθορίζει την κωδικοποίηση των χαρακτήρων, δηλαδή την αντιστοίχισή τους με μια δυαδική ακολουθία (1, 2 ή περισσότερα bytes) και υπάρχουν διάφορα **encodings (character sets)** που καλύπτουν διάφορες γλώσσες. Το **collation** καθορίζει τους κανόνες σύμφωνα με τους οποίους τα δεδομένα χαρακτήρων συγκρίνονται, ταξινομούνται κλπ. Όπως θα δούμε στη συνέχεια μπορώ να ορίσω σύνολο χαρακτήρων και collation σε επίπεδο Β.Δ., πίνακα και πεδίου πίνακα. **Η ρύθμιση που κυριαρχεί είναι αυτή που είναι πιο εξειδικευμένη**, πχ αν ορίσω άλλο σύνολο για την Β.Δ. και άλλο για τον πίνακα, επικρατεί η ρύθμιση του πίνακα. Όπως θα δούμε και στις επόμενες υποενότητες αλλά και στο επόμενο κεφάλαιο **είναι σημαντικό να δημιουργούμε τη Β.Δ. με τους πίνακές της με σύνολο χαρακτήρων utf8 και collation utf8_unicode_ci**, γιατί με αυτό τον τρόπο οι Β.Δ. μας είναι πολυγλωσσικές και δεν θα έχουμε πρόβλημα όταν οι χρήστες μας αποθηκεύουν στοιχεία στις εγγραφές που είναι σε πολλές γλώσσες.

Μπορώ να δω όλα τα encodings (character sets) καθώς και τα αντίστοιχα default collations (δηλαδή αν επιλέξω ένα encoding και δεν θέσω collation άμεσα θα τεθεί το default που του αντιστοιχεί) εκτελώντας την εντολή **show character set**. Πχ

```
mysql> show character set;
```

<i>Charset</i>	<i>Description</i>	<i>Default collation</i>	<i>Maxlen</i>
<i>big5</i>	<i>Big5 Traditional Chinese</i>	<i>big5_chinese_ci</i>	<i>2</i>
<i>dec8</i>	<i>DEC West European</i>	<i>dec8_swedish_ci</i>	<i>1</i>
<i>cp850</i>	<i>DOS West European</i>	<i>cp850_general_ci</i>	<i>1</i>
<i>hp8</i>	<i>HP West European</i>	<i>hp8_english_ci</i>	<i>1</i>
<i>koi8r</i>	<i>KOI8-R Relcom Russian</i>	<i>koi8r_general_ci</i>	<i>1</i>
<i>latin1</i>	<i>cp1252 West European</i>	<i>latin1_swedish_ci</i>	<i>1</i>
<i>latin2</i>	<i>ISO 8859-2 Central European</i>	<i>latin2_general_ci</i>	<i>1</i>
<i>swe7</i>	<i>7bit Swedish</i>	<i>swe7_swedish_ci</i>	<i>1</i>
<i>ascii</i>	<i>US ASCII</i>	<i>ascii_general_ci</i>	<i>1</i>
<i>ujis</i>	<i>EUC-JP Japanese</i>	<i>ujis_japanese_ci</i>	<i>3</i>
<i>sjis</i>	<i>Shift-JIS Japanese</i>	<i>sjis_japanese_ci</i>	<i>2</i>
<i>hebrew</i>	<i>ISO 8859-8 Hebrew</i>	<i>hebrew_general_ci</i>	<i>1</i>
<i>tis620</i>	<i>TIS620 Thai</i>	<i>tis620_thai_ci</i>	<i>1</i>
<i>euckr</i>	<i>EUC-KR Korean</i>	<i>euckr_korean_ci</i>	<i>2</i>
<i>koi8u</i>	<i>KOI8-U Ukrainian</i>	<i>koi8u_general_ci</i>	<i>1</i>
<i>gb2312</i>	<i>GB2312 Simplified Chinese</i>	<i>gb2312_chinese_ci</i>	<i>2</i>
<i>greek</i>	<i>ISO 8859-7 Greek</i>	<i>greek_general_ci</i>	<i>1</i>
<i>cp1250</i>	<i>Windows Central European</i>	<i>cp1250_general_ci</i>	<i>1</i>
<i>gbk</i>	<i>GBK Simplified Chinese</i>	<i>gbk_chinese_ci</i>	<i>2</i>
<i>latin5</i>	<i>ISO 8859-9 Turkish</i>	<i>latin5_turkish_ci</i>	<i>1</i>
<i>armscii8</i>	<i>ARMSCII-8 Armenian</i>	<i>armscii8_general_ci</i>	<i>1</i>
<i>utf8</i>	<i>UTF-8 Unicode</i>	<i>utf8_general_ci</i>	<i>3</i>
<i>ucs2</i>	<i>UCS-2 Unicode</i>	<i>ucs2_general_ci</i>	<i>2</i>
<i>cp866</i>	<i>DOS Russian</i>	<i>cp866_general_ci</i>	<i>1</i>
<i>keybcs2</i>	<i>DOS Kamenicky Czech-Slovak</i>	<i>keybcs2_general_ci</i>	<i>1</i>
<i>macce</i>	<i>Mac Central European</i>	<i>macce_general_ci</i>	<i>1</i>
<i>macroman</i>	<i>Mac West European</i>	<i>macroman_general_ci</i>	<i>1</i>
<i>cp852</i>	<i>DOS Central European</i>	<i>cp852_general_ci</i>	<i>1</i>
<i>latin7</i>	<i>ISO 8859-13 Baltic</i>	<i>latin7_general_ci</i>	<i>1</i>

<i>cp1251</i>	<i>Windows Cyrillic</i>	<i>cp1251_general_ci</i>	<i>1</i>	
<i>cp1256</i>	<i>Windows Arabic</i>	<i>cp1256_general_ci</i>	<i>1</i>	
<i>cp1257</i>	<i>Windows Baltic</i>	<i>cp1257_general_ci</i>	<i>1</i>	
<i>binary</i>	<i>Binary pseudo charset</i>	<i>binary</i>	<i>1</i>	
<i>geostd8</i>	<i>GEOSTD8 Georgian</i>	<i>geostd8_general_ci</i>	<i>1</i>	
<i>cp932</i>	<i>SJIS for Windows Japanese</i>	<i>cp932_japanese_ci</i>	<i>2</i>	
<i>ucjps</i>	<i>UJIS for Windows Japanese</i>	<i>ucjps_japanese_ci</i>	<i>3</i>	
+-----+-----+-----+-----+				

36 rows in set (0.02 sec)

mysql>

Για να διαλέξω να εργαστώ με μια Β.Δ., δηλαδή ότι εντολές δίνω από εκεί και πέρα και αφορούν τη δημιουργία/αλλαγή/διαγραφή πινάκων και ερωτήματα επιλογής/εισαγωγής/ενημέρωσης κλπ θα γίνονται στους πίνακες της συγκεκριμένης Β.Δ., γράφω την εντολή **use <όνομα ΒΔ>**. Πχ

mysql> use newsdb;

Database changed

mysql>

Μετά από αυτή τη εντολή ότι ενέργειες κάνω θα αφορούν τη συγκεκριμένη Β.Δ., μέχρι να επιλέξω, πάλι με την ίδια εντολή, να χρησιμοποιήσω κάποια άλλη Β.Δ..

Για να διαγράψω μια υπάρχουσα Β.Δ., γράφω την εντολή **drop database <όνομα ΒΔ>**. Πχ

mysql> drop database newsdb;

Η πλήρης σύνταξη της εντολής είναι:

DROP {DATABASE | SCHEMA} [IF EXISTS] db_name

4.2.3.2 Ερωτήματα SQL σε Πίνακες

Αφού έχουμε επιλέξει να εργαστούμε με μια Β.Δ. μπορούμε να αρχίσουμε να δίνουμε ερωτήματα SQL για ενέργειες πάνω σε πίνακες. Τέτοια ερωτήματα περιλαμβάνουν τη δημιουργία πίνακα (**CREATE**), αλλαγή δομής πίνακα (**ALTER TABLE**), διαγραφή πίνακα (**DROP TABLE**), επιλογή και προβολή εγγραφών πίνακα που πληρούν συγκεκριμένες συνθήκες (**SELECT**), εισαγωγή στοιχείων σε πίνακα (**INSERT**), ενημέρωση στοιχείων πίνακα σε νέες τιμές (**UPDATE**) κλπ. Στις επόμενες υποενότητες θα δούμε με τη σειρά τις εντολές αυτές με παραδείγματα.

Πριν την παρουσίαση των εντολών πρέπει να γνωρίζουμε ότι η MySQL υποστηρίζει δύο διαφορετικά είδη πινάκων. Τους **Πίνακες Ασφαλών Συναλλαγών TST (Transaction Safe Tables)** που είναι οι πίνακες τύπου **InnoDB** και **BDB** και του **Πίνακες Μη-Ασφαλών Συναλλαγών NTST (Not Transaction Safe Tables)** που είναι οι πίνακες τύπων **HEAP**, **ISAM**, **MERGE** και **MyISAM**.

Οι **Πίνακες Ασφαλών Συναλλαγών** έχουν μια σειρά πλεονεκτήματα όπως το ότι είναι πιο ασφαλής σε περίπτωση καταστροφής, μπορείς να συνδυάσεις πολλές συναλλαγές και να αποδεχτείς τη ομαδική πραγματοποίησή τους με την εντολή **COMMIT** και ότι μπορείς να αναστρέψεις (**ROLLBACK**) αλλαγές που έχουν γίνει (αρκεί να εργάζεσαι με auto commit). Φυσικά οι **Πίνακες Μη-Ασφαλών Συναλλαγών** είναι πιο γρήγοροι, απαιτούν λιγότερο χώρο αποθήκευσης και λιγότερη μνήμη για τις ενημερώσεις.

Ο προεπιλεγμένος τύπος πινάκων είναι ο **MyISAM** (αν και αλλάζει με κατάλληλη επιλογή εκκίνησης του εξυπηρετητή). Προτείνεται στον αναγνώστη, αν απαιτεί μεγαλύτερη ασφάλεια, να δημιουργεί πίνακες τύπου **InnoDB**, οι οποίοι υποστηρίζουν και εφαρμογή περιορισμών με δηλώσεις **ξένων κλειδιών - FOREIGN KEYS CONSTRAINTS** (που δεν υποστηρίζουν οι MyISAM). Επίσης τα χαρακτηριστικά που έχουν οι τύπου **InnoDB** αυξάνουν την απόδοση αλλά και την ταυτόχρονη πρόσβαση πολλών χρηστών. Για να ορίσεις τον τύπο πινάκων πρέπει να ορίσεις τη Μηχανή Αποθήκευσης (Storage Engine) το οποίο γίνεται κατά τη δημιουργία κάθε πίνακα (στην εντολή **CREATE TABLE**), είτε εκτελείς την εντολή **SET storage_engine= ΤΥΠΟΣ** και ο τύπος θα είναι ο συγκεκριμένος που καθόρισες για όλα τα ερωτήματα δημιουργίας πίνακα που θα ακολουθήσεις την τρέχουσα σύνοδο. Πχ:

```
mysql> SET storage_engine= INNODB;
```

Επίσης, αφού έχεις επιλέξει μία Β.Δ. με την εντολή use, μπορείς να χρησιμοποιήσεις την εντολή **show tables** για να προβάλεις τους πίνακες που ήδη υπάρχουν στην επιλεγμένη Β.Δ.. Πχ

```
mysql> show tables;
```

4.2.3.2.1 Ερωτήματα Δημιουργίας, Περιγραφής και Διαγραφής πίνακα

Η εντολή δημιουργίας πίνακα (**CREATE**) έχει τη ακόλουθη σύνταξη:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
(create_definition,...)
[table_options]
```

```
[partition_options]
```

ή

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
[(create_definition,...)]
[table_options]
[partition_options]
```

```
select_statement
```

ή

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
{ LIKE old_tbl_name | (LIKE old_tbl_name) }
```

όπου

create_definition:

col_name column_definition

| [CONSTRAINT [*symbol*]] PRIMARY KEY [*index_type*] (*index_col_name*,...)

[*index_option*] ...

| {INDEX|KEY} [*index_name*] [*index_type*] (*index_col_name*,...)

[*index_option*] ...

| [CONSTRAINT [*symbol*]] UNIQUE [INDEX|KEY] [*index_name*] [*index_type*] (*index_col_name*,...)

[*index_option*] ...

| {FULLTEXT|SPATIAL} [INDEX|KEY] [*index_name*] (*index_col_name*,...)

[*index_option*] ...

| [CONSTRAINT [*symbol*]] FOREIGN KEY [*index_name*] (*index_col_name*,...) *reference_definition*

| CHECK (*expr*)

column_definition:

data_type [NOT NULL | NULL] [DEFAULT *default_value*]

[AUTO_INCREMENT] [UNIQUE [KEY] | [PRIMARY] KEY]

[COMMENT '*string*']

[COLUMN_FORMAT {FIXED|DYNAMIC|DEFAULT}]

[STORAGE {DISK|MEMORY}]

data_type:

BIT[(*length*)]
 | TINYINT[(*length*)] [UNSIGNED] [ZEROFILL]
 | SMALLINT[(*length*)] [UNSIGNED] [ZEROFILL]
 | MEDIUMINT[(*length*)] [UNSIGNED] [ZEROFILL]
 | INT[(*length*)] [UNSIGNED] [ZEROFILL]
 | INTEGER[(*length*)] [UNSIGNED] [ZEROFILL]
 | BIGINT[(*length*)] [UNSIGNED] [ZEROFILL]
 | REAL[(*length*,*decimals*)] [UNSIGNED] [ZEROFILL]
 | DOUBLE[(*length*,*decimals*)] [UNSIGNED] [ZEROFILL]
 | FLOAT[(*length*,*decimals*)] [UNSIGNED] [ZEROFILL]
 | DECIMAL(*length*,*decimals*) [UNSIGNED] [ZEROFILL]
 | NUMERIC(*length*,*decimals*) [UNSIGNED] [ZEROFILL]
 | DATE
 | TIME
 | TIMESTAMP
 | DATETIME
 | YEAR
 | CHAR(*length*)
 | CHARACTER SET *charset_name* [COLLATE *collation_name*]
 | VARCHAR(*length*)
 | CHARACTER SET *charset_name* [COLLATE *collation_name*]
 | BINARY[(*length*)]
 | VARBINARY(*length*)
 | TINYBLOB
 | BLOB
 | MEDIUMBLOB
 | LONGBLOB
 | TINYTEXT [BINARY]
 | CHARACTER SET *charset_name* [COLLATE *collation_name*]
 | TEXT [BINARY]
 | CHARACTER SET *charset_name* [COLLATE *collation_name*]
 | MEDIUMTEXT [BINARY]
 | CHARACTER SET *charset_name* [COLLATE *collation_name*]
 | LONGTEXT [BINARY]
 | CHARACTER SET *charset_name* [COLLATE *collation_name*]
 | ENUM(*value1*,*value2*,*value3*,...)
 | CHARACTER SET *charset_name* [COLLATE *collation_name*]
 | SET(*value1*,*value2*,*value3*,...)
 | CHARACTER SET *charset_name* [COLLATE *collation_name*]
 | *spatial_type*

index_col_name:

col_name [(*length*)] [ASC | DESC]

index_type:

USING {BTREE | HASH | RTREE}

index_option:

KEY_BLOCK_SIZE [=] *value*

| *index_type*

| WITH PARSER *parser_name*

reference_definition:

REFERENCES *tbl_name* (*index_col_name*,...)
 [MATCH FULL | MATCH PARTIAL | MATCH SIMPLE]
 [ON DELETE *reference_option*]
 [ON UPDATE *reference_option*]

reference_option:

RESTRICT | CASCADE | SET NULL | NO ACTION

table_options:

table_option [, *table_option*] ...

table_option:

ENGINE [=] *engine_name*
 | AUTO_INCREMENT [=] *value*
 | AVG_ROW_LENGTH [=] *value*
 | [DEFAULT] CHARACTER SET [=] *charset_name*
 | CHECKSUM [=] {0 | 1}
 | [DEFAULT] COLLATE [=] *collation_name*
 | COMMENT [=] '*string*'
 | CONNECTION [=] '*connect_string*'
 | DATA DIRECTORY [=] '*absolute path to directory*'
 | DELAY_KEY_WRITE [=] {0 | 1}
 | INDEX DIRECTORY [=] '*absolute path to directory*'
 | INSERT_METHOD [=] { NO | FIRST | LAST }
 | KEY_BLOCK_SIZE [=] *value*
 | MAX_ROWS [=] *value*
 | MIN_ROWS [=] *value*
 | PACK_KEYS [=] {0 | 1 | DEFAULT}
 | PASSWORD [=] '*string*'
 | ROW_FORMAT [=] {DEFAULT|DYNAMIC|FIXED|COMPRESSED|REDUNDANT|COMPACT}
 | TABLESPACE *tablespace_name* [STORAGE {DISK|MEMORY|DEFAULT}]
 | UNION [=] (*tbl_name* [, *tbl_name*]...)

partition_options:

PARTITION BY
 { [LINEAR] HASH(*expr*)
 | [LINEAR] KEY(*column_list*)
 | RANGE(*expr*)
 | LIST(*expr*) }
 [PARTITIONS *num*]
 [SUBPARTITION BY
 { [LINEAR] HASH(*expr*)
 | [LINEAR] KEY(*column_list*) }
 [SUBPARTITIONS *num*]
]
 [(*partition_definition* [, *partition_definition*] ...)]

partition_definition:

PARTITION *partition_name*
 [VALUES {LESS THAN {(*expr*) | MAXVALUE} | IN (*value_list*)}]
 [[STORAGE] ENGINE [=] *engine_name*]
 [COMMENT [=] '*comment_text*']

```
[DATA DIRECTORY [=] 'data_dir']
[INDEX DIRECTORY [=] 'index_dir']
[MAX_ROWS [=] max_number_of_rows]
[MIN_ROWS [=] min_number_of_rows]
[TABLESPACE [=] tablespace_name]
[NOGROUP [=] node_group_id]
[(subpartition_definition [, subpartition_definition] ...)]
```

subpartition_definition:

```
SUBPARTITION logical_name
[[STORAGE] ENGINE [=] engine_name]
[COMMENT [=] 'comment_text' ]
[DATA DIRECTORY [=] 'data_dir']
[INDEX DIRECTORY [=] 'index_dir']
[MAX_ROWS [=] max_number_of_rows]
[MIN_ROWS [=] min_number_of_rows]
[TABLESPACE [=] tablespace_name]
[NOGROUP [=] node_group_id]
```

select_statement:

```
[IGNORE | REPLACE] [AS] SELECT ... (Some legal select statement)
```

Όπως βλέπουμε από το συντακτικό της CREATE TABLE μπορούμε να χρησιμοποιήσουμε ένα μεγάλο σύνολο τύπων δεδομένων οι οποίοι καλύπτουν τις περισσότερες ανάγκες. Στο επόμενο πίνακα φαίνονται κάποια βασικά στοιχεία των τύπων δεδομένων:

Τύπος Δεδομένων	Περιγραφή - Χρήση
BIT(M)	Για αποθήκευση bits από 1-64. Το M καθορίζει τον αριθμό bits (1 αν δεν δοθεί)
TINYINT(M) ,	Πολύ μικρός ακέραιος (-128 έως 127), αν είναι unsigned 0 έως 255
SMALLINT(M) ,	Μικρός ακέραιος (-32768 έως 32767), αν είναι unsigned 0 έως 65536
MEDIUMINT(M) ,	Μεσαίος ακέραιος (-8388608 έως 8388607) , αν είναι unsigned 0 έως 16777215
INT(M) ,	Κανονικός ακέραιος (-2147483648 έως 2147483647), αν είναι unsigned 0-4294967295
INTEGER(M) ,	Alias για το INT(M)
BIGINT(M)	Μεγάλος ακέραιος (-9223372036854775808 έως 9223372036854775807), αν είναι unsigned 0 έως 18446744073709551615 Σε κάθε περίπτωση με τον καθορισμό του M καθορίζουμε τον αριθμό των bits που θα χρησιμοποιηθεί (με πάνω όριο ανάλογα τον κάθε τύπο, πχ ο INT χρησιμοποιεί 1-32 bits άρα αν βάλουμε INT(16) άμεσα τον κάνουμε μικρότερο)
SERIAL	Είναι ένα alias του BIGINT UNSIGNED NOT NULL AUTO_INCREMENT UNIQUE οπότε είναι ιδανικός τύπος για πεδίο που θέλουμε να χρησιμοποιηθεί σαν κλειδί.
FLOAT(M,D) ,	Απλής ακρίβειας πραγματικός αριθμός
DOUBLE(M,D)	Διπλής ακρίβειας πραγματικός αριθμός Με το M καθορίζουμε το συνολικό αριθμό ψηφίων (όχι bits) και με το D τον αριθμό ψηφίων μετά την υποδιαστολή. Άρα αν βάλουμε 10,4 θα έχουμε 6 ψηφία το ακέραιο μέρος και 4 το δεκαδικό.
BOOL, BOOLEAN	Το ίδιο με το TINYINT(1), επίσης έχουν οριστεί aliases FALSE=0 και TRUE=1
DATE ,	Ημερομηνία με εύρος από '1000-01-01' έως '9999-12-31'. Τυπώνεται με μορφή 'YYYY-MM-DD'

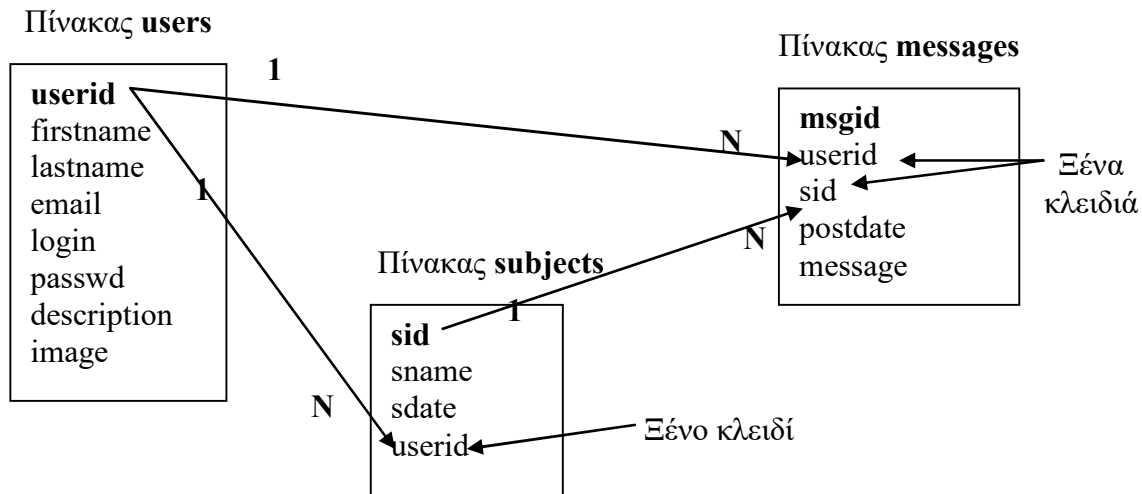
DATETIME,	Συνδυασμός ημερομηνία και ώρας με εύρος από '1000-01-01 00:00:00' έως '9999-12-31 23:59:59'. Τυπώνεται με μορφή 'YYYY-MM-DD HH:MM:SS'
TIMESTAMP,	Χρονοσφραγίδα με εύρος '1970-01-01 00:00:01' UTC έως '2038-01-19 03:14:07' . Αποθηκεύεται ως δευτερόλεπτα που έχουν περάσει από την '1970-01-01 00:00:00' UTC. Είναι πολύ χρήσιμος τύπος για να καταγράφουμε πότε έγινε μια δοσοληψία. Αρκεί να θέσουμε την τιμή NULL και αυτή θα πάρει την τρέχουσα ημερομηνία και ώρα.
TIME,	Η ώρα με εύρος από '-838:59:59' έως '838:59:59'. Τυπώνεται με μορφή 'HH:MM:SS'
YEAR[(2 4)],	Το έτος σε μορφή 2 ή 4 ψηφίων (προεπιλεγμένη τιμή το 4). Σε μορφή 4 ψηφίων έχει εύρος από 1901 έως 2155. Σε μορφή 2 ψηφίων έχει εύρος 70 to 69 (αναπαριστά έτη από το 1970 μέχρι το 2069. Τυπώνεται σε μορφή YYYY. Σε όλα αυτά τα πεδία οι τιμές μπορούν να εισαχθούν είτε ως αριθμοί είτε ως αλφαριθμητικά (που έχουν όμως την αντίστοιχη μορφή).
CHAR(M)	Αλφαριθμητικό σταθερού μήκους που γεμίζει με κενά στα δεξιά μη χρησιμοποιούμενα bytes. Το M είναι το μήκος σε χαρακτήρες από 0 έως 255 (αν δεν οριστεί είναι 1).
VARCHAR(M),	Αλφαριθμητικό μεταβλητού μήκους. Με το M μπορούμε να καθορίσουμε το μέγιστο μήκος (0 έως 65,535). Το χρήσιμο μήκος εξαρτάται από διάφορους παράγοντες όπως το σύνολο χαρακτήρων που χρησιμοποιείται.
BINARY(M),	Ίδιο με το CHAR(M) μόνο που αποθηκεύει δυαδικά δεδομένα. Το M είναι το μήκος.
VARBINARY(M),	Ίδιο με το VARCHAR(M) μόνο που αποθηκεύει δυαδικά δεδομένα. Το M είναι το μήκος.
TINYTEXT,	Αλφαριθμητικό με μέγιστο μήκος 255. Το χρήσιμο μήκος εξαρτάται από διάφορους παράγοντες όπως το σύνολο χαρακτήρων που χρησιμοποιείται.
TEXT[(M)]	Αλφαριθμητικό μέγιστου μήκους 65.535. Το χρήσιμο μήκος εξαρτάται από διάφορους παράγοντες όπως το σύνολο χαρακτήρων που χρησιμοποιείται. Αν δοθεί το M δημιουργείται το πεδίο ως ο μικρότερος τύπος TEXT που είναι αρκετά μεγάλος για να χωράει M χαρακτήρες.

Ακολουθούν παραδείγματα δημιουργίας πινάκων στην Β.Δ. **newsdb** που δημιουργήθηκε στην προηγούμενη υποενότητα με την εντολή **create database newsdb DEFAULT CHARACTER SET utf8 COLLATE utf8_unicode_ci**. Αν δεν έχω βάλει στην δημιουργία της Β.Δ. το σύνολο χαρακτήρων και το collation τότε θα πρέπει να το βάλω σε κάθε πίνακα μετά το τέλος της CREATE γράφοντας μετά το κλείσιμο της παρένθεσης των πεδίων **DEFAULT CHARACTER SET utf8 COLLATE utf8_unicode_ci**.

Η βάση αυτή έχει 3 πίνακες:

1. **users**, εισάγονται τα στοιχεία του χρήστη (κωδικός χρήστη, όνομα, επώνυμο, e-mail, όνομα και κωδικός πρόσβασης, περιγραφή, φωτογραφία χρήστη)
2. **subjects**, εισάγονται θέματα συζητήσεων (κωδικός θέματος, όνομα θέματος, ημερομηνία/ώρα δημιουργίας, κωδικός χρήστη που το δημιούργησε)
3. **messages**, εισάγονται τα μηνύματα των χρηστών (κωδικός μηνύματος, κωδικός χρήστη που το εισήγαγε, κωδικός θέματος στο οποίο εισήχθη, ημερομηνία/ώρα εισαγωγής, κείμενο μηνύματος)

Οι σχέσεις των πινάκων φαίνονται στο επόμενο σχήμα. Ένας χρήστης μπορεί να εισάγει πολλά θέματα αλλά ένα θέμα εισάγεται μόνο από ένα χρήστη ($1 \rightarrow N$), ένας χρήστης μπορεί να εισάγει πολλά μηνύματα αλλά ένα μήνυμα εισάγεται μόνο από ένα χρήστη ($1 \rightarrow N$), ένα θέμα περιέχει πολλά μηνύματα αλλά ένα μήνυμα ανήκει σε ένα μόνο θέμα ($1 \rightarrow N$).



Εικόνα 3: Σχέσεις πινάκων Β.Δ. newsdb

Δημιουργώ τους πίνακες της Β.Δ.

```
mysql> use newsdb;
Database changed
```

```
mysql> CREATE TABLE users (
->  userid int(32) unsigned NOT NULL auto_increment,
->  firstname varchar(50) NOT NULL,
->  lastname varchar(50) NOT NULL,
->  email varchar(50) NOT NULL,
->  login varchar(20) NOT NULL,
->  passwd varchar(100) NOT NULL,
->  description varchar(200) NOT NULL,
->  image varchar(200) NOT NULL,
->  PRIMARY KEY (userid)
-> ) ENGINE=INNODB AUTO_INCREMENT=1 ;
```

Query OK, 0 rows affected (0.06 sec)

```
mysql> CREATE TABLE subjects (
->  sid int(32) unsigned NOT NULL auto_increment,
->  sname varchar(200) NOT NULL,
->  sdate datetime NOT NULL,
->  userid int(32) unsigned NOT NULL,
->  PRIMARY KEY (sid),
->  CONSTRAINT suserid FOREIGN KEY suserid (userid) REFERENCES users (userid)
->  ON DELETE RESTRICT ON UPDATE CASCADE
-> ) ENGINE=INNODB AUTO_INCREMENT=1 ;
```

Query OK, 0 rows affected (0.06 sec)

```
mysql> CREATE TABLE messages (
->  msgid int(32) unsigned NOT NULL auto_increment,
->  userid int(32) unsigned NOT NULL,
->  sid int(32) unsigned NOT NULL,
->  postdate datetime NOT NULL,
->  message text NOT NULL,
->  PRIMARY KEY (msgid),
```

```
-> CONSTRAINT musetid FOREIGN KEY musetid (userid) REFERENCES users (userid)
-> ON DELETE RESTRICT ON UPDATE CASCADE,
-> CONSTRAINT msid FOREIGN KEY msid (sid) REFERENCES subjects (sid)
-> ON DELETE RESTRICT ON UPDATE CASCADE
-> ) ENGINE=INNODB AUTO_INCREMENT=1 ;
```

Query OK, 0 rows affected (0.00 sec)

mysql>

Για να δούμε τους πίνακες που δημιουργήσαμε στη Β.Δ. που έχουμε επιλέξει δίνουμε την εντολή **show tables**. Πχ

mysql> show tables;

```
+-----+
| Tables_in_newsdb |
+-----+
| messages          |
| subjects          |
| users             |
+-----+
```

3 rows in set (0.00 sec)

mysql>

Για να δούμε την περιγραφή (τα πεδία του και τα χαρακτηριστικά τους) ενός πίνακα δίνουμε την εντολή **describe <όνομα πίνακα>**. Πχ

mysql> describe messages;

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type                | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| msgid      | integer unsigned    | NO   | PRI | NULL    | auto_increment |
| userid     | integer unsigned    | NO   | MUL |          |                 |
| sid        | integer unsigned    | NO   | MUL |          |                 |
| postdate   | datetime            | NO   |     |          |                 |
| message    | text                | NO   |     |          |                 |
+-----+-----+-----+-----+-----+-----+
```

5 rows in set (0.03 sec)

mysql>

Για να διαγράψουμε έναν υπάρχον πίνακα (και φυσικά όλα τα δεδομένα του) αρκεί να δώσουμε την εντολή **drop table <όνομα πίνακα>**. Πχ

mysql> drop table messages;

Query OK, 0 rows affected (0.00 sec)

4.2.3.2 Ερωτήματα Εισαγωγής και Ενημέρωσης εγγραφών πίνακα

Η εντολή εισαγωγής εγγραφών σε ένα πίνακα (**INSERT**) έχει τη ακόλουθη σύνταξη:

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name [(col_name,...)]
{VALUES | VALUE} ({expr | DEFAULT},...),(...),...
[ ON DUPLICATE KEY UPDATE
```

```

col_name=expr
[, col_name=expr] ... ]
ή
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name
SET col_name={expr | DEFAULT}, ...
[ ON DUPLICATE KEY UPDATE
col_name=expr
[, col_name=expr] ... ]
ή
INSERT [LOW_PRIORITY | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name [(col_name,...)]
SELECT ...
[ ON DUPLICATE KEY UPDATE
col_name=expr
[, col_name=expr] ... ]

```

Με την πρώτη μορφή INSERT ... VALUES εισάγουμε τιμές που γνωρίζουμε, όπως και με τη δεύτερη μορφή που είναι μια εναλλακτική της πρώτης και με την οποία δίνουμε ένα-ένα τα ζεύγη πεδίο=τιμή. Με την τρίτη μορφή INSERT ... SELECT εισάγουμε τιμές που επιλέγουμε από έναν άλλον πίνακα/πίνακες. Μπορούμε να χρησιμοποιήσουμε το REPLACE αντί του INSERT και τότε θα είναι σαν να χρησιμοποιούμε το INSERT .. IGNORE όπου αν υπάρχει ήδη η ίδια εγγραφή (με τις ίδιες τιμές στα κλειδιά) θα την αντικαταστήσουμε.

Για να εισάγουμε έναν χρήστη στον πίνακα users που δημιουργήσαμε στην προηγούμενη υποενότητα θα γράφαμε (το πεδίο που είναι AUTO_INCREMENT παίρνει μόνο του τιμή):

```

mysql> insert into users
-> (firstname, lastname, email, login, passwd, description, image)
-> values
-> ('Νίκος', 'Κατσάλης', 'nkatsalis@yahoo.gr', 'nkats', 'nkats!wpasswd',
-> 'Καθηγητής Πληροφορικής', 'images/nkatsalis.jpg');
Query OK, 1 row affected (0.00 sec)
mysql> select * from users;
+-----+-----+-----+-----+-----+-----+-----+
| userid | firstname | lastname | email | login | passwd | description | image |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Νίκος | Κατσάλης | nkatsalis@yahoo.gr | nkats | nkats!wpasswd | Καθηγητής Πληροφορικής | images/nkatsalis.jpg |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
mysql>

```

Η εντολή ενημέρωσης εγγραφών σε ένα πίνακα (UPDATE) έχει τη ακόλουθη σύνταξη:

```
UPDATE [LOW_PRIORITY] [IGNORE] table_reference
```

```
SET col_name1={expr1|DEFAULT} [, col_name2={expr2|DEFAULT}] ...
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
```

ή

```
UPDATE [LOW_PRIORITY] [IGNORE] table_references
SET col_name1={expr1|DEFAULT} [, col_name2={expr2|DEFAULT}] ...
[WHERE where_condition]
```

Η πρώτη μορφή χρησιμοποιείται για την ενημέρωση υπαρχόντων εγγραφών ενός πίνακα (το όνομα του οποίου γράφουμε στο *table_reference*). Με την έκφραση SET καθορίζουμε ποια πεδία του (στήλες) θα ενημερωθούν, ενώ με το WHERE βάζουμε συνθήκες που καθορίζουν ποιες εγγραφές θα ενημερωθούν (αν δεν γραφεί το WHERE ενημερώνονται όλες οι εγγραφές). Με το ORDER BY μπορούμε να καθορίσουμε τη σειρά με την οποία θα γίνουν οι ενημερώσεις και με το LIMIT να περιορίσουμε τον αριθμό των εγγραφών που θα ενημερωθούν σε μία μέγιστη τιμή.

Η δεύτερη μορφή χρησιμοποιείται για την ενημέρωση υπαρχόντων εγγραφών σε καθένα από ένα σύνολο πινάκων (τα ονόματα των οποίων γράφουμε στο *table_references*). Σε αυτή τη μορφή δεν μπορούν να χρησιμοποιηθούν το ORDER BY και το LIMIT.

Η χρήση του WHERE και στις δύο μορφές είναι ίδια με τη χρήση του στο ερώτημα επιλογής (SELECT).

Υποθέτοντας ότι ο χρήστης με επώνυμο 'Κατσάλης' , όνομα 'Νίκος' και e-mail 'nkatsalis@yahoo.gr', άλλαξε e-mail και το έκανε 'nkatsalis@gmail.com', μπορούμε να κάνουμε την αλλαγή με το επόμενο ερώτημα:

```
mysql> update users
-> set email='nkatsalis@gmail.com'
-> where firstname='Νίκος' and lastname='Κατσάλης' and email='nkatsalis@yahoo.gr';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
mysql>
```

Υποθέτουμε ότι θέλω σε όλους τους χρήστες που δεν έχουν βάλει φωτογραφία (image='') να βάλω μία προεπιλεγμένη φωτογραφία 'images/default_img.jpg'. Το ερώτημα θα είναι (ταιριάζουν και αλλάζουν 3 εγγραφές):

```
mysql> update users
-> set image='images/default_img.jpg'
-> where image='';
Query OK, 3 rows affected (0.01 sec)
Rows matched: 3 Changed: 3 Warnings: 0
mysql>
```

4.2.3.2.3 Ερώτημα Επιλογής εγγραφών πίνακα

Η εντολή επιλογής εγγραφών από ένα πίνακα (SELECT) με ή χωρίς κριτήρια έχει τη ακόλουθη σύνταξη:

```
SELECT
[ALL | DISTINCT | DISTINCTROW ]
```

```

[HIGH_PRIORITY]
[STRAIGHT_JOIN]
[SQL_SMALL_RESULT] [SQL_BIG_RESULT] [SQL_BUFFER_RESULT]
[SQL_CACHE] [SQL_NO_CACHE] [SQL_CALC_FOUND_ROWS]
select_expr [, select_expr ...]
[FROM table_references]
[WHERE where_condition]
[GROUP BY {col_name | expr | position}
[ASC | DESC], ... [WITH ROLLUP]]
[HAVING where_condition]
[ORDER BY {col_name | expr | position}
[ASC | DESC], ...]
[LIMIT [{offset,] row_count | row_count OFFSET offset}]
[PROCEDURE procedure_name(argument_list)]
[INTO OUTFILE 'file_name'
[CHARACTER SET charset_name]
export_options
| INTO DUMPFILE 'file_name'
| INTO var_name [, var_name]]
[FOR UPDATE | LOCK IN SHARE MODE]]

```

Το **SELECT** χρησιμοποιείται για να επιλέξουμε εγγραφές (γραμμές) από έναν ή περισσότερους πίνακες και μπορεί να περιλαμβάνει και υποερωτήματα (subqueries) και ενώσεις (**UNION**). Στις πιο συνηθισμένες περιπτώσεις έχουμε τα εξής:

1. Κάθε *select_expr* είναι και ένα πεδίο (στήλη) ενός πίνακα που θέλουμε να επιλέξουμε. Πρέπει να υπάρχει τουλάχιστον ένα *select_expr*. Μπορούμε να χρησιμοποιήσουμε το ***** ή το *table_reference*.*
2. Το *table_references* είναι ένας ή περισσότεροι πίνακες (οι οποίοι μπορεί να ενώνονται – **JOIN**) από τους οποίους θα επιλεγούν τα πεδία. Για περισσότερες πληροφορίες μπορεί ο αναγνώστης να δει στο εγχειρίδιο αναφοράς του MySQL το **JOIN Syntax**.
3. Με το **WHERE** καθορίζουμε, προαιρετικά, συνθήκες που πρέπει να πληρούν οι εγγραφές των οποίων τα πεδία θέλουμε να επιλέξουμε. Αν δεν χρησιμοποιηθεί θα επιλεγούν όλες οι εγγραφές. Στην έκφραση του **WHERE** μπορούμε να χρησιμοποιήσουμε οποιαδήποτε από τις συναρτήσεις (functions) και τελεστές (operators) υποστηρίζει η MySQL (εκτός από τις συναρτήσεις ομαδοποίησης – aggregate functions). Για περισσότερες πληροφορίες μπορεί ο αναγνώστης να δει στο εγχειρίδιο αναφοράς του MySQL την ενότητα **Functions and Operators**.

Φυσικά το **SELECT** μπορεί, όπως είδαμε και στην υποενότητα **Σύνδεση με τον MySQL Εξυπηρετητή και εκτέλεση απλών εντολών** να χρησιμοποιηθεί και χωρίς να επιλέξουμε τίποτα από κάποιον πίνακα (πχ για απλούς υπολογισμούς).

Για να δω όλα τα στοιχεία όλων των χρηστών μπορώ να γράψω:

```
mysql> select * from users;
```

Για να δω το επώνυμο και το όνομα των χρηστών που το e-mail τους είναι στην περιοχή (domain) gmail.com μπορώ να γράψω:

```
mysql> select firstname, lastname from users where email like '%gmail.com';
```

Για να δω την ημερομηνία/ώρα εισαγωγής και το κείμενο όλων των μηνυμάτων που υπάρχουν στο θέμα με όνομα (sname) 'ΤΠΕ' μπορώ να γράψω:

```
mysql> select postdate, message from subjects, messages
-> where subjects.sid=messages.sid and sname='ΤΠΕ';
```

Φυσικά το ίδιο θα μπορούσα να κάνω χρησιμοποιώντας την έκφραση JOIN αντί να κάνω join τους πίνακες μέσω του WHERE. Δηλαδή:

```
mysql> select postdate, message from
-> subjects INNER JOIN messages ON subjects.sid=messages.sid
-> where sname='ΤΠΕ';
```

Με τη SELECT μπορούμε να χρησιμοποιήσουμε στο [select_expr](#) ένα σύνολο από συναρτήσεις ομαδοποίησης – aggregate functions οι οποίες με κατάλληλη χρήση του όρου GROUP BY μπορούν να μας δώσουν μια σειρά από χρήσιμη πληροφορία από τους πίνακές μας. Οι πιο γνωστές φαίνονται στον επόμενο πίνακα:

Συνάρτηση Ομαδοποίησης	Περιγραφή - Χρήση
AVG([DISTINCT] expr)	Υπολογίζει και επιστρέφει τον μέσο όρο της τιμής της expr. Αν βάλω το DISTINCT επιστρέφει τον μέσο όρο των διαφορετικών τιμών της expr (δεν προσμετράει τις ίδιες τιμές για τον υπολογισμό).
COUNT([DISTINCT] expr)	Υπολογίζει και επιστρέφει το πλήθος των μη NULL τιμών της expr. Αν καθορίζουμε το DISTINCT επιστρέφει το πλήθος των διαφορετικών μη NULL τιμών της expr. Το COUNT(*) έχει ειδική χρήση και μετράει το πλήθος όλων (και των NULL).
MAX([DISTINCT] expr)	Υπολογίζει και επιστρέφει την μέγιστη τιμή της expr. Αν βάλω το DISTINCT στην πραγματικότητα έχω το ίδιο αποτέλεσμα. Μπορεί και λειτουργεί και για αλφαριθμητικά. Αν με το SELECT δεν επιλεγούν γραμμές επιστρέφει NULL.
MIN([DISTINCT] expr)	Υπολογίζει και επιστρέφει την ελάχιστη τιμή της expr. Αν βάλω το DISTINCT στην πραγματικότητα έχω το ίδιο αποτέλεσμα. Μπορεί και λειτουργεί και για αλφαριθμητικά. Αν με το SELECT δεν επιλεγούν γραμμές επιστρέφει NULL.
SUM([DISTINCT] expr)	Υπολογίζει και επιστρέφει το άθροισμα των τιμών της expr. Αν βάλω το DISTINCT αθροίζει μόνο τις διαφορετικές τιμές της expr. Αν με το SELECT δεν επιλεγούν γραμμές επιστρέφει NULL.

Αν θέλω να μετρήσω πόσα μηνύματα έχω θα μπορούσα να γράψω:

```
mysql> select count(*) from messages;
```

Αν θέλω να βρω τον χρήστη με το μικρότερο επώνυμο θα μπορούσα να γράψω:

```
mysql> select min(firstname) from users;
```

Αν θέλω να βρω τον αριθμό μηνυμάτων που έχει εισάγει κάθε χρήστης θα μπορούσα να γράψω:

```
mysql> select login, count(msgid)
-> from users INNER JOIN messages ON users.userid=messages.userid
-> GROUP BY login;
```

4.3 Διαχείριση Β.Δ. MySQL με τον Query Browser

Στην ενότητα αυτή θα δούμε τις εντολές διαχείρισης Β.Δ. MySQL με την χρήση του γραφικού περιβάλλοντος του Query Browser. Οι εντολές θα παρουσιαστούν μέσα από ένα παράδειγμα δημιουργίας Β.Δ. και των πινάκων της και εντολές σε αυτούς τους πίνακες. Συγκεκριμένα θα

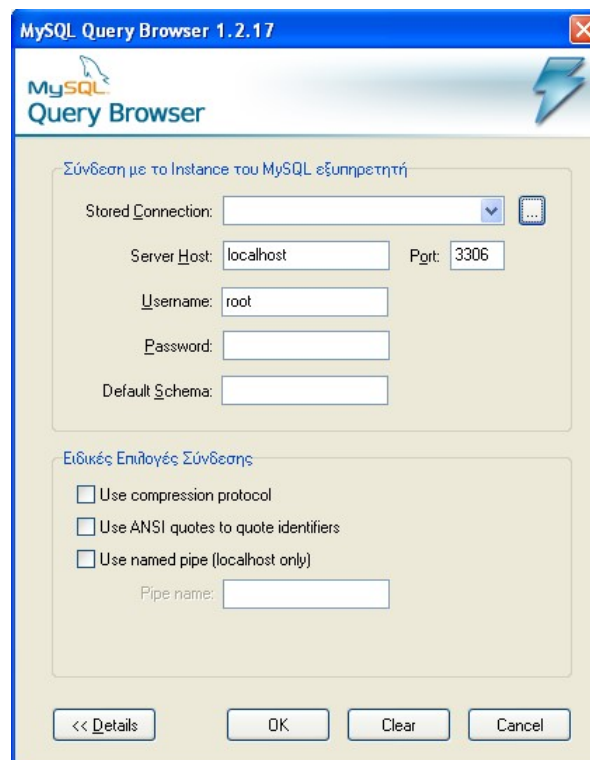
δημιουργήσουμε τη Β.Δ. newstdb, που είδαμε και στην προηγούμενη ενότητα, και τους πίνακές της και θα εκτελέσουμε μια σειρά ερωτήματα εισαγωγής, διαγραφής, ενημέρωσης, επιλογής κλπ.

4.3.1 Σύνδεση με τον MySQL Εξυπηρετητή

Αφού εκτελέσουμε το πρόγραμμα του Query Browser το πρώτο παράθυρο που μας εμφανίζεται μας προτρέπει να δώσουμε τα απαραίτητα στοιχεία για να συνδεθούμε στον MySQL εξυπηρετητή. Όπως φαίνεται και στο επόμενο σχήμα μπορούμε να δώσουμε τις βασικές παραμέτρους σύνδεσης καθώς και κάποιες επιπλέον αν έχουμε πατήσει το κουμπί «**Details >>>**».

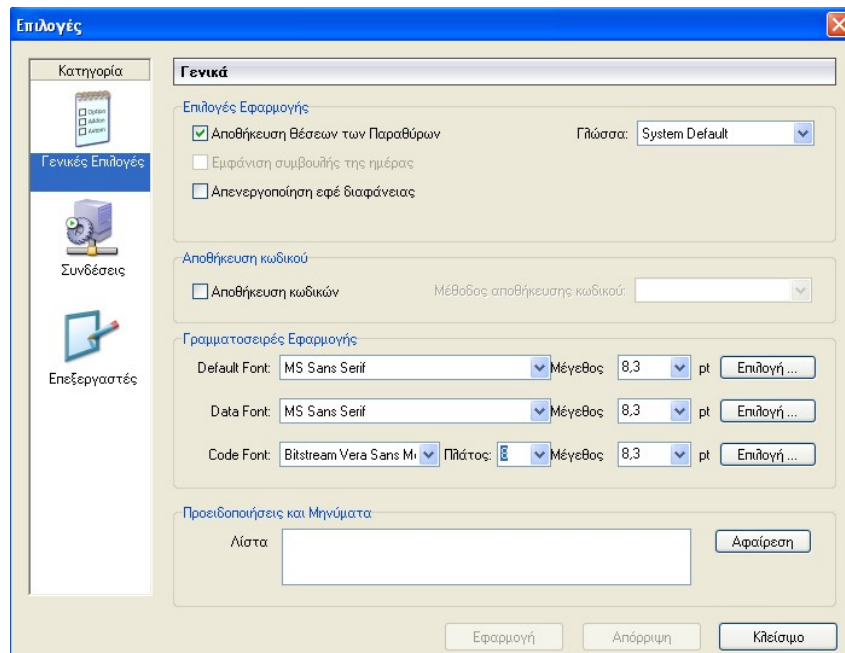
Στο πεδίο «Default Schema» μπορούμε να βάλουμε την Β.Δ. στην οποία θέλουμε να συνδεθούμε αλλά μπορούμε και να το αφήσουμε κενό και να επιλέξουμε αργότερα με ποια Β.Δ. θα δουλέψουμε (πολλές φορές ανάλογα τα δικαιώματα που έχει ο χρήστης με τα στοιχεία του οποίου συνδεόμαστε, ίσως είναι απαραίτητο να καθορίσουμε τη Β.Δ.). Αν το αφήσουμε κενό ίσως μας εμφανιστεί ένα μήνυμα στο οποίο μπορούμε απλώς να πατήσουμε το κουμπί «Αγνόηση».

Στο πεδίο **Stored Connections** μπορούμε να επιλέξουμε κάποια σύνδεση της οποίας τα στοιχεία έχουμε αποθηκεύσει. Από το κουμπί «...» που είναι δίπλα στο πεδίο μας εμφανίζεται ένας διάλογος όπου μπορούμε να δούμε και να αλλάξουμε στοιχεία που αφορούν τις «**Γενικές Επιλογές**» της Εφαρμογής, τις «**Συνδέσεις**» και τους «**Επεξεργαστές**» ανάλογα το τι έχουμε επιλέξει από το αριστερό μέρος τους παραθύρου.

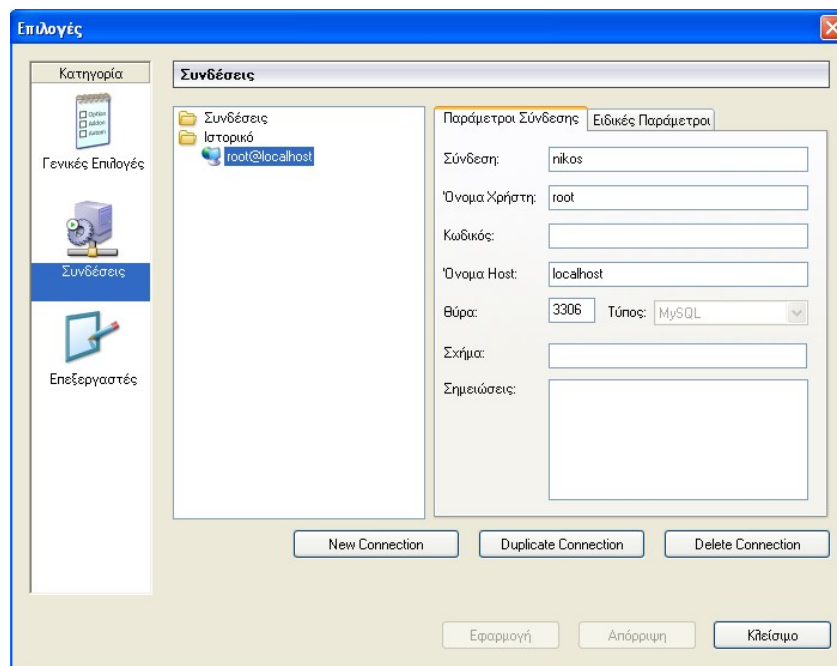


Εικόνα 4: Διάλογος σύνδεσης στον MySQL Εξυπηρετητή με τον Query Browser

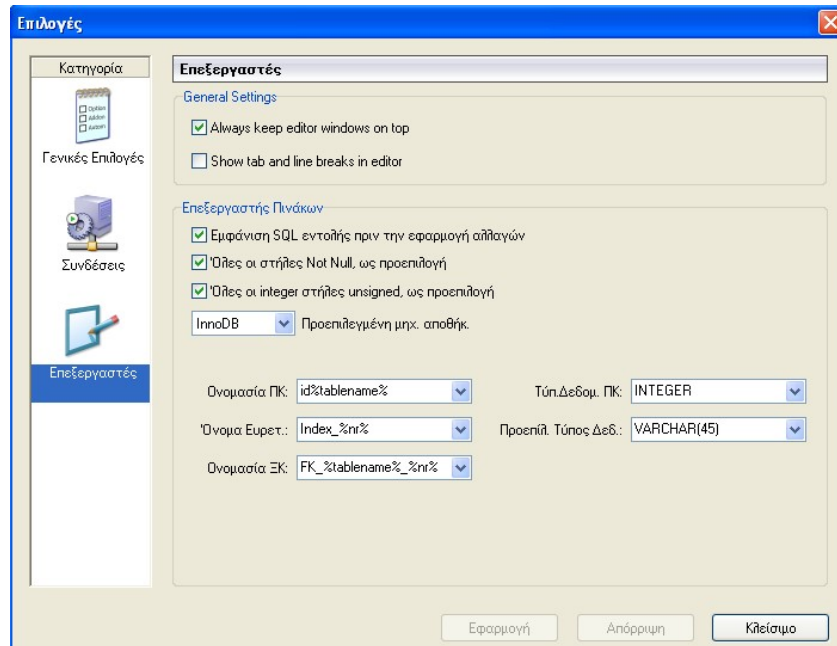
Στο «**Επεξεργαστές**» καθορίζουμε και την προεπιλεγμένη μηχανή αποθήκευσης (InnoDB, MyISAM κλπ) καθώς και μια σειρά άλλων επιλογών όπως προεπιλεγμένους τύπους και ονόματα πεδίων στους πίνακες που θα δημιουργούμε, το αν θα εμφανίζεται παράθυρο επιβεβαίωσης εκτέλεσης ερωτήματος πριν από αλλαγές κλπ. Στα επόμενα σχήματα φαίνονται οι 3 αυτοί διάλογοι.



Εικόνα 5: Γενικές Ρυθμίσεις Εφαρμογής



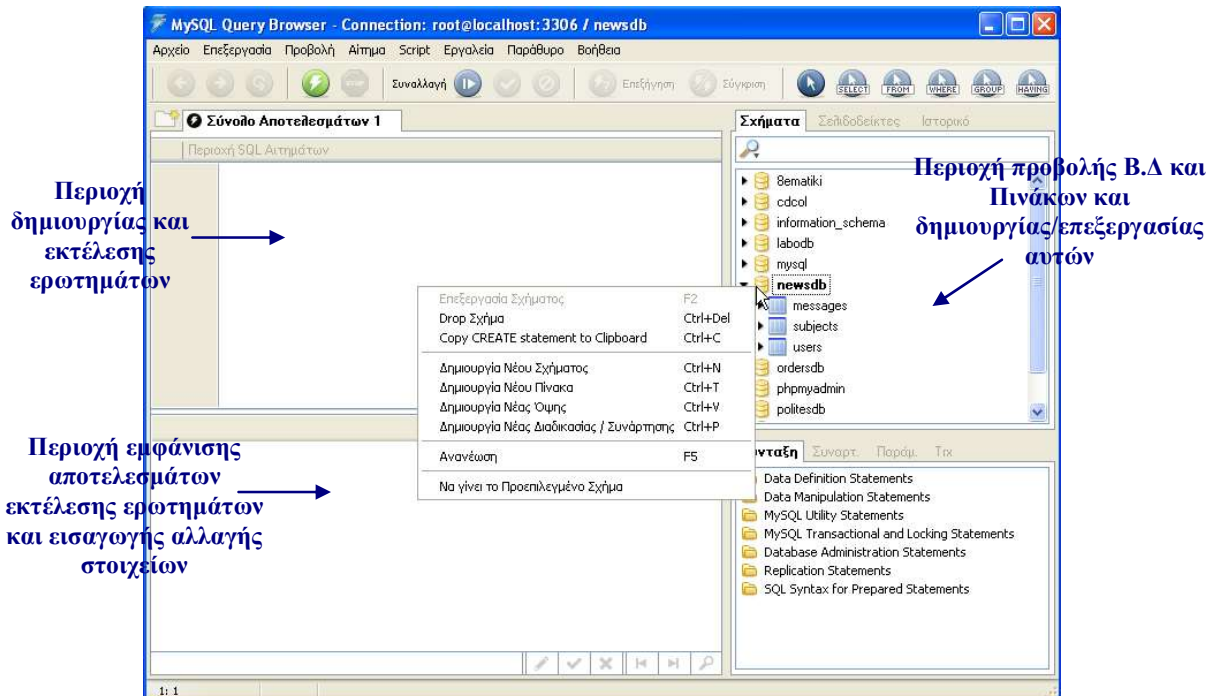
Εικόνα 6: Ρυθμίσεις Συνδέσεων (Δημιουργία, Διαγραφή, Αλλαγή στοιχείων) και Ιστορικό



Εικόνα 7: Ρυθμίσεις που αφορούν τους Επεξεργαστές

4.3.2 Προβολή, Επιλογή και Δημιουργία Β.Δ.

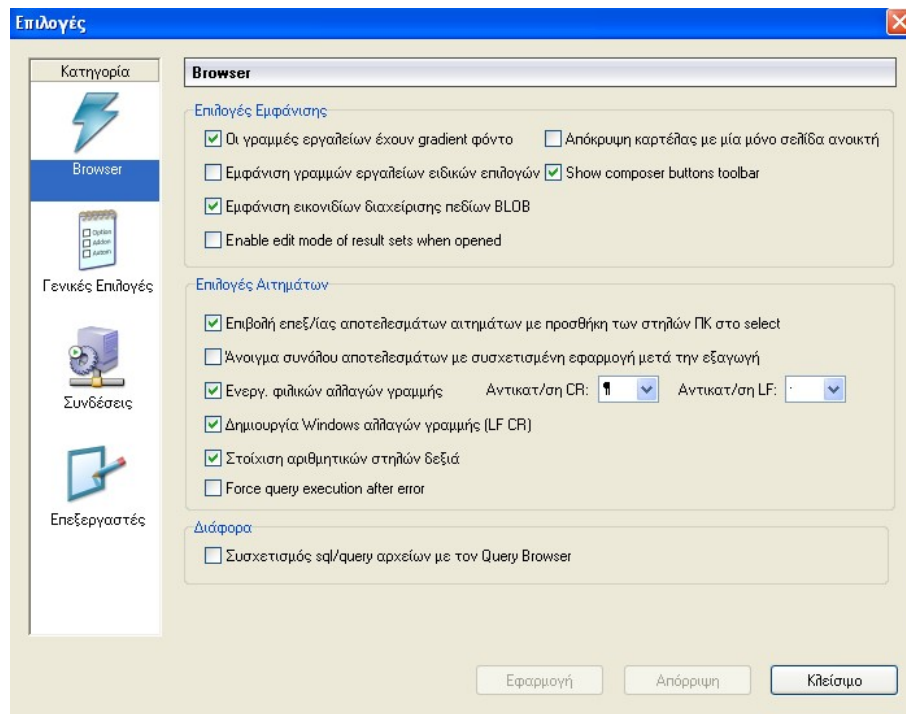
Αφού συνδεθούμε μας εμφανίζεται η βασική οθόνη εργασίας, που φαίνεται στο επόμενο σχήμα, και από την οποία μπορούμε να κάνουμε όλες τις ενέργειες που θέλουμε.



Εικόνα 8: Βασική Οθόνη Εργασίας στο Query Browser

Φυσικά στο πάνω μέρος του παραθύρου έχουμε το κυρίως μενού με τις συνηθισμένες εντολές. Προσοχή πρέπει να δοθεί στο μενού «Προβολή» από το οποίο καθορίζεται τι θα φαίνεται και τι όχι και οπότε μπορεί αν κάτι δεν φαίνεται να το εμφανίσετε. Επίσης στο μενού «Βοήθεια» θα βρείτε

βοήθεια για όλες τις λειτουργίες της εφαρμογής (αν κάποιος έχει πρόβλημα με την ανάγνωση στα Αγγλικά μπορεί να δει το ελληνικό εγχειρίδιο, όπως ήδη αναφέραμε, στην ιστοσελίδα της MySQL). Επίσης από το μενού «Εργαλεία»→»Επιλογές ...» εμφανίζεται ένα παράθυρο όπου ρυθμίζουμε όλες τις επιλογές που είδαμε και από το αρχικό παράθυρο σύνδεσης αλλά και κάποιες επιπλέον όπως βλέπετε στο επόμενο σχήμα:

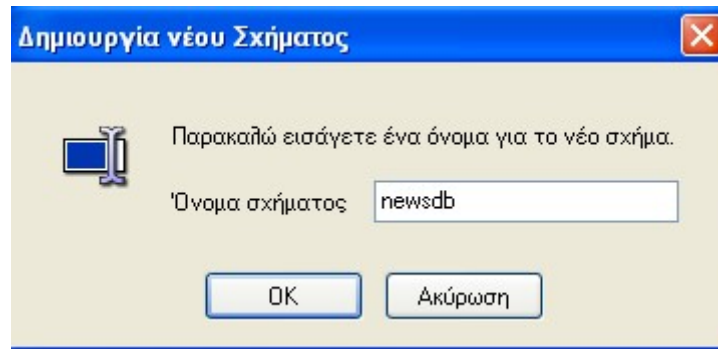


Εικόνα 9: Επιλογές Εφαρμογής από το μενού Εργαλεία

Τα κύρια μέρη της Βασικής Οθόνης της εφαρμογής είναι:

1. Στο αριστερό μέρος και πάνω έχουμε την «Περιοχή SQL Αιτημάτων» όπου γράφουμε τα ερωτήματα που θα εκτελεστούν (σε SQL όπως είδαμε στην προηγούμενη ενότητα).
2. Στο αριστερό μέρος και κάτω εμφανίζονται τα αποτελέσματα του ερωτήματος που γράψαμε και εκτελέσαμε στο πάνω αριστερό μέρος. Μάλιστα υπάρχει η δυνατότητα προσθήκης ή αλλαγής στο αποτέλεσμα αυτό.
3. Στο δεξιό μέρος και πάνω μπορούμε να δούμε τις Β.Δ. που υπάρχουν και πατώντας διπλό κλικ πάνω σε κάποια να την κάνουμε τρέχουσα (όπως με την εντολή `use <όνομα Β.Δ.>`). Επίσης με δεξί κλικ στο τμήμα των Β.Δ. μας ανοίγει ένα σύντομο μενού επιλογών (ανάλογα που κάνουμε δεξί κλικ, στο κενό με επιλεγμένη κάποια Β.Δ. ως τρέχουσα ή στο κενό χωρίς κάποια Β.Δ. επιλεγμένη ως τρέχουσα ή πάνω σε κάποια Β.Δ. το μενού έχει κάποιες επιλογές ενεργοποιημένες και κάποιες απενεργοποιημένες και αναφέρεται στην επιλεγμένη/τρέχουσα Β.Δ. ή σε καμία). Μάλιστα στο μενού συντόμευσης μας δίνονται και οι συντομεύσεις που μπορούμε να χρησιμοποιήσουμε για να κάνουμε κάποιες ενέργειες από το πληκτρολόγιο (πχ Ctrl-T η Δημιουργία Νέου Πίνακα). Επίσης υπάρχουν και άλλες δύο καρτέλες σε αυτό το μέρος «Σελιδοδείκτες» και «Ιστορικό» που βλέπουμε τα αντίστοιχα στοιχεία.
4. Στο δεξιό μέρος και κάτω υπάρχει η ενσωματωμένη βοήθεια οργανωμένη σε τέσσερις καρτέλες. Πχ στην δεύτερη καρτέλα υπάρχουν όλες οι συναρτήσεις της MySQL οργανωμένες κατά κατηγορία και πατώντας διπλό κλικ σε μία από αυτές θα μας εμφανιστεί μία νέα μεγάλη καρτέλα στο αριστερό μέρος με βοήθεια για αυτή τη συνάρτηση.

Το μενού επιλογών που εμφανίζεται όταν πατάμε δεξί κλικ, στο αριστερό και πάνω τμήμα της εφαρμογής, μας επιτρέπει να διαγράψουμε μία Β.Δ. αλλά και να δημιουργήσουμε μία νέα. Επιλέγοντας «**Drop Σχήμα**» από το σύντομο μενού επιλογών διαγράφουμε την επιλεγμένη (ή την τρέχουσα αν δεν πατάμε πάνω σε κάποια Β.Δ.). Επιλέγοντας «**Δημιουργία Νέου Σχήματος**» δημιουργούμε μία νέα Β.Δ.. Θεωρώντας ότι διαγράφουμε τη Β.Δ. newsdb και αμέσως μετά επιλέγουμε τη δημιουργία νέου σχήματος μας εμφανίζεται ο διάλογος του επόμενου σχήματος στον οποίο βάζουμε το όνομα της νέας Β.Δ. που θέλουμε να δημιουργήσουμε και πατώντας «ΟΚ» άμεσα δημιουργείται και εμφανίζεται στο δεξιό τμήμα της βασικής οθόνης.



Εικόνα 10: Εισαγωγή ονόματος της νέας Β.Δ. (νέου Σχήματος)

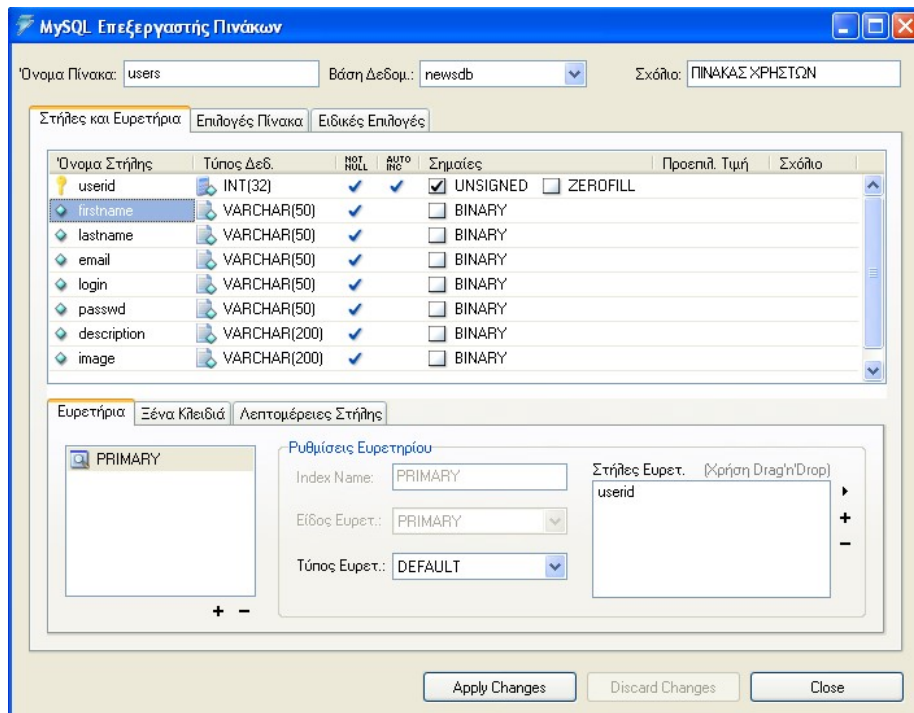
4.3.3 Προβολή, Δημιουργία και Επεξεργασία Πινάκων Β.Δ.

Αφού έχουμε δημιουργήσει τη Β.Δ., για να δημιουργήσουμε έναν πίνακα (CREATE TABLE) σε αυτή, αρκεί και πάλι να πατήσουμε δεξί κλικ πάνω της και να επιλέξουμε από το μενού συντόμευσης τη «**Δημιουργία Νέου Πίνακα**». Τότε εμφανίζεται το παράθυρο διαλόγου «**Επεξεργαστής Πινάκων**» του επόμενου σχήματος, το οποίο διαθέτει τρεις καρτέλες.

Στην πρώτη καρτέλα «**Στήλες και Ευρετήρια**» γράφουμε το όνομα του πίνακα, κάποιο σχόλιο και στην κύρια περιοχή, που βρίσκεται στην μέση της καρτέλας, ορίζουμε ένα-ένα τα πεδία που έχει ο πίνακας και τις ιδιότητές τους (με διπλό κλικ μπαίνουμε στο όνομα του πεδίου για να δημιουργηθεί/αλλάξει ένα πεδίο). Οι ιδιότητες αλλάζουν με απλό κλικ πάνω τους, μάλιστα αν βρισκόμαστε σε ένα πεδίο (τρέχον πεδίο) τότε μπορούμε στην περιοχή που βρίσκεται στο κάτω μέρος της καρτέλας, και η οποία και αυτή έχει τρεις υποκαρτέλες καρτέλες (**Ευρετήρια**, **Ξένα Κλειδιά**, **Λεπτομέρειες Στήλης**), να ορίσουμε περισσότερες ιδιότητες για το τρέχον πεδίο. Πχ το αν ένα πεδίο είναι NOT NULL, τι σύνολο χαρακτήρων και collate θα έχει, τι προεπιλεγμένη τιμή κλπ τα καθορίζουμε από την υποκαρτέλα «**Λεπτομέρειες Στήλης**». Η υποκαρτέλα «**Ξένα Κλειδιά**» δεν αναφέρεται στο τρέχον πεδίο αλλά είναι ίδια όποιο και να έχουμε επιλέξει αφού εκεί έχει μια λίστα που μπορούμε με το «+» και «-» κάτω από αυτήν να ορίσουμε ποιο πεδίο είναι ξένο κλειδί καθώς και τους περιορισμούς για το κάθε ένα πεδίο στη λίστα αυτή. Ομοίως, ούτε η υποκαρτέλα «**Ευρετήρια**» αναφέρεται στο τρέχον πεδίο, αλλά μπορούμε από αυτήν να δημιουργήσουμε ευρετήρια (indexes) για ταχύτερη εύρεση με βάση κάποια πεδία (τα πεδία που ανήκουν στα κλειδιά έχουν πάντα προορισμένα ευρετήρια).

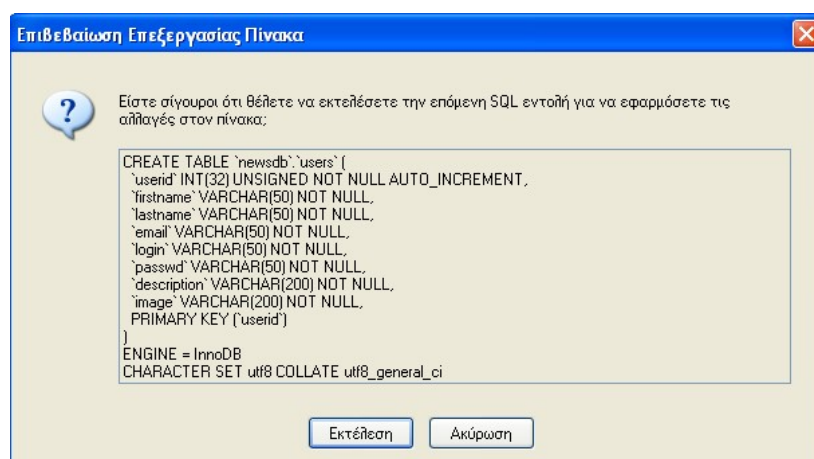
Στην δεύτερη καρτέλα «**Επιλογές Πίνακα**» ορίζουμε την μηχανή για όλο τον πίνακα, καθώς και το σύνολο χαρακτήρων και collation για όλο τον πίνακα (εκτός αν για κάποιο πεδίο έχω βάλει κάτι ιδιαίτερο). Όπως είδαμε και στις προηγούμενες υποενότητες, αλλά και εξηγούμε στο επόμενο κεφάλαιο, είναι πολύ σημαντικό να έχουμε πολυγλωσσικές Β.Δ. γι' αυτό προτείνεται να ορίσετε σύνολο χαρακτήρων **utf8** και collation **utf8_unicode_ci** (ή **utf8_general_ci**) σε αυτή τη καρτέλα όταν δημιουργείτε τους πίνακες. Με αυτές τις επιλογές δεν θα έχετε πρόβλημα με την αποθήκευση οποιασδήποτε πληροφορίας (σε οποιαδήποτε γλώσσα) στην Β.Δ. σας.

Στην τρίτη καρτέλα «**Ειδικές Επιλογές**» ορίζουμε αν θα έχουμε συμπίεση δεδομένων, από ποια τιμή θα ξεκινάει η αυτόματη αύξηση (auto_increment), τη μορφή των εγγραφών, το ελάχιστο και μέγιστο πλήθος εγγραφών, επιλογές αποθήκευσης κλπ.



Εικόνα 11: Επεξεργαστής Πινάκων - Καρτέλα δημιουργίας και διαμόρφωσης πεδίων (στηλών) πίνακα

Αφού βάλω τις επιλογές που θέλω και πατήσω «Apply Changes» εμφανίζεται ένας διάλογος (αν έχουμε βάλει την επιλογή στις «**Γενικές Ρυθμίσεις**»), που φαίνεται στο επόμενο σχήμα, που μου ζητάει να επιβεβαιώσω την εκτέλεση του ερωτήματος που δημιουργήθηκε από τις επιλογές που έβαλα. Όπως θα καταλάβατε, ουσιαστικά οι επιλογές που βάζω «καθοδηγούν» τον Query Browser στην δημιουργία ενός ερωτήματος, το οποίο δυσκολεύομαι να γράψω εγώ και προτιμώ τον γραφικό τρόπο, δηλαδή ότι δημιουργείται/αλλάζει γίνεται μόνο με την εκτέλεση ερωτημάτων, όπως κάναμε στην προηγούμενη ενότητα. Πατώντας εκτέλεση εκτελείται το ερώτημα και δημιουργείται ο πίνακας.

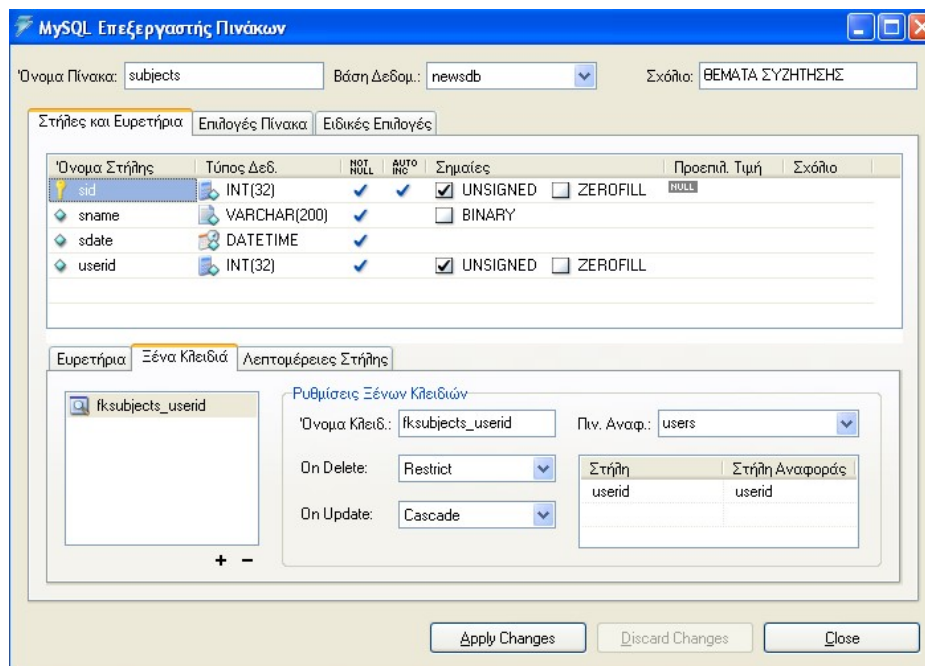


Εικόνα 12: Επιβεβαίωση εκτέλεσης ερωτήματος, που δημιουργήθηκε από τις επιλογές μας, για την δημιουργία του πίνακα

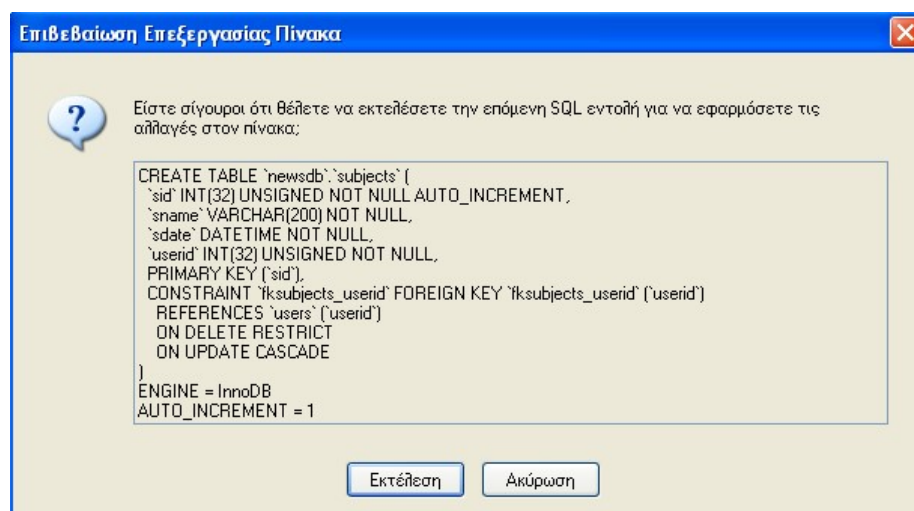
Τώρα αν θα δούμε στην βασική οθόνη κάτω από τη Β.Δ. **newsdb** θα υπάρχει ο πίνακας **users**. Αν θέλουμε να αλλάξουμε κάτι στη σχεδιάσή του (ALTER TABLE) αρκεί να πατήσουμε δεξί κλικ

πάνω του και από το μενού συντόμευσης να επιλέξουμε «**Επεξ/σία Πίνακα**», τότε θα ανοίξει πάλι το παράθυρο «**Επεξεργαστής Πινάκων**» όπου μπορούμε να κάνουμε ότι αλλαγές θέλουμε όπως όταν τον δημιουργούσαμε. Όμως τώρα μόλις πατήσουμε «**Apply Changes**» θα εμφανιστεί ένα παράθυρο επιβεβαίωσης που θα έχει μέσα ένα ερώτημα ALTER TABLE γιατί ο πίνακας υπάρχει ήδη.

Στα επόμενα 2 σχήματα φαίνεται η δημιουργία του πίνακα **subjects** με ενεργοποιημένη την πρώτη καρτέλα με τα πεδία και σε αυτή στο κάτω μέρος η υποκαρτέλα «**Ξένα Κλειδιά**» και το μήνυμα επιβεβαίωσης της εκτέλεσης του ερωτήματος δημιουργίας του.



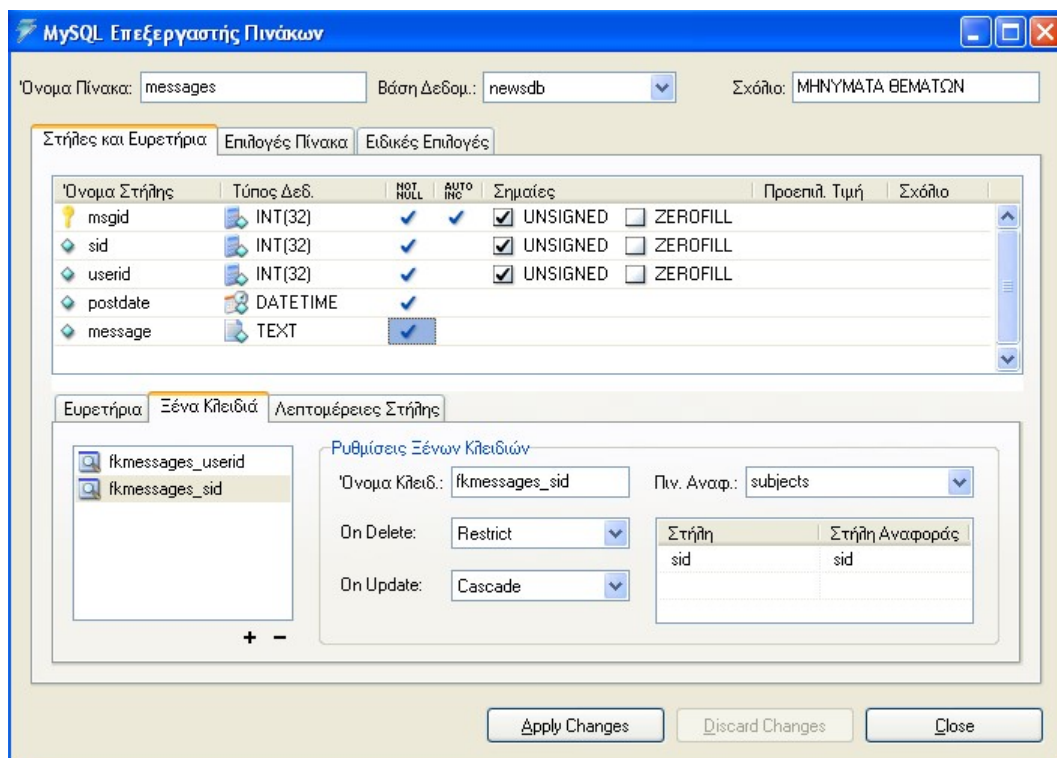
Εικόνα 13: Δημιουργία του πίνακα subjects και προβολή της υποκαρτέλας ορισμού των Ξένων Κλειδιών



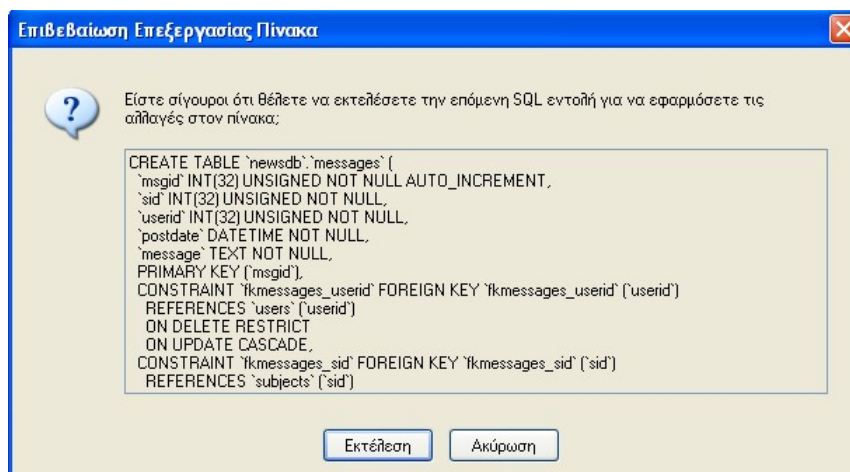
Εικόνα 14: Μήνυμα επιβεβαίωσης εκτέλεσης ερωτήματος δημιουργίας πίνακα subjects

Προσοχή πρέπει να δίνουμε στο όνομα του ξένου κλειδιού (στο όνομα στη λίστα και όχι το όνομα του πεδίου) γιατί πρέπει να είναι μοναδικό ανάμεσα στα ξένα κλειδιά που ορίζονται σε όλους τους πίνακες (στον συγκεκριμένο πίνακα το ονόμασα **fksubjects_userid** (fk subjects userid).

Στα επόμενα 2 σχήματα φαίνεται η δημιουργία του πίνακα **messages** με ενεργοποιημένη την πρώτη καρτέλα με τα πεδία και σε αυτή στο κάτω μέρος η υποκαρτέλα «Ξένα Κλειδιά» και το μήνυμα επιβεβαίωσης της εκτέλεσης του ερωτήματος δημιουργίας του.



Εικόνα 15: Δημιουργία του πίνακα messages και προβολή της υποκαρτέλας ορισμού των Ξένων Κλειδιών



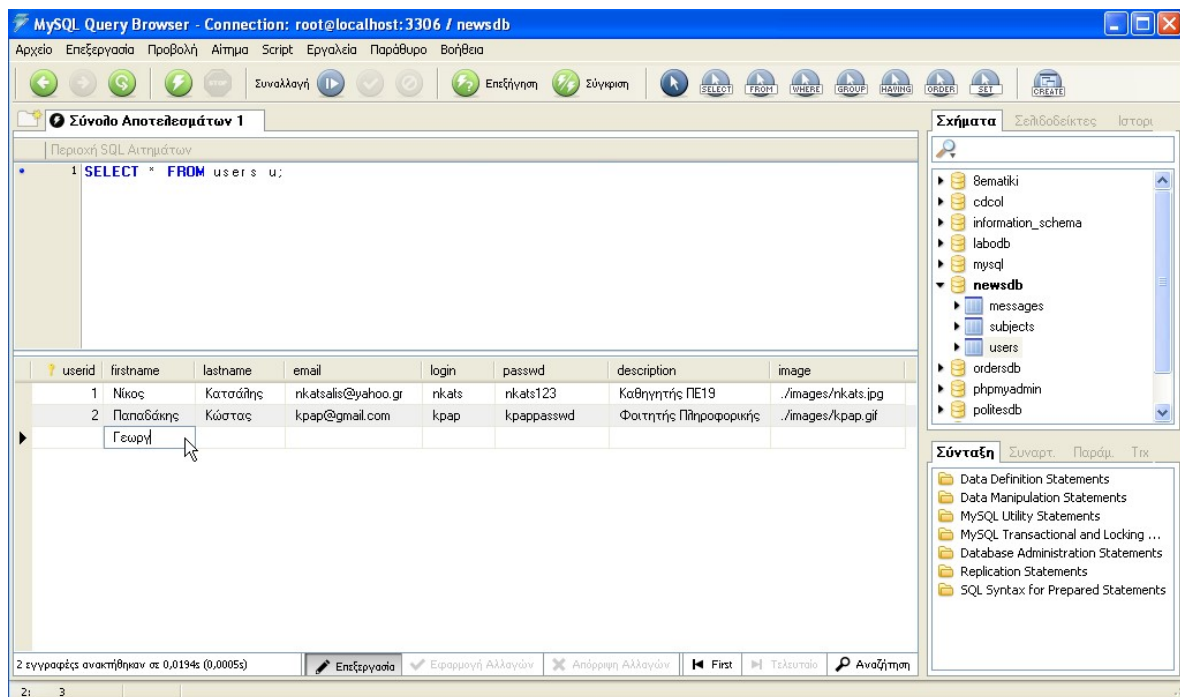
Εικόνα 16: Μήνυμα επιβεβαίωσης εκτέλεσης ερωτήματος δημιουργίας πίνακα messages

4.3.4 Ερωτήματα Εισαγωγής, Ενημέρωσης και Επιλογής στοιχείων Πινάκων Β.Δ.

Για να εκτελέσουμε ερωτήματα για την εισαγωγή, ενημέρωση και επιλογή στοιχείων μπορούμε πολύ απλά να γράψουμε σε SQL τα ερωτήματα, όπως είδαμε και στην προηγούμενη ενότητα, στην «Περιοχή SQL Αιτημάτων» και να τα εκτελέσουμε.

Για τα ερωτήματα επιλογής (SELECT) αυτός είναι ο τρόπος εργασίας, δηλαδή γράφουμε το ερώτημα και από το μενού συντομεύσεων, κάτω από το κυρίως μενού, πατάμε το κουμπί με τον

κεραυνό και το ερώτημα εκτελείται και τα αποτελέσματα εμφανίζονται από κάτω. Μάλιστα αν θέλουμε να επιλέξουμε όλα τα στοιχεία ενός πίνακα αρκεί να κάνουμε ένα διπλό κλικ πάνω του από τη δεξιά περιοχή και άμεσα θα γραφεί στην περιοχή ερωτημάτων ένα ερώτημα επιλογής όλων των γραμμών. Επίσης όπως αναφέραμε όλα τα ερωτήματα που έχουμε εκτελέσει είναι στην αριστερή περιοχή και πάνω στην καρτέλα «Ιστορικό» και από εκεί με διπλό κλικ σε κάποιο το φορτώνουμε στην «Περιοχή SQL Αιτημάτων».



Εικόνα 17: Εισαγωγή εγγραφής χρησιμοποιώντας την προβολή εγγραφών στο αριστερό κάτω τμήμα

Για τα ερωτήματα όμως εισαγωγής (INSERT) και ενημέρωσης (UPDATE) στοιχείων μπορούμε να κάνουμε ένα ερώτημα επιλογής και στα αποτελέσματα που θα μας εμφανιστούν κάτω από την «Περιοχή SQL Αιτημάτων» να μετακινηθούμε, να προσθέσουμε ή να αλλάξουμε τα στοιχεία των εγγραφών χρησιμοποιώντας και τα κουμπιά στο κάτω μέρος της περιοχής (**Επεξεργασία, Εφαρμογή Αλλαγών, Απόρριψη Αλλαγών, Μετακίνηση και Αναζήτηση**). Η διαδικασία αυτή φαίνεται στο προηγούμενο σχήμα. Συγκεκριμένα:

1. Για να εισάγουμε πατάμε το κουμπί «Επεξεργασία» κάτω από τις προβαλλόμενες εγγραφές και πατάμε ένα διπλό κλικ στην κενή εγγραφή κάτω από την τελευταία. Μετά πρέπει να πατήσουμε είτε «Εφαρμογή Αλλαγών» είτε «Απόρριψη Αλλαγών» ανάλογα αν θέλουμε να εφαρμοστούν ή όχι οι αλλαγές.
2. Για να ενημερώσουμε πάλι πατάμε το κουμπί «Επεξεργασία» κάτω από τις προβαλλόμενες εγγραφές και πατάμε ένα διπλό κλικ στο πεδίο της εγγραφής που θέλουμε να ενημερώσουμε. Μετά πρέπει και πάλι να πατήσουμε είτε «Εφαρμογή Αλλαγών» είτε «Απόρριψη Αλλαγών» ανάλογα αν θέλουμε να εφαρμοστούν ή όχι οι αλλαγές.
3. Σε κάθε περίπτωση μπορούμε να μετακινηθούμε είτε με το ποντίκι είτε με τα κουμπιά μετακίνησης που υπάρχουν.
4. Τέλος με το κουμπί αναζήτησης εμφανίζεται ένα παράθυρο (το κλασικό αναζήτησης/αντικατάστασης) που εισάγουμε την τιμή που ψάχνουμε για να μας πάει στην εγγραφή που τη βρίσκει ή στο αντικατάσταση βάζουμε τιμή αναζήτησης και τιμή αντικατάστασης και μπορούμε να υλοποιήσουμε (όπως σε ένα κειμενογράφο) μια έξυπνη

ενημέρωση (UPDATE). Προσοχή πρέπει να δοθεί να επιλέξουμε αν θα ψάξει/αντικαταστήσει στην επιλεγμένη ή σε όλες τις στήλες (και να έχουμε αν χρειαστεί επιλέξει κάποια).

Υπάρχει περίπτωση να έχουμε πρόβλημα με τα Ελληνικά και ενώ εισάγουμε Ελληνικούς χαρακτήρες στα πεδία των εγγραφών που εισάγουμε ή ενημερώνουμε, όταν κάνουμε τις ενημερώσεις και ξανακάνουμε επιλογή των εγγραφών από τη Β.Δ. εμφανίζονται χαρακτήρες «σκουπίδια». Στην περίπτωση αυτή πρέπει να φροντίσουμε να δούμε τι encoding (**character set**) έχει το πρόγραμμα πελάτης μας και να ενημερώσουμε τον MySQL εξυπηρετητή. Αυτό γίνεται αν γράψουμε και να εκτελέσουμε μία φορά το εξής ερώτημα στην «**Περιοχή SQL Αιτημάτων**» για τη συγκεκριμένη σύνδεση:

set names encoding;

Στην θέση του encoding θα βάλουμε αυτό που χρησιμοποιεί η εφαρμογή πελάτης μας. Πχ αν χρησιμοποιεί **utf8** πρέπει να εκτελέσουμε το ερώτημα ***set names utf8;*** ενώ αν χρησιμοποιούμε **ISO 8859-7 Greek** πρέπει να εκτελέσουμε το ερώτημα ***set names greek;*** Περισσότερες λεπτομέρειες θα βρείτε στο επόμενο κεφάλαιο για την PHP στην ενότητα «**Ελληνικοί χαρακτήρες και PHP-MySQL**».

4.4 MySQL και PHP

Επειδή το εγχειρίδιο αυτό δεν έχει σκοπό να αποτελέσει πλήρη οδηγό του MySQL, αλλά να παρουσιάσει τη χρήση του μέσα από την PHP, είδαμε σύντομα κάποιες βασικές εντολές Δημιουργίας της Β.Δ. και των Πινάκων, Εισαγωγής, Επιλογής και Ενημέρωσης στοιχείων. Ο τρόπος και παραδείγματα για τη χρήση του MySQL μέσα από την PHP παρατίθενται στο επόμενο κεφάλαιο που παρουσιάζεται η PHP. Για πλήρη παρουσίαση της χρήσης και των δυνατοτήτων του MySQL ο αναγνώστης μπορεί να διαβάσει το Εγχειρίδιο Αναφοράς του (<https://dev.mysql.com/doc/>).

5 Η γλώσσα PHP

Στην ενότητα αυτή παρουσιάζονται συνοπτικά τα βασικά στοιχεία της PHP (τι είναι, τρόπος χρήσης, συντακτικό, μεταβλητές, δομές, συχνά χρησιμοποιούμενες συναρτήσεις) καθώς και οι συναρτήσεις, του MySQL Module, για τη σύνδεση με MySQL. Επίσης, στο τέλος της ενότητας παρατίθενται ένας οδηγός για την προτεινόμενη (από το εγχειρίδιο αυτό) μεθοδολογία επικοινωνίας με τον MySQL καθώς και κάποια παραδείγματα έτοιμων προγραμμάτων. Στο εγχειρίδιο αυτό τα παραδείγματα αφορούν την PHP 5.X.

Για πλήρη οδηγό της γλώσσας, το σύνολο των συναρτήσεων και των modules σύνδεσης με άλλες Β.Δ. και εφαρμογές, λεπτομέρειες όσον αφορά τη χρήση καθώς και παραδείγματα μπορείτε να δείτε το Εγχειρίδιο Αναφοράς της PHP (<http://www.php.net/docs.php>).

5.1 Τι είναι η PHP

Η PHP είναι μια γενικού σκοπού scripting γλώσσα προγραμματισμού, η οποία είναι ειδικά κατάλληλη για ανάπτυξη εφαρμογών για το Web και μπορεί να ενσωματωθεί στην HTML. Τα αρχικά στο όνομά της αντιπροσωπεύουν το "PHP: Hypertext Preprocessor". Ένα σημαντικό στοιχείο της PHP είναι ότι είναι ΕΛ/ΛΑΚ με όλα τα πλεονεκτήματα που αυτό προσφέρει. Επίσης η PHP ενώ είναι πολύ απλή για ένα αρχάριο προγραμματιστή συγχρόνως προσφέρει πολλά προχωρημένα χαρακτηριστικά για έναν επαγγελματία προγραμματιστή.

Ο κώδικας PHP είναι εσώκλειστος σε HTML κείμενο σε ειδικά tags (ετικέτες) αρχής και τέλους (δείτε παρακάτω υποενότητα «**Βγαίνοντας από την HTML και μπαίνοντας στον κώδικα PHP**») που σας επιτρέπουν να μεταφέρεστε μέσα και έξω από το "PHP mode" (PHP τρόπο λειτουργίας).

Παράδειγμα Κώδικα PHP

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title>Τίτλος Σελίδας</title>
</head>
<body>
    <?php
    echo "<h1> Παράδειγμα PHP script ! </h1>\n";
    ?>
</body>
</html>
```

Αποτέλεσμα Εκτέλεσης Κώδικα PHP

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title>Τίτλος Σελίδας</title>
</head>
<body>

<h1> Παράδειγμα PHP script ! </h1>

</body>
</html>
```

Ο κώδικας της PHP εκτελείται στον Web Εξυπηρετητή. Έτσι αν το παραπάνω πρόγραμμα (script) βρισκόταν στον εξυπηρετητή και το ζητούσε ο πελάτης, τελικά θα λάμβανε ένα αρχείο που θα

περιείχε το αποτέλεσμα της εκτέλεσης του κώδικα, δηλαδή ένα HTML κείμενο. Μάλιστα ο πελάτης δεν έχει κανέναν τρόπο να καταλάβει τι κώδικας υπάρχει από πίσω (αν φροντίσουμε το αρχείο με τον κώδικα PHP να έχει κατάληξη .html και να έχουν γίνει οι κατάλληλες ρυθμίσεις στον εξυπηρετητή, μέσω οδηγίας στο http.conf ή το .htaccess, ώστε να εκτελεί PHP και σε αρχεία με κατάληξη .html). Φυσικά θα πρέπει να φροντίσουμε και άλλες «λεπτομέρειες», όπως να τυπώνουμε κατάλληλα την HTML μέσω PHP (πχ να βάζουμε αλλαγή γραμμή κλπ) ώστε ο κώδικας να φαίνεται γραμμένος με το χέρι και όχι αυτόματα παραγόμενος κλπ.

Με την χρήση της PHP, λοιπόν, αντί να γράφουμε κάποιο πρόγραμμα σε κάποια γλώσσα προγραμματισμού το οποίο να παράγει σαν έξοδο ένα HTML αρχείο, γράφουμε ένα HTML αρχείο μέσα στο οποίο όποτε θέλουμε γράφουμε PHP κώδικα ο οποίος τυπώνει HTML στο σημείο του HTML αρχείου που γράφουμε τον κώδικα. Φυσικά με την PHP δεν είμαστε περιορισμένοι να εξάγουμε μόνο HTML αλλά και οποιοδήποτε άλλο κείμενο, XHTML, XML, εικόνες, αρχεία PDF, ακόμη και ταινίες Flash (χρησιμοποιώντας τα libswf και Ming). Επίσης όλα αυτά μπορεί να εξάγονται και να αποθηκεύονται στο σύστημα αρχείων. Επίσης ένα από τα μεγαλύτερα πλεονεκτήματα της PHP είναι η δυνατότητά της για επικοινωνία με ένα μεγάλο πλήθος Συστημάτων Διαχείρισης Βάσεων Δεδομένων όπως:

Adabas D	Ingres	Oracle (OCI7 and OCI8)
dBase	InterBase	Ovrimos
Empress	FrontBase	PostgreSQL
FilePro (read-only)	mSQL	Solid
Hyperwave	Direct MS-SQL	Sybase
IBM DB2	MySQL	Velocis
Informix	ODBC	Unix dbm

Πίνακας 1: ΣΑΒΔ με τα οποία μπορεί και επικοινωνεί η PHP

Παρόλο που η κύρια χρήση της PHP, όπως ήδη αναφέρθηκε, είναι για εκτέλεση προγραμμάτων (scripts) στον εξυπηρετητή, δεν παύει να είναι μια γενικού σκοπού scripting γλώσσα και μπορεί να κάνει και πολλά άλλα όπως:

- **Command line scripting** με τη χρήση του μεταγλωττιστή της PHP (χωρίς Web εξυπηρετητή και Web browser)
- **Client-side GUI** με τη χρήση του **PHP-GTK**.

Το συγκεκριμένο εγχειρίδιο περιορίζεται στην χρήση της μόνο για εκτέλεση προγραμμάτων (scripts) στον εξυπηρετητή. Για τις άλλες χρήσεις μπορείτε να πάρετε πληροφορίες από την ιστοσελίδα της PHP (<https://www.php.net>).

5.2 Πως λειτουργεί η PHP

5.2.1 Βασικός Τρόπος Λειτουργίας

Όπως ήδη αναφέραμε η PHP είναι βασικά μια scripting γλώσσα προγραμματισμού που εκτελείται στον εξυπηρετητή (τουλάχιστον αυτό μας ενδιαφέρει στο συγκεκριμένο εγχειρίδιο). Η γενική της λειτουργία είναι η εξής:

1. Ένας πελάτης ζητάει μια ιστοσελίδα από τον εξυπηρετητή
2. Εφόσον η συγκεκριμένη ιστοσελίδα έχει κατάληξη (πχ .php, .htm κλπ) που εμείς έχουμε ορίσει ότι θέλουμε να ελέγχεται από τον PHP parser/interpreter (ο διερμηνευτής, δηλαδή το πρόγραμμα που διατρέχει την ιστοσελίδα και εκτελεί τα scripts PHP που βρίσκει) πριν αποσταλεί στον χρήστη δίνεται στον PHP parser/interpreter.

3. Το αποτέλεσμα του PHP parser/ interpreter αποστέλλεται στον πελάτη.

Μάλιστα αυτό μπερδεύει πολλούς αρχάριους οι οποίοι έχουν συνηθίσει όταν εκτελείται κώδικας να τυπώνει κάπου αλλού (π.χ. στην οθόνη) και τους δυσκολεύει να αντιληφθούν ότι αυτό που τυπώνουμε με PHP τυπώνεται στο σημείο της ιστοσελίδας που έχουμε βάλει τον κώδικα. Γιαυτό άλλωστε συνήθως αυτό που τυπώνουμε είναι με ετικέτες HTML αφού τελικά θα διαμορφώσει την ιστοσελίδα που βρίσκεται ο κώδικας και θα προβληθεί ως HTML ιστοσελίδα από ένα browser.

Ανάλογα με το Λειτουργικό Σύστημα στο οποίο εκτελείται ο εξυπηρετητής καθώς και το είδος του εξυπηρετητή υπάρχουν πολλοί τρόποι εγκατάστασης και λειτουργίας της PHP. Οι πιο γνωστοί είναι:

- Σαν ένα module
- Σαν ένα CGI executable

Η εγκατάσταση σαν module είναι η πιο αποδοτική και ασφαλής.

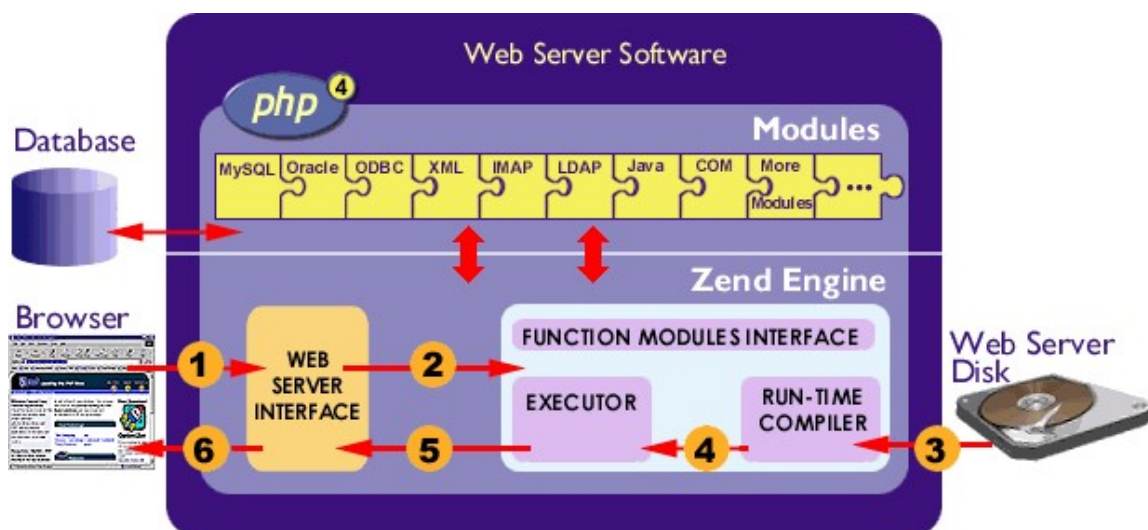
5.2.2 Δομή συνολικού συστήματος

Η δομή της PHP όπως φαίνεται από έξω, αν τη δούμε σαν συνολικό σύστημα, καθώς και τα βήματα λειτουργίας (1, 2, 3 ...) παρουσιάζονται στο επόμενο σχήμα.

Το βασικό κομμάτι στο οποίο στηρίζεται η PHP είναι η μηχανή της γλώσσας (language engine) η οποία ονομάζεται Zend και ουσιαστικά αποτελεί τον πυρήνα της. Για να δημιουργηθεί ένας λειτουργικός Web Server Interpreter (διερμηνευτής) απαιτούνται τρία τμήματα:

1. Το **τμήμα του διερμηνευτή (interpreter part)** το οποίο αναλύει τον κώδικα, τον μεταγλωττίζει και τον εκτελεί.
2. Το **τμήμα λειτουργιών (functionality part)** που υλοποιεί τη λειτουργικότητα της γλώσσας (συναρτήσεις κλπ).
3. Το **τμήμα διεπαφής (interface part)** που επικοινωνεί με τον Web Εξυπηρετητή.

Όπως φαίνεται και στο σχήμα η μηχανή Zend περιλαμβάνει όλο το πρώτο τμήμα (διερμηνευτή) και ένα μέρος του δεύτερου (τμήμα λειτουργιών), κυρίως το βασικό τμήμα των λειτουργιών της γλώσσας και των ενσωματωμένων συναρτήσεων. Μαζί με τα modules επέκτασης (το υπόλοιπο δεύτερο τμήμα) και το τρίτο τμήμα (τμήμα διεπαφής) έχουμε το ολοκληρωμένο σύστημα της PHP.



Εικόνα 18: Η δομή του συστήματος της PHP

5.3 Κανόνες σύνταξης – Εντολές

5.3.1 Βγαίνοντας από την HTML και μπαίνοντας στον κώδικα PHP

Υπάρχουν διάφοροι τρόποι για να καθορίσεις πως οριοθετείτε (αρχίζει και τελειώνει) ο κώδικας PHP μέσα σε ένα αρχείο HTML. Ο καλύτερος τρόπος είναι χρησιμοποιώντας τις HTML ετικέτες:

```
<?php . . . εδώ μπαίνει ο κώδικας . . . ?>
```

Δηλαδή με το tag **<?php** δηλώνουμε ότι ξεκινάει ο κώδικας και με το **?>** ότι τελειώνει. Μόλις ο parser της PHP συναντήσει την αρχή, ξεκινάει την εκτέλεση του κώδικα μέχρι να συναντήσει το κλείσιμο. Δηλαδή με το άνοιγμα **<?php** μπαίνουμε σε PHP κατάσταση και με το κλείσιμο **?>** ξαναμπαίνουμε σε HTML κατάσταση.

Ο τρόπος αυτός είναι ο καλύτερος γιατί είναι πάντα διαθέσιμος και υποστηρίζεται και σε κώδικα συμβατό με XML όπως XHTML. Επίσης με κατάλληλη ενεργοποίηση (μέσω οδηγίας στο php.ini) μπορεί να χρησιμοποιείς και τις σύντομες ετικέτες **<?** για άνοιγμα και **?>** για κλείσιμο.

Άλλοι τρόποι είναι με τις ετικέτες:

```
<script language="php">. . . εδώ μπαίνει ο κώδικας . . . </script>
```

Επίσης μπορεί να χρησιμοποιηθούν ASP-style ετικέτες **<%** για άνοιγμα και **%>** για κλείσιμο αλλά πρέπει να ενεργοποιηθούν με κατάλληλες ρυθμίσεις.

Παραδείγματα κώδικα με τη χρήση διαφόρων τρόπων:

```
<?php echo ("Για υποστήριξη HTML εγγράφων καλύτερα με αυτές τις  
ετικέτες\n"); ?>
```

```
<?php  
echo ("Είναι ο απλούστερος και ο γενικότερος τρόπος (SGML εντολή  
επεξεργασίας)\n");  
?>
```

```
<script language="php">  
echo ("Μερικοί editors προτιμούν αυτόν τον τρόπο !");  
</script>
```

```
<% echo ("Φυσικά μπορείς να χρησιμοποιήσεις και ASP-style tags"); %>
```

Φυσικά στην PHP μπορείς να κάνεις και πιο προχωρημένο κώδικα όπως στο επόμενο παράδειγμα που αναμιγνύουμε PHP mode με HTML mode. Ουσιαστικά βγαίνουμε από PHP mode στην μέση ενός PHP μπλοκ (μίας if) και μετά ξαναμπαίνουμε για να ολοκληρώσουμε το μπλοκ (την if στην προκειμένη περίπτωση):

```
<?php  
if ($expression) {  
?>  
<strong>This is true.</strong>  
<?php  
} else {  
?>  
<strong>This is false.</strong>  
<?php  
}  
?>
```

Στο παραπάνω παράδειγμα αν η μεταβλητή \$expression είναι αληθής τότε θα μείνει το:

```
<strong>This is true.</strong>
```

ενώ αν είναι ψευδής θα μείνει το:

```
<strong>This is false.</strong>
```

Με αυτόν τον τρόπο μπορούμε να γλιτώσουμε τη χρήση των εντολών εκτύπωσης της PHP (echo, print κ.λ.π.) που απαιτούν εισαγωγικά και μας δυσκολεύουν και άρα να μην γράφουμε πολύ κώδικα, Φυσικά για έναν αρχάριο είναι δύσκολο γιατί μπερδεύεται και πολλές φορές ξεχνάει τμήμα κώδικα με αποτέλεσμα να έχουμε λάθη στην εκτέλεση.

Ο τρόπος με τον οποίο, στην PHP, εκτυπώνουμε το κείμενο που θέλουμε να μείνει είναι με τη χρήση των συναρτήσεων **echo** και **print**. Και οι δύο συναρτήσεις θα παρουσιαστούν σε επόμενη ενότητα που αφορά τις συναρτήσεις, αλλά επειδή χρησιμοποιούνται διαρκώς στα παραδείγματα, παρουσιάζονται και εδώ.

Η **echo** συντάσσεται ως εξής:

```
void echo ( string arg1 [, string argn...])
```

και εμφανίζει ένα ή περισσότερα strings τα οποία παίρνει ως παράμετρο. Αν το όρισμα είναι μόνο ένα δεν χρειάζονται ούτε και οι παρενθέσεις. Αν μέσα σε ένα string έχουμε μεταβλητές της PHP (δες παρακάτω για μεταβλητές) και χρησιμοποιηθούν μονά εισαγωγικά τότε δεν τυπώνεται η τιμή των μεταβλητών αλλά ότι υπάρχει μέσα στα μονά εισαγωγικά. Ενώ με τη χρήση διπλών εισαγωγικών, οι μεταβλητές είναι διακριτές μέσα στο string και τυπώνονται οι τιμές τους (και τα υπόλοιπα όπως είναι).

Η **print** συντάσσεται ως εξής:

```
int print ( string arg)
```

και εμφανίζει ένα string και επιστρέφει και την τιμή 1. Και εδώ ισχύουν τα ίδια με την echo για τις παρενθέσεις και τη χρήση των μονών – διπλών εισαγωγικών.

Παράδειγμα

```
$x = "nikos";  
echo 'καλημέρα $x <p />'; // τυπώνει καλημέρα $x  
echo "καλημέρα $x <p />"; // τυπώνει καλημέρα nikos  
print 'καλημέρα $x <p />'; // τυπώνει καλημέρα $x  
print "καλημέρα $x <p />"; // τυπώνει καλημέρα nikos
```

5.3.2 Διαχωρισμός εντολών και σχόλια

Ο διαχωρισμός εντολών στην PHP γίνεται όπως και στην C, με τη χρήση του ερωτηματικού «;». Για να γράψουμε σχόλια υποστηρίζονται ότι και στη C, στη C++ και στο Unix Shell. Δηλαδή:

```
/* ...Σχόλιο μίας ή περισσότερων γραμμών ... */  
// η υπόλοιπη γραμμή είναι σχόλιο  
# η υπόλοιπη γραμμή είναι σχόλιο
```

Παράδειγμα:

```
<?php  
echo "Hello !"; // σχόλιο στυλ C++  
echo "Hello again !";  
/* Σχόλιο πολλών γραμμών με  
το στυλ της C */  
echo "Hello for the third time"; # Σχόλιο στυλ Unix Shell  
?>
```


5.4 Τύποι Δεδομένων

Η PHP υποστηρίζει τους εξής πρωταρχικούς τύπους:

1. **boolean** (για λογικούς, TRUE ή FALSE)
2. **integer** (για ακεραίους)
3. **float** και **double** (για πραγματικούς αριθμούς και είναι το ίδιο)
4. **string** (για αλφαριθμητικά)
5. **array** (για πίνακες)
6. **object** (για αντικείμενα)
7. **Resource** (για αναφορές σε εξωτερικούς πόρους)

Επίσης υπάρχουν και τρεις σύνθετοι τύποι, οι οποίοι χρησιμοποιούνται κυρίως για ορισμό παραμέτρων:

1. **mixed** (για παράμετρο που μπορεί να είναι πολλών τύπων)
2. **number** (για παράμετρο που μπορεί να είναι είτε integer είτε float)
3. **callback** (για πραγματικούς)

Αυτό που πρέπει να κατανοήσουμε είναι ότι στην PHP δεν χρειάζεται να ορίσουμε εμείς τον τύπο μιας μεταβλητής, αλλά αυτός ορίζεται έμμεσα από την τιμή που δίνουμε στην μεταβλητή κατά τη διάρκεια εκτέλεσης του προγράμματος (κοίτα επόμενη υποενότητα «Μεταβλητές»). Γι' αυτό υπάρχουν μια σειρά συναρτήσεις σχετικές με τον έλεγχο και τη μετατροπή τύπων. Μερικές από αυτές είναι:

- **var_dump()**, η οποία παίρνει σαν όρισμα μία ή περισσότερες εκφράσεις και τυπώνει τον τύπο και την τιμή τους.
- **gettype()**, η οποία παίρνει σαν όρισμα μία μεταβλητή και επιστρέφει τον τύπο της (ένα αλφαριθμητικό, πχ Integer). Χρησιμοποιείται κυρίως για debugging.
- **is_integer()**, **is_string()**, **is_boolean()** κλπ, οι οποίες επιστρέφουν TRUE ή FALSE αν η μεταβλητή που τους δίνεται ως όρισμα είναι του συγκεκριμένου τύπου.
- **cast()** και **settype()**, με τις οποίες κάνουμε μετατροπή τύπων (typecasting).

Παραδείγματα:

```
<?php
$bool = TRUE;    // ένας boolean
$str   = "foo";  // ένας string
$int   = 12;     // ένας integer

echo gettype($bool); // εκτυπώνει "boolean"
echo gettype($str);  // εκτυπώνει "string"

// If this is an integer, increment it by four
if (is_int($int)) {
    $int += 4;
}

// Αν η μεταβλητή $bool είναι τύπου string, τύπωσέ την αλλιώς μην τυπώνει τίποτα
if (is_string($bool)) {
    echo "String: $bool";
}
?>
```

Στην PHP υπάρχει μια ειδική τιμή **NULL** (που γράφεται και με οποιονδήποτε συνδυασμό μικρών-κεφαλαίων) και η οποία σημαίνει κενό. Την τιμή **NULL** την παίρνει μια μεταβλητή στις εξής περιπτώσεις:

- Της έχουμε αναθέσει την τιμή NULL
 - Δεν της έχουμε αναθέσει ακόμα τιμή
 - Έχει κληθεί η συνάρτηση **unset()** με όρισμα τη μεταβλητή
- Χρήσιμη συνάρτηση για να ελέγξεις αν κάποια μεταβλητή είναι NULL είναι η **is_null()**.

Ο επόμενος πίνακας δείχνει μια σειρά μεταβλητών και τι επιστρέφουν οι πιο γνωστές συναρτήσεις (που θα χρησιμοποιούμε και εμείς) για αυτές:

Expression	gettype()	empty()	is_null()	isset()	boolean : if(\$x)
\$x = "";	<u>string</u>	TRUE	FALSE	TRUE	FALSE
\$x = NULL	<u>NULL</u>	TRUE	TRUE	FALSE	FALSE
var \$x;	<u>NULL</u>	TRUE	TRUE	FALSE	FALSE
\$x is undefined	<u>NULL</u>	TRUE	TRUE	FALSE	FALSE
\$x = array();	<u>array</u>	TRUE	FALSE	TRUE	FALSE
\$x = false;	<u>boolean</u>	TRUE	FALSE	TRUE	FALSE
\$x = true;	<u>boolean</u>	FALSE	FALSE	TRUE	TRUE
\$x = 1;	<u>integer</u>	FALSE	FALSE	TRUE	TRUE
\$x = 42;	<u>integer</u>	FALSE	FALSE	TRUE	TRUE
\$x = 0;	<u>integer</u>	TRUE	FALSE	TRUE	FALSE
\$x = -1;	<u>integer</u>	FALSE	FALSE	TRUE	TRUE
\$x = "1";	<u>string</u>	FALSE	FALSE	TRUE	TRUE
\$x = "0";	<u>string</u>	TRUE	FALSE	TRUE	FALSE
\$x = "-1";	<u>string</u>	FALSE	FALSE	TRUE	TRUE
\$x = "php";	<u>string</u>	FALSE	FALSE	TRUE	TRUE
\$x = "true";	<u>string</u>	FALSE	FALSE	TRUE	TRUE
\$x = "false";	<u>string</u>	FALSE	FALSE	TRUE	TRUE

5.5 Μεταβλητές και Σταθερές

5.5.1 Μεταβλητές

Οι μεταβλητές στην PHP αναπαρίστανται από το σύμβολο του δολαρίου ακολουθούμενο από το όνομα της μεταβλητής.

Το όνομα της μεταβλητής είναι **case-sensitive**. Τα ονόματα των μεταβλητών ακολουθούν τους ίδιους κανόνες όπως και οι labels στην PHP. Ένα έγκυρο όνομα μεταβλητής αρχίζει με ένα γράμμα ή underscore, ακολουθούμενο από οποιονδήποτε αριθμό από γράμματα, αριθμούς, ή underscores.

Παραδείγματα:

```
<?php
$var = "Bob";
$Var = "Joe";
echo "$var, $Var";      // τυπώνει "Bob, Joe"

$4site = 'not yet';    // Μη αποδεκτή μεταβλητή αφού ξεκινάει με αριθμό
$_4site = 'not yet';   // Αποδεκτή μεταβλητή αφού ξεκινάει με underscore
$tδyte = 'mansikka';   // Αποδεκτό αφού ο 'δ' είναι (Extended) ASCII 228.
?>
```

Στην PHP όταν θέτουμε μια μεταβλητή ίση με μία άλλη τότε απλώς το περιεχόμενο της δεύτερης αντιγράφεται στην πρώτη. Πχ.

```
<?php $a = 10; $b = $a; ?>
```

Τώρα και η μεταβλητή \$b είναι μεταβλητή τύπου Integer με τιμή 10 αλλά είναι μια διαφορετική μεταβλητή από την \$a, δηλαδή αν αλλάξουμε την τιμή της \$a δεν θα επηρεαστεί η \$b.

Στην PHP υποστηρίζεται και η **ανάθεση με αναφορά** στην οποία η διαφορά είναι ότι βάζουμε μπροστά από τη δεύτερη μεταβλητή το & και τότε απλώς η πρώτη μεταβλητή αντιστοιχίζεται στην δεύτερη. Δηλαδή αλλάζοντας την μία αλλάζει και η άλλη αφού στην ουσία είναι η ίδια μεταβλητή.

Πχ

```
<?php $a = 10; $b = &$a; $b = 5; ?>
```

Τώρα και η μεταβλητή \$a και η \$b έχουν τιμή 5. Αφού βάλαμε την \$b να αντιστοιχιστεί στην \$a (**ανάθεση με αναφορά**) και συνεπώς όταν αλλάξαμε την \$b άλλαξε και η \$a. Η **ανάθεση με αναφορά** εφαρμόζεται μόνο ανάμεσα σε μεταβλητές με όνομα. Αν θέλουμε να πάψει μία από αυτές να δείχνει στην τιμή αυτή πρέπει να χρησιμοποιηθεί η συνάρτηση **unset()**. Έτσι αν θέλαμε η \$a να πάψει να έχει αντιστοιχιστεί στην συγκεκριμένη τιμή θα γράφαμε **unset(\$a)**, κάτι όμως που δεν θα επηρέαζε την \$b.

Πρέπει να επισημάνουμε ότι η ανάθεση με αναφορά δεν είναι όπως οι δείκτες σε κάποιες γλώσσες (πχ στη C) και γι' αυτό ενώ κάποιες χρήσεις της είναι όπως των δεικτών (πχ πέρασμα σε συνάρτηση τιμής με αναφορά), δεν έχουν την ίδια συμπεριφορά σε όλες τις περιπτώσεις. Στην πραγματικότητα αυτό που γίνεται είναι να δημιουργείται ένα alias στον πίνακα συμβόλων (symbol table) που και αυτή δείχνει στον ίδιο χώρο μνήμης και όχι να έχουμε μια νέα μεταβλητή (όπως στους δείκτες).

Παρατήρηση: Στην PHP υπάρχουν και μια σειρά από δεσμευμένες μεταβλητές, οι οποίες εξαρτώνται από το περιβάλλον στο οποίο εκτελείται η PHP, αλλά η αναφορά τους είναι πέρα από τους σκοπούς του εγχειριδίου αυτού.

5.5.1.1 Μεταβλητές Μεταβλητών

Στην PHP μπορείς να έχεις μεταβλητά ονόματα μεταβλητών, δηλαδή το όνομα μιας μεταβλητής να αλλάζει δυναμικά. Αυτό γίνεται με τις μεταβλητές μεταβλητών όπου στην πραγματικότητα παίρνεις την τιμή μιας μεταβλητής και της συμπεριφέρεσαι ως όνομα μεταβλητής. Άρα αλλάζοντας την τιμή της μεταβλητής μεταβλητής αλλάζει το όνομα της μεταβλητής. Με ένα παράδειγμα θα γίνει πιο εύκολα κατανοητό:

```
<?php
$var = "Bob"; // Εδώ δηλώνω και δίνω τιμή στη μεταβλητή $var
$$var = "Hello"; // Εδώ δηλώνω και δίνω τιμή στη μεταβλητή $Bob

// Πρέπει να χρησιμοποιήσω {} αλλιώς αν έγραφα $$var θα τυπώσει το πρώτο
// $ σαν απλό χαρακτήρα $ και μετά θα τυπώσει την τιμή της $var
echo "{$$var} $var"; // τυπώνει "Hello Bob"
// Είναι το ίδιο σαν να έγραφα (αφού υπάρχει η μεταβλητή $Bob)
echo "$Bob $var";
?>
```

5.5.1.2 Μεταβλητές Πίνακα (Arrays)

Στην PHP ο πίνακας είναι μια ιδιαίτερη δομή που στην πραγματικότητα αποτελεί μια «αντιστοίχιση» που αντιστοιχίζει τιμές σε κλειδιά. Τα κλειδιά είναι οι δείκτες του πίνακα και μπορεί να είναι οποιοσδήποτε βασικός τύπος (πχ ακέραιος, string) και οι τιμές είναι τα περιεχόμενα

του πίνακα και μπορεί να είναι οτιδήποτε (ακόμα και άλλος πίνακας) **χωρίς να είναι υποχρεωτικά του ίδιου τύπου μεταξύ τους**.

Αυτή η ιδιαίτερη δομή σου δίνει μεγάλη ευελιξία προγραμματιστικά αφού χρησιμοποιώντας έναν πίνακα μπορείς να υλοποιήσεις άλλες δομές όπως **Συλλογές (Collections)**, **Λίστες (Lists)**, **Ουρές (Queues)**, **Στοίβες (Stacks)**, **Λεξικά (Dictionaries)** κλπ.

Ο τρόπος για τη δημιουργία ενός πίνακα είναι πολλοί αλλά ο βασικός είναι με τη γλωσσική δομή **array()**. Συντάσσεται ως εξής:

```
array([key =>] value , ... )
```

Παράδειγμα

```
<?php
$arr = array("foo" => "bar", 12 => true);

echo $arr["foo"]; // τυπώνει bar
echo $arr[12];    // τυπώνει 1
?>
```

Επίσης μπορείς να δημιουργήσεις πίνακα με τον κλασικό τρόπο των άλλων γλωσσών (χρησιμοποιώντας τα []).

Παράδειγμα

```
<?php
//είναι το ίδιο με το προηγούμενο παράδειγμα
$arr ["foo"] = "bar";
$arr [12] = true;
?>
```

Αν χρησιμοποιήσεις τη δομή arr() αλλά δεν βάλεις κλειδιά παρά μόνο τιμές τότε μπαίνουν αυτόματα κλειδιά οι ακέραιοι 0, 1, 2, Αν βάλεις κάποια κλειδιά και ξεχάσεις κάποιο από αυτά (δηλαδή βάλεις μόνο την τιμή) τότε αυτό παίρνει άμεσα την τιμή του μεγαλύτερου ακεραίου κλειδιού που υπάρχει ήδη συν 1 ή την τιμή 0 αν δεν υπάρχει μέχρι εκείνο το σημείο κλειδί που να είναι ακέραιος.

Παράδειγμα

```
<?php
$arr = array("foo" => "bar", 25, 5 => 43, "car", 12 => true);

//Δημιουργούνται τα εξής στοιχεία:
// $arr["foo"] = "bar", $arr[0] = 25, $arr[5] = 43
// $arr[6] = "car", $arr[12] = true
?>
```

Για να αλλάξεις ένα στοιχείο απλώς του βάζεις στο συγκεκριμένο δείκτη (κλειδί) μια άλλη τιμή. Για να σβήσεις ένα στοιχείο χρησιμοποιείς τη συνάρτηση **unset()** με όρισμα το στοιχείο. Επίσης υπάρχουν και μία σειρά συναρτήσεις που είναι χρήσιμες για πίνακες όπως οι παρακάτω:

Συνάρτηση

array_values()

Χρήση

array_values (array \$array) : array

Επιστρέφει έναν πίνακα με αριθμητικούς δείκτες και τιμές τις τιμές του πίνακα.

array_keys()

array_keys (array \$array) : array

array_keys (array \$array , mixed \$search_value [, bool \$strict = FALSE]) : array

Επιστρέφει έναν πίνακα με αριθμητικούς δείκτες και τιμές τα κλειδιά του πίνακα. Αν χρησιμοποιηθεί η δεύτερη μορφή με τη `$search_value` επιστρέφει μόνο τα κλειδιά με αυτή την τιμή.

array_key_exists()

array_key_exists ([mixed](#) \$key , array \$array) : bool

Επιστρέφει TRUE αν υπάρχει το συγκεκριμένο κλειδί στον πίνακα.

asort()

asort (array &\$array [, int \$sort_flags = SORT_REGULAR]) : bool

arsort()

Ταξινομεί τον πίνακα με βάση τις τιμές διατηρώντας την αντιστοίχιση κλειδιών και τιμών.

arsort (array &\$array [, int \$sort_flags = SORT_REGULAR]) : bool

Ταξινομεί αντίστροφα τον πίνακα με βάση τις τιμές διατηρώντας την αντιστοίχιση κλειδιών και τιμών.

print_r()

print_r ([mixed](#) \$expression [, bool \$return = FALSE]) : [mixed](#)

Τυπώνει τη μεταβλητή \$expression με τρόπο ώστε να είναι αναγνώσιμη από έναν άνθρωπο. Εξυπηρετεί για να τυπωθούν τα περιεχόμενα ενός πίνακα για έλεγχο.

Χρήσιμη για τη διαχείριση πινάκων είναι η δομή **foreach** (βλέπε ενότητα «Δομές Ελέγχου» παρακάτω).

5.5.1.3 Εμβέλεια Μεταβλητών

Στην PHP η εμβέλεια μιας μεταβλητής αναφέρεται στο περιεχόμενο στο οποίο ορίζεται και μπορεί να χρησιμοποιηθεί (είναι ορατή). Αν μια μεταβλητή χρησιμοποιηθεί μέσα σε ένα αρχείο είναι διαθέσιμη σε όλο το αρχείο αυτό καθώς και σε όλα τα αρχεία που αυτό κάνει include. Μία μεταβλητή που ορίζεται μέσα σε μια συνάρτηση είναι τοπική σε αυτή τη συνάρτηση μόνο. Επίσης για να αναφερθούμε μέσα σε μια συνάρτηση, σε μια μεταβλητή που έχει οριστεί έξω από αυτή, πρέπει να χρησιμοποιήσουμε τη λέξη **global**. Παραδείγματα:

```
<?php
```

```
$a = 10; // καθολική εμβέλεια
```

```
include "b.inc"; // εισάγω στο σημείο αυτό τα περιεχόμενα του αρχείου b.inc
```

```
function Test()
```

```
{
```

```
    echo $a; // αναφέρεται σε τοπική μεταβλητή (η οποία δεν έχει δηλωθεί)
```

```
}
```

```
echo $a; // τυπώνει την τιμή της καθολικής μεταβλητής $a
```

```
Test(); // βγάζει λάθος για μη ορισμένη μεταβλητή
```

```
?>
```

Εδώ η μεταβλητή \$a είναι γνωστή σε όλο το αρχείο καθώς και στο αρχείο «b.inc», αλλά μέσα στη συνάρτηση για να τη χρησιμοποιήσουμε θα έπρεπε να χρησιμοποιήσουμε τη λέξη global. Έτσι στο συγκεκριμένο παράδειγμα θα τυπωθεί από την προτελευταία εντολή (την echo) η τιμή 10 αλλά από την κλήση της Test() θα τυπωθεί μήνυμα λάθους (εφόσον έχουμε ενεργοποιημένα τα λάθη) γιατί η \$a μέσα στη συνάρτηση θεωρείται ότι είναι μια άλλη local μεταβλητή η οποία μάλιστα δεν έχει δηλωθεί. Για να αναφερθούμε στην έξω \$a θα έπρεπε η συνάρτηση να γραφεί:

```
function Test()
```

```
{
```

```
    global $a;
```

```
    echo $a; // αναφέρεται στην καθολική μεταβλητή $a
```

```
}
```

Ένας άλλος τρόπος για να χρησιμοποιήσουμε global μεταβλητές είναι να χρησιμοποιήσουμε τον ειδικά ορισμένο από την PHP πίνακα **\$GLOBALS**, ο οποίος περιέχει όλες τις global μεταβλητές και μπορούμε να αναφερθούμε στην κάθε μία χρησιμοποιώντας το όνομά της σαν δείκτη του πίνακα. Έτσι θα μπορούσαμε να γράψουμε:

```
function Test()  
{  
    echo $GLOBALS["a"]; // αναφέρεται στην καθολική μεταβλητή $a  
}
```

Μάλιστα ο συγκεκριμένος πίνακας **\$GLOBALS** είναι διαθέσιμος παντού, δηλαδή είναι ένας superglobal (υπεκαθολικός) πίνακας (βλέπε παρακάτω).

5.5.2 Σταθερές

Μια σταθερά είναι απλώς ένα όνομα που αντιστοιχεί σε μια απλή τιμή και δεν μπορεί να αλλάξει. Όπως φαίνεται και από το ότι λέγεται σταθερά, αυτή η τιμή δεν μπορεί να αλλάξει κατά την εκτέλεση του script. Για το όνομά των σταθερών ακολουθούνται οι κανόνες των μεταβλητών.

Όπως οι superglobals, η εμβέλεια (scope) μιας σταθεράς είναι global. Μπορείτε να έχετε πρόσβαση σ' αυτή από οποιοδήποτε μέρος στο script χωρίς να λαμβάνετε υπόψη την εμβέλεια.

Για να οριστεί μια σταθερά μπορεί να χρησιμοποιηθεί η συνάρτηση **define()**. Για να χρησιμοποιήσουμε την τιμή μιας σταθεράς αρκεί να γράψουμε το όνομά της (χωρίς \$ μπροστά !!) ή να χρησιμοποιήσουμε τη συνάρτηση **constant()**. Πχ.

```
<?php  
define("HI", "Hello world.");  
echo HI; // τυπώνει "Hello world."  
echo constant("HI"); // τυπώνει "Hello world."  
echo Hi; // τυπώνει "Hi" αλλά βγάζει ειδοποίηση λάθους χρήσης  
?>
```

5.6 Τελεστές και Εκφράσεις

5.6.1 Τελεστές

Οι τελεστές μοιάζουν με τους τελεστές της C.

5.6.1.1 Τελεστής Ανάθεσης

Ο βασικός τελεστής ανάθεσης είναι ο **=**. Αυτός ο τελεστής δεν σημαίνει «ισούται με» αλλά «ανατίθεται σε». Η τιμή μιας έκφρασης ανάθεσης είναι η τιμή που της ανατίθεται. Δηλαδή, η τιμή του **\$a = 3** είναι 3. Αυτό σας επιτρέπει να κάνετε μερικά περίπλοκα πράγματα:

```
$a = ($b = 4) + 5; // το $a είναι ίσο με 9 και το $b ίσο πήρε την τιμή 4.
```

Πέρα από το βασικό τελεστή ανάθεσης, υπάρχουν "σύνθετοι τελεστές" για όλους τους δυαδικούς αριθμητικούς και αλφαριθμητικούς τελεστές που σας επιτρέπουν να χρησιμοποιήσετε μια έκφραση και στη συνέχεια να θέσετε την τιμή στο αποτέλεσμα της έκφρασης. Πχ:

```
$a = 3;  
$a += 5; // sets $a to 8, as if we had said: $a = $a + 5;  
$b = "Hello ";  
$b .= "There!"; // θέτει το $b ίσο με "Hello There!", όμοια με  
                //το να εκτελούσα την εντολή $b = $b . "There!";
```


Φυσικά όπως ήδη αναφέραμε σε προηγούμενη ενότητα («Μεταβλητές») υπάρχει και η ανάθεση με αναφορά.

5.6.1.2 Αριθμητικοί Τελεστές

Παράδειγμα	Όνομα	Αποτέλεσμα
$\$a + \b	Πρόσθεση	Αποτέλεσμα του $\$a$ και του $\$b$.
$\$a - \b	Αφαίρεση	Διαφορά του $\$a$ και του $\$b$.
$\$a * \b	Πολλαπλασιασμός	Γινόμενο του $\$a$ και του $\$b$.
$\$a / \b	Διαίρεση	Αποτέλεσμα του $\$a$ και του $\$b$.
$\$a \% \b	Modulus	Υπόλοιπο του $\$a$ διαιρεμένου από το $\$b$.

Πίνακας 2: Αριθμητικοί τελεστές στην PHP

5.6.1.3 Λογικοί Τελεστές

Παράδειγμα	Όνομα	Αποτέλεσμα
$\$a \& \b	And	Τα bits που είναι 1 και στο $\$a$ και στο $\$b$ ενεργοποιούνται (1), ενώ αν κάποιο είναι 0 έστω και σε ένα από τα $\$a$, $\$b$ απενεργοποιείται (0).
$\$a \b	Or	Τα bits που είναι 1 είτε στο $\$a$ είτε στο $\$b$, ενεργοποιούνται (1).
$\$a \wedge \b	Xor	Τα bits που είναι 1 είτε στο $\$a$ είτε στο $\$b$, αλλά όχι και στα δύο συγχρόνως, ενεργοποιούνται (1).
$\sim \$a$	Not	Τα bits που δεν είναι ενεργοποιημένα στο $\$a$, ενεργοποιούνται (1) και όσα είναι ενεργοποιημένα απενεργοποιούνται (0).
$\$a << \b	Shift left	Μετακίνηση των bits του $\$a$ κατά $\$b$ βήματα προς τα αριστερά (κάθε βήμα σημαίνει "πολλαπλασιασμός επί δύο")
$\$a >> \b	Shift right	Μετακίνηση των bits του $\$a$ κατά $\$b$ βήματα προς τα δεξιά (κάθε βήμα σημαίνει "ακέραια διαίρεση δια δύο")

Πίνακας 3: Λογικοί τελεστές στην PHP

5.6.1.4 Συγκριτικοί Τελεστές

Παράδειγμα	Όνομα	Αποτέλεσμα
$\$a == \b	Ισότητα	TRUE αν το $\$a$ είναι ίσο με το $\$b$.
$\$a === \b	Ομοιότητα	TRUE αν το $\$a$ είναι ίσο με το $\$b$, και είναι επιπλέον του ίδιου τύπου.
$\$a != \b	Ανισα	TRUE αν το $\$a$ δεν είναι ίσο με το $\$b$.
$\$a <> \b	Όχι ίσα	TRUE αν το $\$a$ δεν είναι ίσο με το $\$b$.
$\$a !== \b	Ανόμοια	TRUE αν το $\$a$ δεν είναι ίσο με το $\$b$ ή αν δεν είναι του ίδιου τύπου.
$\$a < \b	Μικρότερο από	TRUE αν το $\$a$ είναι ακριβώς μικρότερο από το $\$b$.
$\$a > \b	Μεγαλύτερο από	TRUE αν το $\$a$ είναι αυστηρά μεγαλύτερο από το $\$b$.
$\$a <= \b	Μικρότερο από ή ίσο με	TRUE αν το $\$a$ είναι μικρότερο από ή ίσο με το $\$b$.
$\$a >= \b	Μεγαλύτερο από ή ίσο με	TRUE αν το $\$a$ είναι μεγαλύτερο από ή ίσο με το $\$b$.

Πίνακας 4: Τελεστές σύγκρισης στην PHP

Σε ορισμένες συγκρίσεις γίνεται πρώτα μετατροπή τύπων όπως στην "1"==1. Όμως σε όσες έχουν και σύγκριση τύπου π.χ. "1"===1 τότε συγκρίνονται και με τον τύπο τους. Οι κανόνες αυτοί ισχύουν στις δομές ελέγχου και επανάληψης που θα δούμε παρακάτω. Γενικά πρέπει να προσέχουμε τον τύπο των συγκρινόμενων γιατί μπορούν να δημιουργηθούν διάφορα προβλήματα (π.χ. "hello"==0 είναι αληθής !!!).

5.6.1.5 Τελεστής Εκτέλεσης

Η PHP παρέχει τα **ανάποδα μονά αυτάκια (backticks)** `` τα οποία παίζουν τον ρόλο του τελεστή εκτέλεσης, δηλαδή ότι βάλουμε μέσα θα προσπαθήσει να το εκτελέσει σαν εντολή φλοιού. Πχ.:

```
$output = `ls -al`; // ή $output = `dir` σε windows
echo "<pre>$output</pre>";
```

Παρόμοια αποτέλεσμα μπορούμε να έχουμε με τη συνάρτηση `shell_exec()`.

5.6.1.6 Τελεστές Αύξησης-Μείωσης

Παράδειγμα	Όνομα	Αποτέλεσμα
++\$a	Προ-αύξηση	Αυξάνει το \$a κατά ένα και επιστρέφει την τιμή του \$a.
\$a++	Μετά-αύξηση	Επιστρέφει την τιμή του \$a και μετά το αυξάνει κατά ένα.
--\$a	Προ-μείωση	Μειώνει την τιμή του \$a κατά 1 και μετά επιστρέφει το \$a.
\$a--	Μετά-μείωση	Επιστρέφει την τιμή του \$a και μετά το μειώνει κατά 1.

Πίνακας 5: Τελεστές Αύξησης-Μείωσης στην PHP

5.6.1.7 Λογικοί Τελεστές

Παράδειγμα	Όνομα	Αποτέλεσμα
\$a and \$b	And	TRUE αν και το \$a και το \$b είναι TRUE .
\$a or \$b	Or	TRUE αν είτε το \$a είτε το \$b είναι TRUE .
\$a xor \$b	Xor	TRUE αν είτε το \$a είτε το \$b είναι TRUE , αλλά όχι και τα δυο.
! \$a	Not	TRUE αν το \$a δεν είναι TRUE .
\$a && \$b	And	TRUE αν και το \$a και το \$b είναι TRUE .
\$a \$b	Or	TRUE αν είτε το \$a είτε το \$b είναι TRUE .

Πίνακας 6: Λογικοί τελεστές στην PHP

5.6.1.8 Τελεστές για Strings

Υπάρχουν δύο τελεστές για strings (αλφαριθμητικών). Ο πρώτος είναι η τελεία (.) και αποτελεί τον τελεστή συνένωσης, ο οποίος επιστρέφει τη συνένωση των τιμών των αριστερών και των δεξιών παραμέτρων. Ο δεύτερος είναι το τελεία-ίσον (.=) και αποτελεί τον τελεστή ανάθεσης και συνένωσης, ο οποίος συνενώνει την τιμή της παραμέτρου της δεξιάς πλευράς στην τιμή της παραμέτρου (που είναι μια μεταβλητή) της αριστερής πλευράς και την αναθέτει στην παράμετρο της αριστερής πλευράς. Πχ.:

```
$a = "Hello ";
$b = $a . "World!"; // το $b έχει την τιμή Hello World!

$a = "Hello ";
$a .= "World!";      // το $a έχει την τιμή Hello World!
```

Αν θέλω σε ένα string μέσα να βάλω κάποιον ειδικό χαρακτήρα (πχ ") χρησιμοποιώ τον χαρακτήρα διαφυγής \. Πχ

```
$a = "Hello \"World\""; // το $a έχει την τιμή Hello "World!"
```

5.6.1.9 Τελεστές Πινάκων

Ο μόνος τελεστής πίνακα στην PHP είναι ο + τελεστής. Αυτός προσθέτει τη δεξιά μεριά του πίνακα στην αριστερή μεριά, ενώ τα διπλά κλειδιά ΔΕΝ γίνονται overwrite (δεν γράφονται από πάνω). Πχ.:

```
$a = array("a" => "apple", "b" => "banana");
$b = array("a" => "pear", "b" => "strawberry", "c" => "cherry");
$c = $a + $b;
```

Το αποτέλεσμα θα είναι ο πίνακας \$c να έχει τρία στοιχεία, το a και b του πίνακα \$a και το c του πίνακα \$b. Διατηρούνται οι τιμές για το a και b από τον πίνακα \$a γιατί ο συγκεκριμένος τελεστής δεν κάνει override τα κοινά κλειδιά.

5.6.1.10 Προτεραιότητα Τελεστών

Ο παρακάτω πίνακας έχει τους τελεστές και την προτεραιότητα που έχουν. Γενικά προτείνεται να χρησιμοποιούμε παρενθέσεις για να διασφαλίζουμε ρητά την προτεραιότητα που επιθυμούμε:

Σχετικότητα	Τελεστές
αριστερή	,
αριστερή	or
αριστερή	xor
αριστερή	and
δεξιά	print
αριστερή	= += -= *= /= .= %= &= = ^= <<= >>=
αριστερή	? :
αριστερή	
αριστερή	&&
αριστερή	
αριστερή	^
αριστερή	&
Χωρίς σύνδεση	== != === !==
Χωρίς σύνδεση	< <= > >=
Αριστερή	<< >>
Αριστερή	+ - .
Αριστερή	* / %
Δεξιά	! ~ ++ -- (int) (float) (string) (array) (object) @
Δεξιά	[
Χωρίς σύνδεση	new

Πίνακας 7: Προτεραιότητα Τελεστών στην PHP

5.6.2 Εκφράσεις

Οι εκφράσεις (expressions) είναι το πιο σημαντικό κομμάτι της PHP. Στην PHP, σχεδόν όλα όσα γράφετε είναι εκφράσεις. Ο απλούστερος και συγχρόνως ο πιο ακριβής τρόπος για να ορίσουμε μια έκφραση είναι "οτιδήποτε έχει τιμή".

5.7 Δομές Ελέγχου

Οι δομές ελέγχου που υποστηρίζονται από την γλώσσα παρουσιάζονται στις επόμενες υποενότητες.

5.7.1 If – If-else - If-elseif-else

Η if στην PHP είναι ίδια με την C, δηλαδή έχει τους παρακάτω τρόπους σύνταξης, ανάλογα με την χρήση που θέλεις να κάνεις:

If	If με else	If με elseif και else
<pre>if (expr) statement</pre>	<pre>if (expr) { statements } else { statements }</pre>	<pre>if (expr) { statements } elseif (expr) { statements } // όσα elseif θέλω else { statements }</pre>

Παραδείγματα

<pre><?php if (\$a==5) { print "ίσο με 5"; \$a = \$a + 1 print "τώρα με 6"; } ?></pre>	<pre><?php if (\$a==5) { print "ίσο με 5"; } else { print "όχι ίσο με 5"; } ?></pre>	<pre><?php if (\$a > 5) { print "μεγαλύτερο από 5"; } elseif (\$a == 5) { print "ίσο με 5"; } else { print "μικρότερο του 5"; } ?></pre>
--	--	---

5.7.2 switch

Η switch είναι και αυτή όπως στη C:

```
switch (expr) {
    case value1:
        statements
        break;
    case value2:
        statements
        break;
```

```

    . . .
    default:
        statements
        break;
}

```

Το τμήμα default είναι προαιρετικό και αν το βάλουμε εκτελείται μόνο αν δεν ισχύει κανένα case. Φυσικά η χρήση break είναι σημαντική γιατί αλλιώς αν εκτελεστεί ένα case Που δεν έχει break θα εκτελεστεί και το από κάτω του κ.ο.κ. μέχρι να βρεθεί το πρώτο break.

Παράδειγμα:

```

switch ($x) {
    case 0:
        print "είναι ίσο με 0";
        break;
    case 1:
        print "είναι ίσο με 1";
        break;
    case 2:
        print "είναι ίσο με 2";
        break;
}

```

5.7.3 while

Η while στην PHP είναι και αυτή ίδια με την C, δηλαδή συντάσσεται ως εξής:

```

while (expr)
    statement
while (expr)
{
    statements
}

```

Με μία μόνο εντολή μέσα της δεν απαιτεί να βάλω { }

Βάζοντας { } μπορούμε να βάλουμε μία ή και περισσότερες εντολές.

Παράδειγμα

```

$i = 1;
while ($i <= 10) {
    print $i;
    $i++;
}

```

5.7.4 do..while

Η do ... while στην PHP είναι και αυτή ίδια με την C. Η διαφορά της με τη while είναι ότι ο βρόγχος εκτελείται τουλάχιστον μία φορά αφού η «όσο (συνθήκη)» εξετάζεται μετά το τέλος της εκτέλεσης του βρόγχου οπότε και λάθος να είναι θα εκτελεστεί τουλάχιστον μία φορά. Συντάσσεται ως εξής:

```

do {
    statements
} while (expr);

```

Παράδειγμα

```

$i = 1;
do {
    print $i;
} while ($i <= 10);

```

```

    $i++;
} while ($i <= 10);

```

5.7.5 for

Η for στην PHP είναι και αυτή ίδια με την C, δηλαδή συντάσσεται ως εξής:

```

for (expr1; expr2; expr3) {
    statements
}

```

Παράδειγμα

```

for ($i = 1; $i <= 10; $i++) {
    print $i;
}

```

5.7.6 foreach

Στην PHP παρέχεται η δομή foreach όπως και στη γλώσσα Perl, η οποία μας επιτρέπει να προσπελάζουμε εύκολα πίνακες. Φυσικά δεν μπορεί να χρησιμοποιηθεί με απλές μεταβλητές αλλά μόνο με πίνακες. Η σύνταξή έχει τις εξής δύο μορφές:

```

foreach (array_expression as $value) statement

```

Στη μεταβλητή \$value μπαίνουν μία μία η τιμή του πίνακα και εκτελείται το σώμα της foreach.

```

foreach (array_expression as $key => $value) statement

```

Στη μεταβλητή \$key και \$value μπαίνουν μία μία το κλειδί και η τιμή του πίνακα και εκτελείται το σώμα της foreach.

Παράδειγμα

```

$arr = array("one", "two", "three");

// Τυπώνω τις τιμές
foreach ($arr as $value) {
    echo "Value: $value<br />\n";
}

//Αλλάζω τις τιμές
foreach ($arr as $k => $value) {
    $arr[$k] = "number " . $value;
}

```

5.7.7 break και continue

Η **break** χρησιμοποιείται για τον τερματισμό της εκτέλεσης της τρέχουσας εντολής for, foreach while, do...while καθώς και της δομής switch. Μάλιστα δέχεται και ένα προαιρετικό αριθμό (όρισμα) αμέσως μετά το λεκτικό break που καθορίζει πόσες εμφωλευμένες δομές θα τερματιστούν.

Η **continue** χρησιμοποιείται σε δομές επανάληψης για να παρακαμφτεί το υπόλοιπο της τρέχουσας επανάληψης και να συνεχίσει από την αρχή της επόμενης. Μάλιστα δέχεται και αυτή ένα προαιρετικό αριθμό (όρισμα) αμέσως μετά το λεκτικό continue που καθορίζει πόσα επίπεδα εμφωλευμένων επαναλήψεων θα παρακαμφτούν.

Παράδειγμα break

```

$i = 0;
while (++$i) {
    switch ($i) {
        case 5:
            echo "At 5<br />\n";

```

Παράδειγμα Continue

```

$i = 0;
while ($i++ < 5) {
    echo "Εξωτερική !<br />\n";
    while (1) {
        echo "&nbsp;Μεσαία!<br />\n";

```



```

        break 1; //Έξοδος από switch
    case 10:
        echo "At 10; quitting<br />\n";
        break 2; //Έξοδος από switch
και while
    default:
        break;
}
}

while (1) {
    echo "&nbsp;Εσωτερική!<br />\n";
    continue 3;
}
echo "Δεν τυπώνεται ποτέ !<br />\n";
}
echo "Δεν τυπώνεται ποτέ !<br />\n";
}

```

5.8 require(), require_once() και include_once(), include()

Η PHP είναι μία γλώσσα που ευνοεί τη χρήση συναρτήσεων για να φτιάχνεις επαναχρησιμοποιούμενο κώδικα, να γλιτώνεις επανάληψη του κώδικα και να μπορείς να συντηρείς το πρόγραμμά σου ευκολότερα. Για αυτό είναι προτιμότερο να γράφεις τις συναρτήσεις που θα χρησιμοποιείς σε ένα ή περισσότερα ξεχωριστά αρχεία και όχι στο αρχείο της ιστοσελίδας. Όποτε χρειάζεσαι να χρησιμοποιήσεις κάποιες συναρτήσεις αρκεί να «ενσωματώσεις» το αρχείο με τις συναρτήσεις στο αρχείο της ιστοσελίδας σου (στον κώδικα που έχεις στην ιστοσελίδα) και απλώς να καλέσεις τις συναρτήσεις που θέλεις. Για να γίνει η «ενσωμάτωση» χρησιμοποιείς τις συναρτήσεις **require()** και **include()**. Παράδειγμα:

```

<?php
// μπαίνει όλος ο κώδικας που υπάρχει στο αρχείο math_functions.php
require 'math_functions.php';
// μπαίνει όλος ο κώδικας που υπάρχει στο αρχείο του οποίου το όνομα έχει
//τιμή η μεταβλητή $afile
include $afile;

// μπαίνει όλος ο κώδικας που υπάρχει στο αρχείο functions.php
include('functions.php');
?>

```

Οι συναρτήσεις **require_once()** και **include_once()** είναι παρόμοιες με τη διαφορά ότι αν έχει ήδη ενσωματωθεί το αρχείο τότε δεν θα ξαναενσωματωθεί.

5.9 Συναρτήσεις

Η PHP, όπως ήδη αναφέρθηκε, ευνοεί τη χρήση συναρτήσεων για να μπορεί ο κώδικάς μας να έχει όλα τα πλεονεκτήματα που η χρήση τους προσφέρει. Έτσι παρέχεται η δυνατότητα για να γράψει ο χρήστης τις δικές του συναρτήσεις αλλά παρέχεται και ένα μεγάλο σύνολο έτοιμων (ενσωματωμένων) συναρτήσεων μέσω των οποίων γίνονται οι πιο συνηθισμένες λειτουργίες.

5.9.1 Συναρτήσεις οριζόμενες από τον χρήστη

Η σύνταξη για την ορισμό μιας συνάρτησης από τον χρήστη είναι η εξής:

```

function όνομα (παράμετροι)
{
    statements
}

```

Μία συνάρτηση πρέπει να οριστεί πριν χρησιμοποιηθεί ακριβώς όπως και οι μεταβλητές. Μέσα σε μια συνάρτηση μπορούν να μουν ότι εντολές υπάρχουν και σε ένα πρόγραμμα της PHP. Μάλιστα μπορούμε μέχρι και να ορίσουμε συναρτήσεις μέσα σε μια συνάρτηση. Φυσικά οι εσωτερικές, στη συνάρτηση, συναρτήσεις θα υπάρξουν μόνο όταν κληθεί η εξωτερική.

Επειδή στις περισσότερες περιπτώσεις σκοπός της συνάρτησης είναι η επιστροφή κάποιας τιμής, τότε πρέπει μία από της εντολές να είναι η:

return τιμή;

Μόλις εκτελεστεί η συγκεκριμένη εντολή, τερματίζεται η συνάρτηση και επιστρέφεται η συγκεκριμένη τιμή, που ακολουθεί τη λέξη return, στο σημείο που κλήθηκε η συνάρτηση. Η τιμή αυτή μπορεί να είναι σταθερά ή μεταβλητή οποιουδήποτε τύπου (ακόμα και πίνακας, αντικείμενο κλπ).

Οι παράμετροι που ακολουθούν το όνομα μιας συνάρτησης χρησιμεύουν για να περάσουμε τιμές κατά την κλήση και είναι μια λίστα μεταβλητών ή/και σταθερών που διαχωρίζονται με κόμματα. Για να περάσουμε μεταβλητού αριθμού παραμέτρους γίνεται χρησιμοποιώντας τις συναρτήσεις **func_num_args()**, **func_get_arg()**, και **func_get_args()**.

Το όνομα μιας συνάρτησης είναι όπως ακριβώς το όνομα μιας μεταβλητής αλλά είναι case-insensitive. Η PHP υποστηρίζει φυσικά και αναδρομή.

Πολλές φορές στις συναρτήσεις χρησιμοποιούμε **static** μεταβλητές για να μπορούμε να διατηρούμε τιμές μεταξύ των κλήσεων.

Παραδείγματα:

```
<?php
//Παράδειγμα με ακέραιο
function print_upto($i)
{
    for ($i = 1; $i <= 10; $i++)
        print $i;
}
//Παράδειγμα με πέρασμα παραμέτρου με αναφορά
function concatVer1(&$str1, $str2)
{
    $str1 .= $str2;
}
function concatVer2($str1, $str2)
{
    return $str1 . $str2;
}

//Έχουν το ίδιο αποτέλεσμα
$a = 'Hello ';
$b = 'World !!';
concatVer1($a, $b);

$a = 'Hello ';
$b = 'World !!';
$a = concatVer2($a, $b);

//Παράδειγμα με πίνακα
$c = array("one", "two", "three");
function printArray($inarr)
{
    foreach ($inarr as $value) {
        echo "Value: $value<br>\n";
    }
}

//Παράδειγμα με προκαθορισμένη τιμή εφόσον κληθεί χωρίς να δοθεί όρισμα
function HelloWithDefault($name = 'χωρίς όνομα')
{
```

```

    return "Hello $name.\n";
}
echo HelloWorldWithDefault ();
echo HelloWorldWithDefault ("Νίκος");

//Παράδειγμα με μεταβλητή συνάρτηση
$x = 'HelloWithDefault';
echo $x(); //θα κληθεί η HelloWorldWithDefault ();

//Παράδειγμα με στατική μεταβλητή σε συνάρτηση
function funcstat($setval = false)
{
    static $a;

    if ($setval)
        $a = 1;
    else
        $a += 1;
    return $a;
}
$initializeVal = TRUE;
funcstat($initializeVal);
funcstat();
funcstat();
echo funcstat(); //τυπώνει 4
?>

```

Όπως αναφέρθηκε και σε προηγούμενη ενότητα, αν η συνάρτηση πρόκειται να χρησιμοποιηθεί σε περισσότερα από ένα αρχεία-ιστοσελίδες, τότε θα ήταν καλύτερο να ορίζεται σε ξεχωριστό αρχείο που θα ενσωματώνεται στον κώδικα της ιστοσελίδας όποτε χρειάζεται κάποια από τις συναρτήσεις που περιέχει.

5.9.2 Ενσωματωμένες Συναρτήσεις

Όπως αναφέρθηκε υπάρχει ένα μεγάλο σύνολο ενσωματωμένων συναρτήσεων από τις οποίες κάποιες ήδη αναφέρθηκαν στις προηγούμενες παραγράφους. Ορισμένες από τις συχνότερα χρησιμοποιούμενες, μαζί με τις συναρτήσεις που παρέχονται από το MySQL Module, παρατίθενται σε επόμενη υποενότητα («**Παραπομπή Συναρτήσεων**»).

Η αναλυτική παρουσίασή όλων των συναρτήσεων (ενσωματωμένων και από τα διάφορα modules) είναι πέρα από τους σκοπούς αυτού του εγχειριδίου.

5.10 Κλάσεις και Αντικείμενα

Η PHP είναι μια αντικειμενοστραφής (object-oriented) γλώσσα και υποστηρίζει τη δημιουργία αντικειμένων μέσω του γλωσσικού **class** (κλάση). Η κλάση χρησιμοποιείται για τον ορισμό ενός αντικειμένου που έχει κάποιες ιδιότητες-χαρακτηριστικά (είναι η συλλογή μεταβλητών της κλάσης) και συμπεριφορά-μεθόδους χειρισμού (είναι οι συναρτήσεις που ορίζονται στην κλάση). Αφού ορίσεις το αντικείμενο (γράφοντας την κλάση), για να δημιουργήσεις ένα νέο αντικείμενο αυτής της κλάσης (ένα νέο instance) χρησιμοποιείς το **new**.

```

Παράδειγμα
<?php
class Cart
{

```

```
var $items; // πίνακας για να τα αντικείμενα του καλαθιού αγοράς
var $customerid; // ένα id για τον πελάτη

//κατασκευαστής ο οποίος καλείται κατά τη δημιουργία
//κάθε αντικειμένου Cart. Εδώ κάνουμε αρχικοποιήσεις
public function __construct($cid) {
    $this->customerid = $cid;
    $this->items = array();
}

// Πρόσθεσε $num αντικείμενα τύπου $antik στο καλάθι
public function addItem ($antik, $num)
{
    //αν υπάρχει στον πίνακα αύξησέ το
    if (array_key_exists($antik, $this->items))
        $this->items[$antik] += $num;
    else //αν δεν υπάρχει βάλτο και θέσε του την τιμή
        $this->items[$antik] = $num;
}

// Αφαίρεσε $num αντικείμενα τύπου $antik από το καλάθι
public function removeItem ($antik, $num)
{
    //Αν υπάρχει το αντικείμενο
    if (array_key_exists($antik, $this->items)) {
        //Αν υπάρχει τόση ποσότητα
        if ($this->items[$antik] >= $num) {
            $this->items[$antik] -= $num;
            //Αν μηδενίστηκε αφαίρεσέ το από τον πίνακα
            if ($this->items[$antik]==0)
                unset($this->items[$antik]);
            return true;
        } else { //Αν δεν υπάρχει τόση ποσότητα
            return false;
        }
    }
    else
        return false;
}

// Αφαίρεσε $num αντικείμενα τύπου $antik από το καλάθι
public function printItems()
{
    echo "<table><tr><th>Προϊόν</th><th>Συνολική Ποσότητα</th></tr>";
    foreach ($this->items as $k => $v) {
        echo "<tr><td>$k</td><td>$v</td></tr>";
    }
    echo "</table>";
}

}

$c = new Cart(123);
$c->addItem("lemonia", 1);
$c->addItem("portokalia", 2);
$c->addItem("lemonia", 2);
$c->printItems();
```

?>

Φυσικά υποστηρίζεται και κληρονομικότητα με τη χρήση της λέξης **extends**.

Παράδειγμα:

```
<?php
class VisitorCart extends Cart
{
    var $customername;

    //Νέος κατασκευαστής
    public function __construct($cid, $cname) {
        $this->customerid = $cid;
        $this->customername = $cname;
        $this->items = array();
    }

    //Μέθοδος για αλλαγή του ονόματος του πελάτη
    public function setName($cname)
    {
        $this->customername = $cname;
    }
    //Μέθοδος για να πάρουμε το όνομα του πελάτη
    public function getName()
    {
        return $this->customername;
    }
}

$vc = new VisitorCart(222, "Νίκος");
$vc->addItem("patates", 2);
echo "<p>Όνομα: " . $vc->getName() . "</p>";
$vc->printItems();
```

?>

Επίσης υποστηρίζονται και πολλά άλλα χαρακτηριστικά που έχουν οι αντικειμενοστραφείς γλώσσες, τα οποία όμως ξεφεύγουν από τους στόχους του εγχειριδίου αυτού. Για περισσότερες πληροφορίες δείτε το εγχειρίδιο στον σύνδεσμο <https://www.php.net/manual/en/language.oop5.php>.

5.11 Παραπομπή Συναρτήσεων

Οι ενσωματωμένες συναρτήσεις που παρέχει η PHP είναι πάρα πολλές. Στην επόμενη υποενότητα παρουσιάζονται επιλεκτικά μερικές που χρησιμοποιούνται πολύ συχνά. Αμέσως μετά παρουσιάζονται συνοπτικά μόνο αυτές που αφορούν τη σύνδεση PHP με MySQL και σε επόμενη ενότητα («Μεθοδολογία Σύνδεσης PHP-MySQL») θα παρουσιαστεί η χρήση των πιο σημαντικών από αυτές με παραδείγματα.

5.11.1 Συχνά Χρησιμοποιούμενες Ενσωματωμένες Συναρτήσεις

Μερικές από τις πιο συχνά χρησιμοποιούμενες συναρτήσεις, οι οποίες δεν έχουν ήδη αναφερθεί στις προηγούμενες ενότητες, παρουσιάζονται στον επόμενο πίνακα. Το εγχειρίδιο αυτό δεν σκοπεύει να παρουσιάσει όλες τις συναρτήσεις ή να δώσει αναλυτική χρήση των σημαντικότερων, γι'αυτό προτείνεται ο χρήστης να μεταβεί στο εγχειρίδιο αναφοράς της PHP (<https://www.php.net/manual/en/funcref.php>) όπου θα βρεί πλήρη λίστα των συναρτήσεων καθώς και παραδείγματα.

Συνάρτηση header	Χρήση header (string \$header [, bool \$replace = TRUE [, int \$http_response_code]]) : void Αποστέλλουμε ακατέργαστους (raw) headers στον πελάτη. Είναι πολύ χρήσιμη για μια σειρά από εφαρμογές (ανακατεύθυνση, αποστολή αρχείου κλπ) και γι' αυτό παρουσιάζεται παρακάτω σε ξεχωριστή υποενότητα.
headers_sent	headers_sent ([string &\$file [, int &\$line]]) : bool Ελέγχουμε αν ήδη στείλαμε τους header για να αποφύγουμε τα λάθη σχετικά με την αποστολή headers ενώ ήδη έχουν σταλεί.
count	count (mixed \$array_or_countable [, int \$mode = COUNT_NORMAL]) : int Μετράει τα στοιχεία μιας μεταβλητής και πίνακα και object. Ομοίως και η sizeof.
min, max, sin, cos, tan, round, sqrt, pi, pow, log, exp, rand echo	Μαθηματικές συναρτήσεις echo (string \$arg1 [, string \$...]) : void Εμφανίζει ένα ή περισσότερα strings. Αν μέσα σε ένα string έχουμε μεταβλητές της PHP (δες παρακάτω για μεταβλητές) και χρησιμοποιηθούν μονά εισαγωγικά τότε δεν τυπώνεται η τιμή των μεταβλητών αλλά ότι υπάρχει μέσα στα μονά εισαγωγικά. Ενώ με τη χρήση διπλών εισαγωγικών, οι μεταβλητές είναι διακριτές μέσα στο string και τυπώνονται οι τιμές τους (και τα υπόλοιπα όπως είναι).
print	print (string \$arg) : int Εμφανίζει ένα string επιστρέφει 1. Και εδώ ισχύουν τα ίδια με την echo όσων αφορά τις παρενθέσεις και τη χρήση των μονών – διπλών εισαγωγικών.
printf	printf (string \$format [, mixed \$...]) : int Εμφανίζει ένα μορφοποιημένο string (όπως στη C)
sprintf	sprintf (string \$format [, mixed \$...]) : string Επιστρέφει ένα μορφοποιημένο string (όπως στη C)
fprintf	fprintf (resource \$handle , string \$format [, mixed \$...]) : int Τυπώνει σαν την printf αλλά στο resource (αρχείο).
sscanf	sscanf (string \$str , string \$format [, mixed &\$...]) : mixed Διαβάζει από ένα string με βάση το format
fscanf	fscanf (resource \$handle , string \$format [, mixed &\$...]) : mixed Διαβάζει από ένα resource (αρχείο) με βάση το format
flush	flush (void) : void Σπρώχνει στην έξοδο ότι υπάρχει στον buffer.
number_format	number_format (float \$number [, int \$decimals = 0]) : string number_format (float \$number , int \$decimals = 0 , string \$dec_point = ".", string \$thousands_sep = ",") : string Μορφοποιεί κατάλληλα έναν αριθμό.
count_chars	count_chars (string \$string [, int \$mode = 0]) : mixed Μετρά το πλήθος των εμφανίσεων κάθε χαρακτήρα και το επιστρέφει με ποικίλους τρόπους (ανάλογα την τιμή του mode)
convert_uuencode convert_uuencode	convert_uuencode (string \$data) : string Αποκωδικοποιεί ένα uuencoded string convert_uuencode (string \$data) : string Κωδικοποιεί ένα string σε uuencoded
trim	trim (string \$str [, string \$character_mask = "\t\n\r\0\x0B"]) : string

ltrim	Βγάζει τα κενά από ένα string
rtrim	ltrim (string \$str [, string \$character_mask]) : string Βγάζει τα κενά από τα αριστερά ενός string
	rtrim (string \$str [, string \$character_mask]) : string Βγάζει τα κενά από τα δεξιά ενός string
strchr, strstr, strlen, strcmp, strtok, strtolower, strtoupper, substr	Όπως και στη C χειρίζομαστε αλφαριθμητικά.
	substr (string \$string , int \$start [, int \$length]) : string Επιστρέφει ένα μέρος του string.
strrev	strrev (string \$string) : string Επιστρέφει το string αντεστραμμένο.
str_word_count	str_word_count (string \$string [, int \$format = 0 [, string \$charlist]]) : mixed Μετράει τις λέξεις που έχει ένα string και επιστρέφει τον αριθμό τους. Ανάλογα την τιμή του \$format επιστρέφει άλλες τιμές (και πίνακα)
wordwrap	wordwrap (string \$str [, int \$width = 75 [, string \$break = "\n" [, bool \$cut = FALSE]]]) : string Αναδιπλώνει ένα string μετά από έναν αριθμό χαρακτήρων.
empty	empty (mixed \$var) : bool Ελέγχει και επιστρέφει TRUE αν μια μεταβλητή είναι empty (είτε NULL είτε άδεια π.χ. για string "").
system	system (string \$command [, int &\$return_var]) : string Εκτελεί ένα εξωτερικό πρόγραμμα και επιστρέφει το αποτέλεσμα του.
parse_url	parse_url (string \$url [, int \$component = -1]) : mixed Αναλύει ένα URL και επιστρέφει τα τμήματά του σε πίνακα
ctype_alnum	Παίρνουν σαν όρισμα ένα string και ελέγχουν (επιστρέφουν TRUE ή FALSE) αν περιλαμβάνει μόνο το συγκεκριμένο τύπο χαρακτήρων. Πχ
ctype_alpha	ctype_digit (string \$text) : bool Επιστρέφει TRUE αν όλοι οι χαρακτήρες του ορίσματος είναι αριθμητικοί.
ctype_cntrl	ctype_graph (string \$text) : bool Επιστρέφει TRUE αν όλοι οι χαρακτήρες του ορίσματος είναι εκτυπώσιμοι, δηλαδή ορατοί (όχι το space).
ctype_digit	ctype_cntrl (string \$text) : bool Επιστρέφει TRUE αν όλοι οι χαρακτήρες του ορίσματος είναι control χαρακτήρες.
ctype_lower	date (string \$format [, int \$timestamp = time()]) : string Επιστρέφει μια ημερομηνία με βάση το format
ctype_upper	getdate ([int \$timestamp = time()]) : array Επιστρέφει έναν πίνακα που περιέχει πληροφορίες για την ημερομηνία του timestamp ή την τρέχουσα αν δεν δοθεί timestamp
ctype_print	time (void) : int Επιστρέφει το τρέχον timestamp (δευτερόλεπτα από 1/1/1970) GMT
ctype_space	localtime ([int \$timestamp = time() [, bool \$is_associative = FALSE]]) : array Επιστρέφει έναν πίνακα που περιέχει πληροφορίες για την τοπική ώρα του timestamp ή την τρέχουσα αν δεν δοθεί timestamp
ctype_punct	strtotime (string \$time [, int \$now = time()]) : int Μετατρέπει οποιοδήποτε string που περιέχει ημερομηνία ή ώρα με αγγλικό
ctype_xdigit	
date	

	κείμενο σε timestamp.
exit	exit ([string \$status]) : void exit (int \$status) : void Προκαλεί το τέλος του προγράμματος
die	die ([string \$status]) : void Όπως η exit
password_hash	password_hash (string \$password , int \$algo [, array \$options]) : string Δημιουργεί ένα νέο password hash με δικό του salt. Για να ελέγξουμε αν ο χρήστης έδωσε το σωστό password χρησιμοποιούμε τη password_verify() .
password_verify	password_verify (string \$password , string \$hash) : bool Εξετάζουμε αν το password που έδωσε ο χρήστης (όπως το πληκτρολόγησε) είναι σωστό-ταιριάζει με το hash που έχουμε αποθηκεύσει για τον χρήστη (χρησιμοποιώντας τη password_hash()).

Πίνακας 8: Συχνά χρησιμοποιούμενες συναρτήσεις της PHP

Επίσης υπάρχει μια πολύ σημαντική συνάρτηση, η **phpinfo()** που μπορεί να μας επιστρέψει ένα σύνολο από πληροφορίες (για το σύστημα στο οποίο εκτελείται η PHP, για τις ρυθμίσεις διαμόρφωσης της PHP, για τα φορτωμένα modules, για τις ρυθμίσεις του Web εξυπηρετητή, για τις επικεφαλίδες HTTP κλπ). Με την εκτέλεσή της τυπώνονται ένα σύνολο από πίνακες με πολλές πληροφορίες και είναι πολύ σημαντικό να τη χρησιμοποιούμε για να ελέγξουμε ότι όλες οι ρυθμίσεις έχουν γίνει όπως θέλουμε. Προτείνεται ένα από τα πρώτα παραδείγματα που θα κάνει ο αναγνώστης αφού εγκαταστήσει τα απαραίτητα (Web Εξυπηρετητή με υποστήριξη PHP) είναι να καλέσει τη συγκεκριμένη συνάρτηση. Το συντακτικό της είναι:

phpinfo ([int \$what = INFO_ALL]) : bool

και παίρνει σαν παράμετρο (προαιρετική) τις επόμενες σταθερές οι οποίες μάλιστα μπορεί να συνδυαστούν με bitwise OR ώστε να πάρουμε αποτελέσματα πολλών κατηγοριών:

Όνομα Σταθεράς	Τιμή	Περιγραφή
INFO_GENERAL	1	Την γραμμή διαμόρφωσης (κατά την κλήση), την τοποθεσία του php.ini, την ημερομηνία δημιουργίας του περιβάλλοντος, τον Web εξυπηρετητή, το σύστημα κλπ
INFO_CREDITS	2	PHP Credits (ομοίως και η phpcredits()).
INFO_CONFIGURATION	4	Τρέχουσες Local και Master τιμές για τις PHP directives (ομοίως και η ini_get()).
INFO_MODULES	8	Τα φορτωμένα modules και οι ρυθμίσεις τους (ομοίως και η get_loaded_modules()).
INFO_ENVIRONMENT	16	Πληροφορίες για τις μεταβλητές περιβάλλοντος (οι οποίες υπάρχουν και στην \$_ENV)
INFO_VARIABLES	32	Δείχνει όλες τις προ-ορισμένες/δεσμευμένες μεταβλητές (Environment , GET , POST , Cookie , Server) που θα δούμε παρακάτω.
INFO_LICENSE	64	PHP πληροφορίες άδειας χρήσης
INFO_ALL	-1	Δείχνει όλα τα παραπάνω και είναι το προκαθορισμένο όρισμα αν δεν δοθεί.

Παράδειγμα

```
<?php
```

```
// Δείχνει όλες τις πληροφορίες όπως και η phpinfo(INFO_ALL)
```

```
// και η phpinfo(-1)
phpinfo();
```

```
// Δείχνει μόνο τις πληροφορίες των module όπως και η phpinfo(8)
phpinfo(INFO_MODULES);
?>
```

5.11.2 Συναρτήσεις της επέκτασης MySQL improved

Στον επόμενο πίνακα φαίνονται μερικές από τις πιο σημαντικές συναρτήσεις της PHP (ένα από τα modules επέκτασης της γλώσσας – το βελτιωμένο-improved) για χρήση στην επικοινωνία με MySQL εξυπηρετητή. Φυσικά υπάρχει και η δυνατότητα χρήσης της κλάσης `mysqli` και `mysqli_result` και των μεθόδων τους ώστε να χρησιμοποιούμε τον αντικειμενοστραφή τρόπο προγραμματισμού αντί να χρησιμοποιήσουμε το δομημένο τρόπο με τη χρήση των συναρτήσεων. Γενικά προτείνεται στους αναγνώστες να μεταβούν όσο το δυνατό συντομότερα στον αντικειμενοστραφή τρόπο. Αναλυτικές πληροφορίες για το σύνολο των συναρτήσεων και για τις κλάσεις με τις μεθόδους και ιδιότητές τους θα βρείτε στο <https://www.php.net/manual/en/book.mysqli>.

Συνάρτηση	Χρήση
<code>mysqli_change_user</code>	<code>mysqli_change_user</code> (mysqli <code>\$link</code> , string <code>\$user</code> , string <code>\$password</code> , string <code>\$database</code>) : bool Αλλάζει τον τρόπο χρήσης της ενεργής σύνδεσης με τον MySQL server
<code>mysqli_character_set_name</code>	<code>mysqli_character_set_name</code> (mysqli <code>\$link</code>) : string Επιστρέφει το όνομα του συνόλου χαρακτήρων της ενεργής σύνδεσης με τη ΒΔ
<code>mysqli_get_charset</code>	<code>mysqli_get_charset</code> (mysqli <code>\$link</code>) : object Επιστρέφει ένα αντικείμενο που παρέχει διάφορες ιδιότητες για το σύνολο χαρακτήρων της ενεργής σύνδεσης
<code>mysqli_set_charset</code>	<code>mysqli_set_charset</code> (mysqli <code>\$link</code> , string <code>\$charset</code>) : bool Θέτει το σύνολο χαρακτήρων που θα χρησιμοποιείται όταν στέλνουμε δεδομένα στο DB server
<code>mysqli_close</code>	<code>mysqli_close</code> (mysqli <code>\$link</code>) : bool Κλείνει τη σύνδεση με τον MySQL server.
<code>mysqli_connect</code>	<code>mysqli_connect</code> ([string <code>\$host</code> = ini_get("mysqli.default_host") [, string <code>\$username</code> = ini_get("mysqli.default_user") [, string <code>\$passwd</code> = ini_get("mysqli.default_pw") [, string <code>\$dbname</code> = "" [, int <code>\$port</code> = ini_get("mysqli.default_port") [, string <code>\$socket</code> = ini_get("mysqli.default_socket")]]]]) : mysqli Ανοίγει μια νέα σύνδεση με τον MySQL server
<code>mysqli_query</code>	<code>mysqli_query</code> (mysqli <code>\$link</code> , string <code>\$query</code> [, int <code>\$resultmode</code> = MYSQLI_STORE_RESULT]) : mixed Στέλνει ένα ερώτημα για εκτέλεση στη ΒΔ της ενεργής σύνδεσης. Σε περίπτωση αποτυχίας επιστρέφει FALSE ενώ σε περίπτωση επιτυχίας για <i>SELECT</i> , <i>SHOW</i> , <i>DESCRIBE</i> ή <i>EXPLAIN</i> queries επιστρέφει ένα <code>mysqli_result</code> object. Για άλλα είδη ερωτημάτων (Update κ.λ.π.) επιστρέφει TRUE . Αν επιστραφεί ένα <code>mysqli_result</code> object, αφού το χρησιμοποιήσουμε (με την mysqli_fetch_array κ.λ.π.) να ελευθερώσουμε τη μνήμη που έχει δεσμευτεί με την mysqli_free_result (<code>\$result</code>) αλλιώς υπάρχει περίπτωση να μην μπορούμε

	να εκτελέσουμε άλλο ερώτημα.
mysqli_free_result	mysqli_free_result (mysqli_result \$result) : void Απελευθερώνει τη μνήμη που έχει δεσμευτεί μετά την mysqli_query από το αποτέλεσμα που έχει αποθηκευτεί στο mysqli_result object. Πρέπει να καλείτε πάντα μετά τη χρήση του αποτελέσματος.
mysqli_connect_errno	mysqli_connect_errno (void) : int Επιστρέφει τον αριθμό λάθους από την τελευταία κλήση σύνδεσης (mysqli_connect) στον MySQL server
mysqli_connect_error	mysqli_connect_error (mysqli \$link) : string Επιστρέφει το μήνυμα λάθους από την τελευταία κλήση σύνδεσης (mysqli_connect) στον MySQL server
mysqli_errno	mysqli_errno (mysqli \$link) : int Επιστρέφει τον αριθμό λάθους από την προηγούμενη λειτουργία στον MySQL server
mysqli_error	mysqli_error (mysqli \$link) : string Επιστρέφει το μήνυμα λάθους από την προηγούμενη λειτουργία στον MySQL server
mysqli_real_escape_string	mysqli_real_escape_string (mysqli \$link , string \$escapestr) : string Προσθέτει χαρακτήρες διαφυγής στους ειδικούς χαρακτήρες ενός αλφαριθμητικού που αναπαριστά ένα ερώτημα και επιστρέφει το νέο string για να χρησιμοποιηθεί για το ερώτημα.
mysqli_affected_rows	mysqli_affected_rows (mysqli \$link) : int Αριθμός γραμμών που επηρεάστηκαν από το προηγούμενο ερώτημα που εκτελέστηκε (για ερώτημα που επηρεάζει τα δεδομένα του πίνακα, δηλ. UPDATE, INSERT, DELETE κ.λ.π.).
mysqli_num_rows	mysqli_num_rows (mysqli_result \$result) : int Επιστρέφει τον αριθμό των γραμμών (εγγραφών) στο αποτέλεσμα του ερωτήματος που εκτελέστηκε (για ερώτημα SELECT).
mysqli_fetch_array	mysqli_fetch_array (mysqli_result \$result [, int \$resulttype = MYSQLI_BOTH]) : mixed Φέρνει μία γραμμή από τα αποτελέσματα ενός ερωτήματος σαν έναν πίνακα. Ο πίνακας αυτός είναι και associative (οι δείκτες είναι τα ονόματα των στηλών) και με αριθμητικούς δείκτες (οι δείκτες είναι οι αριθμοί των στηλών). Αν δεν υπάρχει άλλη γραμμή επιστρέφει NULL. Την καλούμε επαναλαμβανόμενα για να πάρουμε όλες τις γραμμές.
mysqli_fetch_assoc	mysqli_fetch_assoc (mysqli_result \$result) : array Φέρνει μία γραμμή από τα αποτελέσματα ενός ερωτήματος σαν έναν associative πίνακα. Αν δεν υπάρχει άλλη γραμμή επιστρέφει NULL. Την καλούμε επαναλαμβανόμενα για να πάρουμε όλες τις γραμμές.
mysqli_fetch_row	mysqli_fetch_row (mysqli_result \$result) : mixed Φέρνει μια γραμμή από τα αποτελέσματα σαν πίνακα με αριθμητικούς δείκτες. Αν δεν υπάρχει άλλη γραμμή επιστρέφει NULL. Την καλούμε επαναλαμβανόμενα για να πάρουμε όλες τις γραμμές.
mysqli_fetch_object	mysqli_fetch_object (mysqli_result \$result [, string \$class_name = "stdClass" [, array \$params]]) : object

	Επιστρέφει την τρέχουσα γραμμή του αποτελέσματος σαν ένα object όπου οι ιδιότητές του έχουν το όνομα των πεδίων του αποτελέσματος.
mysqli_fetch_all	mysqli_fetch_all (mysqli_result \$result [, int \$resulttype = MYSQLI_NUM]) : mixed Φέρνει όλες τις γραμμές από τα αποτελέσματα σαν πίνακα. Ο πίνακας αυτός είναι και associative (οι δείκτες είναι τα ονόματα των στηλών) και με αριθμητικούς δείκτες (οι δείκτες είναι οι αριθμοί των στηλών).
mysqli_fetch_field	mysqli_fetch_field (mysqli_result \$result) : object Φέρνει πληροφορίες για μια στήλη από τα αποτελέσματα σαν αντικείμενο με συγκεκριμένες ιδιότητες. Την καλούμε επαναλαμβανόμενα για να πάρουμε πληροφορίες για όλες τις στήλες.
mysqli_fetch_fields	mysqli_fetch_fields (mysqli_result \$result) : array Φέρνει πληροφορίες για όλες τις στήλες από τα αποτελέσματα σαν πίνακα αντικειμένων με συγκεκριμένες ιδιότητες.
mysqli_fetch_lengths	mysqli_fetch_lengths (mysqli_result \$result) : array Επιστρέφει έναν πίνακα με το μήκος κάθε στήλης της τρέχουσας γραμμής ενός αποτελέσματος.
mysqli_num_fields	mysqli_num_fields (mysqli_result \$result) : int Επιστρέφει τον αριθμό των πεδίων (στηλών) από το συγκεκριμένο σύνολο αποτελεσμάτων.
mysqli_insert_id	mysqli_insert_id (mysqli \$link) : mixed Επιστρέφει το ID (για το πεδίο AUTO_INCREMENT) που δημιουργήθηκε από το προηγούμενο ερώτημα εισαγωγής (INSERT) ή ερώτημα UPDATE όταν έχει χρησιμοποιηθεί η συνάρτηση LAST_INSERT_ID() του MySQL. Αν δεν εκτελέστηκε κάποιο INSERT ή δεν υπάρχει πεδίο AUTO_INCREMENT επιστρέφει 0.
mysqli_get_host_info	mysqli_get_host_info (mysqli \$link) : string Επιστρέφει ένα string που περιγράφει το host του MySQL server και τον τύπο σύνδεσης για τη σύνδεση.
mysqli_get_proto_info	mysqli_get_proto_info (mysqli \$link) : int Επιστρέφει τον αριθμό της έκδοσης του πρωτοκόλλου που χρησιμοποιείται για τη σύνδεση.

Πίνακας 9: Συναρτήσεις του MySQL Module της PHP

Φυσικά υπάρχουν και άλλες συναρτήσεις με τις οποίες έχεις τη δυνατότητα για πρόσβαση σε πολύ μεγαλύτερο φάσμα της λειτουργικότητας του MySQL (**MySQLi - Improved MySQL Extensions**) που προσφέρονται από τις εκδόσεις 4.1 και μετά. Ο αναγνώστης μπορεί να μελετήσει τις συναρτήσεις αυτές, έχουν πολύ μικρές διαφορές από αυτές που παρουσιάζονται εδώ, και να τις χρησιμοποιήσει αντί τις συγκεκριμένες.

5.12 Υπερκαθολικές (Superglobal) μεταβλητές

Στην PHP περιλαμβάνονται μια σειρά από **Υπερκαθολικές (Superglobal)** μεταβλητές οι οποίες είναι άμεσα διαθέσιμες σε οποιοδήποτε σημείο του κώδικά μας και μας παρέχουν διάφορες πληροφορίες για το περιβάλλον εκτέλεσης καθώς και για τα δεδομένα του χρήστη. Αυτές περιλαμβάνουν τις: **\$GLOBALS**, **\$_SERVER**, **\$_POST**, **\$_GET**, **\$_COOKIE**, **\$_FILES**,

`$_ENV`, **`$_REQUEST`**, **`$_SESSION`** και είναι πίνακες. Στις επόμενες υποενότητες αναλύονται κάποιες από αυτές.

5.12.1 Αποστολή δεδομένων από τον πελάτη/χρήστη

Τις περισσότερες φορές για να μπορέσουμε να δημιουργήσουμε ένα δυναμικό αποτέλεσμα απαιτούνται δεδομένα εισόδου από τον πελάτη. Τέτοια δεδομένα μπορεί να περιλαμβάνουν στοιχεία προτιμήσεων, στοιχεία εισαγωγής ή ενημέρωσης πινάκων Βάσης Δεδομένων, στοιχεία για την ρύθμιση της επιλογής ή/και τρόπου προβολής των δεδομένων που θα επιστραφούν στον πελάτη κλπ.

Ο συνήθης τρόπος για τη μετάδοση των δεδομένων αυτών από τον πελάτη είναι η χρήση φορμών με τις μεθόδους **GET** και **POST** του **HTTP** καθώς και συνδέσμων που στο URL ακολουθούν ζεύγη πεδίων/τιμών, δηλαδή ουσιαστικά χρήση και πάλι της μεθόδου **GET**.

Για να μπορούμε να χρησιμοποιήσουμε τα δεδομένα αυτά στα PHP προγράμματα (scripts) πρέπει να μπορούμε να τα λάβουμε μέσα από τον κώδικα της PHP. Ανάλογα με το πώς μας στέλνονται (**POST** ή **GET**) τα δεδομένα αυτά, ο τρόπος λήψης στην PHP είναι ο εξής:

- Αν τα δεδομένα στέλνονται με τη μέθοδο **GET** τότε άμεσα εισάγονται στον πίνακα **`$_GET`**.
- Αν τα δεδομένα στέλνονται με τη μέθοδο **POST** τότε άμεσα εισάγονται στον πίνακα **`$_POST`**.
- Επίσης τα στοιχεία που αφορούν τα cookies εισάγονται στον πίνακα **`$_COOKIE`**.
- Σε κάθε περίπτωση τα δεδομένα εισάγονται και στον πίνακα **`$_REQUEST`**.

Συγκεκριμένα πρόκειται για associative πίνακες με δείκτες τα ονόματα των πεδίων (που έχουν καθοριστεί στη φόρμα) και τιμές τις τιμές που συμπλήρωσε ο χρήστης (στη φόρμα), οι οποίοι είναι Υπερκαθολικές (Superglobal) μεταβλητές, δηλαδή είναι άμεσα διαθέσιμοι παντού στον κώδικά μας. Έτσι αν ο χρήστης πατήσει αποστολή (submit) σε μια φόρμα στην οποία υπάρχει ένα πεδίο με το όνομα **name** και στο οποίο συμπλήρωσε, πριν κάνει submit, την τιμή **”Κατσάλης Νίκος”** τότε στον κώδικά μας μπορούμε να κάνουμε τα εξής:

Αν η φόρμα στάλθηκε με τη μέθοδο GET:

```
$k = $_GET['name'];
echo $k; // τυπώνεται Κατσάλης Νίκος
```

Αν η φόρμα στάλθηκε με τη μέθοδο POST:

```
$k = $_POST['name'];
echo $k; // τυπώνεται Κατσάλης Νίκος
```

Φυσικά όπως αναφέρθηκε, όπως και να στάλθηκαν μπορούμε να κάνουμε:

```
$k = $_REQUEST['name'];
echo $k; // τυπώνεται Κατσάλης Νίκος
```

Παράδειγμα

Αρχείο με τη φόρμα χρήστη
(πχ frm.html)

```
<form action="action.php" method="post">
  Your name: <input type="text"
name="name" />
  Your age: <input type="text" name="age"
/>
  <input type="submit">
</form>
```

Αρχείο στο οποίο στέλνεται η φόρμα
(action.php)

```
Hi
<?php echo $_POST["name"]; ?>.
You are
<?php echo $_POST["age"]; ?>
years old.
```


Επίσης αν θέσουμε σε on, στο php.ini (βλέπε παράρτημα), τη μεταβλητή **register_globals** τότε άμεσα δημιουργείται στο PHP script μας μια καθολική μεταβλητή με το όνομα του πεδίου της φόρμας. Δηλαδή στο προηγούμενο παράδειγμα (είτε με GET είτε με POST) θα δημιουργηθούν οι καθολικές μεταβλητές \$name και \$age με τις αντίστοιχες τιμές που συμπλήρωσε ο χρήστης στη φόρμα. Φυσικά αυτό είναι πολύ επικίνδυνο αφού επιτρέπει στον πελάτη να δημιουργεί ότι μεταβλητές θέλει στο πρόγραμμά σου (αφού μπορεί πολύ εύκολα να φτιάξει μια δική του φόρμα και να της θέσει ως action το δικό σου script γεμίζοντάς σε με τις δικές του μεταβλητές, που μπορεί να συμπίπτουν και με τις ήδη υπάρχουσες). Το ίδιο ισχύει και για τη μεταφορά αρχείου και τον υπερκαθολικό πίνακα **\$_FILES** που θα δούμε παρακάτω.

Είναι πολύ σημαντικό να δίνεται **μεγάλη προσοχή στον έλεγχο των δεδομένων που μας στέλνονται**. Όλα τα δεδομένα που μας στέλνονται από φόρμες είναι τύπου string και πρέπει να κάνουμε κατάλληλους ελέγχους (αν έχουν συμπληρωθεί, αν έχουν τις κατάλληλες τιμές κλπ) αλλά και κατάλληλες μετατροπές (βλέπε στο εγχειρίδιο της PHP τον χειρισμό για τις μετατροπές σε άλλους τύπους). Επίσης πρέπει να προσέξουμε γιατί ο πελάτης μπορεί στα δεδομένα που θα στείλει να έχει ειδικούς χαρακτήρες (όπως ' , " και \) οι οποίοι μπορεί να προκαλέσουν προβλήματα αν προσπαθήσουμε να τα χρησιμοποιήσουμε σε κάποιες ενέργειες, όπως η εισαγωγή τους σε ένα πίνακα της MySQL). Υπάρχουν συναρτήσεις της PHP με τις οποίες μπορούμε να κάνουμε escape τους επικίνδυνους αυτούς χαρακτήρες (βλέπε συναρτήσεις **addslashes()**, **stripslashes()**, **htmlspecialchars()** κλπ). Φυσικά όταν είναι για εισαγωγή σε MySQL είναι προτιμότερο η χρήση ειδικών συναρτήσεων που παρέχει η επέκταση της PHP για MySQL (**mysqli_real_escape_string()**).

Για να γίνουν όλα αυτά υπάρχουν μία σειρά χρήσιμες συναρτήσεις της PHP που χρησιμοποιούμε συχνά. Ορισμένες από αυτές φαίνονται στον επόμενο πίνακα:

Συνάρτηση	Χρήση
isset (mixed \$var [,mixed \$...]) : bool	Εξετάζει αν μία μεταβλητή έχει τεθεί, δηλαδή αν έχει δηλωθεί και είναι διαφορετική από NULL. Αν ναι επιστρέφει TRUE αλλιώς επιστρέφει FALSE.
empty (mixed \$var) : bool	Εξετάζει αν μία μεταβλητή είναι NULL ή είναι άδεια (π.χ. αν είναι string και είναι ίση με "" και αν είναι επιστρέφει TRUE αλλιώς επιστρέφει FALSE
is_null (mixed \$var) : bool	Εξετάζει αν μία μεταβλητή είναι NULL και αν είναι επιστρέφει TRUE αλλιώς επιστρέφει FALSE.
is_numeric (mixed \$var) : bool	Εξετάζει αν μία μεταβλητή είναι αριθμός ή string που περιέχει αριθμό και αν είναι επιστρέφει TRUE αλλιώς επιστρέφει FALSE.
intval (mixed \$var [, int \$base=10]) : int	Επιστρέφει την ακέραια τιμή μίας μεταβλητής (τη μετατρέπει).
float floatval (mixed \$var)	Επιστρέφει την πραγματική τιμή μίας μεταβλητής (τη μετατρέπει).
string trim (string \$str [, string \$character_mask = "\t\n\r\0\x0B"])	Επιστρέφει ένα νέο string που είναι όπως το πρώτο όρισμά της αφού αφαιρεθούν τα κενά στην αρχή και στο τέλος του ορίσματος.
mixed filter_var (mixed \$variable [, int \$filter = FILTER_DEFAULT [, mixed \$options]])	Φιλτράρει μία μεταβλητή με το συγκεκριμένο φίλτρο και επιστρέφει FALSE αν το φίλτρο αποτύχει ή το αποτέλεσμα του φίλτρου.

Στο επόμενο παράδειγμα θεωρούμε ότι ο χρήστης μας στέλνει μία φόρμα με τη μέθοδο post με το πεδίο με όνομα name. Παρακάτω βλέπουμε το λάθος και το σωστό χειρισμό:

//ΛΑΘΟΣ ΤΡΟΠΟΣ

```
$val = $_POST['name']; // Συμβαίνει ΛΑΘΟΣ αν δεν υπάρχει το στοιχείο
```

//ΣΩΣΤΟΣ ΤΡΟΠΟΣ

```
if (isset($_POST['name'])) { //αν έχει τεθεί (άρα υπάρχει)
    if (!empty(trim($_POST['name']))) { //αν δεν είναι NULL ή μόνο κενά
        $val = $_POST['name'];
        .....
    }
}
```

5.12.1.1 Διαφορά GET με POST

Οι δύο μέθοδοι για την αποστολή δεδομένων από τον πελάτη στον εξυπηρετητή έχουν σημαντικές διαφορές που πρέπει να γνωρίζουμε για να μην δημιουργηθούν προβλήματα στις εφαρμογές που αναπτύσσουμε.

Συγκεκριμένα αν χρησιμοποιήσουμε τη μέθοδο **get** τότε τα δεδομένα της φόρμας (ζεύγη όνομα πεδίου=τιμή) μπαίνουν μετά το τέλος του url που έχουμε στην ιδιότητα **target** της φόρμας (αφού μπει πρώτα ο χαρακτήρας ?). Π.χ.:

- Αν **target="https://www.test.gr/form.html"**
- και τα πεδία της φόρμας έχουν ιδιότητα **name= "on1"** και **name= "on2"** και πληκτρολογήσουμε τις τιμές **timi1** και **timi2** αντίστοιχα.

Τότε θα δημιουργηθεί και θα αποσταλεί ως αίτημα στον web server το url:

https://www.test.gr/form.html?on1=timi1&on2=timi2

Αυτό έχει τρεις σημαντικές συνέπειες:

1. Υπάρχει περιορισμός μήκους (που εξαρτάται από τον web server) και άρα αν τα πεδία είναι πολλά ή έχουν μεγάλα ονόματα-τιμές μπορεί να μην αποσταλούν σωστά.
2. Το url που δημιουργείται διατηρείται στο ιστορικό του browser με αποτέλεσμα να μπορεί να το δει κάποιος και αν κατά λάθος ή εσκεμμένα το επιλέξει να προκαλέσει την επαναμετάδοση της φόρμας το οποίο μπορεί να έχει ανεπιθυμητές συνέπειες (π.χ. μία φόρμα που εισάγει στοιχεία θα τα ξαναεισάγει).
3. Δεν είναι δυνατή η μετάδοση κάποιων δεδομένων (π.χ. **input type="file"**)

Αντιθέτως με την **post** τα δεδομένα της φόρμας στέλνονται μέσα στο σώμα του HTTP αιτήματος που στέλνεται στον web server χωρίς να δημιουργείται πρόβλημα με το μέγεθος και το είδος, δεν είναι ορατό σε άλλους χρήστες και δεν μπορούμε να βρούμε το αίτημα αργότερα και να το ξαναστείλουμε, συνεπώς είναι και πιο ασφαλές. Οπότε για φόρμες που δεν είμαστε σίγουροι για το μήκος ή προκαλούν αλλαγές που έχουν σημασία θα πρέπει να προτιμάμε την μέθοδο **post**. Αντιθέτως η **get** χρησιμοποιείται είτε για φόρμες που απλώς προβάλλουν κάποια στοιχεία χωρίς να προκαλούν αλλαγές είτε κατευθείαν στους συνδέσμους για να παρέχουμε στοιχεία μέσω του url που βοηθάνε στο καθορισμό των ιστοσελίδων-πόρων που ζητάμε σε έναν ιστότοπο.

5.12.2 Πληροφορίες Εξυπηρετητή, Πελάτη και Περιβάλλοντος

Η Υπερκαθολική (Superglobal) μεταβλητή `$_SERVER` είναι ένας associative πίνακας με δείκτες ένα σύνολο strings που αντιπροσωπεύουν στοιχεία του εξυπηρετητή, του πελάτη και του περιβάλλοντος της PHP και τιμές τις τιμές των συγκεκριμένων στοιχείων.

Παράδειγμα:

```
<?php
// Τυπώνει το αλφαριθμητικό που περιγράφει το είδος του φυλλομετρητή του
πελάτη
echo $_SERVER['HTTP_USER_AGENT'];

// Ελέγχει αν στο αλφαριθμητικό που περιγράφει το είδος του φυλλομετρητή
// του πελάτη υπάρχει η λέξη MSIE, και αν ναι τυπώνει ότι χρησιμοποιεί
// MS Internet Explorer καθώς και το αλφαριθμητικό.
if (strstr($_SERVER['HTTP_USER_AGENT'], "MSIE")) {
    echo '<h3>Χρησιμοποιείς Internet Explorer</h3>';
    echo 'Συγκεκριμένα χρησιμοποιείς τον: <br />';
    echo '<b>' . $_SERVER["HTTP_USER_AGENT"] . '</b>';
} else { // αλλιώς τυπώνει ότι δεν χρησιμοποιεί MS IE καθώς
    //και το αλφαριθμητικό που περιγράφει το είδος του φυλλομετρητή
    echo '<h3>Δεν χρησιμοποιείς Internet Explorer</h3>';
    echo 'Χρησιμοποιείς τον: <br />';
    echo '<b>' . $_SERVER["HTTP_USER_AGENT"] . '</b>';
}
?>
```

5.13 Συνάρτηση header για προχωρημένες λειτουργίες

Μία πολύ χρήσιμη συνάρτηση που μας δίνεται στην PHP είναι η **header** με την οποία στέλνουμε στον πελάτη έναν ακατέργαστο (raw) header (για περισσότερες πληροφορίες όσον αφορά τους **HTTP headers**, ο αναγνώστης μπορεί να διαβάσει τις προδιαγραφές του HTTP/1.1 <https://www.w3.org/Protocols/>). Το συντακτικό της έχει ως εξής:

```
void header ( string $string [, bool $replace= true [, int
$http_response_code ]] )
```

Στην υποχρεωτική παράμετρο string γράφουμε τον header, μαζί με τις επιλογές που επιθυμούμε, που θέλουμε να αποστείλουμε. Η προαιρετική παράμετρος **replace** καθορίζει αν η συγκεκριμένη επικεφαλίδα θα αντικαταστήσει (προεπιλεγμένη τιμή **TRUE**) κάποια προηγούμενη που γράψαμε ή θα σταλεί μαζί με τις προηγούμενες (**FALSE**). Δηλαδή βάζοντας **FALSE** Μπορούμε να στείλουμε μαζί πολλούς headers (του ίδιου όμως τύπου). Πχ

```
<?php
header('WWW-Authenticate: Negotiate');
header('WWW-Authenticate: NTLM', false);
?>
```

Η προαιρετική παράμετρος **http_response_code** υποχρεώνει τον HTTP κώδικα απάντησης να είναι ότι και η τιμή του.

Προσοχή πρέπει να δοθεί στον τρόπο κλήσης της συνάρτησης αφού αυτή **πρέπει να κληθεί στο script μας πριν από οποιοδήποτε έξοδο στην ιστοσελίδα, ούτε καν ένα <space>** (αλλιώς οι headers έχουν ήδη σταλεί). Πχ το παρακάτω θα είναι λάθος:

```
<html>
<?php
// Λάθος αφού ήδη έχουμε βγάλει έξοδο την ετικέτα <html>
header (...);
```

Επειδή πολλές φορές, ιδιαίτερα αν κάνουμε include άλλα PHP αρχεία, μπορεί να έχουμε ήδη βγάλει έξοδο αλλά να μας είναι δύσκολο να βρούμε σε ποιο σημείο του κώδικα για να το διορθώσουμε, μπορούμε να χρησιμοποιήσουμε τη συνάρτηση **headers_sent** η οποία ελέγχει αν ήδη έχουν σταλεί οι headers και μάλιστα με κατάλληλα ορίσματα μας λέει και το αρχείο και σε ποια γραμμή. Η σύνταξή της είναι:

```
bool headers_sent ([ string &$file [, int &$line ] ] )
```

Μας επιστρέφει TRUE αν ήδη έχουν σταλεί αλλιώς FALSE. Βάζοντας τα δύο ορίσματα (όπως βλέπετε είναι call-by-reference) μας επιστρέφεται το όνομα του αρχείου και η γραμμή που έγινε πρώτη φορά έξοδος.

Υπάρχουν πολλά που μπορούμε να κάνουμε με τη **header**, αλλά εδώ θα περιοριστούμε σε τέσσερις πολύ χρήσιμες κλήσεις της στις οποίες χρησιμοποιούμε ορισμένους headers (η παράμετρος **string** έχει συγκεκριμένη μορφή). Αυτές αναλύονται στις τέσσερις υποενότητες που ακολουθούν.

5.13.1 Αποστολή Κωδικών Κατάστασης

Όταν το string ξεκινάει με **HTTP/** χρησιμοποιείται για να αποσταλούν HTTP κωδικοί κατάστασης (status codes). Επίσης το ίδιο πετυχαίνεις χρησιμοποιώντας τον header **Status:**. Πχ

```
<?php
header("HTTP/1.0 404 Not Found");
?>
ή
<?php
header("Status: 404 Not Found");
?>
```

5.13.2 Ανακατεύθυνση σε URL

Όταν χρησιμοποιηθεί ο header **Location:**, το string ξεκινάει με Location: και μετά ακολουθεί ένα URL που χρησιμοποιείται για να σταλεί πίσω στον πελάτη ο κωδικός κατάστασης REDIRECT (ανακατεύθυνση) 302 (εκτός και αν ήδη έχει σταλεί κάποιος άλλος κωδικός κατάστασης 3XX) και ο πελάτης ανακατευθύνεται σε αυτό το URL.

Η χρήση αυτή είναι πολύ σημαντική αφού μας επιτρέπει μετά από κάποιους ελέγχους των στοιχείων του πελάτη να αποφασίσουμε να τον ανακατευθύνουμε εκεί που θέλουμε (πχ σε μια ιστοσελίδα πληροφοριών και να μην αναγκάζομαστε να βάζουμε μηνύματα λάθους/πληροφόρησης σε κάθε script, ή σε μια σελίδα για να κάνει login αφού κρίνουμε ότι χρειάζεται πιστοποίηση για να κάνει κάποια λειτουργία που προσπάθησε κλπ). Πχ

```
<?php
// Ανακατευθύνω τον πελάτη στη σελίδα www.php.net
header("Location: http://www.php.net/");
?>
```

Μάλιστα επειδή κάποιοι πελάτες απαιτούν πλήρες URL μπορούμε να χρησιμοποιήσουμε το **\$_SERVER['HTTP_HOST']**, **\$_SERVER['PHP_SELF']** και τη συνάρτηση **dirname** για να δημιουργήσουμε δυναμικά ένα πλήρες URL για τις δικές μας ιστοσελίδες. Έτσι αν θέλαμε να τον κατευθύνουμε στο αρχείο **start.html** που βρίσκεται στον υποφάκελο **help** του φακέλου που βρίσκεται το script που εκτελείται θα μπορούσαμε να γράψουμε:

```
<?php
// Ανακατευθύνω τον πελάτη στη σελίδα start.html που βρίσκεται στον
// υποφάκελο help του φακέλου που βρίσκεται το script στο server μου.
header("Location: http://" . $_SERVER['HTTP_HOST'] .
dirname($_SERVER['PHP_SELF']) . "/help/start.html");
?>
```

5.13.3 Αποστολή Αρχείου και Προτροπή του χρήστη για αποθήκευσή

Μία άλλη χρήσιμη λειτουργία είναι όταν θέλουμε να στείλουμε στον πελάτη ένα αρχείο και θέλουμε να του εμφανίσουμε ένα παράθυρο που θα προτρέπει (τον χρήστη) να αποθηκευτεί το αρχείο. Στην περίπτωση αυτή χρησιμοποιούμε δύο κλήσεις της **header** όπου χρησιμοποιούνται οι headers **Content-type** (για τα είδη **Content Types/Subtypes** που μπορείτε να χρησιμοποιήσετε δείτε τα **MIME Media Types**) και **Content-Disposition**. Ένα παράδειγμα είναι:

```
<?php
// Καθορίζουμε ότι θα σταλεί αρχείο τύπου PDF
// για περισσότερους τύπους Content Types/Subtypes, όπως το
// application/pdf, δείτε τα MIME Media Types
header('Content-type: application/pdf');

// Καθορίζουμε το προτεινόμενο όνομα ως downloaded.pdf. Για περισσότερες
// επιλογές δείτε τους HTTP headers στις προδιαγραφές του HTTP/1.1
header('Content-Disposition: attachment; filename="downloaded.pdf"');

// Διαβάζετε το αρχείο (του οποίου το πραγματικό όνομα είναι
// original.pdf) και στέλνεται (αν πατήσει αποδοχή ο χρήστης στον πελάτη)
readfile('original.pdf');
?>
```

5.14 HTTP αναγνώριση με τη PHP

Μια άλλη λειτουργία είναι όταν θέλουμε να **πιστοποιήσουμε-αναγνωρίσουμε** τον πελάτη μας για να δει κάποια ιστοσελίδα (ή απλώς για να μας δώσει τους κωδικούς του). Τότε μπορούμε, με κατάλληλη κλήση της **header**, να μην φορτωθεί η ιστοσελίδα αλλά να υποχρεώσουμε τον πελάτη (του στέλνουμε ένα **Authentication Required** μήνυμα) να ανοίξει στον χρήστη ένα παράθυρο για να συμπληρώσει όνομα χρήστη και κωδικό και να μας σταλούν στην ίδια ιστοσελίδα (ξαναεκτελείται ο κώδικας της ιστοσελίδας μας με τα στοιχεία όνομα χρήστη και κωδικό που συμπλήρωσε ο χρήστης). Όταν ξαναεκτελείται ο κώδικάς μας τότε στον υπερκαθολικό πίνακα **\$_SERVER** έχουν προστεθεί τα στοιχεία με δείκτες **PHP_AUTH_USER** και **PHP_AUTH_PW** στα οποία έχει μπει το όνομα χρήστη και ο κωδικός. Αν ο χρήστης στον πελάτη πατήσει **Άκυρο** (Cancel) στο παράθυρο που του εμφανιστεί για να βάλει τους κωδικούς, τότε δεν θα υπάρχουν αυτά τα στοιχεία (άρα με την **isset** μπορούμε να το εξετάσουμε). Η διαδικασία θα φανεί καλύτερα με το επόμενο παράδειγμα:

```
<?php
// Έλεγχε αν πάτησε Cancel
if (!isset($_SERVER['PHP_AUTH_USER'])) {
    header('WWW-Authenticate: Basic realm="Secure Access"');
    header('HTTP/1.0 401 Unauthorized');
    echo 'Πρέπει να δώσεις τα πιστοποιητικά σου !!!';
    exit;
} else {
    //Παρακάτω απλώς εκτυπώνω τα στοιχεία που έδωσε ο χρήστης. Φυσικά
    //θα μπορούσα να συνδεθώ με μια ΒΔ MySQL (δες παρακάτω) και να ελέγξω
    //αν υπάρχει (κάνοντας SELECT στον πίνακα με τα στοιχεία των χρηστών)
    //ο χρήστης και ανάλογα να συνεχίσω τον κώδικά μου
    echo "Γεια σου " . $_SERVER['PHP_AUTH_USER'];
    echo "<br />";
    echo "Ο κωδικός που έδωσες είναι: " . $_SERVER['PHP_AUTH_PW'];
}
?>
```

Μόνο ο βασικός (Basic) τύπος πιστοποίησης υποστηρίζεται. Για περισσότερα δείτε τις προδιαγραφές του HTTP 1.1.

5.15 Χειρισμός Αρχείων στην PHP

Για να χρησιμοποιήσουμε ένα αρχείο στην PHP (για διάβασμα ή γράψιμο ή προσθήκη) μέσα σε ένα script πρέπει να γίνουν τα εξής βήματα:

1. Άνοιγμα του αρχείου για την κατάλληλη ενέργεια (διάβασμα ή γράψιμο ή προσθήκη)
2. Ενέργειες στο αρχείο (διάβασμα, γράψιμο κλπ).
3. Κλείσιμο του αρχείου

Για να ανοίξουμε ένα αρχείο χρησιμοποιούμε την συνάρτηση `fopen`:

```
resource fopen ( string $filename , string $mode [, bool
$use_include_path = false [, resource $context ]] )
```

Στην `fopen` το πρώτο όρισμα είναι το όνομα του αρχείου (με το μονοπάτι αν δεν είναι στον ίδιο φάκελο με το script) και το δεύτερο όρισμα καθορίζει γιατί θα ανοιχτεί το αρχείο ("r" για διάβασμα, "w" για γράψιμο που σημαίνει ότι αν το αρχείο έχει περιεχόμενα διαγράφονται, "a" για συμπλήρωση στο τέλος του αρχείου κλπ). Η `fopen` αν αποτύχει να ανοίξει το αρχείο (πχ όταν πάμε να ανοίξουμε για διάβασμα ένα αρχείο που δεν υπάρχει) επιστρέφει `FALSE`.

Για να διαβάσουμε σε ένα αρχείο, το οποίο φυσικά έχουμε ανοίξει για διάβασμα, συνήθως χρησιμοποιούμε τις συναρτήσεις `fscanf`, `fgets` και για να γράψουμε `fprintf`, `fwrite`:

```
mixed fscanf ( resource $handle , string $format [, mixed &$... ] )
string fgets ( resource $handle [, int $length ] )
int fprintf ( resource $handle , string $format [, mixed $args [, mixed
$... ]] )
int fwrite ( resource $handle , string $string [, int $length ] )
```

Η `fscanf` επιστρέφει τον αριθμό των τιμών που διάβασε (άρα 0 δηλαδή `FALSE` αν αποτύχει) και η `fgets` αν αποτύχει επιστρέφει `FALSE` και αυτό είναι χρήσιμο για να καταλάβουμε αν δεν υπάρχουν άλλα στοιχεία να διαβάσουμε (το αρχείο έφτασε στο τέλος του).

Παράδειγμα διαβάσματος αρχείου `test.txt` (είναι στον ίδιο φάκελο με το αρχείο `php`) το οποίο περιέχει ονόματα:

```
//Άνοιξε το αρχείο για διάβασμα
$fp = fopen("test.txt", "r");
//Αν δεν καταφέρουμε να το ανοίξουμε βγάλε μήνυμα
if (!$fp)
echo "<b>Cannot open file !</b>";
else //Αν το αρχείο άνοιξε διάβασε τα στοιχεία του και τύπωσε τα σε λίστα
{
    echo "<ul>";
    //Διάβαζε μέχρι να μην υπάρχουν άλλα δεδομένα. Κάθε ένα string
    //που διαβάζεις τύπωσε το σαν στοιχείο της λίστας
    while (fscanf($fp, "%s", $val))
        echo "<li>$val</li>";
}
```

```

        echo "</ul>";
    }
    //Κλείσε το αρχείο
    fclose($fp);

```

Παράδειγμα γραψίματος στο τέλος αρχείου **test.txt** (είναι στον ίδιο φάκελο με το αρχείο php):

```

//Ανοίξε το αρχείο για γράψιμο στο τέλος (append)
$fp = fopen("test.txt", "a");
if (!$fp) //Αν δεν καταφέραμε να το ανοίξουμε
echo "<b>Cannot open file !</b>";
else
{
    //Αν το αρχείο άνοιξε σωστά γράψε την τιμή που σου έστειλε ο χρήστης
    //(πεδίο name μίας φόρμας που έχει αποσταλεί με post)
    if (isset($_POST['name']) && !empty(trim($_POST['name'])))
    {
        $val = $_POST['name'];
        fprintf($fp, "%s\n", $val);
        echo "<b>Η τιμή αποθηκεύτηκε !</b>";
    }
    fclose($fp);
}

```

5.16 Μεθοδολογία Σύνδεσης PHP-MySQL

Για τη σύνδεση με ένα MySQL εξυπηρετητή, την εκτέλεση ερωτημάτων και τον έλεγχο και χρήση των αποτελεσμάτων υπάρχουν διάφορες προσεγγίσεις που μπορεί κάποιος να χρησιμοποιήσει. Στην υποενότητα αυτή θα παρουσιάσουμε μία προτεινόμενη μεθοδολογία.

Τα βήματα που προτείνονται στην μεθοδολογία αυτή περιλαμβάνουν με τη σειρά:

1. Σύνδεση με τον MySQL εξυπηρετητή και επιλογή της Βάσης Δεδομένων
2. Εκτέλεση Ερωτημάτων
3. Έλεγχος και εκτύπωση αποτελεσμάτων ερωτημάτων
4. Κλείσιμο της σύνδεσης

Ακολουθώντας τα παραπάνω βήματα που αναλύονται στις επόμενες υποενότητες θα μπορείτε πολύ εύκολα να δημιουργήσετε ισχυρές εφαρμογές. Επίσης μετά την ολοκλήρωση της ανάλυσης των βημάτων δίνεται ένα ολοκληρωμένο παράδειγμα.

5.16.1 Σύνδεση με τον MySQL εξυπηρετητή και επιλογή Β.Δ.

Συνδεόμαστε στον MySQL server χρησιμοποιώντας τη συνάρτηση **mysql_connect** η οποία παίρνει σαν παραμέτρους το μηχάνημα στο οποίο τρέχει ο MySQL server, το όνομα χρήστη και τον κωδικό πρόσβασης. Η συνάρτηση αυτή μας επιστρέφει τη σύνδεση που δημιουργείται και την οποία χρειαζόμαστε για τα επόμενα βήματα. Η σύνταξή της έχει ως εξής:

```

mysql_connect ([ string $host = ini_get("mysqli.default_host") [, string
$username = ini_get("mysqli.default_user") [, string $passwd =
ini_get("mysqli.default_pw") [, string $dbname = "" [, int $port =
ini_get("mysqli.default_port") [, string $socket =
ini_get("mysqli.default_socket") ]]]]] ) : mysqli

```

Στην περίπτωση επιτυχίας επιστρέφει ένα αντικείμενο **mysqli** που προσδιορίζει τη σύνδεση (το οποίο χρησιμοποιείται για να καθορίζει τη σύνδεση στις επόμενες εντολές), ενώ σε περίπτωση αποτυχίας επιστρέφει **NULL**. Το αλφαριθμητικό **server** είναι ο H/Y (όνομα ή IP διεύθυνση) στον

οποίο εκτελείται ο MySQL εξυπηρετητής και μπορεί να περιλαμβάνει και port, πχ "localhost:1234".

Παράδειγμα:

```
// Σύνδεση με τον mysql server που τρέχει στον Η/Υ μας (localhost)
// με όνομα χρήστη "root" και κωδικό "nikos" και σύνδεση στη ΒΔ "testdb"
$link = mysqli_connect("localhost", "root", "nikos", "testdb");
if (!$link)
    die ('Αδυναμία σύνδεσης: ' . mysqli_connect_error());

// ακολουθεί ο κώδικας με δεδομένη την επιτυχία σύνδεσης και με χρήση
// μεταβλητής $link που αποτελεί ένα mysqli object
```

5.16.2 Εκτέλεση Ερωτημάτων

Εκτελούμε ερωτήματα στη βάση χρησιμοποιώντας τη συνάρτηση **mysqli_query**. Η συνάρτηση αυτή μας επιστρέφει το αποτέλεσμα του ερωτήματος. Η σύνταξή της έχει ως εξής:

```
mysqli_query ( mysqli $link , string $query [, int $resultmode =
MYSQLI_STORE_RESULT ] ) : mixed
```

Παίρνει σαν παράμετρο το αντικείμενο **mysqli** που επέστρεψε η σύνδεση στον server (η **mysqli_connect**), το ερώτημα (**string \$query**), σε SQL, που θέλουμε να εκτελέσουμε και (προαιρετικά) τον τρόπο χειρισμού του αποτελέσματος (**int \$resultmode**) που αν δεν δοθεί θα πάρει προεπιλεγμένη τιμή **MYSQLI_STORE_RESULT**. Σε περίπτωση αποτυχίας επιστρέφει **FALSE** ενώ σε περίπτωση επιτυχίας για **SELECT**, **SHOW**, **DESCRIBE** ή **EXPLAIN** queries επιστρέφει ένα **mysqli_result** object. Για άλλα είδη ερωτημάτων (UPDATE, INSERT κ.λ.π.) επιστρέφει **TRUE**.

Η επιλογή **MYSQLI_STORE_RESULT** προκαλεί τη μεταφορά όλου του αποτελέσματος από τον server στον πελάτη και επιτρέπει να γίνουν διάφορες άλλες ενέργειες (όπως αναζήτηση στα αποτελέσματα με τη **mysqli_data_seek()**). Η **MYSQLI_USE_RESULT** συνδέει τα αποτελέσματα που έχουν μεταφερθεί στον πελάτη με αυτά που παραμένουν στον server αλλά αυτό έχει σαν αποτέλεσμα να μην μπορούν να εκτελεστούν άλλα ερωτήματα μέχρι να απελευθερώσουμε τη μνήμη που έχει δεσμευτεί με την **mysqli_free_result(\$result)**. Γενικά προτείνεται αν επιστραφεί ένα **mysqli_result** object τότε αφού το χρησιμοποιήσουμε (με την **mysqli_fetch_array** κ.λ.π.) να το ελευθερώσουμε με την **mysqli_free_result(\$result)**.

Προσοχή πρέπει να δοθεί ώστε το αλφαριθμητικό του ερωτήματος να μην τερματίζεται με ; (ερωτηματικό).

Παράδειγμα εκτέλεσης ερωτήματος:

```
// Εκτέλεση SELECT στη Β.Δ. που συνδεθήκαμε στο προηγούμενο βήμα
$sql = "SELECT fid, title, company FROM films where title='$title'";
$result = mysqli_query($link, "SELECT * FROM films where
title='ALIENS'");
//αν απέτυχε η εκτέλεση του ερωτήματος βγάλε μήνυμα και τερμάτισε
if (!$result)
    die ('Cannot select from table: ' . mysqli_error($link));
// Ακολουθούν εντολές με δεδομένη την επιτυχία του ερωτήματος (βλέπε
επόμενο βήμα)
```

Μεγάλη προσοχή πρέπει να δοθεί γιατί ο πελάτης μπορεί στα δεδομένα που θα στείλει να έχει ειδικούς χαρακτήρες (όπως ' , " και \) οι οποίοι μπορεί να προκαλέσουν προβλήματα αν

προσπαθήσουμε να τα εισάγουμε σε ένα πίνακα της MySQL. Υπάρχουν συναρτήσεις της PHP με τις οποίες μπορούμε να κάνουμε escape τους επικίνδυνους αυτούς χαρακτήρες (βλέπε συναρτήσεις `addslashes()`, `stripslashes()`, `htmlspecialchars()` κλπ). Όμως καλύτερα είναι να χρησιμοποιήσουμε τις συναρτήσεις που παρέχει η επέκταση της PHP για το MySQL όπως τη `mysqli_real_escape_string` ή η βιβλιοθήκη της αντίστοιχης Β.Δ. που χρησιμοποιούμε, αν δεν χρησιμοποιούμε MySQL.

Σημαντική Παρατήρηση: Συνήθως μετά από την σύνδεση στον MySQL server και πριν εκτελέσουμε οποιοδήποτε ερώτημα πρέπει να ενημερώσουμε το server για την κωδικοποίηση χαρακτήρων που χρησιμοποιεί ο πελάτης (ότι κωδικοποίηση είναι η ιστοσελίδα μας). Αν δεν γίνει αυτό θα έχουμε σημαντικό πρόβλημα με την αποθήκευση των δεδομένων μας στην Β.Δ.. Δείτε την επόμενη υποενότητα για το πώς γίνεται αυτή η ενημέρωση.

5.16.2.1 Ελληνικοί και άλλοι Χαρακτήρες και PHP-MySQL

Στον MySQL εξυπηρετητή το **default encoding (character set)** είναι **latin1** και το **default collation** είναι **latin1_swedish_ci**. Αυτό σημαίνει ότι όταν στέλνω δεδομένα στον MySQL εξυπηρετητή τα περιμένει σε αυτό το encoding και συνεπώς αν του τα στείλω από την εφαρμογή πελάτη σε ένα άλλο encoding θα προσπαθήσει να τα ερμηνεύσει με το δικό του και θα τα μετατρέψει σε διαφορετικά («σκουπίδια») από αυτά που του έστειλα. Μετά θα τα μετατρέψει στο κατάλληλο encoding της Β.Δ. ώστε να τα αποθηκεύσει σε αυτή με αποτέλεσμα να αποθηκευτούν στη Β.Δ. τα «σκουπίδια».

Αυτό που πρέπει λοιπόν να γίνει είναι να εξασφαλίσω 2 πράγματα:

1. Ότι η Β.Δ. θα έχει κατάλληλο encoding ώστε να μπορεί να αποθηκευτούν σε αυτή τα Ελληνικά και ότι άλλα δεδομένα στέλνω. Αυτό όπως αναφέραμε και στο προηγούμενο κεφάλαιο το λύνουμε επιλέγοντας για τις Β.Δ. **UTF-8** και συγκεκριμένα:
 - a. encoding-character set **utf8mb4** και collation **utf8mb4_unicode_ci**
 - b. ή encoding-character set **utf8** και collation **utf8_unicode_ci** ή **utf8_general_ci**και έτσι εξασφαλίζουμε πολυγλωσσική υποστήριξη. Φυσικά μπορούμε να βάλουμε και κάποιο άλλο encoding και collation που υποστηρίζει αυτά που θέλουμε.
2. Ότι αν το πρόγραμμα πελάτης χρησιμοποιεί διαφορετικό encoding (character set) από το latin1 που είναι το default που περιμένει ο MySQL εξυπηρετητής να φροντίσω να ενημερώσω, με το που γίνει η σύνδεσή μου με τον MySQL εξυπηρετητή, τι encoding χρησιμοποιώ ώστε:
 - Να γνωρίζει ο MySQL εξυπηρετητής σε τι encoding θα του στέλνω τα δεδομένα ώστε να κάνει τις σωστές μετατροπές κωδικοποίησης
 - Να καταλάβει πάλι πως θα τα μετατρέψει όταν θα μου τα στέλνει πίσω ώστε να προβάλλονται από τον πελάτη μου σωστά.

Το πρώτο από τα παραπάνω γίνεται κατά τη δημιουργία της Β.Δ. στον MySQL εξυπηρετητή. Το δεύτερο πρέπει να γίνει αφού συνδεθώ με το MySQL εξυπηρετητή και πριν εκτελέσω οποιοδήποτε ερώτημα σε αυτόν. Αυτό μπορεί να γίνει με δύο τρόπους:

1. Με απλή εκτέλεση ερωτήματος όπως φαίνεται στο επόμενο:

```
$str = "SET NAMES 'utf8' ";  
mysqli_query($link, $str);
```

2. Με τη χρήση της όπως φαίνεται στο επόμενο:

```
mysqli_set_charset($link, "utf8");
```

Ο δεύτερος τρόπος είναι προτιμότερος καθώς επηρεάζει και τη λειτουργία της συνάρτησης `mysqli_real_escape_string()` καθώς θέτει σωστά τις ρυθμίσεις της σύνδεσης ενώ ο πρώτος δεν την επηρεάζει οπότε μπορεί να μην λειτουργήσει σωστά όταν τη χρησιμοποιούμε.

Στα παραπάνω παραδείγματα χρησιμοποιώ ως encoding το utf8 όμως αυτό θα αντικατασταθεί από το encoding που χρησιμοποιεί ο πελάτης μου, που στην περίπτωση των ιστοσελίδων που εξετάζουμε είναι το encoding της ιστοσελίδας μέσα στην οποία βρίσκεται το PHP script. Ουσιαστικά με το παραπάνω ερώτημα/κλήση συνάρτησης θέτω μια σειρά μεταβλητές που αφορούν τη συγκεκριμένη σύνδεση με τις οποίες πληροφορώ με ότι χρειάζεται να ξέρει το MySQL εξυπηρετητή. Για να δείτε τις μεταβλητές που επηρεάζουν την επικοινωνία μπορείτε να εκτελέσετε με τον πελάτη σας τα παρακάτω ερωτήματα στον MySQL εξυπηρετητή:

```
show variables like "character_set%"
```

και

```
show variables like "collation%"
```

Επίσης μπορείτε να χρησιμοποιήσετε τη συνάρτηση της MySQL επέκτασης της PHP

```
mysqli_character_set_name ( mysqli $link ) : string
```

Με την οποία επιστρέφεται το τι έχει τεθεί ως encoding στον πελάτη σας και αν δεν είναι αυτό που χρησιμοποιείται να θέσετε το σωστό encoding με τους δύο τρόπους (ερώτημα και `mysqli_set_charset()`) που είδαμε παραπάνω. Π.χ.:

```
$link = mysqli_connect("localhost", "root", "nikos", "testdb");  
$charset = mysqli_character_set_name($link);  
echo "The current character set is: $charset\n";
```

Μάλιστα δεν χρειάζεται να αλλάξω και το collation γιατί όταν αλλάξω το encoding (character set) ουσιαστικά αλλάζει και το collation στο default για το συγκεκριμένο encoding. Όπως είδαμε και στο προηγούμενο κεφάλαιο μπορούμε να δούμε τα encodings και τα default collation που τους αντιστοιχούν εκτελώντας την εντολή **show character set**; στον MySQL εξυπηρετητή.

Ακολουθούν δύο παραδείγματα με προγράμματα πελάτη ιστοσελίδες που χρησιμοποιούνται συχνά.

Παράδειγμα 1:

Πολλές φορές ο πελάτης μου είναι ιστοσελίδες που χρησιμοποιούν το encoding ISO Ελληνικά (ISO-8859-7). Μάλιστα για να το καταλαβαίνουν οι φυλλομετρητές (browsers) και να αλλάζουν αυτόματα ώστε να την φορτώνουν σωστά το δηλώνω στις ιστοσελίδες μου στο τμήμα `<head>...</head>` με την ετικέτα `<meta ..>` ως εξής:

```
<meta charset="ISO-8859-7" />
```

Τότε θα πρέπει να εκτελέσω μετά από κάθε σύνδεση (μετά από τη `mysqli_connect()`) και πριν εκτελέσω άλλα ερωτήματα, το ερώτημα:

```
mysqli_query($link, "greek");
```

ή να εκτελέσω τη συνάρτηση:

```
mysqli_set_charset($link, "greek");
```

Παράδειγμα 2:

Πολλές φορές ο πελάτης μου είναι ιστοσελίδες που χρησιμοποιούν το encoding **utf8** (είναι συνηθισμένο, για λόγους υποστήριξης όλων των γλωσσών, τα τελευταία χρόνια να τις γράφουμε σε αυτό το encoding). Μάλιστα για να το καταλαβαίνουν οι φυλλομετρητές (browsers) και να αλλάζουν αυτόματα ώστε να την φορτώνουν σωστά το δηλώνω στις ιστοσελίδες μου στο τμήμα `<head>...</head>` με την ετικέτα `<meta ..>` ως εξής:

```
<meta charset="utf-8" />
```

Τότε θα πρέπει να εκτελέσω μετά από κάθε σύνδεση (μετά από τη `mysqli_connect()`) και πριν εκτελέσω άλλα ερωτήματα, το ερώτημα:

```
mysqli_query($link, "utf8");
```

ή να εκτελέσω τη συνάρτηση:

```
mysqli_set_charset($link, "utf8");
```

Φυσικά για να μην γράφω κάθε φορά που κάνω σύνδεση (`mysqli_connect`) το ερώτημα, προτείνεται στον αναγνώστη να γράψει μία παραμετρική συνάρτηση με μία παράμετρο που θα δίνει το encoding που θέλει (μπορεί να βάλει προκαθορισμένη τιμή "greek" ή όποια αυτός θέλει ώστε αν δεν τη δώσει να πάρει αυτή). Στην συνάρτηση αυτή θα γίνεται η σύνδεση και θα εκτελείται το προηγούμενο `mysqli_query` για το encoding του πελάτη και θα επιστρέφεται η σύνδεση για να τη χρησιμοποιήσει όπως θέλει αυτός που την κάλεσε.

Με τον τρόπο αυτό αρκεί να καλώ τη συνάρτηση αυτή, αντί για την `mysqli_connect`, στον κώδικά μου (αφού φυσικά την έχω βάλει σε ένα αρχείο που το έχω κάνει `include` ή `require`). Μία τέτοια συνάρτηση μπορεί να έχει ως εξής:

```
function connectToServer($enc = "greek")
{
    // Μπορώ και τα host, username και passwd να τα έχω παραμετρικά
    $link = mysqli_connect("somehost", "a user", "a password", "a db");
    if (!$link)
        die("Cannot connect to server " . mysqli_connect_error());

    //εκτελώ το mysqli_query($link, "SET NAMES '...'");
    $str = "SET NAMES '" . $enc . "'";
    mysqli_query($link, $str);
    return $link;
}
```

Φυσικά καλό είναι να γίνονται και όλοι οι έλεγχοι για τυχόν που θα αποτύχει κάτι (η σύνδεση ή το ερώτημα) όπως έχουμε δείξει στα προηγούμενα.

Αρα για ένα πελάτη, που έχει encoding `ISO-8859-7`, για να συνδεθώ θα καλέσω:

```
$link = connectToServer();
// Από εδώ και πέρα χρησιμοποιώ το $link για να συνδεθώ με τις Β.Δ. που
// θέλω και να εκτελέσω τα ερωτήματα χωρίς πρόβλημα με τα ελληνικά
```

Για ένα πελάτη, που έχει `utf-8`, για να συνδεθώ θα καλέσω:

```
$link = connectToServer("utf8");
// Από εδώ και πέρα χρησιμοποιώ το $link για να συνδεθώ με τις Β.Δ. που
// θέλω και να εκτελέσω τα ερωτήματα χωρίς πρόβλημα με τα ελληνικά
```

5.16.3 Έλεγχος και εκτύπωση αποτελεσμάτων ερωτημάτων

Στο προηγούμενο βήμα αν το ερώτημα είναι επιλογής (`SELECT`) μπορούμε χρησιμοποιώντας τη συνάρτηση `mysqli_num_rows`, να δούμε πόσες γραμμές επιλέχτηκαν. Η σύνταξή της έχει ως εξής:

```
mysqli_num_rows ( mysqli_result $result ) : int
```

Η παράμετρος `mysqli_result $result` είναι το αποτέλεσμα της `mysqli_query` με την οποία εκτελέσαμε το `SELECT` ερώτημα.

Σε ερωτήματα που επηρεάζουν τις εγγραφές (UPDATE, INSERT κλπ) μπορούμε να δούμε τον αριθμό των γραμμών που επηρεάστηκαν (στις οποίες έγινε αλλαγή) με την `mysqli_affected_rows`. Αν η εκτέλεση του ερωτήματος έχει αποτύχει επιστρέφει -1 (αν και αυτό πρέπει να το έχουμε ήδη ελέγξει όπως είδαμε στο προηγούμενο βήμα). Η σύνταξή της έχει ως εξής:

```
mysqli_affected_rows ( mysqli $link ) : int
```

Παίρνει σαν παράμετρο τον προσδιοριστή σύνδεσης (`mysqli $link`) που επιστρέφει η `mysqli_connect()`, αν έχουμε πολλές συνδέσεις ανοικτές και θέλουμε να επιλέξουμε σε ποια αναφέρεται η εντολή, και επιστρέφει τον αριθμό των γραμμών που επηρεάστηκαν. Προσοχή πρέπει να δοθεί και εδώ γιατί σε κάποια ερωτήματα ίσως δεν λάβουμε το αποτέλεσμα που περιμέναμε. Πχ αν εκτελέσουμε ένα DELETE χωρίς WHERE συνθήκες και διαγραφούν όλες οι εγγραφές θα επιστραφεί τιμή 0, όπως και αν εκτελέσουμε ένα UPDATE στο οποίο ενώ μπορεί να ταιριάζει N εγγραφές, αλλάζει μόνο K (<N) γιατί είχαν ήδη την ίδια τιμή με την οποία θέλαμε να τις ενημερώσουμε (και η MySQL δεν τα αλλάζει), τότε θα επιστρέψει τιμή K και όχι N.

Όταν το ερώτημα είναι SELECT, και αφού δούμε πόσες εγγραφές επιλέχτηκαν (με την `mysqli_num_rows`), μπορούμε να κάνουμε μια επανάληψη τόσες φορές όσες οι εγγραφές που επιλέχτηκαν και μέσα στην επανάληψη να εκτελούμε κάθε φορά τη `mysqli_fetch_array` ή την `mysqli_fetch_assoc` ή την `mysqli_fetch_row` ή την `mysqli_fetch_object` που μας επιστρέφει την επόμενη γραμμή (με διάφορες μορφές πίνακα οι τρεις πρώτες και ως αντικείμενο η τελευταία) την οποία μπορούμε να τυπώνουμε με HTML. Συγκεκριμένα η `mysqli_fetch_row` επιστρέφει πίνακα που είναι και με αριθμητικούς δείκτες (με πρώτο το 0) και associative, η `mysqli_fetch_assoc` μόνο associative ενώ η `mysqli_fetch_object` μόνο με αριθμητικούς δείκτες. Η σύνταξή τους έχει ως εξής:

```
mysqli_fetch_array ( mysqli\_result $result [, int $resulttype =  
MYSQLI_BOTH ] ) : mixed  
mysqli_fetch_assoc ( mysqli\_result $result ) : array  
mysqli_fetch_row ( mysqli\_result $result ) : mixed  
mysqli_fetch_object ( mysqli\_result $result [, string $class_name =  
"stdClass" [, array $params ]] ) : object
```

Όλες παίρνουν ως παράμετρο το αποτέλεσμα που επιστρέφει το ερώτημα SELECT (από την `mysqli_query`) καθώς και ένα άλλο (η `mysqli_fetch_array()`) που καθορίζει αν ο πίνακας που θα επιστραφεί θα είναι associative (θα έχει δείκτες τα ονόματα των πεδίων) `MYSQLI_ASSOC`, ή μόνο με αριθμητικούς δείκτες `MYSQLI_NUM` ή και τα δύο `MYSQLI_BOTH` και αν δεν δοθεί θεωρείται η `MYSQLI_BOTH`. Άρα για να πάρουμε όλες τις εγγραφές που επιλεχτήκαν πρέπει να καλέσουμε τόσες φορές τη συνάρτηση που θα επιλέξουμε όσες και οι γραμμές που επιλεχτήκαν (`mysqli_num_rows`). Φυσικά η επανάληψη δεν απαιτείται να είναι συγκεκριμένες φορές αφού μπορεί να γίνει μέχρι να μας επιστραφεί NULL που σημαίνει ότι δεν υπάρχει άλλη εγγραφή-γραμμή στα αποτελέσματα του ερωτήματος.

Επίσης μπορούμε αν θέλουμε να πάρου όλα τα αποτελέσματα με την `mysqli_fetch_all()` η σύνταξη της οποίας είαν η εξής:

```
mysqli_fetch_all ( mysqli\_result $result [, int $resulttype = MYSQLI_NUM ] ) : mixed
```

Αυτή μας επιστρέφει όλες τις γραμμές από τα αποτελέσματα σαν πίνακα (associative και με αριθμητικούς δείκτες ανάλογα το δεύτερο όρισμα). Η συγκεκριμένη συνάρτηση έχει κάποιους περιορισμούς που πρέπει να λάβουμε υπόψη μας και γιαυτό συνήθως χρησιμοποιούμε τις άλλες και παίρνουμε μία μία τις εγγραφές του αποτελέσματος.

Ακολουθεί ένα παράδειγμα χρήσης:

```
// Μετά την εκτέλεση ερωτήματος SELECT με την τη mysqli_query που
// επέστρεψε $result
$num_results = mysqli_num_rows($result);
if ($num_results > 0) // αν επιλέχθηκαν γραμμές
{
    echo "<table>";
    echo "<tr><th>Τίτλος</th><th>Εταιρεία</th></tr>";
    // κάνε επανάληψη και κάλεσε κάθε φορά τη mysqli_fetch_array για να
    // πάρεις την επόμενη εγγραφή που επιλέχτηκε με τη μορφή πίνακα
    for ($i=0; $i < $num_results; ++$i)
    {
        // Επέστρεψε στο $r έναν πίνακα με τα στοιχεία της επόμενης γραμμής
        $r = mysqli_fetch_array($result);
        echo "<tr><td>" . $r['title'] . "</td><td>" . $r['company'] .
"</td></tr>";
    }
    echo "</table>";
}
else // αν δεν επιλέχθηκαν γραμμές
    echo "<h2> Δεν βρέθηκε κάποιο film !!</h2>";

// Μετά την εκτέλεση ερωτήματος INSERT που εισάγω μία (1) νέα γραμμή
$num = mysqli_affected_rows($link);
//αν έγινε η εισαγωγή θα επηρεάστηκε μόνο μια γραμμή (αφού μία έβαλα)
if ($num != 1)
    echo "<h2> Δεν έγινε η εισαγωγή του film !!</h2>";
else
    echo "<h2> Έγινε η εισαγωγή του film !!</h2>";
```

5.16.4 Κλείσιμο της σύνδεσης

Κλείνουμε τη σύνδεση που ανοίξαμε στο βήμα 1 χρησιμοποιώντας τη συνάρτηση **mysqli_close** που παίρνει σαν παράμετρο τη σύνδεση αυτή. Η σύνταξή της έχει ως εξής:

```
mysqli_close ( mysqli $link ) : bool
```

Κλείνει τη σύνδεση με τον MySQL server. Παίρνει σαν παράμετρο τον προσδιοριστή σύνδεσης που έχει επιστρέψει η **mysqli_connect()** και επιστρέφει TRUE αν κλείσει επιτυχώς ή FALSE αν δεν κλείσει.

Παράδειγμα:

```
// Κλείσιμο σύνδεσης με MySQL Εξυπηρετητή
mysqli_close($link);
```

5.16.5 Ολοκληρωμένο παράδειγμα των βημάτων

Στο ολοκληρωμένο παράδειγμα αυτό υποθέτω ότι έχω έναν MySQL εξυπηρετητή που τρέχει στο ίδιο H/Y με τον Web Εξυπηρετητή και ο διαχειριστής (username «root») έχει κωδικό «nikos». Σε αυτόν τον MySQL εξυπηρετητή έχω δημιουργήσει μία ΒΔ με όνομα «**videoclub**» η οποία έχει έναν πίνακα «**films**» με 5 πεδία, **fid** (autoincrement), **title**, **company**, **rentperiod** και **cost**, στον οποίο αποθηκεύονται οι ταινίες. Οπότε κάνω τα εξής:

1. Συνδέομαι στον MySQL Εξυπηρετητή στο τρέχον μηχάνημα (μην ξεχνάτε ότι ο κώδικας εκτελείται στον Web Εξυπηρετητή) με το όνομα χρήστη root και κωδικό nikos.
2. Αν η σύνδεση πετύχει επιλέγω την ΒΔ videoclub.

3. Εφόσον συνδέθηκε, εκτελώ ένα ερώτημα επιλογής (SELECT) όπου βρίσκω (τα πεδία fid, title και company) όλες τις ταινίες με τίτλο «CRASH» και τις τυπώνω δημιουργώντας έναν πίνακα HTML ή τυπώνω ένα μήνυμα ότι δεν βρήκα.
4. Μετά εκτελώ ένα ερώτημα εισαγωγής (INSERT) με το οποίο εισάγω μια ταινία με τίτλο «EDISON», εταιρία «COLUMBIA PICTURES», περίοδο ενοικίασης 7 ημέρες και κόστος 1.5€, και ελέγχω αν έγινε σωστά η εισαγωγή.
5. Τέλος κλείνω τη σύνδεση με τον MySQL Εξυπηρετητή.

Κώδικας:

```
/* Βήμα 1: Σύνδεση με τον mysql server που βρίσκεται στο μηχάνημά μας
(localhost) και η οποία απαιτεί όνομα χρήστη "root" και κωδικό "nikos"
και φυσικά και στη Β.Δ. videoclub */

$mysqluser = "root";
$password = "nikos";
$db = "videoclub";

$link = mysqli_connect("localhost", $mysqluser, $password,
$db);
// Αν απέτυχε η σύνδεση βγάλε μήνυμα και τερμάτισε το πρόγραμμα
if (!$link)
    die ('Cannot connect to server: ' . mysqli_connect_error());

/* -----*/
/* Βήμα 2 και 3 για ερώτημα SELECT */
/* Βήμα 2: Εκτέλεση ερωτήματος SELECT στη ΒΔ με δεδομένη τη σύνδεση από
το προηγούμενο βήμα ($link). Υποθέτουμε ότι έχουμε έναν πίνακα με ταινίες
και θέλουμε να επιλέξουμε όλες τις ταινίες με τίτλο 'CRASH' */

$title = "CRASH";
/* Προσέξτε τη χρήση των διπλών εισαγωγικών μέσα στα οποία βάζω την
μεταβλητή $title σε μονά εισαγωγικά. Άρα θα τυπωθούν τα μονά εισαγωγικά
και μέσα τους η τιμή της $title και θα γίνει η σύγκριση με το πεδίο title
στο WHERE clause */
$sql = "SELECT fid, title, company FROM films where title='$title'";
$result = mysqli_query($link, $sql);
//αν απέτυχε η εκτέλεση του ερωτήματος βγάλε μήνυμα και τερμάτισε
if (!$result)
    die ('Cannot select from table: ' . mysqli_error());

/* Βήμα 3: έλεγχος αποτελεσμάτων προηγούμενου ερωτήματος SELECT και
εκτύπωσή τους */

$num_results = mysqli_num_rows($result);
//αν επιστράφηκαν ταινίες, τύπωσε τις δημιουργώντας έναν πίνακα σε HTML
if ($num_results > 0)
{
    echo "<table>";
    echo "<tr><th>Κωδικός</th><th>Τίτλος</th><th>Εταιρεία</th></tr>";
    for ($i=0; $i < $num_results; ++$i)
    {
        // Πάρε στο $r έναν πίνακα με τα στοιχεία της επόμενης γραμμής
        $r = mysql_fetch_array($result);
```



```

        echo "<tr><td>" . $r['fid'] . "</td><td>" . $r['title'] .
"</td><td>" . $r['company'] . "</td></tr>";
    }
    echo "</table>";
}
else
{
    echo "<h2> Δεν βρήκα ταινία που να ταιριάζει!</h2>";
}

/* -----*/
/* Βήμα 2 και 3 για ερώτημα INSERT */
/* Βήμα 2: Εκτέλεση ερωτήματος στη Β.Δ. που συνδεθήκαμε. Υποθέτουμε ότι
έχουμε έναν πίνακα με ταινίες και θέλουμε να εισάγουμε μια νέα ταινία με
τίτλο 'EDISON', της εταιρίας 'COLUMBIA PICTURES', με περίοδο ενοικίασης 7
ημέρες και κόστος 1.5€ */

$title = "EDISON";
$company = "COLUMBIA PICTURES";
$rentperiod = 7;
$cost = 1.5;
//Προσοχή όλα τα πεδία που είναι string (char, varchar, text κ.λ.π.)
πρέπει να μουν σε εισαγωγικά ενώ τα αριθμητικά όχι
$sql = "insert into films (title, company, rentperiod, cost) values
('$title', '$company', $rentperiod, $cost)";

$result = mysqli_query($link, $sql);
//Αν απέτυχε η εισαγωγή του ερωτήματος βγάλε μήνυμα και τερμάτισε
if (!$result)
    die ('Cannot insert in to table : ' . mysqli_error());

/* Βήμα 3: έλεγχος αποτελέσματος προηγούμενου ερωτήματος INSERT και
εκτύπωσή του */
$num = mysqli_affected_rows($link);
//αν έγινε η εισαγωγή θα επηρεάστηκε μια γραμμή (αφού μία έβαλα)
if ($num != 1)
    echo "<h2> Δεν μπόρεσε να εισαχθεί το film!</h2>";
else
    echo "<h2> Το film εισήχθει!</h2>";

/* -----*/
/*Βήμα 4: Κλείσιμο σύνδεσης με MySQL Εξυπηρετητή */
mysqli_close($link);

```

5.17 Μεταφορά Αρχείου από τον Πελάτη στον Εξυπηρετητή

Η μεταφορά ενός αρχείου από τον Εξυπηρετητή στον πελάτη (**Download**) είναι μια τετριμμένη διαδικασία και αρκεί να τυπώσουμε με PHP στην ιστοσελίδα-αποτέλεσμα ένα σύνδεσμο στο αρχείο αυτό.

Πολλές φορές όμως είναι σημαντικό να μπορούμε να κάνουμε την αντίστροφη διαδικασία (**UPLOAD**), δηλαδή να μεταφέρουμε ένα αρχείο από τον Η/Υ του Πελάτη και να το αποθηκεύουμε στον Η/Υ του εξυπηρετητή. Ένα απλό παράδειγμα χρήσης θα ήταν η αποστολή της φωτογραφίας

ενός χρήστη, κατά την εγγραφή του σε ένα forum συζητήσεων, ενώ ένα πιο πολύπλοκο θα περιλάμβανε το κατέβασμα αρχείων, την ενημέρωσή τους και τη μεταφορά τους πάλι στον Εξυπηρετητή σε ένα σύστημα διαμοιρασμού εγγράφων ή σε ένα ηλεκτρονικό πρωτόκολλο.

Για να γίνει αυτό με τη PHP πρέπει να δημιουργήσουμε μια κατάλληλη φόρμα (με συγκεκριμένες προδιαγραφές) για να επιλέξει ο Πελάτης, από το σύστημα αρχείων του, το αρχείο που θέλει να στείλει στον Εξυπηρετητή. Η φόρμα αυτή θα έχει τις προδιαγραφές:

```
<form enctype="multipart/form-data" action="PHP script file"
method="post">
<input type="hidden" name="MAX_FILE_SIZE" value="30000">
Επέλεξε αρχείο που θα στείλεις: <input name="userfile" type="file">
<input type="submit" value="Αποστολή" />
</form>
```

Πρέπει σαν τιμή του **action** να βάλουμε το όνομα του αρχείου που έχει το PHP script στο οποίο θα χειριστούμε την λήψη του αρχείου στον Εξυπηρετητή, το **enctype** να είναι **"multipart/form-data"** και η μέθοδος να είναι **POST**. Το πεδίο με όνομα **"MAX_FILE_SIZE"** έχει σκοπό να περιορίσει το μέγεθος (σε bytes) του αρχείου που στέλνει ο χρήστης και να μην τον αφήσει να στείλει μεγαλύτερο, αν και αυτό, λόγω του ότι δεν είναι ασφαλές, ελέγχεται από την PHP μέσω μιας καθολικής μεταβλητής που θέτουμε στο script, απλώς με τον συγκεκριμένο τρόπο κόβεται αμέσως η μεταφορά χωρίς να περιμένει ο χρήστης να μεταφερθούν τόσα bytes όσα το όριο για να του το διακόψει η PHP.

Το αποτέλεσμα της φόρμας αυτής το στέλνουμε σε ένα PHP script το οποίο θα χρησιμοποιήσει την υπερκαθολική μεταβλητή **\$ _FILES** και τη συνάρτηση **move_uploaded_file** για να πάρει το αρχείο από τον Πελάτη και να το βάλει στη σύστημα αρχείων του εξυπηρετητή. Η σύνταξη της συνάρτησης είναι:

bool move_uploaded_file (string filename, string destination)

Στην πραγματικότητα η συνάρτηση αυτή δεν κάνει τη μεταφορά, η οποία έχει ήδη γίνει με την αποστολή της φόρμας, αλλά ελέγχει αν είναι έγκυρο αρχείο που έχει μεταφερθεί με τον μηχανισμό της PHP (μέσω HTTP POST). Παίρνει σαν παράμετρο το όνομα του αρχείου **string filename** που είναι το προσωρινό όνομα που έχει πάρει αρχείο όταν μεταφέρθηκε, και το **string destination** που είναι το όνομα του αρχείου (μαζί με το μονοπάτι που καθορίζει το φάκελο) στο σύστημα αρχείων του εξυπηρετητή στο οποίο θα αποθηκευτεί το αρχείο.

Για να δούμε τις πληροφορίες του αρχείου που έκανε upload ο Πελάτης χρησιμοποιούμε την υπερκαθολική **\$ _FILES**. Συγκεκριμένα, εφόσον, όπως στην προηγούμενη φόρμα, ο πελάτης έχει χρησιμοποιήσει το όνομα **name="userfile"** στο πεδίο τύπου **file** στη φόρμα, η υπερκαθολική μεταβλητή (πίνακας) **\$ _FILES** περιέχει τα εξής:

Στοιχείο Πίνακα **\$ _FILES**

\$ _FILES['userfile']['name']

\$ _FILES['userfile']['type']

\$ _FILES['userfile']['size']

\$ _FILES['userfile']['tmp_name']

\$ _FILES['userfile']['error']

Περιεχόμενο

Το αρχικό όνομα του αρχείου στο μηχανήμα του Πελάτη.

Ο MIME τύπος του αρχείου, αν ο φυλλομετρητής του πελάτη έχει στείλει αυτή τη πληροφορία. Ένα παράδειγμα θα ήταν **"image/gif"**.

Το μέγεθος (bytes) του αρχείου που έγινε Upload.

Το προσωρινό όνομα του αρχείου στο οποίο έχει αποθηκευτεί το αρχείο στον Εξυπηρετητή.

Ο κωδικός σφάλματος που σχετίζεται με το Upload του

αρχείου. Το ['error'] έχει προστεθεί στην PHP 4.2.0

Οπότε αρκεί να καλέσουμε την **move_uploaded_file**, στην οποία θα δώσουμε σαν πρώτη παράμετρο το προσωρινό όνομα και σαν δεύτερη το όνομα (μαζί με το μονοπάτι φακέλων) με το οποίο θα αποθηκευτεί τελικά το αρχείο, και εφόσον μας επιστρέψει TRUE η μεταφορά και η αντιγραφή είχε επιτυχία, αλλιώς μπορούμε να τυπώσουμε το λάθος.

Ακολουθεί ένα παράδειγμα μιας φόρμας και ενός script για τη μεταφορά ενός αρχείου (θεωρούμε ότι είναι μια εικόνα).

Φόρμα (αρχείο **filesend.html**):

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title> Παράδειγμα Αποστολής Αρχείου </title>
</head>
<body>
<form enctype="multipart/form-data" action="uploadfile.php"
method="POST">
Send this file: <input name="userfile" type="file" /><br />
<input type="submit" value="Αποστολή" />
</form>
</body>
</html>
```

Προσέξτε ότι έχω χρησιμοποιήσει σαν όνομα του πεδίου με τύπο "file" το όνομα "userfile", και ότι στο action έχω πει ότι η φόρμα θα αποστέλλεται στο αρχείο **uploadfile.php**.

PHP script (αρχείο **uploadfile.php**):

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title> Αποτέλεσμα Αποστολής Αρχείου </title>
</head>
<body>
<?php
/* Αν η έκδοση PHP είναι πριν τη 4.1.0, θα πρέπει να χρησιμοποιηθεί η
υπερκαθολική $HTTP_POST_FILES αντί για τη $_FILES */
/* Ορίζω ότι τα αρχεία θα μπαίνουν στον φάκελο ./images/ και θα έχουν το
όνομα που είχαν και στον H/Y του Πελάτη */
$uploadaddir = './images/';
$uploadfile = $uploadaddir . basename($_FILES['userfile']['name']);

/* Ελέγχω αν έγινε μέσω του PHP μηχανισμού (και της HTTP POST μεθόδου) η
μεταφορά του αρχείου και συγχρόνως το αντιγράψω από την προσωρινή του
θέση (προσωρινό όνομα) στο σημείο που όρισα προηγουμένως.
Τυπώνω και ένα μήνυμα ότι όλα πήγαν καλά αν επιστραφεί TRUE αλλιώς τυπώνω
ότι δεν έγινε η μεταφορά σωστά */
if (move_uploaded_file($_FILES['userfile']['tmp_name'], $uploadfile)) {
    echo "<b>Το αρχείο είναι έγκυρο και μεταφέρθηκε επιτυχώς !</b>\n";
} else {
    echo "<b>Αποτυχία μεταφοράς, πιθανή επίθεση upload !</b>\n";
}
```

```

/* Μάλιστα εδώ τυπώνω και όλα τα στοιχεία του πίνακα $_FILES για
debugging (σε κανονικό κώδικα δεν πρέπει να τα βλέπει ο χρήστης).
Η print_r τυπώνει πληροφορίες για μια μεταβλητή σε μορφή εύκολα
αναγνώσιμη για άνθρωπο και είναι πολύ χρήσιμη για να τυπώσεις έναν
ολόκληρο πίνακα και να δεις τα περιεχόμενά του */
echo 'Here is some more debugging info: <br />';
echo "<pre>";
print_r($_FILES);
echo "</pre>";
?>
</body>
</html>

```

5.18 Σύνοδοι (Sessions) για διατήρηση στοιχείων επικοινωνίας

Μία πολύ σημαντική λειτουργία που απαιτείται από τις σημερινές εφαρμογές στον παγκόσμιο ιστό είναι η δυνατότητα να αναγνωρίζεται και να μπορεί να διατηρούνται δεδομένα για έναν πελάτη μεταξύ διαδοχικών προσβάσεων του. Με αυτόν τον τρόπο μπορούμε να έχουμε ελέγξει (authenticate) μία φορά ένα πελάτη (πχ μέσω κωδικών) για τα δικαιώματά του να έχει πρόσβαση σε ένα σύνολο σελίδων/λειτουργιών και από εκεί και πέρα να του επιτρέπεται η πρόσβαση σε αυτές (και μόνο) τις σελίδες/λειτουργίες που έχει δικαίωμα χωρίς τον διαρκή έλεγχο σε κάθε σελίδα (δηλαδή να χρειάζεται να παρέχει τους κωδικούς του σε κάθε νέα σελίδα που προβάλλει).

Η υποστήριξη Συνόδων (Sessions) στην PHP (από την PHP 4.0 και μετά) έχει ακριβώς στόχο τη αναγνώριση και διατήρηση ενός συνόλου δεδομένων μεταξύ διαδοχικών προσβάσεων του ίδιου χρήστη/πελάτη. Η υλοποίηση του συγκεκριμένου μηχανισμού από την PHP, γίνεται είτε μέσω cookies είτε διαδίδεται μέσω του URL (ορίζεται με την κατάλληλη οδηγία στο `php.ini`) και μάλιστα η υποστήριξη είναι ενσωματωμένη και δεν απαιτεί τη χρήση κάποιας εξωτερικής βιβλιοθήκης.

Με τις συνόδους για κάθε πελάτη που έχει πρόσβαση στις ιστοσελίδες μας ανατίθεται ένα μοναδικό αναγνωριστικό συνόδου (session id), μάλιστα μπορούμε να ορίσουμε και ένα σύνολο άλλων δεδομένων που μπορούμε να διατηρούμε για κάθε πελάτη. Η διαδικασία αυτή μπορεί να γίνεται αυτόματα αν έχουμε ορίσει στο `php.ini` την εντολή/οδηγία `session.auto_start` ή να την προκαλέσουμε καλώντας μία συνάρτηση (`session_start()`). Πρέπει να γνωρίζουμε ότι ενώ με τις συνόδους μας δίνεται ένα ισχυρό προγραμματιστικό εργαλείο, **δυστυχώς δεν μας παρέχει ασφάλεια για τα μεταδιδόμενα δεδομένα** (η διάδοση μέσω cookies είναι λίγο πιο ασφαλής από τη διάδοση μέσω του URL), οπότε αν επιθυμούμε ασφάλεια θα πρέπει να εφαρμόσουμε επιπλέον μέτρα όπως η χρήση **HTTPS (SSL/TLS)** για την πρόσβαση στον Web Εξυπηρετητή.

Στις δύο υποενοτήτες που ακολουθούν, δίνονται οι βασικές συναρτήσεις που χρησιμοποιούνται για την υλοποίηση των συνόδων και περιγράφεται η διαδικασία χρήσης των συνόδων, μέσα από ένα τυπικό σενάριο, θεωρώντας ότι δεν έχει ενεργοποιηθεί η αυτόματη εκκίνηση (στο **php.in** η οδηγία είναι `session.auto_start = 0`).

5.18.1 Συναρτήσεις για τις Συνόδους

Στον επόμενο πίνακα φαίνονται οι συναρτήσεις της PHP για χρήση στις συνόδους. Η συμπεριφορά των συναρτήσεων επηρεάζεται από τις ρυθμίσεις που έχουμε για τις συνόδους στο **php.ini**, μάλιστα με πολλές από αυτές μπορούμε να δούμε τις ρυθμίσεις και να τις αλλάξουμε.

Συνάρτηση	Χρήση
<code>session_cache_expire</code> ([string \$new_cache_expire]) : int	Αν δεν δοθεί όρισμα, επιστρέφει το χρόνο (σε λεπτά) που έχουμε ορίσει να

διατηρούνται οι σελίδες μας στην cache του πελάτη και των ενδιαμέσων proxies. Αν δοθεί όρισμα θέτει το χρόνο ίσο με αυτή την τιμή. Φυσικά για να έχει νόημα η τιμή πρέπει το **\$cache_limiter** (δες επόμενη συνάρτηση) να μην είναι **nocache** γιατί αν είναι **nocache** δεν γίνονται cache οι σελίδες μας από τον πελάτη και τους ενδιαμέσους proxies. Η αντίστοιχη οδηγία στο αρχείο **php.ini** είναι η **session.cache_expire**

session_cache_limiter ([string \$cache_limiter]) : **string**

Αν δεν δοθεί όρισμα, επιστρέφει το όνομα του cache_limiter, ενώ αν δοθεί {nocache, private, private_no_expire, public} το θέτει σε αυτή τη τιμή. Με το cache_limiter ορίζουμε (στέλνονται μέσω των HTTP headers) πως ο πελάτης (αλλά και ενδιαμέσοι proxies) μπορούν να κάνουν cache τα περιεχόμενα των σελίδων. Η προεπιλεγμένη τιμή (nocache) δεν τους επιτρέπει να τα κάνουν cache. Η αντίστοιχη οδηγία στο αρχείο **php.ini** είναι η **session.cache_limiter**.

session_write_close (void) : **bool**

Γράφει τα δεδομένα της τρέχουσας συνόδου και την τερματίζει.

session_commit (void) : **bool**

Είναι alias της **session_write_close()**.

session_encode (void) : **string**

και

session_decode (string \$data) : **bool**

Η πρώτη επιστρέφει ένα string με τα δεδομένα της συνόδου, ενώ η δεύτερη θέτει τα δεδομένα της τρέχουσας συνόδου στο string. Θα μπορούσαμε να χρησιμοποιήσουμε την πρώτη, για λόγους debugging, για να δούμε τι μεταβλητές (και τις τιμές τους) έχουμε φυλάξει στην τρέχουσα σύνοδο.

session_get_cookie_params (void) : **array**

Επιστρέφει ένα πίνακα με τις παραμέτρους του cookie της τρέχουσας συνόδου. Συγκεκριμένα επιστρέφει 4 στοιχεία:

- “lifetime”, η ζωή του cookie σε δευτερόλεπτα
- “path”, το μονοπάτι που αποθηκεύονται οι πληροφορίες
- “domain”, η περιοχή του cookie
- “secure”, αν το cookie πρέπει να μεταφέρεται μόνο πάνω από ασφαλή σύνδεση

Οι αντίστοιχες οδηγίες στο αρχείο **php.ini** είναι οι **session.cookie_lifetime**, **session.cookie_path**, **session.cookie_domain** και **session.cookie_secure**.

session_set_cookie_params (int \$lifetime [, string \$path [, string \$domain [, bool \$secure = FALSE [, bool \$httponly = FALSE]]]) : **bool**

ή

session_set_cookie_params (array \$options) : **bool**

Θέτει τις παραμέτρους του cookie της τρέχουσας συνόδου στις τιμές των ορισμάτων τους.

Οι αντίστοιχες οδηγίες στο αρχείο **php.ini** είναι οι **session.cookie_lifetime**, **session.cookie_path**, **session.cookie_domain** και **session.cookie_secure**.

session_id ([string \$id]) : **string**

Αν πάρει όρισμα τότε θέτει το id της τρέχουσας συνόδου, αλλιώς το επιστρέφει. Για να τεθεί το νέο id πρέπει να κληθεί πριν τη **session_start()**.

session_is_registered (string \$name) : **bool**

Επιστρέφει TRUE αν έχει εγγραφεί (register) μεταβλητή με όνομα το όρισμά της στην τρέχουσα σύνοδο, αλλιώς επιστρέφει FALSE. Φυσικά μπορείς να εξετάσεις κατευθείαν και τον πίνακα (υπερκαθολική) **\$_SESSION** για να δεις αν έχει τη μεταβλητή που θέλεις.

session_register ([mixed](#) \$name [, [mixed](#) \$...]) : **bool**

Παίρνει ένα μεταβλητό αριθμό ορισμάτων που μπορεί να είναι string ή πίνακες με ονόματα και εγγράφει τις αντίστοιχες μεταβλητές στην τρέχουσα σύνοδο. Επιστρέφει TRUE αν εγγραφούν όλες οι μεταβλητές. Φυσικά μπορείς να χρησιμοποιείς κατευθείαν τον πίνακα (υπερκαθολική) \$_SESSION για να βάζεις νέες μεταβλητές.

session_module_name ([string \$module]) : **string**

Αν πάρει όρισμα θέτει το όνομα του module σε αυτό αλλιώς το επιστρέφει.

session_name ([string \$name]) : **string**

Αν πάρει όρισμα θέτει το όνομα της συνόδου σε αυτό αλλιώς το επιστρέφει. Η αντίστοιχη οδηγία στο αρχείο php.ini είναι η **session.name**.

session_regenerate_id ([bool \$delete_old_session = **FALSE**]) : **bool**

Ανανεώνει το id της συνόδου με ένα καινούριο και επιστρέφει TRUE αν έγινε επιτυχώς η ανανέωση.

session_save_path ([string \$path]) : **string**

Αν πάρει όρισμα τότε θέτει το μονοπάτι αποθήκευσης δεδομένων των συνόδων, αλλιώς το επιστρέφει. Για να τεθεί το νέο μονοπάτι πρέπει να κληθεί πριν τη session_start(). Η αντίστοιχη οδηγία στο αρχείο php.ini είναι η **session.save_path**.

session_set_save_handler ([callable](#) \$open , [callable](#) \$close , [callable](#) \$read , [callable](#) \$write , [callable](#) \$destroy , [callable](#) \$gc [, [callable](#) \$create_sid [, [callable](#) \$validate_sid [, [callable](#) \$update_timestamp]]) : **bool**

ή (για (PHP >=5.4):

session_set_save_handler (object \$sessionhandler [, bool \$register_shutdown = **TRUE**]) : **bool**)

Θέτει τις συναρτήσεις που θα καλούνται σε επίπεδο χρήστη για να γίνεται η αποθήκευση και ανάκτηση των δεδομένων. Τα ορίσματα είναι τα ονόματα των συναρτήσεων, τις οποίες πρέπει να έχουμε υλοποιήσει. Η PHP έχει έναν προκαθορισμένο **χειριστή (handler)** που είναι ο **files** και έχει ορισμένες τις δικές του συναρτήσεις. Η οδηγία στο αρχείο php.ini που ορίζεις τον χειριστή είναι είναι η **session.save_handler**

session_start ([array \$options = array()]) : **bool**

Η βασικότερη συνάρτηση που χρειαζόμαστε. Ξεκινάει τη δημιουργία μίας νέας συνόδου ή στην περίπτωση που έχει δημιουργηθεί, ανακτά όλα τα δεδομένα της συνόδου που υφίσταται (για να δει αν υφίσταται και να πάρει το id εξετάζει ανάλογα με το μηχανισμό υλοποίησης το cookie που στέλνει ο πελάτης ή τα δεδομένα που αποστέλλει στην αίτησή του). Καλείται πριν από κάθε (σχεδόν) ενέργεια που αφορά τον έλεγχο της συνόδου και τον χειρισμό των δεδομένων της). Προσοχή πρέπει να δοθεί στο ότι **πρέπει να καλείται πριν οποιοσδήποτε κώδικας HTML έχει βγει ως έξοδο** γιατί αλλιώς έχουν ήδη σταλεί οι headers στον πελάτη (οπότε δεν μπορεί να δημιουργήσει/αποκαταστήσει τα δεδομένα της συνόδου). Επιστρέφει TRUE αν όλα πάνε καλά αλλιώς επιστρέφει FALSE.

session_unset (void) : **bool**

Ελευθερώνει όλες τις μεταβλητές που έχουν εγγραφεί στην τρέχουσα σύνοδο. Αν για να δημιουργήσουμε τις μεταβλητές έχουμε χρησιμοποιήσει κατευθείαν τον πίνακα \$_SESSION για να δημιουργούμε τις μεταβλητές, τότε πρέπει να τις ελευθερώνουμε με χρήση της απλής συνάρτησης unset (πχ unset(\$_SESSION['όνομα μεταβλητής'])).

Προσοχή πρέπει να δοθεί, όπως αναφέραμε, στην κλήση της **session_start** η οποία **πρέπει να γίνει πριν την έξοδο οποιουδήποτε κώδικά μας, δεν επιτρέπεται ούτε ένα <space> πριν**. Πχ το παρακάτω είναι λάθος:

```
<html>
<?php
```



```
echo " ";  
/* Λάθος αφού ήδη έχουμε βγάλει έξοδο την ετικέτα <html> αλλά και το  
space " ", άρα έχουν σταλεί οι headers */  
session_start();  
?>
```

Όπως είδαμε και με τη **header**, μπορούμε να εξετάσουμε αν έχουν σταλεί ήδη οι headers με τη συνάρτηση **headers_sent** και να βρούμε και σε ποιο αρχείο και γραμμή έγινε πρώτη φορά.

5.18.2 Διαδικασία χρήσης των συνόδων

Στην υποενότητα αυτή θα δοθεί η διαδικασία με την οποία μπορούμε να χρησιμοποιήσουμε τις συνόδους. Η διαδικασία αυτή διαφέρει αν οι ιστοσελίδες μας είναι μη προστατευμένες από την περίπτωση που είναι προστατευμένες και γι' αυτό θα παρουσιαστούν ξεχωριστά σαν δύο υποενότητες. Στα παραδείγματα δεν χρησιμοποιώ καθόλου τις **session_register** και **session_is_registered** αλλά προτιμώ να δουλεύω άμεσα (και για μεγαλύτερη ασφάλεια) με τον πίνακα **\$_SESSION**. Σε κάθε περίπτωση θα παρουσιάσουμε ένα απλό μοντέλο που θα μπορούμε να ελέγχουμε ποιος είναι ο χρήστης, αν έχει πιστοποιηθεί και άρα μπορεί να βλέπει κάποιες σελίδες κ.λ.π.. Όμως με τον ίδιο τρόπο που θα δούμε εδώ μπορεί ο χρήστης να εφαρμόσει και επίπεδα εξουσιοδότησης και γενικότερα συνθετότερα σχήματα ελέγχου και διαχείρισης του τι ενέργειες μπορεί να κάνει ο χρήστης αλλά και τι περιεχόμενο του παρουσιάζετε.

5.18.2.1 Μη προστατευμένες ιστοσελίδες

Όταν οι ιστοσελίδες μας είναι μη προστατευμένες, πχ σε ένα e-shop, ή σε κρατήσεις εισιτηρίου στην πραγματικότητα αυτό που θέλουμε να κάνουμε είναι να αναγνωρίζουμε ποιος είναι ο πελάτης ώστε να του παρουσιάζουμε/ζητάμε συγκεκριμένα δεδομένα που τον αφορούν. Μία τέτοια περίπτωση θα ήταν σε ένα e-shop να εμφανίζουμε στον πελάτη το καλάθι με τις αγορές που μέχρι εκείνη τη στιγμή έχει κάνει, καθώς και τη λίστα των προϊόντων που στην συγκεκριμένη στιγμή έχει διαλέξει να προβάλλει για να επιλέξει να αγοράσει.

Ο κώδικας που θα έχουμε για τις συνόδους θα πρέπει να μπει σε όλες τις ιστοσελίδες μας. Με τον κώδικα αυτό πρέπει να καταλαβαίνουμε **αν είναι η πρώτη φορά** που βλέπει τις ιστοσελίδες μας και να κάνουμε τις κατάλληλες ενέργειες (να δημιουργήσουμε τις κατάλληλες μεταβλητές για τη σύνοδο) ώστε τις άλλες φορές να καταλαβαίνουμε ότι δεν είναι η πρώτη φορά και με κάποιο τρόπο να τον αναγνωρίζουμε (συνήθως το session id), ώστε πχ να δείχνουμε το δικό του καλάθι αγορών μέχρι τη στιγμή αυτή.

Ένας απλοϊκός τρόπος που θα μπορούσαμε να το χειριστούμε σε αυτή την περίπτωση φαίνεται στο επόμενο παράδειγμα. Όπως αναφέραμε ο συγκεκριμένος κώδικας που ακολουθεί θα πρέπει να μπει σε όλες τις ιστοσελίδες (φυσικά η κάθε ιστοσελίδα θα έχει και δικό της κώδικα PHP και XHTML κλπ):

```
<?php  
/* Δημιουργώ τη σύνοδο (αν είναι η πρώτη φορά που βλέπει τις ιστοσελίδες  
μου) ή αν έχει ήδη μπει, απλώς φορτώνονται όλα τα δεδομένα που έχει η  
σύνοδος */  
session_start();  
  
/* Πρέπει να καταλάβω αν είναι η πρώτη φορά ή όχι. Για να το κάνω αυτό  
εξετάζω αν έχει ήδη γίνει register μια συγκεκριμένη μεταβλητή την οποία  
την πρώτη φορά την κάνω πάντα register και την χρησιμοποιώ για να  
ξεχωρίσω τον πελάτη. Αποφασίζω αυτή η μεταβλητή να έχει το όνομα "user"  
*/
```

```

if (!isset ($_SESSION["user"])) // Είναι η πρώτη φορά
{
/* Αφού είναι η πρώτη φορά πρέπει να δημιουργήσω τις μεταβλητές που θα
περιέχουν τα δεδομένα που θα χρειάζομαι για τον χρήστη. Οποσδήποτε σε
αυτά θα είναι και η μεταβλητή με όνομα "user". Βάζω στη μεταβλητή user
την τιμή του session id που θα είναι μοναδική (φυσικά μπορώ να βάλω μια
άλλη μοναδική τιμή) για να τον αναγνωρίζω */
    $_SESSION["user"] = session_id();
}
/* Από εδώ και πέρα, είτε ήταν η πρώτη φορά είτε όχι, ότι θέλω να κάνω θα
το κάνω χρησιμοποιώντας τη μεταβλητή $_SESSION["user"] ώστε να το κάνω
για τον συγκεκριμένο χρήστη.
Πχ αν ήταν ιστοσελίδες ενός e-shop:
Τότε αν επέλεξε να κάνει μια αγορά, μπορώ να καλώ μια συνάρτηση με την
οποία θα συνδέομαι σε μια ΒΔ MySQL και θα εισάγω το προϊόν που αγόρασε
(τα στοιχεία του οποίου όπως έχουμε μάθει θα μου έχουν σταλεί από την
προηγούμενη ιστοσελίδα με POST ή GET και θα τα πάρω από τους αντίστοιχους
πίνακες $_POST και $_GET) σε έναν πίνακα της Β.Δ. φροντίζοντας ότι αυτός
θα έχει και ένα πεδίο στο οποίο θα αποθηκεύω την τιμή $_SESSION["user"]
για τη συγκεκριμένη αγορά.
Επίσης μπορώ κάθε φορά να καλώ μια συνάρτηση που θα συνδέομαι στη ΒΔ
MySQL και θα επιλέγω όλα τα προϊόντα από τον συγκεκριμένο πίνακα αγορών
που στο συγκεκριμένο πεδίο έχουν τιμή $_SESSION["user"] και θα του τα
προβάλλω ως το καλάθι που μέχρι τώρα έχει.
. . . */
?>

```

5.18.2.2 Προστατευμένες ιστοσελίδες

Όταν οι ιστοσελίδες μας είναι προστατευμένες, πχ στο web-mail ή σε ένα forum κλπ, τότε η εισαγωγή του χρήστη (login) γίνεται μέσω συγκεκριμένης ιστοσελίδας στην οποία στέλνονται τα πιστοποιητικά του (συνήθως όνομα χρήστη και κωδικός που έχει βάλει σε μια φόρμα) και η οποία έχει τον κατάλληλο κώδικα για να πιστοποιήσει τον χρήστη και να ξεκινήσει τη σύννοδο. Στην περίπτωση αυτή λοιπόν θα ήταν καλύτερα να διακρίνουμε τη διαδικασία σε τρία βήματα:

1. Ο πελάτης εισάγεται στις συγκεκριμένες ιστοσελίδες.
2. Ο πελάτης περιηγείται στις συγκεκριμένες ιστοσελίδες.
3. Ο πελάτης φεύγει από τις ιστοσελίδες.

Στις επόμενες υποενότητες θα αναλύσουμε τα βήματα και θα δώσουμε παραδείγματα.

5.18.2.2.1 Ο πελάτης εισάγεται στις συγκεκριμένες ιστοσελίδες

Ένας τρόπος που θα μπορούσαμε να το χειριστούμε σε αυτή την περίπτωση φαίνεται στο επόμενο παράδειγμα. Ο κώδικας που ακολουθεί **θα μπει μόνο στην ιστοσελίδα που θα πιστοποιεί τον χρήστη**. Αν τα στοιχεία του ο χρήστης τα βάζει σε μια φόρμα τότε η ιστοσελίδα αυτή θα είναι αυτή που είναι στο action της φόρμας. Φυσικά μπορεί για να παίρνουμε τα στοιχεία του χρήστη να χρησιμοποιούμε και την HTTP αναγνώριση μέσω της συνάρτησης header (βλέπε υποενότητα **HTTP αναγνώριση με τη PHP παραπάνω**). Στον κώδικα που ακολουθεί υποθέτουμε ότι μας έχουν σταλεί από μια φόρμα:

```

<?php
/* Πάρε τα στοιχεία του χρήστη (από τον $_POST ή τον $_GET, ανάλογα τη
μέθοδο που εστάλησαν) και πιστοποίησε τα.

```

Τα στοιχεία των χρηστών θα μπορούσαμε να τα έχουμε σε έναν πίνακα μιας ΒΔ της MySQL, οπότε τότε θα μπορούσαμε να συνδεθούμε σε αυτή και να δούμε αν υπάρχει χρήστης με το συγκεκριμένο όνομα και κωδικό και από εκεί να ανακτήσουμε και το μοναδικό id του χρήστη */

Κώδικας PHP για τα παραπάνω ...

```
/* Βάζω στη μεταβλητή $pistop το αποτέλεσμα της πιστοποίησης. Αν ο
χρήστης δεν πιστοποιήθηκε (πχ έδωσε λάθος κωδικό) τότε τυπώνουμε ένα
μήνυμα λάθους και ξαναβγάζουμε τη φόρμα για να ξαναπροσπαθήσει */
if ($pistop == FALSE)
{

}
/* Αν πιστοποιήθηκε δημιουργούμε τη σύνοδο και εγγράφουμε τις μεταβλητές
που θέλουμε στην σύνοδο και τους δίνουμε τις τιμές εκείνες που θα μας
επιτρέψουν στις υπόλοιπες ιστοσελίδες μας να αναγνωρίζουμε τον χρήστη και
να έχουμε και τα στοιχεία του που χρειαζόμαστε. Πχ θα μπορούσαμε να
έχουμε το μοναδικό id του χρήστη στον πίνακα των χρηστών, το όνομά του,
καθώς και μια φωτογραφία του (το μονοπάτι της) την οποία μας είχε κάνει
upload κατά την εγγραφή του. Όλα αυτά θα μπορούσαμε να τα κρατήσουμε σε
μεταβλητές από την διαδικασία της πιστοποίησης προηγουμένως που
συνδεθήκαμε και είδαμε τα στοιχεία του πίνακα χρηστών.*/
else
{
/* Δημιουργώ τη σύνοδο (αν είναι η πρώτη φορά που βλέπει τις ιστοσελίδες
μου) ή αν έχει ήδη μπει, απλώς φορτώνονται όλα τα δεδομένα που έχει η
σύνοδος */
session_start();

/* Οι μεταβλητές $user_id, $user_name, $user_photo_path έχουν τεθεί από
την διαδικασία της πιστοποίησης προηγουμένως στις τιμές που είχε η
εγγραφή του χρήστη με το συγκεκριμένο όνομα και κωδικό που μας στάλθηκε
από τη φόρμα */
$_SESSION["userid"] = $user_id;
$_SESSION["username"] = $user_name;
$_SESSION["userphoto"] = $user_photo_path;
/* Από εδώ και κάτω μπορούμε να βγάλουμε ένα μήνυμα καλωσορίσματος ή/και
ένα μενού επιλογών για τους πιστοποιημένους χρήστες (προστατευμένες
σελίδες) κ.λ.π. */
Κώδικας PHP για τα παραπάνω ...

}
?>
```

5.18.2.2.2 Ο πελάτης περιηγείται στις συγκεκριμένες ιστοσελίδες

Σε όλες τις προστατευμένες ιστοσελίδες θα πρέπει να γίνεται έλεγχος αν ο πελάτης έχει πιστοποιηθεί. Αυτό μπορεί να γίνει ξεκινώντας τη σύνοδο και κοιτώντας αν μέσα στα δεδομένα της υπάρχουν οι μεταβλητές (μία είναι αρκετή) τις οποίες προσθέσαμε όταν έγινε σωστά η πιστοποίηση του κάποια χρονική στιγμή πριν. Ακολουθεί ένας απλός τρόπος, σε σχέση με την προηγούμενη παράγραφο, με κώδικα που πρέπει να υπάρχει σε όλες τις προστατευμένες ιστοσελίδες:

<?php

```

/* Αν υπήρχε η σύννοδος φόρτωσε τα δεδομένα της, αλλιώς δημιούργησε μια
νέα σύννοδο με καθόλου δεδομένα */
session_start();

/* Αν δεν υπάρχει η μεταβλητή $_SESSION["userid"] πάει να πει ότι δεν
υπήρχε η σύννοδος δηλαδή δεν είναι εξουσιοδοτημένος και άρα δεν έχει
δικαίωμα να δει τις ιστοσελίδες */
if (!isset($_SESSION["userid"]))
{
?>
<html>
...

<?php echo "<h1> Απαγορεύεται η πρόσβαση !!</h1>"; ?>
...
</html>

<?php
}
/* Αν είναι εξουσιοδοτημένος κάνε ότι θέλεις έχοντας και τα στοιχεία που
θέλεις (στον πίνακα $_SESSION) για να αναγνωρίζεις τον συγκεκριμένο
χρήστη */
else
{
...

/* Καλό είναι σε κάθε σελίδα να υπάρχει και ένας σύνδεσμος που θα οδηγεί
σε μια σελίδα που θα τερματίζει τη σύννοδο */
echo "[ <a href='logout.php'>Logout " . $_SESSION['username'] . " </a>
]\n";
}
?>

```

5.18.2.2.3 Ο πελάτης φεύγει από τις ιστοσελίδες

Εδώ υποθέτουμε ότι ο χρήστης θέλει να φύγει (logout) και πατάει σε κάποιον σύνδεσμο που τον οδηγεί στην συγκεκριμένη ιστοσελίδα (όπως τον σύνδεσμο παραπάνω) ενδεικτικό κώδικα της οποίας παραθέτουμε παρακάτω. Ο σκοπός του κώδικα εδώ είναι να καταστρέψουμε την σύννοδο ώστε να μην υπάρχουν τα δεδομένα και να αναγνωρίζουμε ότι δεν είναι πια πιστοποιημένος και πρέπει να ξαναπιστοποιηθεί.

```

<?php
/* Αν υπήρχε η σύννοδος φόρτωσε τα δεδομένα της, αλλιώς δημιούργησε μια
νέα σύννοδο με καθόλου δεδομένα */
session_start();

/* Αν δεν υπάρχει η μεταβλητή $_SESSION["userid"] πάει να πει ότι δεν
υπήρχε η σύννοδος δηλαδή δεν είναι πιστοποιημένος και άρα δεν έχει
δικαίωμα να δει τις ιστοσελίδες */
if (!isset($_SESSION["userid"]))
{
?>
<html>
...

<?php echo "<h1> Απαγορεύεται η πρόσβαση !!</h1>"; ?>

```

```

...
</html>

<?php
}
/* Αν είναι εξουσιοδοτημένος επέλεξε να κάνει logout οπότε πρέπει να
καταστρέψουμε τη σύνοδο */
else
{
$user_name = $_SESSION["username"];
unset($_SESSION["userid"]);
unset($_SESSION["username"]);
unset($_SESSION["userphoto"]);

session_destroy();
?>
<html>
...

<?php
echo "<h1>You have successfully logged out $user_name </h1>\n";
?>
...
</html>

<?php
}
?>

```

5.19 Παραδείγματα Εφαρμογών

5.19.1 Παράδειγμα 1

Στο παράδειγμα αυτό υπάρχει μια ιστοσελίδα **example1.html** με μία φόρμα στην οποία δίνεις ένα κείμενο και δύο αριθμούς που καθορίζουν τις διαστάσεις του πίνακα που θέλεις να δημιουργήσεις.

Τα στοιχεία αυτά στέλνονται στο αρχείο **example1.php** με τη μέθοδο **POST** το οποίο τα διαβάζει χρησιμοποιώντας PHP και εμφανίζει μια επικεφαλίδα με το κείμενο που έδωσες καθώς και έναν πίνακα με τόσες γραμμές και στήλες όσες οι δύο αριθμοί που έδωσες και περιεχόμενο τον αριθμό της γραμμής και της στήλης.

Αρχείο example1.html

```

<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title> Παράδειγμα 1 HTML Page </title>
</head>
<body>
<h1>Παράδειγμα 1 HTML Page</h1>
<form action="example1.php" method="post">
Κείμενο: <input type="text" maxlength="50" name="txt1" /> <br />
Γραμμές: <input type="text" maxlength="2" name="rows" /> <br />
Στήλες: <input type="text" maxlength="2" name="cols" /> <br />
<input type="submit" value="Αποστολή" />

```

```
</form>
</body>
</html>
```

Αρχείο example1.php

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title> Παράδειγμα 1 PHP Page </title>
</head>
<body>
<?php
/* Παίρνω τις 3 τιμές που μου στάλθηκαν με POST από τον Υπερκαθολικό
πίνακα $_POST. Εδώ για απλότητα δεν κάνω ελέγχους αλλά κανονικά θα έπρεπε
να ελέγξω με isset(), empty() κ.λ.π. */
$txt = $_POST['txt1'];
$r = $_POST['rows'];
$c = $_POST['cols'];

// Τυπώνω το κείμενο επικεφαλίδα 1
echo "<h1>" . $txt . "</h1>";

// Δημιουργώ έναν πίνακα με $r γραμμές και $c στήλες με κείμενο τον
τρέχον αριθμό
// γραμμής και στήλης
echo "<table>";
for ($i=1 ; $i<=$r ; ++$i)
{
    echo "<tr>";
    for ($j=1;$j<=$c;++$j)
    {
        echo "<td>";
        echo "row " . $i . ", column " . $j;
        echo "</td>";
    }
    echo "</tr>";
}
echo "</table>";
?>
</body >
</html>
```

5.19.2 Παράδειγμα 2

Στο παράδειγμα αυτό υπάρχει μια ιστοσελίδα **example2.html** με μία φόρμα στην οποία δίνεις το όνομα, το επώνυμο και την ηλικία ενός ατόμου. Τα στοιχεία αυτά στέλνονται στο αρχείο **example2.php**, με τη μέθοδο POST, το οποίο τα διαβάζει χρησιμοποιώντας PHP και τα εισάγει σε έναν πίνακα επικοινωνώντας με μία βάση MySQL. Μάλιστα κάνω και κάποιους ελέγχους μήπως δεν έχει συμπληρωθεί σωστά κάποιο από τα πεδία ή έχει συμπληρωθεί με κενά. Επιπλέον μετατρέπω κατάλληλα τα περιεχόμενα των μεταβλητών που χρησιμοποιώ στα ερωτήματα για να μην έχω πρόβλημα με τυχόν ειδικούς χαρακτήρες που μπορεί να μου δημιουργήσουν πρόβλημα (μονά, διπλά quotes και \).

Ο MySQL Εξυπηρετητής στον οποίο συνδέεται βρίσκεται στον ίδιο H/Y (localhost) με τον Web Εξυπηρετητή και ο λογαριασμός του root δεν έχει κωδικό (κάτι που δεν είναι ασφαλές για πραγματική χρήση). Επίσης, για να λειτουργήσει το παράδειγμα, πρέπει στον MySQL Εξυπηρετητή να υπάρχει μία ΒΔ με όνομα **test** στην οποία θα υπάρχει ένας πίνακας με όνομα **testtable** που θα έχει τα εξής πεδία:

onoma char(50), primary key

eponimo char(50)

age int

Αρχείο example2.html

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title> Παράδειγμα 2 PHP Page </title>
</head>
<body>
<form action="example2.php" method="post">
Όνομα: <input type="text" maxlength="50" name="firstname" /> <br />
Επώνυμο: <input type="text" maxlength="50" name="lastname" /> <br />
Ηλικία: <input type="text" maxlength="2" name="age" /> <br />
<input type="submit" value="Εισαγωγή" />
</form>
</body>
</html>
```

Αρχείο example2.php

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title> Παράδειγμα 2 PHP Page </title>
</head>
<body>
<?php
//Βάζω σε μεταβλητές τα στοιχεία σύνδεσης στον MySQL server
$mysqluser = "root";
$password = "";
$mysqladb = "test";

/* Ελέγχω μήπως κάποιο στοιχείο της φόρμα δεν συμπληρώθηκε ή δεν
συμπληρώθηκε σωστά. Αν υπάρχει λάθος βάζω ένα μήνυμα στην $error
την οποία έχω στην αρχή ίση με "", οπότε μετά θα ξέρω αν υπήρχε
λάθος και ποιο ή ποια ήταν. */
$error = "";

// Ελέγχω μήπως δεν στάλθηκε ή δεν συμπληρώθηκε το όνομα
if (!isset($_POST['firstname']) or empty(trim($_POST['firstname'])))
{
    $error = $error . "<li> Το όνομα είναι κενό. </li>";
}
else
    $on = $_POST['firstname'];

// Ελέγχω μήπως δεν στάλθηκε ή δεν συμπληρώθηκε το επώνυμο
```

```
if (!isset($_POST ['lastname']) or empty(trim($_POST ['lastname'])))
{
    $error = $error . "<li> Το επώνυμο είναι κενό. </li>";
}
else
    $sep = $_POST ['lastname'];

// Ελέγχω για την ηλικία εξετάζοντας επιπλέον μήπως δεν είναι
// θετικός αριθμός (>0)
if (!isset($_POST ['age']) or empty(trim($_POST ['age'])))
{
    $error = $error . "<li> Η ηλικία είναι κενή. </li>";
}
else
{
    $ilikia = trim($_POST ['age']);
    if (ctype_digit($ilikia)== 0)
    {
        $error = $error . "<li> Η ηλικία δεν είναι αποδεκτή. </li>";
    }
    else
    {
        $ilkiaval = intval($ilikia);
        if ($ilkiaval <= 0)
            $error = $error . "<li> Η ηλικία πρέπει να είναι μεγαλύτερη
από 0</li>";
    }
}

// Αν συνέβη λάθος τότε η μεταβλητή $error δεν είναι κενή
if (trim($error)!="")
{
    echo "<ol>";
    echo $error;
    echo "</ol>";
    echo "<br /><b> Η εισαγωγή απέτυχε !! </b>";
}
else // αλλιώς ξεκινάω τη διαδικασία για εισαγωγή στην Β.Δ.
{
    // Συνδέομαι με τον mysql server και τη Β.Δ.
    $link = mysqli_connect("localhost", $mysqluser, $mysqlpassword,
$mysqldb);
    if (!$link)
        die ('Αποτυχία σύνδεσης στον MySQL server: ' .
mysqli_connect_error());

    // Θέτω σωστά το encoding
    mysqli_set_charset($link, "utf8");

    // Πρώτα κάνω μετατροπή για να μην έχω πρόβλημα με ειδικούς
    // χαρακτήρες που ίσως έχουν προστεθεί στις μεταβλητές
    $on = mysqli_real_escape_string($link, $on);
    $sep = mysqli_real_escape_string($link, $sep);

    // Επιλέγω τα στοιχεία του testtable με το όνομα που μου δόθηκε
```

```

$sql = "SELECT * FROM testtable where onoma='$on'";
$result = mysqli_query($link, $sql);

if (!$result)
    die ('Δεν μπορώ να κάνω SELECT: ' . mysqli_error($link));

// Παίρνω τον αριθμό των εγγραφών που επιλέχτηκαν
$num = mysqli_num_rows($result);
// Αν επιλέχτηκε έστω και μία σημαίνει ότι υπάρχει και δεν
// πρέπει να το ξαναεισάγω αφού το όνομα είναι κλειδί
if ($num >= 1)
{
    // close connection
    mysqli_close($link);
    $error = $error . "<li> Υπάρχει ήδη αυτό το όνομα. </li>";
    echo "<ol>" . $error . "</ol>";
    echo "<br /><b> Η εισαγωγή απέτυχε. </b>";
}
else // Αλλιώς (δεν υπάρχει ήδη το όνομα) το εισάγω
{
    $sql = "insert into testtable (onoma, eponimo, age) values
('$on', '$sep', $ilikiaval)";
    $result = mysqli_query($link, $sql);
    if (!$result)
        die ('Δεν μπορώ να κάνω INSERT : ' .
mysqli_error($link));

    // Ελέγχω πόσες εγγραφές εισαχθήκαν
    $num = mysqli_affected_rows($link);
    if ($num != 1)
    {
        die ('Δεν μπορώ να Εισάγω: ' . mysqli_error($link));
    }
    else
    {
        // close connection
        mysqli_close($link);
        echo "<b> Η εισαγωγή πέτυχε !!</b> για την εγγραφή
με τα στοιχεία: <br />";
        echo $on . "<br />";
        echo $sep . "<br />";
        echo $ilikiaval . "<br />";
    }
}
}
?>
</body>
</html>

```

5.19.3 Παράδειγμα 3

Στο παράδειγμα αυτό υπάρχει μια ιστοσελίδα **example3.html** με μία φόρμα στην οποία δίνεις το όνομα ή μια έκφραση με την οποία συγκρίνεται το όνομα του ατόμου (πχ % για όλους ή N% για όσους το όνομα αρχίζει από N). Τα στοιχεία αυτά στέλνονται στο αρχείο **example3.php**, με τη μέθοδο GET, το οποίο τα διαβάζει χρησιμοποιώντας PHP και κάνει ένα ερώτημα στον MySQL

server και επιστρέφει (προβάλλει) όλες τις εγγραφές που ταιριάζουν δημιουργώντας έναν πίνακα σε HTML όπου σε κάθε γραμμή είναι τα στοιχεία και μίας από τις εγγραφές που ταιριάζουν στην έκφραση αναζήτησης.

Ο MySQL Εξυπηρετητής στον οποίο συνδέεται βρίσκεται στον ίδιο H/Y (localhost) και ο λογαριασμός του root δεν έχει κωδικό (κάτι που δεν είναι ασφαλές για πραγματική χρήση). Επίσης, για να λειτουργήσει το παράδειγμα, πρέπει στον MySQL Εξυπηρετητή να υπάρχει μία Β.Δ. με όνομα test στην οποία θα υπάρχει ένας πίνακας με όνομα testtable που θα έχει τα εξής πεδία:

onoma char(50), primary key

eponimo char(50)

age int

Αρχείο example3.html

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title> Παράδειγμα 2 HTML Page </title>
</head>
<body>
<form action="example3.php" method="get">
Όνομα ή λογική έκφραση που θα συγκριθεί με το πεδίο όνομα: <input
type="text" maxlength="50" name="firstname" />
<br />
<input type="submit" value="Αναζήτηση-Προβολή" />
</form>
</body>
</html>
```

Αρχείο example3.php

```
<!DOCTYPE html>
<html lang="el">
<head>
<meta charset="utf-8" />
<title> Παράδειγμα 2 PHP Page </title>
</head>
<body>
<?php
// Βάζω σε μεταβλητές τα στοιχεία με τα οποία θα συνδεθώ
$username = "root";
$password = "";
$db = "test";

/* Ελέγχω μήπως κάποιο στοιχείο της φόρμα δεν συμπληρώθηκε ή δεν
συμπληρώθηκε σωστά. Αν υπάρχει λάθος βάζω ένα μήνυμα στην $error
την οποία έχω στην αρχή ίση με "", οπότε μετά θα ξέρω αν υπήρχε
λάθος και ποιο ή ποια ήταν. */
$error = "";

// Ελέγχω μήπως δεν συμπληρώθηκε το όνομα
if (!isset($_GET['firstname']) or empty(trim($_GET ['firstname'])))
{
    $error = $error . "<li>Το όνομα είναι κενό.</li>";
}
else
    $on = trim($_GET['firstname']);
```

```

// Αν συνέβη λάθος τότε η μεταβλητή $error δεν είναι κενή
if (trim($error)!="")
{
    echo "<ol>" . $error . "</ol>";
    echo "<br /><b>Η αναζήτηση-προβολή απέτυχε !!</b>";
}
else //αλλιώς ξεκινάω τη διαδικασία για αναζήτηση-προβολή
{
    //Συνδέομαι με τον mysql server
    $link = mysqli_connect("localhost", $mysqluser, $mysqlpassword,
    $mysqldb);
    if (!$link)
        die ('Αποτυχία σύνδεσης στον MySQL server: ' .
    mysqli_connect_error());

    // Θέτω σωστά το encoding
    mysqli_set_charset($link, "utf8");

    // Πρώτα κάνω μετατροπή για να μην έχω πρόβλημα με ειδικούς
    // χαρακτήρες που ίσως έχουν προστεθεί στις μεταβλητές
    $on = mysqli_real_escape_string($link, $on);

    // Επιλέγω όλα τα στοιχεία από τον testtable με το όνομα που μου
    // δόθηκε, χρησιμοποιώντας like
    $sql = "SELECT * FROM testtable where onoma like '$on'";
    $result = mysqli_query($link, $sql);

    if (!$result)
        die ('Δεν μπορώ να κάνω SELECT: ' . mysqli_error($link));
    // Παίρνω τον αριθμό των εγγράφων που επιλέχτηκαν
    $num = mysqli_num_rows($result);

    // Αν επιλέχτηκε έστω και μία τις προβάλω με έναν πίνακα HTML
    if ($num >= 1)
    {
        echo "<center>Ταιριάζουν " . $num . " εγγραφές:</center>";
        echo "<table>";
        echo "<tr><th>Όνομα</th> <th>Επώνυμο</th>";
        echo "<th>Ηλικία</th></tr>";
        for ($i=0; $i < $num; ++$i)
        {
            $r = mysqli_fetch_array($result);
            echo "<tr><td>" . $r['onoma'] . "</td> <td>" .
            $r['eponimo'] . "</td> <td>" . $r['age'] . "</td></tr>";
        }
        echo "</table>";

        // close connection
        mysqli_close($link);
    }
    else // Αλλιώς (δηλαδή δεν ταιριάζει καμία)
    {
        // close connection
        mysqli_close($link);
    }
}

```

```
        echo "<b>Δεν υπάρχει εγγραφή που να ταιριάζει με το  
όνομα/λογική έκφραση που δώσατε !<br />";  
    }  
}  
?>  
</body>  
</html>
```


6 ΠΑΡΑΡΤΗΜΑ Α – ΡΥΘΜΙΣΕΙΣ WEB ΕΞΥΠΗΡΕΤΗΤΗ, PHP ΚΑΙ MySQL

Στο παράρτημα αυτό θα δούμε ορισμένες από τις ρυθμίσεις που μπορούμε να κάνουμε στο Web Εξυπηρετητή καθώς και στην λειτουργία της PHP μέσα από αυτόν και στη λειτουργία του MySQL.

Παρά το ότι αν εγκαταστήσουμε ένα από τα πακέτα που αναφέρθηκαν στο 3ο Κεφάλαιο («**Τι χρειαζόμαστε για την PHP-MySQL**») οι περισσότερες ρυθμίσεις που χρειάζονται, τουλάχιστον για να ξεκινήσουμε, έχουν γίνει, είναι σημαντικό να μπορούμε να αλλάξουμε ορισμένες ρυθμίσεις για να βελτιώσουμε τη λειτουργία του συστήματός μας αλλά και την ασφάλεια των εφαρμογών μας.

Οι ρυθμίσεις που θα δούμε αφορούν τη λειτουργία του MySQL και την εγκατάσταση του Web Εξυπηρετητή **Apache** και της λειτουργίας της PHP με αυτόν και γίνονται με αλλαγές στα εξής αρχεία:

Αρχείο	Χρήση
http.conf	Το βασικό αρχείο των ρυθμίσεων διαμόρφωσης του HTTP Εξυπηρετητή Apache
.htaccess	Αρχεία που επιτρέπουν την εφαρμογή ρυθμίσεων του HTTP Εξυπηρετητή Apache ανά φάκελο
php.ini	Το βασικό αρχείο ρυθμίσεων διαμόρφωσης της λειτουργίας της PHP
my.ini ή my.cnf	Το βασικό αρχείο ρυθμίσεων διαμόρφωσης της λειτουργίας του MySQL

6.1 Αρχείο *http.conf*

Το αρχείο **http.conf** αποτελεί το βασικό αρχείο των ρυθμίσεων διαμόρφωσης του HTTP Εξυπηρετητή Apache. Είναι ένα απλό αρχείο κειμένου στο οποίο υπάρχουν εντολές-οδηγίες (directives) για την διαμόρφωση του τρόπου λειτουργίας του Εξυπηρετητή. Στο αρχείο αυτό όταν μία γραμμή αρχίζει με τον χαρακτήρα «#» σημαίνει ότι είναι σχόλιο και θα βρείτε αρκετά σχόλια τα οποία σας βοηθάνε πάρα πολύ στον ορισμό των σωστών οδηγιών.

Ο φάκελος που βρίσκεται το αρχείο αυτό είναι καθορισμένος κατά την μεταγλώττιση του εκτελέσιμου του εξυπηρετητή αλλά μπορεί να καθοριστεί και κατά την εκτέλεση με την σημαία **-f**. Αν έχετε εγκαταστήσει ένα από τα έτοιμα πακέτα εγκατάστασης (xampp ή winamp) το αρχείο αυτό συνήθως βρίσκεται στον φάκελο:

Φάκελος Εγκατάστασης/*apache/conf*

Για λόγους μεγέθους και καλύτερης οργάνωσης δεν περιέχονται όλες οι οδηγίες ρυθμίσεων σε αυτό το αρχείο αλλά σε διάφορα σημεία του υπάρχει μια εντολή Include με την οποία ενσωματώνει άλλα αρχεία με οδηγίες. Έτσι, συνήθως, υπάρχει μια σειρά άλλων αρχείων διαμόρφωσης τα οποία το **http.conf** κάνει Include για να γράφονται όλες οι οδηγίες σε αυτό. Πχ στην εγκατάσταση του xampp, υπάρχει μέσα στον φάκελο **Φάκελος Εγκατάστασης/*apache/conf*** (που βρίσκεται και το **http.conf**) ένας φάκελος **extra** στον οποίο υπάρχουν διάφορα άλλα αρχεία διαμόρφωσης (**httpd-xampp.conf**, **httpd-ssl.conf**, **httpd-languages.conf** κλπ) τα οποία κάνει Include το **http.conf**.

Στο παρακάτω πίνακα παρουσιάζονται σε συντομία οι σημαντικότερες ρυθμίσεις που μας αφορούν. Πριν δοκιμάσετε να αλλάξετε οτιδήποτε καλό είναι να κρατήσετε ένα αντίγραφο ασφαλείας του αρχικού **http.conf**. Επίσης για να λειτουργήσουν οτιδήποτε αλλαγές κάνετε πρέπει να επανεκκινήσετε τον Apache.

Οδηγία - Εντολή	Χρήση
ServerRoot <Φάκελος>	Καθορίζουμε τον αρχικό φάκελο κάτω από τον οποίο διατηρούνται τα αρχεία διαμόρφωσης, καταγραφής συμβάντων, λαθών κλπ του εξυπηρετητή. Πχ για το xampp θα δείτε το εξής: <i>ServerRoot "<Φάκελος εγκατάστασης xampp>/apache"</i>
Listen PORT	Καθορίζουμε σε ποια θύρα TCP θα ακούει ο εξυπηρετητής. Συνήθως η θύρα αυτή είναι η 80. Πχ <i>Listen 80</i> Αν όμως τρέχει στον υπολογιστή μας ήδη κάποια εφαρμογή που χρησιμοποιεί τη συγκεκριμένη θύρα θα έχουμε προβλήματα και δεν θα εκκινεί ο εξυπηρετητής, οπότε μπορούμε να βάλουμε μία άλλη θύρα. Φυσικά για να συνδεθεί ο πελάτης πρέπει να βάλει στο URL και τη θύρα. Επίσης αν ο Η/Υ μας έχει πολλές IP μπορούμε να καθορίσουμε να ακούει μόνο στις αιτήσεις που γίνονται σε μία από αυτές (αλλιώς θα απαντάει στη συγκεκριμένη θύρα σε αιτήσεις που γίνονται προς όλες μας τις IP). Πχ <i>Listen 193.46.54.230:80</i> Τέλος μπορούμε να κάνουμε και άλλες ρυθμίσεις.
DocumentRoot <Φάκελος>	Ο αρχικός φάκελος του δέντρου φακέλων του Ιστοτόπου. Μέσα σε αυτόν θα τοποθετούμε τις σελίδες μας. Πχ για το xampp θα δείτε το εξής: <i>DocumentRoot "<Φάκελος εγκατάστασης. xampp>/htdocs"</i> Αν θέλετε να είναι το C:/Data/htdocs αρκεί να θέσετε: <i>DocumentRoot " C:/Data/htdocs"</i>
ErrorLog <Φάκελος>/error.log και LogLevel επίπεδο	Το αρχείο στο οποίο θα γίνεται η καταγραφή συμβάντων. Ο Φάκελος είναι σε σχέση με το ServerRoot. Με το LogLevel καθορίζεται το επίπεδο από το οποίο και πάνω θα γίνεται η καταγραφή ενός συμβάντος. Πχ για το xampp θα δείτε το εξής: <i>ErrorLog logs/error.log</i> <i>LogLevel warn</i>
LoadModule <module> <αρχείο> και Action <application> <file>	Ο Apache είναι ένας αρθρωτός (modular) εξυπηρετητής ο οποίος υποστηρίζει τη διασύνδεση νέων δομών-τμημάτων (Modules) που επιτρέπουν την επέκταση της λειτουργικότητάς του. Ο οδηγίες LoadModule και Action είναι δύο πολύ σημαντικές οδηγίες μέσω των οποίων ενεργοποιούμε/διαμορφώνουμε τον τρόπο με τον οποίο θα εκτελείται η PHP. Μάλιστα οι οδηγίες αυτές είναι πιθανότερο να βρεθούνε μέσα σε μια άλλη οδηγία την <ifModule Module> ώστε να εξασφαλιστεί ότι έχουν ήδη συνδεθεί άλλα modules που επηρεάζουν. Όπως αναφέρθηκε και στο κεφάλαιο « Η γλώσσα PHP », η PHP μπορεί να εκτελείται: 1. Είτε σαν Module 2. Είτε σαν cgi-εκτελέσιμο Για λόγους απόδοσης αλλά και ασφάλειας (https://www.php.net/manual/en/security.php) είναι προτιμότερο να

εκτελείται σαν module. Η πρώτη εντολή (**LoadModule**) θα χρησιμοποιηθεί για την ενεργοποίηση της PHP σαν Module και η δεύτερη δεν θα χρησιμοποιηθεί καθόλου (θα είναι σε σχόλιο «#»).

Πχ για το xampp θα δείτε το εξής:

```
LoadModule      php5_module      "<φάκελος      εγκατάστασης
Xampp>/apache/bin/php5apache2.dll"
```

Και

```
#Action application/x-httpd-php "/php/php-cgi.exe"
```

Οι εντολές αυτές μπορεί να είναι σε ξεχωριστό αρχείο το οποίο κάνει include το http.conf. Πχ στο xampp βρίσκονται στο αρχείο httpd-xampp.conf που βρίσκεται στον φάκελο **<Φάκελος εγκατάστασης xampp>/conf/extra**

AddType <application> <file>

Χρησιμοποιείται για διάφορους λόγους όπως για τη συσχέτιση μιας εφαρμογής με μια σειρά αρχείων με συγκεκριμένες καταλήξεις. Έτσι αν θέλαμε όλα τα αρχεία που έχουν κατάληξη .html .htm .php .php4 .php5 να παρνάνε από τον **PHP parser/interpreter**, ώστε να μπορούμε και σε αυτά να γράφουμε PHP scripts που θα εκτελούνται, τότε θα βάζαμε την οδηγία:

```
AddType application/x-httpd-php .html .htm .php .php4 .php5
```

Και οι εντολές αυτές μπορεί να είναι σε ξεχωριστό αρχείο το οποίο κάνει include το http.conf. Πχ στο xampp βρίσκονται στο αρχείο httpd-xampp.conf που βρίσκεται στον φάκελο **<Φάκελος εγκατάστασης xampp>/conf/extra**

Στην εγκατάσταση του xampp έχουμε στο αρχείο httpd-xampp.conf την εντολή:

```
AddType application/x-httpd-php .php .php5 .php4 .php3 .phtml
```

Δηλαδή αν ένα αρχείο είναι .html δεν θα περάσει από τον **PHP parser/interpreter**, αλλά θα σταλεί απευθείας στον πελάτη και οπότε δεν θα εκτελεστεί το PHP script που τυχόν θα έχουμε γράψει.

Περισσότερες πληροφορίες για τις διάφορες οδηγίες μπορεί να βρει ο αναγνώστης μελετώντας το ίδιο το αρχείο http.conf και τα σχόλια που περιλαμβάνει καθώς και τις οδηγίες στην επίσημη ιστοσελίδα του Apache και πιο συγκεκριμένα στον σύνδεσμο <https://httpd.apache.org/docs/2.2/configuring.html> (το 2.2 αναφέρεται στην έκδοση και μπορεί να το αλλάξει ο αναγνώστης για να δει τις πληροφορίες για την έκδοση που τον αφορά).

6.2 Αρχείο .htaccess

Ο Apache εξυπηρετητής επιτρέπει την αποκεντρωμένη διαμόρφωση με την χρήση αρχείων που τοποθετούνται σε φακέλους. Το βασικό αρχείο με αυτή τη χρήση είναι το **.htaccess** (μπορούμε με την οδηγία **AccessFileName filename [filename] ...** να ορίσουμε και άλλα αρχεία). Τοποθετώντας ένα αρχείο με όνομα **.htaccess** σε έναν φάκελο, όταν γίνεται πρόσβαση σε κάποιο αρχείο που βρίσκεται σε αυτόν το φάκελο και στους υποφακέλους του (με την προϋπόθεση ότι δεν περιέχουν και αυτοί κάποιο αρχείο .htaccess), διαβάζονται και εφαρμόζονται οι ρυθμίσεις που έχουμε βάλει (και οι οποίες είναι όπως και στο http.conf) μέσα στο αρχείο αυτό.

Έτσι αν θέλουμε μόνο τα αρχεία με κατάληξη **.php** να περνούν από τον **PHP parser/interpreter**, εκτός από τα αρχεία που βρίσκονται κάτω από ένα **συγκεκριμένο φάκελο** όπου θέλουμε να περνούν τα αρχεία που έχουν κατάληξη **.php .html** και **.htm** τότε αρκεί στο αρχείο **http.conf** (ή σε όποιο κάνει Include) να βάλουμε την οδηγία:

```
AddType application/x-httpd-php .php
```

Και στον **συγκεκριμένο φάκελο** να βάλουμε ένα αρχείο **.htaccess** με την οδηγία:

```
AddType application/x-httpd-php .php .html .htm
```

Περισσότερες πληροφορίες μπορεί να βρει ο αναγνώστης μελετώντας τις οδηγίες στον σύνδεσμο <https://httpd.apache.org/docs/2.2/howto/htaccess.html> (το 2.2 αναφέρεται στην έκδοση και μπορεί να το αλλάξει ο αναγνώστης για να δει τις πληροφορίες για την έκδοση που τον αφορά).

6.3 Αρχείο *php.ini*

Οι ρυθμίσεις διαμόρφωσης που αφορούν την λειτουργία της PHP μπαίνουν με τη μορφή οδηγιών-εντολών σε ένα αρχείο με όνομα *php.ini*. Το αρχείο αυτό είναι, παρόμοια με το *http.conf*, ένα αρχείο κειμένου στο οποίο κάθε γραμμή που αρχίζει με τον χαρακτήρα «;» είναι σχόλιο. Μάλιστα το αρχείο περιέχει πολλά σχόλια που βοηθούν να κατανοήσουμε το νόημα των οδηγιών-εντολών που μπορούμε να κάνουμε.

Το αρχείο αυτό υπάρχει στον φάκελο που βρίσκεται και η PHP στην εγκατάστασή μας (πχ στο *xampp* βρίσκεται στον φάκελο <Φάκελος εγκατάστασης *xampp*>/*php*/).

Στο παρακάτω πίνακα παρουσιάζονται σε συντομία οι σημαντικότερες ρυθμίσεις που μας αφορούν. Πριν δοκιμάσετε να αλλάξετε οτιδήποτε καλό είναι να κρατήσετε ένα αντίγραφο ασφαλείας του αρχικού *php.ini*.

Οδηγία - Εντολή	Χρήση
engine = On/Off	Ενεργοποίηση/Απενεργοποίηση της μηχανής PHP κάτω από τον Apache. Πχ <i>engine = On</i>
short_open_tag = On/Off	Ενεργοποιεί τη χρήση των σύντομων ετικετών <? και ?>. Αλλιώς επιτρέπονται μόνο οι κανονικές <?php και ?>.
asp_tags = On/Off	Ενεργοποιεί/Απενεργοποιεί τη χρήση των ASP-style <% %> ετικετών.
safe_mode = On/Off safe_mode_gid = On/Off safe_mode_include_dir = safe_mode_exec_dir = safe_mode_allowed_env_vars = safe_mode_protected_env_vars =	Σημαντικές ρυθμίσεις που αφορούν την εφαρμογή πολιτικών ασφαλείας στην εκτέλεση των scripts. Φυσικά οι περισσότερες απαιτούν να είναι: <i>safe_mode = On</i> Πχ με την safe_mode_exec_dir επιτρέπουμε την εκτέλεση (με την <i>exec</i>) εντολών που βρίσκονται σε συγκεκριμένους φακέλους. Με το <i>safe_mode=on</i> εφαρμόζεται έλεγχος σε επίπεδο χρήστη (πχ εξετάζεται αν ο ιδιοκτήτης του script είναι και ιδιοκτήτης ενός αρχείου που πάει να ανοίξει το script). Με το <i>safe_mode=off</i> και το <i>safe_mode_gid=on</i> ο έλεγχος αυτός γίνεται σε επίπεδο ομάδας (group). Για περισσότερες πληροφορίες προτείνεται στον αναγνώστη να ελέγξει το κεφάλαιο Security στο επίσημο εγχειρίδιο της PHP.
open_basedir =	Τίθεται στον φάκελο κάτω από τον οποίο και μόνο επιτρέπονται όλες οι λειτουργίες σε αρχεία. Φυσικά μία τέτοια ρύθμιση έχει ουσία μόνο όταν η ρύθμιση γίνεται ανά φάκελο ή virtual-host web server. Προκαθορισμένα η οδηγία αυτή είναι σε σχόλιο.

disable_functions = disable_classes =	Η πρώτη τίθεται ίση με μία λίστα από συναρτήσεις (χωρισμένες με κόμματα) που θέλεις να είναι απενεργοποιημένες. Ομοίως και για τις κλάσεις.
max_execution_time = max_input_time = memory_limit =	Καθορίζουμε αντίστοιχα το μέγιστο χρονικό διάστημα (σε δευτερόλεπτα) εκτέλεσης ενός script, τον μέγιστο χρόνο (σε δευτερόλεπτα) που ένα script θα κάνει parse δεδομένα της αίτησης και την μέγιστη μνήμη που θα καταναλώνει ένα script. Πχ <i>max_execution_time = 60</i> <i>max_input_time = 60</i> <i>memory_limit = 32M</i>
error_reporting =	Τίθεται ίση με ένα σύνολο σταθερών που συνδέονται με δυαδικές λογικές πράξεις (δες σχόλια για το ποιες σταθερές μπορεί να χρησιμοποιήσεις) για να καθοριστεί ποια λάθη θα αναφέρονται.
display_errors = On/Off	Καθορίζεις αν θα εμφανίζονται τα λάθη που συμβαίνουν κατά την εκτέλεση ενός script στην ιστοσελίδα. Είναι πολύ σημαντικό μόλις τελειώσουμε τις δοκιμές στις ιστοσελίδες που θέλουμε να φτιάξουμε και τις δοκιμάσουμε να απενεργοποιούμε την προβολή των λαθών. Αυτό συμβαίνει γιατί μπορεί κάποιος να προκαλέσει λάθη σε ιστοσελίδες με scripts για να βγάλει, από τα μηνύματα που θα εμφανιστούν, συμπεράσματα για τις ρυθμίσεις και την ασφάλεια και να δοκιμάσει επιθέσεις που μπορεί να απειλήσουν τον ιστοτόπο, τα δεδομένα ή και την ίδια τη λειτουργία του Web Εξυπηρετητή.
display_startup_errors = On/Off	Ομοίως με την προηγούμενη αλλά αυτό αφορά λάθη κατά την εκκίνηση της PHP.
variables_order =	Καθορίζεται η σειρά με την οποία εγγράφονται (register) οι μεταβλητές από την GET (G), POST (P), Cookie (C), Environment (E) και Built-in (S). Η εγγραφή γίνεται από αριστερά προς τα δεξιά. Πχ <i>variables_order = "EGPCS"</i>
register_globals = On/Off	Αν τεθεί On τότε όλες οι μεταβλητές που δημιουργούνται μέσω φορμών κλπ δεν μπαίνουν μόνο στον αντίστοιχο πίνακα αλλά δημιουργείται άμεσα μία καθολική μεταβλητή με το συγκεκριμένο όνομα. Πχ αν μια τιμή μας σταλεί με μια φόρμα, το πεδίο της οποίας ονομάζεται username, με τη μέθοδο GET τότε θα δημιουργηθεί αυτόματα μια καθολική μεταβλητή με αυτό το όνομα (και φυσικά θα μπει και στον πίνακα \$_GET['username']). Είναι πολύ σημαντικό για λόγους ασφαλείας να έχουμε τη ρύθμιση: <i>register_globals = Off</i> και να χρησιμοποιούμε τον κατάλληλο πίνακα για να πάρουμε την τιμή μιας μεταβλητής (πχ \$_GET, \$_POST, \$_FILES κλπ).
file_uploads = On/Off upload_tmp_dir = <φάκελος> upload_max_filesize =	Οι ρυθμίσεις αυτές αφορούν τις μεταφορές αρχείων από τον πελάτη στον εξυπηρετητή (upload), όπως είδαμε και στο κεφάλαιο «Η γλώσσα PHP». Ρυθμίζεις, αντίστοιχα, το αν επιτρέπονται οι μεταφορές αρχείων, σε ποιο φάκελο θα αποθηκεύονται προσωρινά τα αρχεία και το μέγιστο μέγεθος αρχείου που θα δεχόμαστε. Πχ

	<pre>file_uploads = On/Off upload_tmp_dir = <φάκελος> upload_max_filesize = 32M</pre>
allow_url_fopen = On/ Off allow_url_include = On/ Off	Το αν θα επιτρέπεται να ανοίξουμε αρχείο από URL (και όχι μόνο από τον τοπικό δίσκο) και αν θα επιτρέπεται να κάνουμε include αρχεία php script πάλι από URL (και όχι μόνο από τον τοπικό δίσκο)
[Session] session.save_handler = session.save_path = session.use_cookies = session.cookie_lifetime = session.auto_start = 1/0 ...	Με την οδηγία [Session] δηλώνεται η έναρξη μιας ομάδας οδηγιών που αναφέρονται στις ρυθμίσεις που αφορούν τις συνόδους, για τις οποίες μιλήσαμε και στο κεφάλαιο «Η γλώσσα PHP». Μάλιστα στις συναρτήσεις για τη διαχείριση των συνόδων που είδαμε, πολλές από αυτές ήταν για να μπορείς να διαβάσεις/αλλάξεις τις ρυθμίσεις που έχεις και στο αρχείο php.ini μέσα από τον κώδικα. Επειδή οι εντολές-οδηγίες είναι αρκετές εδώ αναφέρουμε ενδεικτικά μόνο 5 από αυτές: Με την session.save_handler δηλώνουμε το όνομα του Χειριστή των συνόδων της PHP που είναι υπεύθυνος για να αποθηκεύει/χειρίζεται τα δεδομένα των συνόδων. Ο προκαθορισμένος είναι ο «files» (που κάνει την αποθήκευση/χειρισμό με τη χρήση αρχείων). πχ <pre>session.save_handler = files</pre> Με την session.save_path δηλώνουμε που θα τα αποθηκεύει (φάκελος), πχ στην εγκατάσταση του xampp θα δούμε το εξής: <pre>session.save_path = "<φάκελος εγκατάστασης Xampp>\tmp"</pre> Με την session.use_cookies δηλώνουμε αν θα χρησιμοποιούνται cookies για την μεταφορά και αποθήκευση στοιχείων στον πελάτη, πχ <pre>session.use_cookies = 1</pre> Με την session.cookie_lifetime δηλώνουμε σε δευτερόλεπτα πόση διάρκεια ζωής θα έχουν τα cookies μίας συνόδου, με τιμή 0 δηλώνουμε ότι δεν θα λήγουν, παρά μόνο με το κλείσιμο του φυλλομετρητή, πχ <pre>session.cookie_lifetime = 0</pre> Με την session.auto_start δηλώνουμε αν θα γίνεται αυτόματα η έναρξη συνόδου όταν ένας χρήστης κάνει πρόσβαση και δεν θα χρειάζεται να το προκαλούμε εμείς καλώντας στα scripts μας την session_start(). Πχ με την επόμενη εντολή απενεργοποιούμε την αυτόματη εκκίνηση της συνόδου και πρέπει να κάλεσουμε τη session_start() για να ξεκινήσει: <pre>session.auto_start = 0</pre> Υπάρχουν πολλές ακόμα εντολές-οδηγίες που αφορούν τις συνόδους, τις οποίες θα πρέπει να μελετήσεις ο αναγνώστης αν η εφαρμογή που θέλει να κάνει είναι για εμπορική χρήση.
[MySQL] mysql.allow_persistent = On/Off mysql.max_persistent = mysql.max_links = mysql.default_port = mysql.default_socket = mysql.default_host =	Με την οδηγία [MySQL] δηλώνεται η έναρξη μιας ομάδας οδηγιών που αναφέρονται στις ρυθμίσεις που αφορούν την λειτουργία της PHP σε συνάρτηση με την MySQL. Ορισμένες από αυτές τις οδηγίες είναι: Οι mysql.allow_persistent και mysql.max_persistent με τις οποίες δηλώνεις αν επιτρέπονται οι μόνιμες συνδέσεις (να μην χρειάζεται να κλείνεις τη σύνδεση και να την ξαναανοίγεις στο επόμενο script κοκ) και πόσες μόνιμες μπορείς να έχεις ανοικτές (-1 για άπειρες) αντίστοιχα. Πχ

mysql.default_user = *mysql.allow_persistent = On*
mysql.default_password = *mysql.max_persistent = -1*
mysql.connect_timeout = Η *mysql.max_links* δηλώνει πόσες συνδέσεις γενικά (μόνιμες και μη) μπορεί να έχουμε ανοικτές ταυτόχρονα (-1 για άπειρες). Πχ
mysql.trace_mode = *mysql.max_links = -1*

Οι *mysql.default_port*, *mysql.default_socket*, *mysql.default_host*, *mysql.default_user*, *mysql.default_password* μας επιτρέπουν να ορίσουμε προκαθορισμένες τιμές για τη σύνδεση με το MySQL εξυπηρετητή ώστε να μην χρειάζεται να τις βάζουμε σε κάθε *mysql_connect*. Πχ με τις παρακάτω εντολές δεν δηλώνω προκαθορισμένες τιμές και πρέπει είτε να τις βάζω είτε θα χρησιμοποιηθούν οι προκαθορισμένες του MySQL (πχ για το port και το socket):

mysql.default_port =
mysql.default_socket =
mysql.default_host =
mysql.default_user =
mysql.default_password =

Οι *mysql.connect_timeout* και *mysql.trace_mode* μας επιτρέπουν να ορίσουμε τα δευτερόλεπτα (-1 για άπειρα) στα οποία θα προσπαθήσει μία σύνδεση να πραγματοποιηθεί - connect timeout (αν δεν γίνει σε όσα ορίσουμε παύει η προσπάθεια) και αν θα εμφανίζονται προειδοποιήσεις (warnings) που αφορούν λειτουργίες σε πίνακες αντίστοιχα. Πχ με τις παρακάτω εντολές ορίζω 60 δευτερόλεπτα connect timeout και να μην προβάλλονται προειδοποιήσεις):

mysql.connect_timeout = 60
mysql.trace_mode = Off

Υπάρχουν πολλές άλλες οδηγίες/ρυθμίσεις που δεν είναι δυνατόν να παρατεθούν στα πλαίσια του συγκεκριμένου εγχειριδίου. Προτείνεται ο αναγνώστης να μελετήσει τα σχόλια του αρχείου *php.ini* και να συμβουλευτεί τη λίστα οδηγιών που προσφέρεται στην επίσημη ιστοσελίδα της PHP (<https://www.php.net/manual/en/ini.php>).

6.4 Αρχείο *my.ini* ή *my.cnf*

Όπως αναφέραμε και στο κεφάλαιο που παρουσιάσαμε τον MySQL, πριν εκκινήσει ο εξυπηρετητής διαβάζει μια σειρά από επιλογές από ένα αρχείο επιλογών στο οποίο μπορούμε να ορίσουμε διάφορες παραμέτρους. Το αρχείο αυτό στα Windows είναι το **my.ini** ενώ στο Unix είναι το αρχείο **my.cnf**.

Στα Windows το αρχείο *my.ini* συνήθως βρίσκεται στον φάκελο εγκατάστασης των Windows (μπορούμε να το δούμε με **echo %WINDIR%**) ενώ στο Unix το αρχείο **my.cnf** συνήθως βρίσκεται στο */etc/my.cnf* ή */etc/mysql/my.cnf* κλπ. Στην πραγματικότητα ο MySQL διαβάζει μια σειρά από αρχεία, ανάλογα με το λειτουργικό που τρέχει. Τα αρχεία αυτά αναφέρονται στο εγχειρίδια αναφοράς του MySQL.

Επίσης μπορεί με την επιλογή εκκίνησης **--defaults-file <όνομα αρχείου>** να έχει επιλεγεί και κάποιο άλλο αρχείο. Για παράδειγμα στην εγκατάσταση του xampp στα Windows, ο MySQL εξυπηρετητής στήνεται σαν υπηρεσία (εισάγεται στις υπηρεσίες, Πίνακας Ελέγχου → Εργαλεία Διαχείρισης → Υπηρεσίες) με αυτόματη εκκίνηση και με την επιλογή:

--defaults-file <Φάκελος Εγκατάστασης xampp>/mysql/bin/my.cnf

οπότε το αρχείο που διαβάζει τις επιλογές είναι το συγκεκριμένο.

Ο καλύτερος τρόπος για να δούμε ποια αρχεία διαβάζει, εκτός από αυτά που του δίνουμε με επιλογή, είναι να εκτελέσουμε:

mysql --help

τότε εμφανίζεται μια λίστα βοήθειας στην οποία αναφέρονται και όλα τα αρχεία με τις επιλογές που διαβάζει.

Επειδή οι επιλογές που μπορεί να έχουμε σε ένα αρχείο επιλογών είναι πάρα πολλές εδώ θα δούμε κάποιες βασικές. Το αρχείο αυτό είναι, παρόμοια με τα άλλα, ένα αρχείο κειμένου στο οποίο κάθε γραμμή που αρχίζει με τον χαρακτήρα «;» αλλά και με τον «#» είναι σχόλιο. Μάλιστα το αρχείο περιέχει πολλά σχόλια που βοηθούν να κατανοήσουμε το νόημα των οδηγιών-εντολών που μπορούμε να κάνουμε. Αφού εντοπίσουμε το αρχείο που διαβάζεται μπορούμε να το ανοίξουμε με οποιοδήποτε απλό κειμενογράφο και να το αλλάξουμε, αλλά προτείνεται να κρατήσουμε ένα αντίγραφο ασφαλείας πριν αλλάξουμε οτιδήποτε. Οι περισσότερες από τις εντολές που θα δούμε στο παρακάτω πίνακα μπορούν να τεθούν και από τη γραμμή εντολών κατά την εκκίνηση των προγραμμάτων (εξυπηρετητή, πελατών) βάζοντας ένα -- μπροστά τους, πχ η εντολή `basedir=<όνομα φακέλου>` θα δινόταν στην γραμμή εντολών `--basedir=<όνομα φακέλου>`.

Οδηγία - Εντολή	Χρήση
[ομάδα]	Το αρχείο είναι οργανωμένο σε ομάδες εντολών όπου κάθε ομάδα αναφέρεται σε εντολές που αφορούν συγκεκριμένες εφαρμογές ή ομάδες εφαρμογών. Πχ η ομάδα εντολών [mysqld] διαβάζεται από τον MySQL εξυπηρετητή η ομάδα εντολών [mysql] διαβάζεται από τον MySQL πελάτη η ομάδα εντολών [client] διαβάζεται από όλες τις εφαρμογές πελάτες
[mysqld] basedir = tmpdir = datadir = port = socket = character-set-server = collation-server = #skip-networking #skip-innodb innodb_data_home_dir= innodb_data_file_path=	Όπως αναφέραμε στην ομάδα αυτή εντολών βάζουμε τις εντολές που θα διαβαστούν από τον MySQL εξυπηρετητή. Η εντολή <code>basedir=<όνομα φακέλου></code> καθορίζει το βασικό φάκελο εγκατάστασης του MySQL Η εντολή <code>tmpdir=<όνομα φακέλου></code> καθορίζει το φάκελο που δημιουργούνται προσωρινά αρχεία (δημιουργούνται και στον φάκελο temp των Windows συνήθως). Η εντολή <code>datadir=<όνομα φακέλου></code> καθορίζει το φάκελο που δημιουργούνται τα αρχεία αποθήκευσης των δεδομένων (πίνακες). Αυτός ο φάκελος είναι ο πιο σημαντικός και πρέπει να κρατάμε αντίγραφο ασφαλείας του συχνά. Η εντολή <code>port=<αριθμός></code> καθορίζει τη θύρα στην οποία «ακούει» ο εξυπηρετητής για αιτήσεις. Η εντολή <code>socket=όνομα</code> καθορίζει το όνομα του αρχείου socket (στο Unix) ή της σωλήνωσης (Windows) για τις τοπικές συνδέσεις. Οι εντολές <code>character-set-server</code> και <code>collation-server</code> καθορίζουν αντίστοιχα το σύνολο χαρακτήρων και collation του εξυπηρετητή. Η εντολή <code>skip-networking</code> αν δεν είναι σε σχόλιο καθορίζει ότι μόνο τοπικές συνδέσεις μπορεί να γίνονται (μόνο από το localhost)

innodb_log_group_home_dir =	Η εντολή <i>skip-innodb</i> αν δεν είναι σε σχόλιο καθορίζει ότι δεν θα είναι ενεργοποιημένη η μηχανή InnoDB και έτσι δεν θα μπορείς να δημιουργήσεις InnoDB Β.Δ. και πίνακες (θα βγάλει ένα warning και θα κάνει τον πίνακα MyISAM). Αν το βάλεις σε σχόλιο ενεργοποιείται η μηχανή InnoDB και μπορείς να θέσεις και μια σειρά άλλων εντολών (που είναι ήδη γραμμένες αλλά σε σχόλια και πρέπει να τις βγάλεις).
innodb_log_arch_dir	
[client]	Όπως αναφέραμε στην ομάδα αυτή εντολών βάζουμε τις εντολές που θα διαβαστούν από όλους τους πελάτες.
# password = your_password	Η εντολή <i>password=<τιμή></i> καθορίζει το προεπιλεγμένο password που θα χρησιμοποιείται από τους πελάτες.
port =	Η εντολή <i>port=<αριθμός></i> καθορίζει την προεπιλεγμένη θύρα που θα χρησιμοποιείται από τους πελάτες.
socket =	Η εντολή <i>socket=όνομα</i> καθορίζει το προεπιλεγμένο όνομα του αρχείου socket (στο Unix) ή της σωλήνωσης (Windows) για τις τοπικές συνδέσεις που θα χρησιμοποιεί ο πελάτης.
default-character-set =	Η εντολή <i>default-character-set</i> καθορίζει το σύνολο χαρακτήρων του πελάτη.

Υπάρχουν πολλές άλλες οδηγίες/ρυθμίσεις που δεν είναι δυνατόν να παρατεθούν στα πλαίσια του συγκεκριμένου εγχειριδίου. Προτείνεται ο αναγνώστης να μελετήσει τα σχόλια του αρχείου `my.ini/my.cnf` και να συμβουλευτεί το εγχειρίδιο αναφοράς που προσφέρεται στην επίσημη ιστοσελίδα του MySQL (<https://dev.mysql.com/doc/>).

7 ΠΑΡΑΡΤΗΜΑ Β – Επίδειξη Ολοκληρωμένης Εφαρμογής

Στο παράρτημα αυτό θα δούμε μια ολοκληρωμένη εφαρμογή για λόγους επίδειξης. Η εφαρμογή αφορά τη δυνατότητα μέσω του Παγκόσμιου Ιστού, και με τη χρήση της PHP και MySQL να μπορούμε να εισάγουμε Θέματα Συζήτησης και μηνύματα σε αυτά τα θέματα. Συγκεκριμένα θέλουμε:

- Να μπορεί ο χρήστης να αναζητήσει/προβάλλει μηνύματα σε ένα συγκεκριμένο θέμα ή σε όλα τα θέματα με τη δυνατότητα να αναζητήσει με βάση κάποιο κείμενο που περιέχεται στα μηνύματα.
- Να μπορεί ο χρήστης να εισάγει ένα νέο θέμα. Η δυνατότητα αυτή υπάρχει μόνο για τους πιστοποιημένους χρήστες.
- Να μπορεί ο χρήστης να εισάγει ένα νέο μήνυμα σε ένα υπάρχον θέμα. Η δυνατότητα αυτή υπάρχει μόνο για τους πιστοποιημένους χρήστες.
- Να μπορεί ο χρήστης να εγγραφεί δίνοντας κάποια στοιχεία του και να αποκτάει έτσι λογαριασμό (όνομα χρήστη και κωδικό).
- Να μπορεί ο χρήστης να εισαχθεί-πιστοποιηθεί (login) στο σύστημα με βάση ένα λογαριασμό που έχει δημιουργήσει κατά το προηγούμενο βήμα.
- Να μπορεί ο χρήστης να βγει-αποπιστοποιηθεί (logout) από το σύστημα.

Για να γίνουν αυτά θα μας χρειαστεί η Β.Δ. **newsdb** που δημιουργήσαμε στο κεφάλαιο που παρουσιάσαμε τον MySQL και η οποία αποτελείται από τρεις πίνακες:

1. **users**, εισάγονται τα στοιχεία του χρήστη (κωδικός χρήστη, όνομα, επώνυμο, e-mail, όνομα και κωδικός πρόσβασης, περιγραφή, φωτογραφία χρήστη)
2. **subjects**, εισάγονται θέματα συζητήσεων (κωδικός θέματος, όνομα θέματος, ημερομηνία/ώρα δημιουργίας, κωδικός χρήστη που το δημιούργησε)
3. **messages**, εισάγονται τα μηνύματα των χρηστών (κωδικός μηνύματος, κωδικός χρήστη που το εισήγαγε, κωδικός θέματος στο οποίο εισήχθη, ημερομηνία/ώρα εισαγωγής, κείμενο μηνύματος)

Επίσης θα γράψουμε μια σειρά από ιστοσελίδες με τον κατάλληλο κώδικα PHP για να υλοποιηθεί η λειτουργικότητα που θέλουμε. Ο κώδικας θα συνδέεται με την βάση newsdb στον MySQL εξυπηρετητή για να αποθηκεύσει, επιλέξει, ενημερώσει κλπ τα στοιχεία που χρειάζεται.

7.1 Υλοποίηση Βάση Δεδομένων

Όπως είδαμε η Β.Δ. που θα χρησιμοποιήσουμε είναι η newsdb που έχουμε ήδη δει. Ακολουθούν οι εντολές SQL για τη δημιουργία της Β.Δ. την οποία μπορεί να κάνετε με όποιο τρόπο θεωρείτε καταλληλότερο (γραμμή εντολών ή κάποιο γραφικό περιβάλλον).

```
CREATE DATABASE newsdb DEFAULT CHARACTER SET utf8 COLLATE utf8_unicode_ci;
```

```
USE newsdb;
```

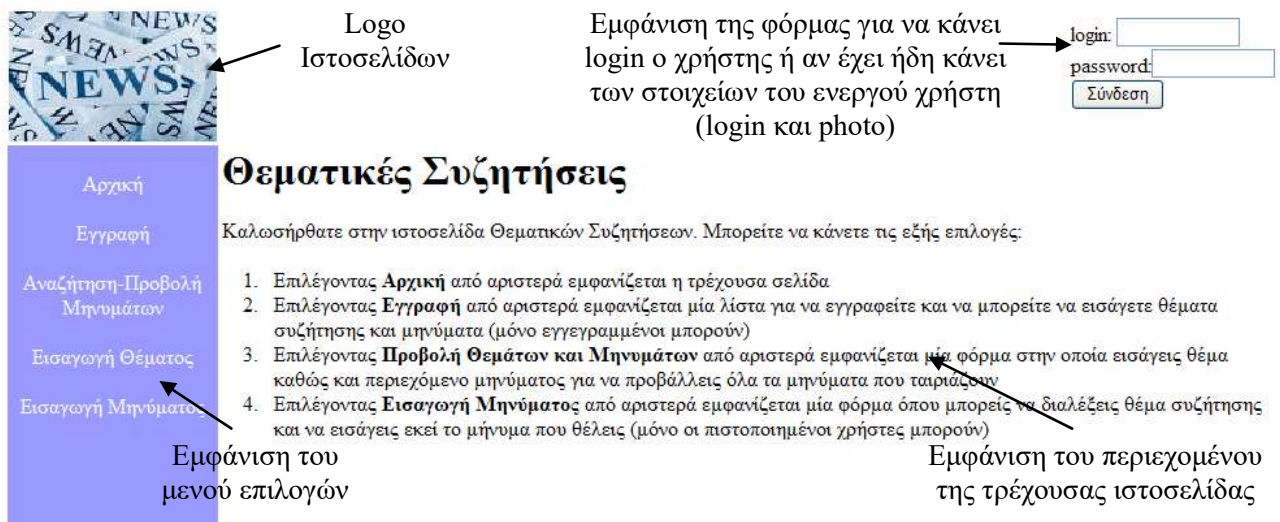
```
CREATE TABLE users (  
  userid int(32) unsigned NOT NULL auto_increment,  
  firstname varchar(50) NOT NULL,
```

```
    lastname varchar(50) NOT NULL,  
    email varchar(50) NOT NULL,  
    login varchar(20) NOT NULL,  
    passwd varchar(250) NOT NULL,  
    description varchar(200) NOT NULL,  
    image varchar(200) NOT NULL,  
    PRIMARY KEY (userid)  
) ENGINE=INNODB AUTO_INCREMENT=1 ;  
  
CREATE TABLE subjects (  
    sid int(32) unsigned NOT NULL auto_increment,  
    sname varchar(200) NOT NULL,  
    sdate datetime NOT NULL,  
    userid int(32) unsigned NOT NULL,  
    PRIMARY KEY (sid),  
    CONSTRAINT suserid FOREIGN KEY suserid (userid) REFERENCES users  
(userid)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=INNODB AUTO_INCREMENT=1 ;  
  
CREATE TABLE messages (  
    msgid int(32) unsigned NOT NULL auto_increment,  
    userid int(32) unsigned NOT NULL,  
    sid int(32) unsigned NOT NULL,  
    postdate datetime NOT NULL,  
    message text NOT NULL,  
    PRIMARY KEY (`msgid`),  
    CONSTRAINT muserid FOREIGN KEY muserid (userid) REFERENCES users  
(userid)  
    ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT msid FOREIGN KEY msid (sid) REFERENCES subjects (sid)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=INNODB AUTO_INCREMENT=1 ;
```

7.2 Υλοποίηση Ιστοσελίδων

Στην υποενότητα αυτή θα δούμε τις ιστοσελίδες που γράψαμε για να υλοποιήσουμε την λειτουργικότητα της εφαρμογής. Οι ιστοσελίδες αυτές είναι συμβατές με **HTML** και χρησιμοποιούν ένα αρχείο **CSS** για να ορίσουμε κάποια βασικά στοιχεία για τη μορφή τους (για λόγους επίδειξης και όχι εκμάθησης).

Πριν ξεκινήσουμε να παρουσιάζουμε τις ιστοσελίδες θα πρέπει να εξηγήσουμε την υλοποίηση του τρόπου τοποθέτησης (layout) που επιλέξαμε να έχουν οι ιστοσελίδες μας. Ο τρόπος τοποθέτησης (layout) φαίνεται στο επόμενο σχήμα και φυσικά υλοποιήθηκε με τη χρήση CSS.



Το layout της ιστοσελίδας αποτελείται:

1. Από μία περιοχή στο πάνω μέρος (<header>) που χωρίζεται σε 3 υποπεριοχές, την αριστερή (logo), την κεντρική και τη δεξιά (login form ή logout link αν ο χρήστης είναι ήδη συνδεδεμένος).
2. Ένα μενού (<nav>) με τις επιλογές που όλες δείχνουν στην ίδια σελίδα αλλά δίνουν άλλη τιμή σε μία ιδιότητα page για να ξεχωρίζουμε τι έχει επιλεγεί από το μενού.
3. Το κυρίως μέρος που εμφανίζονται τα δεδομένα που αφορούν την τρέχουσα επιλογή του χρήστη με βάση τι έχει επιλεγεί στο μενού.
4. Το κάτω μέρος (<footer>) που εμφανίζονται κάποιες πληροφορίες.

Ο τρόπος που υλοποιήθηκε ο ιστότοπος είναι η χρήση παραμέτρων στο url (μέθοδος get) για να καθορίσουμε ποια σελίδα επιθυμεί ο χρήστης. Οπότε στην πραγματικότητα υπάρχουν μόνο δύο αρχεία-ιστοσελίδες η **index.html** και η **functions.php**. Η **index.html** ανάλογα την παράμετρο στο url καλεί τις κατάλληλες συνάρτησεις για να τυπώσει αυτά που χρειάζονται. Οι διαφορετικές συναρτήσεις έχουν υλοποιηθεί όλες στο αρχείο **functions.php** το οποίο γίνεται include στο **index.html**, και στόχο έχουν να διαχειρίζονται την κάθε επιλογή από το μενού υλοποιώντας την αντίστοιχη λειτουργικότητα και φυσικά να εκτελούν τον απαραίτητο κώδικα για τη σύνδεση με τη Β.Δ. και την εκτέλεση ερωτημάτων.

Η ιστοσελίδα **index.html** έχει κώδικα PHP οπότε έχω βάλει κατάλληλο αρχείο **.htaccess** ώστε να εκτελούνται από την PHP και τα αρχεία με κατάληξη **.html** (δείτε προηγούμενο παράρτημα). Επίσης επειδή χρησιμοποιούμε αρχεία **css** έχουμε έναν φάκελο **css** μέσα στον οποίο υπάρχουν το **layout.css** (για το layout κ.λ.π.), το **format.css** (για τη μορφοποίηση αντικειμένων) και το **menu.css** (για το μενού). Επίσης υπάρχει και ο φάκελος **images**, ο οποίος περιέχει έναν υποφάκελο **users** (**images/users**). Μέσα στον υποφάκελο **images** υπάρχουν οι εικόνες **logo.png** και **nophoto.png** που αντιστοιχούν στο logo της σελίδας και στην default φωτογραφία χρήστη (όταν δεν έχει δώσει δική του κατά την εγγραφή του) αντίστοιχα και στον υποφάκελο **images/users** μπαίνουν οι φωτογραφίες των χρηστών που οι ίδιοι ανεβάζουν κατά την εγγραφή τους παίρνοντας κατάλληλο όνομα.

Ο κώδικας των ιστοσελίδων ακολουθεί στις επόμενες υποενότητες και έχει πυκνά σχόλια για τη διευκόλυνση του αναγνώστη.

7.2.1 Αρχείο index.html

Είναι η βασική σελίδα που υλοποιεί την επιλογή του τι έχει κάνει ο χρήστης (μέσω των τιμών που μεταβιβάζονται μέσω του url με τη μέθοδο get) και αυτή που καλεί τις διάφορες συναρτήσεις από το αρχείο functions.php.

```
<?php
/*
Σεκινάω το session. Είναι απαραίτητο γιατί πρέπει να ελέγξω στην starttable()
αν έχει πιστοποιηθεί (κάνει login) ο χρήστης για να εμφανίζω πάνω δεξιά την
εικόνα και το μήνυμα αλλιώς να εμφανίζω τη φόρμα login.
*/
session_start();

include("functions.php");

/* Σεκινάω να δω τι έχει επιλέξει ο χρήστης για να κάνω τις απαραίτητες
ενέργειες */
If (isset($_GET["page"]) && !empty($_GET["page"]))
    $page=trim($_GET["page"]);
else
    $page="1"; //Αν δεν υπάρχει η παράμετρος θέσε την Αρχική σελίδα

$msg="";
switch ($page) {
    case "1": //Επέλεξε Αρχική σελίδα από το μενού
        printStartOfPage('ΘΕΜΑΤΙΚΕΣ ΣΥΖΗΤΗΣΕΙΣ - Αρχική', $msg);
        initialPage();
        printEndOfPage();
        break;

    case "2": //Επέλεξε Εγγραφή από το μενού ή συμπλήρωσε και έστειλε τη φόρμα
εγγραφής
        printStartOfPage('ΘΕΜΑΤΙΚΕΣ ΣΥΖΗΤΗΣΕΙΣ - Εγγραφή Χρήστη', $msg);

        // Για λόγους επίδειξης τη λειτουργία εγγραφής την κάνω με δύο
συναρτήσεις
        // Αν έχει αποσταλεί η φόρμα εγγραφής
        if (isset($_GET["regform"]))
            userRegisterPage();
        else // Αλλιώς εμφάνισε τη φόρμα
            printRegisterForm("index.html?page=2");

        printEndOfPage();
        break;

    case "3": //Επέλεξε Προβολή Μηνυμάτων από το μενού ή συμπλήρωσε και έστειλε
τη φόρμα αναζήτησης-προβολής μηνυμάτων
        printStartOfPage('ΘΕΜΑΤΙΚΕΣ ΣΥΖΗΤΗΣΕΙΣ - Προβολή Μηνυμάτων', $msg);
        viewMessagesPage("index.html?page=3");
        printEndOfPage();
        break;

    case "4": //Επέλεξε Εισαγωγή Θέματος από το μενού ή συμπλήρωσε και έστειλε τη
φόρμα εισαγωγής θέματος
        printStartOfPage('ΘΕΜΑΤΙΚΕΣ ΣΥΖΗΤΗΣΕΙΣ - Εισαγωγή Θέματος', $msg);
        insertSubjectPage("index.html?page=4");
        printEndOfPage();
        break;

    case "5": //Επέλεξε Εισαγωγή Μηνύματος από το μενού ή συμπλήρωσε και έστειλε
τη φόρμα εισαγωγής Μηνύματος
        printStartOfPage('ΘΕΜΑΤΙΚΕΣ ΣΥΖΗΤΗΣΕΙΣ - Εισαγωγή Μηνύματος', $msg);
        insertMessagePage("index.html?page=5");
        printEndOfPage();
        break;
```

```

case "6": //Έκανε σύνδεση από τη φόρμα σύνδεσης
          //Κάνε τη σύνδεση και εμφάνισέ του την αρχική σελίδα
          $msg=userLoginVerify();
          printStartOfPage('ΘΕΜΑΤΙΚΕΣ ΣΥΖΗΤΗΣΕΙΣ - Αρχική', $msg);
          initialPage();
          printEndOfPage();
          break;
case "7": //Έκανε αποσύνδεση με το σύνδεσμο αποσύνδεσης
          //Κάνε την αποσύνδεση και εμφάνισέ του την αρχική σελίδα
          $msg=userLogout();
          printStartOfPage('ΘΕΜΑΤΙΚΕΣ ΣΥΖΗΤΗΣΕΙΣ - Αρχική', $msg);
          initialPage();
          printEndOfPage();
          break;
default:
          printStartOfPage('ΘΕΜΑΤΙΚΕΣ ΣΥΖΗΤΗΣΕΙΣ - Εισαγωγή Θέματος', $msg);
          unknownPage();
          printEndOfPage();
          break;
}
?>

```

7.2.2 Αρχείο functions.php

Περιέχει τις συναρτήσεις που υλοποιούν τη λειτουργικότητα των ιστοσελίδων και καλούνται από την ιστοσελίδα index.html. Κάθε συνάρτηση έχει σχόλια για τη λειτουργικότητά της πριν τη δήλωσή της.

```

<?php
/* -----
Η συνάρτηση τυπώνει την αρχή της ιστοσελίδας με τον τίτλο που παίρνει σαν
όρισμα. Η αρχή της ιστοσελίδας περιλαμβάνει το <header> (που έχει τρία μέρη), το
μενού (<nav>) και το άνοιγμα της κύριας περιχής (<main>). Το δεύτερο όρισμα
είναι ένα μήνυμα που θα τυπωθεί στην αρχή του <main>.
*/
function printStartOfPage($title, $msg)
{
?>
<!DOCTYPE html >
<html lang="el">
<head>
<link rel="stylesheet" type="text/css" href="css/layout.css" />
<link rel="stylesheet" type="text/css" href="css/menu.css" />
<link rel="stylesheet" type="text/css" href="css/format.css" />
<title>
<?php echo $title; ?>
</title>
<meta charset="UTF-8" />
</head>
<body>
<!-- Header, χωρίζεται σε logo αριστερά, κέντρο και φόρμα εισαγωγής ή μήνυμα
καλωσορίσματος δεξιά -->
<header>
<div class="headerleft">

</div>
<div class="headercenter">
<h1>Ιστοσελίδα Θεματικών Συζητήσεων</h1>
</div>
<div class="headerright">

```

```

<?php
/* Αν έχει ήδη κάνει κάποιος login βγάλε μήνυμα καλωσορίσματος, τη φωτογραφία
του (αν έχει) και ένα σύνδεσμο για αποσύνδεση. */
if (isset($_SESSION['userid']))
{
    echo "<b>Καλωσήρθες " . $_SESSION['login'] . "</b><br />\n";
    echo "<a href='index.html?page=7'>Αποσύνδεση</a><br />";
    echo "<img class='userphoto' src='" . $_SESSION['image'] . "' alt='User
Photo' /><br />\n";
}
else
{
    echo "<form class='formlogin' action='index.html?page=6' method='post'>\n";
    echo "<fieldset>\n<legend>Φόρμα Σύνδεσης:</legend>\n";
    echo "<label>login:</label> <input type='text' size='10' name='login' />\n";
    echo "<label>password:</label><input type='password' size='10' name='passwd'
/>\n";
    echo "<input type='submit' value='Σύνδεση' />\n";
    echo "</fieldset></form>\n";
}
?>
</div>
</header>
<!-- Μενού επιλογών με 5 επιλογές -->
<nav>
<ul class="menu">
<li><a href="index.html?page=1">Αρχική</a></li>
<li><a href="index.html?page=2">Εγγραφή Χρήστη</a></li>
<li><a href="index.html?page=3">Προβολή Μηνυμάτων</a></li>
<li><a href="index.html?page=4">Εισαγωγή Θέματος</a></li>
<li><a href="index.html?page=5">Εισαγωγή Μηνύματος</a></li>
</ul>
</nav>
<main>
<?php
echo $msg;
}

/* -----
Η συνάρτηση τυπώνει το τέλος της ιστοσελίδας, δηλαδή το κλείσιμο της περιοχής
<main> και το <footer>
*/
function printEndOfPage ()
{
    // Κλείνω το τμήμα main και εμφανίζω το footer
    echo "</main>";
    echo "<footer>\n Δημιουργία για εκπαιδευτικούς λόγους<br />\n";
    echo "Εγχειρίδιο PHP-MySQL <br />\n&copy 2020 - Νικόλαος Κατσάλης\n</footer>\n";
}

/* -----
Η συνάρτηση κάνει σύνδεση στο MySQL εξυπηρετητή που τρέχει στο ίδιο μηχάνημα και
ανοίγει τη Β.Δ. newsdb. Η συνάρτηση επιστρέφει τη σύνδεση στον εξυπηρετητή για
να εκτελέσει τα ερωτήματα αυτός που την καλεί.
*/
function connectToServer($enc = "utf8")
{
    $link = mysqli_connect("localhost","root","", "newsdb");
    if (!$link)
        die("Cannot connect to server " . mysqli_connect_error($link));
    //έτω το encoding

```

```

        mysqli_set_charset($link, $enc);
        return $link;
    }

    /* -----
    Η συνάρτηση επιστρέφει το αποτέλεσμα της εκτέλεσης του ερωτήματος για να το
    χρησιμοποιήσει όπως θέλει αυτός που την κάλεσε και εκτέλεσε το ερώτημα
    */
    function executeQuery($q, $l)
    {
        //εκτελώ το ερώτημα
        return mysqli_query($l, $q);
    }

    /* -----
    Η συνάρτηση τυπώνει τα απαραίτητα στην περίπτωση που η επιλογή του χρήστη είναι
    η αρχική σελίδα.
    */
    function initialPage()
    {
        ?>
        <h2>Αρχική Σελίδα Θεματικών συζητήσεων</h2>
        <p>Καλωσήρθατε στην ιστοσελίδα Θεματικών Συζητήσεων. Μπορείτε να κάνετε τις
        εξής επιλογές:</p>
        <ol>
        <li>Επιλέγοντας <b>Αρχική</b> από αριστερά εμφανίζεται η τρέχουσα σελίδα</li>
        <li>Επιλέγοντας <b>Εγγραφή</b> από αριστερά εμφανίζεται μία λίστα για να
        εγγραφείτε και να μπορείτε να εισάγετε θέματα συζήτησης και μηνύματα (μόνο
        εγγεγραμμένοι μπορούν)</li>
        <li>Επιλέγοντας <b>Προβολή Θεμάτων και Μηνυμάτων</b> από αριστερά εμφανίζεται
        μία φόρμα στην οποία εισάγεις θέμα καθώς και περιεχόμενο μηνύματος για να
        προβάλλεις όλα τα μηνύματα που ταιριάζουν</li>
        <li>Επιλέγοντας <b>Εισαγωγή Μηνύματος</b> από αριστερά εμφανίζεται μία φόρμα
        όπου μπορείς να διαλέξεις θέμα συζήτησης και να εισάγεις εκεί το μήνυμα που
        θέλεις (μόνο οι πιστοποιημένοι χρήστες μπορούν)</li>
        </ol>
        <?php
    }

    /* -----
    Η συνάρτηση τυπώνει τα απαραίτητα στην περίπτωση που η επιλογή του χρήστη είναι
    η αρχική σελίδα.
    */
    function userRegisterPage()
    {
        ?>
        <h2>Εγγραφή Νέου Χρήστη</h2>
        <?php
        $fname = "";
        $lname = "";
        $email = "";
        $login = "";
        $passwd = "";
        $rpasswd = "";
        $descr = "";

        /* Έλεγχε τα πεδία. Αν δεν έχει δώσει κάτι που πρέπει βάζω στη μεταβλητή $err
        μία τιμή διαφορετική από κενό την οποία ελέγχω παρακάτω. Όλα τα πεδία είναι
        υποχρεωτικά εκτός από την περιγραφή και την φωτογραφία. */
        $error = "";

```

```
if (!isset($_POST['firstname']) || empty(trim($_POST['firstname'])))
    $error .= "<li>Δεν έχετε δώσει όνομα !</li>\n";
else
    $fname = trim($_POST['firstname']);

if (!isset($_POST['lastname']) || empty(trim($_POST['lastname'])))
    $error .= "<li>Δεν έχετε δώσει επώνυμο !</li>\n";
else
    $lname = trim($_POST['lastname']);

if (!isset($_POST['email']) || empty(trim($_POST['email'])))
    $error .= "<li>Δεν έχετε δώσει e-mail !</li>\n";
else
    $email = trim($_POST['email']);

if (!isset($_POST['login']) || empty(trim($_POST['login'])))
    $error .= "<li>Δεν έχετε δώσει login !</li>\n";
else
    $login = trim($_POST['login']);

if (!isset($_POST['passwd']) || empty(trim($_POST['passwd'])))
    $error .= "<li>Δεν έχετε δώσει password !</li>\n";
else
    $passwd = $_POST['passwd'];

if (!isset($_POST['rpasswd']) || empty(trim($_POST['rpasswd'])))
    $error .= "<li>Δεν έχετε δώσει επιβεβαίωση password !</li>\n";
else
    $rpasswd = $_POST['rpasswd'];

//Έλεγξε αν ταιριάζει το password με την επιβεβαίωση
if ($passwd != $rpasswd)
    $error .= "<li>Το password και η επιβεβαίωση διαφέρουν !</li>\n";

if (!isset($_POST['description']) || empty(trim($_POST['description'])))
    $descr = "";
else
    $descr = $_POST['description'];

// Αν δεν έδωσε σωστά στοιχεία βγάλε μήνυμα λάθους
if ($error != "")
    echo "<ol>\n" . $error . "</ol>\n";
else /* Αλλιώς ξεκίνα την εισαγωγή στον πίνακα χρηστών, αφού βεβαιωθείς
      ότι το login δεν χρησιμοποιείται ήδη. */
{
    // Αν έδωσε φωτογραφία
    if (basename($_FILES['userfile']['name']) != NULL)
    {
        /* Δημιουργώ το όνομα της εικόνας από το login του χρήστη και το κανονικό
        της όνομα για μοναδικότητα */
        $uploadaddir = './images/users/';
        $uploadfile = $uploadaddir . $login . "-" .
        basename($_FILES['userfile']['name']);

        if (move_uploaded_file($_FILES['userfile']['tmp_name'], $uploadfile)) {
            echo "<b>Η εικόνα μεταφορτώθηκε επιτυχώς.\n</b>";
        } else {
            die("<b>Πιθανή περίπτωση file upload attack!\n</b>");
        }
    }
    else // αλλιώς βάλε στον χρήστη μια προεπιλεγμένη
```

```

    $uploadfile = "./images/default/nophoto.jpg";

    $link = connectToServer();
    /* Το password το κωδικοποιώ (κρυπτογράφηση) πριν το αποθηκεύσω ώστε να μην
    μπορεί να το δει ούτε ο διαχειριστής ούτε κανένας άλλος. ΠΡΟΣΟΧΗ: Το πεδίο του
    πίνακα πρέπει να είναι μεγάλο για να χωράει το hash (>200 χαρακτήρες) */
    $passwd = password_hash($passwd, PASSWORD_DEFAULT);

    // Κάνε μετατροπή για να μην έχω πρόβλημα με ειδικούς χαρακτήρες
    $fname = mysqli_real_escape_string($link, $fname);
    $lname = mysqli_real_escape_string($link, $lname);
    $email = mysqli_real_escape_string($link, $email);
    $login = mysqli_real_escape_string($link, $login);

    // Έλεγχε αν υπάρχει χρήστης με το ίδιο login
    $sqlq = "select * from users where login='$login'";
    $res = executeQuery($sqlq, $link);
    if (!$res)
        die("Cannot select from table " . mysqli_error($link));

    $num = mysqli_num_rows($res);
    // Αν υπάρχει ήδη χρήστης με το ίδιο login
    if ($num > 0)
        echo "<span class='infomsg'> Το login που δώσατε χρησιμοποιείται, παρακαλώ
    δοκιμάστε με άλλο !</span>\n";
    else // αλλιώς κάνε την εισαγωγή
    {
        // Δημιούργησε το ερώτημα εισαγωγής και εκτέλεσέ το. Πρέπει να επηρέασε
        ακριβώς μία γραμμή
        $sqlq = "insert into users (firstname, lastname, email, login, passwd,
    description, image) ";
        $sqlq .= " values ('$fname', '$lname', '$email', '$login', ";
        $sqlq .= " '$passwd', '$descr', '$uploadfile')";
        $res = executeQuery($sqlq, $link);

        if (!$res)
            die("Cannot insert into table " . mysqli_error($link));

        $num = mysqli_affected_rows($link);

        if ($num == 1) // Αν η εισαγωγή πέτυχε
        {
            echo "<span class='infomsg'>Καλωσορίσατε " . $login . " ! Παρακαλώ κάντε
            σύνδεση από τη φόρμα πάνω δεξιά !</span><br />";
            if (trim($uploadfile) != '')
                echo "<img class='userphoto' src='$uploadfile' alt='IMAGE' />";
        }
        else {
            echo "<h2>Η εγγραφή σας απέτυχε παρακαλώ ξαναπροσπαθήστε !</h2>\n";
            printRegisterForm();
        }
    }
    mysqli_close($link);
}
}

/* -----
Η συνάρτηση τυπώνει τη φόρμα εγγραφής του χρήστη
*/
function printRegisterForm($pageurl) {
?>

```



```

<p />
<form class='formregister' enctype='multipart/form-data' action='<?php echo
$pageurl."&regform=1";?>' method='post'>
<fieldset>
<legend>Φόρμα Εγγραφής:</legend>
<label>Επώνυμο:</label> <input type="text" size="30" name="lastname" /><br />
<label>Όνομα:</label> <input type="text" size="30" name="firstname" /><br />
<label>E-mail:</label> <input type="email" size="30" name="email" /><br /><br />
<label>login:</label> <input type="text" size="30" name="login" /><br />
<label>Password:</label> <input type="password" size="30" name="passwd" /><br />
<label>Repeat Password:</label> <input type="password" size="30" name="rpasswd"
/><br />
<label>Περιγραφή:</label> <textarea name="description" rows="5" cols="40">
</textarea>
<br /><br />
<label>Αρχείο Φωτογραφίας (jpg, gif, png):</label> <input name="userfile"
type="file" /><br />
<input type="submit" value="Εγγραφή" />
<input type="reset" value="Καθαρισμός" />
</fieldset></form>
<?php
}

/* -----
Η συνάρτηση τυπώνει τη φόρμα αναζήτησης και τα αποτελέσματα εφόσον τη στείλει ο
χρήστης
*/
function viewMessagesPage($pageurl) {
??
<h2>Αναζήτηση-Προβολή Μηνυμάτων</h2>
<p>Στην ιστοσελίδα αυτή μπορείτε να αναζητήσετε/προβάλετε τα μηνύματα που
θέλετε. Η διαδικασία είναι να επιλέξετε το θέμα για το οποίο θέλετε να προβάλετε
τα μηνύματα του και να συμπληρώσετε το κείμενο που περιέχουν τα μηνύματα που
θέλετε να προβάλετε. Τότε θα αναζητηθούν και θα προβληθούν όλα τα μηνύματα, του
συγκεκριμένου μόνο θέματος, στα οποία βρίσκετε το συγκεκριμένο κείμενο που
δώσατε.</p>
<p>Αν θέλετε να προβάλετε όλα τα μηνύματα ενός θέματος απλώς επιλέξετε το θέμα
και μην γράψετε τίποτα στο κείμενο μηνύματος.</p>
<p>Αν θέλετε να προβάλετε όλα τα μηνύματα που έχουν κάποιο κείμενο ανεξαρτήτου
θέματος, τότε μην επιλέξετε κανένα θέμα και συμπληρώστε το κείμενο που θέλετε
στο κείμενο μηνύματος.</p>
<p>Αν θέλετε να προβάλετε όλα τα μηνύματα όλων των θεμάτων, τότε αφήστε και τα
δύο πεδία κενά.</p>
<?php
/* Για να καταλάβω αν η φόρμα έχει σταλεί (άρα πρέπει να κάνω αναζήτηση και
προβολή) ή απλώς πρέπει να εμφανίσω τη φόρμα (η σελίδα απλώς φορτώθηκε) έχω
βάλει ένα κρυφό πεδίο στη φόρμα με όνομα "type". Αν υπάρχει το πεδίο στον πίνακα
$_POST πάει να πει ότι στάλθηκε και θέτω τη μεταβλητή formSend=TRUE αλλιώς τη
θέτω FALSE για να ξέρω τι θα κάνω παρακάτω. */
if (isset($_POST['type']))
    $formsend = TRUE;
else
    $formsend = FALSE;

// Συνδέσου στον MySQL Εξυπηρετητή
$link = connectToServer();

/* Επέλεξε όλα τα θέματα και δημιούργησε δυναμικά μία λίστα φόρμας (με <select>
και <option>) στην οποία εμφάνισε το όνομα του θέματος αλλά σαν τιμή μπαίνει το
κλειδί (sid) ώστε αν επιλέξει κάποιο να σταλεί το κλειδί. */
$sqlq = "select * from subjects";

```

```

$res = executeQuery($sqlq, $link);
if (!$res)
    die("Cannot select from table " . mysqli_error($link));

$num = mysqli_num_rows($res);

echo "<form class='formsearch' action='$pageurl' method='post'>\n";
echo "<fieldset><legend>Φόρμα Αναζήτησης-Προβολής Μηνυμάτων:</legend>";
echo "Επιλογή Θέματος: <select name='subject'>\n";
// Βάλε ένα κενό θέμα με τιμή -1 αν δεν θέλει να επιλέξει κανένα
echo "<option value=''></option>\n";
// Βάλε στη λίστα όλα τα θέματα
for ($i=1; $i <= $num; ++$i)
{
    $r = mysqli_fetch_array($res);
    /* Αν έχει αποσταλλεί η φόρμα και το τρέχον θέμα είχε επιλεγεί βάλε το στη
    λίστα ως επιλεγμένο. */
    if ($formsend && $r['sid'] == $_POST['subject'])
        echo "<option value='' . $r['sid'] . \"\" selected>\" . $r['sname'] .
    \"></option>\n";
    else // Αλλιώς βάλε απλώς το θέμα στη λίστα
        echo "<option value='' . $r['sid'] . \"\">\" . $r['sname'] . \"></option>\n";
}
echo "</select><br />\n";
echo "<input type='hidden' name='type' value='type' /><br />\n";

/* Αν δεν έχει αποσταλλεί η φόρμα εμφάνισε το πεδίο για το κείμενο μηνύματος
κενό */
if (!$formsend)
    echo "Κείμενο Μηνύματος: <input type='text' size='50' name='txt' /><br
/>\n";
else // Αλλιώς εμφάνισε το κείμενο που είχε γράψει ο χρήστης όταν την έστειλε
    echo "Κείμενο Μηνύματος: <input type='text' size='50' name='txt' value='\" .
$_POST['txt'] . \"' /><br />\n";

echo "<input type='submit' value='Εμφάνιση' />\n";
echo "<input type='reset' value='Καθαρισμός' />\n";
echo "</fieldset></form>\n";
echo "<p />\n";

/* Αν έχει αποσταλεί η φόρμα κάνε αναζήτηση και πρόβαλλε τα αποτελέσματα κάτω
από τη φόρμα δημιουργώντας ένα HTML πίνακα. */
if ($formsend)
{
    // Κάνε μετατροπή για να μην έχω πρόβλημα με ειδικούς χαρακτήρες
    $subject = mysqli_real_escape_string($link, $_POST['subject']);
    $txt = mysqli_real_escape_string($link, $_POST['txt']);

    /* Δημιούργησε ερώτημα αναζήτησης με βάση την επιλογή του θέματος και το
    κείμενο που έχει συμπληρώσει ο χρήστης. */
    $str = "";
    if (!empty($_POST['subject']))
        $str .= " and messages.sid = '". $subject . "'";

    if (!empty($_POST['txt']))
        $str .= " and message like '%" . $txt . "%'";

    $sqlq = "select sname, msgid, postdate, login, image, message, description
";
    $sqlq .= "from subjects, messages, users ";
    $sqlq .= "where messages.userid = users.userid and messages.sid =

```

```

subjects.sid " . $str;
    $sqlq .= " order by sname ASC, msgid DESC";

    /* Εκτέλεσε το ερώτημα που δημιούργησες δυναμικά με βάση τα δεδομένα της
    φόρμας που συμπλήρωσε ο χρήστης. */
    $res = executeQuery($sqlq, $link);

    if (!$res)
        die("Cannot select from table " . mysqli_error($link));

    // Τύπωσε τα αποτελέσματα σε ένα πίνακα
    $num = mysqli_affected_rows($link);
    if ($num == 0)
        echo "<span class='infomsg'>Δεν βρέθηκαν αποτελέσματα !</span><br
/>\n";
    else
    {
        if ($num == 1)
            echo "<span class='infomsg'>Βρέθηκε " . $num . " αποτέλεσμα
!</span><br />\n";
        else
            echo "<span class='infomsg'>Βρέθηκαν " . $num . " αποτελέσματα
!</span><br />\n";

        /* Ο πίνακας έχει 3 στήλες: Το θέμα στο οποίο ανήκει το μήνυμα, Ο
        χρήστης που το εισήγαγε και το περιεχόμενο του μηνύματος. */
        echo "<table class='tableresults'>\n";
        echo "<th>Θέμα</th><th>Χρήστης</th><th>Μήνυμα</th>\n";
        /* Για κάθε αποτέλεσμα (εγγραφή που επέστρεψε το select) δημιούργησε
        και μία γραμμή με 3 κελιά. */
        for ($i=1; $i <= $num; ++$i)
        {
            /* Φέρε την επόμενη εγγραφή του αποτελέσματος και φόρτωσε τα
            πεδία της στον πίνακα $r. */
            $r = mysqli_fetch_array($res);
            // Ξεκίνα τη γραμμή
            echo "<tr>";
            // Τύπωσε το κελί με το θέμα
            echo "<td>" . $r['sname'] . "<br />posted at " .
            $r['postdate'] . "</td>";
            /* Τύπωσε το κελί με τον χρήστη (login του, φωτογραφία του (αν
            υπάρχει) και περιγραφή του). */
            echo "<td>" . $r['login'] . "<p />";
            if (!empty($r['image']))
                echo "<img src='" . $r['image'] . "' /><br />";
            echo $r['description'] . "</td>";
            // Τύπωσε το κελί με το μήνυμα
            echo "<td>" . $r['message'] . "</td></tr>";
        }
        echo "</table>";
    }
}
mysqli_close($link);

}

/* -----
Η συνάρτηση τυπώνει τη φόρμα εισαγωγής θέματος και εισάγει νέο θέμα εφόσον αυτή
έχει αποσταλλεί.
*/
function insertSubjectPage($pageurl) {

```

```

echo "<h2>Εισαγωγή Θέματος</h2>\n";
echo "<p />\n";

/* Αν ο χρήστης έχει πιστοποιηθεί εμφανίσει τη φόρμα για να εισάγει το θέμα και
εφόσον έχει αποσταλλεί κάνει την εισαγωγή του νέου θέματος. */
if (isset($_SESSION['userid']))
{
    // Η φόρμα πρέπει να σταλλεί στο $pageurl
    ?>
    <form class='formsubject' action='<?php echo $pageurl; ?>' method='post'>
    <fieldset><legend>Φόρμα Εισαγωγής Θέματος:</legend>
    Όνομα Θέματος που θα εισάγετε: <input type='text' size='50' name='sname' /><br
    />
    <input type='submit' value='Εισαγωγή' />
    <input type='reset' value='Καθαρισμός' />
    </fieldset></form>
    <?php

    // Αν έχει αποσταλλεί η φόρμα κάνει την εισαγωγή
    if (isset($_POST['sname']))
    {
        // Έλεγξε αν έδωσε όνομα θέματος. Αν όχι βγάλε μήνυμα λάθους αλλιώς προχώρα
σε εισαγωγή αλλιώς
        if (empty(trim($_POST['sname'])))
            echo "<span class='errmsgsg'>Δεν έχετε δώσει όνομα θέματος
!</span>\n";
        else // αλλιώς ξεκίνα την εισαγωγή
        {
            // Συνδέσου με το server και τη Β.Δ.
            $link = connectToServer();

            // Κάνε μετατροπή για να μην έχω πρόβλημα με ειδικούς χαρακτήρες
            $sname = mysqli_real_escape_string($link, trim($_POST['sname']));

            /* Δημιούργησε το ερώτημα εισαγωγής και εκτέλεσέ το (πρέπει να
επηρρέασει ακριβώς μία γραμμή). Δημιούργησε την Ημερομηνία-ώρα στην κατάλληλη
μορφή. */
            $today = getdate();
            $datetime = $today['year'] . "-" . $today['mon'] . "-" .
            $today['mday'];
            $datetime .= " " . $today['hours'] . ":" . $today['minutes'] . ":" .
            $today['seconds'];

            $sqlq = "insert into subjects (sname, userid, sdate) ";
            $sqlq .= " values ('$sname', " . $_SESSION['userid'] . ",
            '$datetime') ";
            $res = executeQuery($sqlq, $link);
            if (!$res)
                die("Cannot insert into table " . mysqli_error($link));

            $num = mysqli_affected_rows($link);
            if ($num == 1)
                echo "<span class='infomsg'> Το θέμα " . $sname . " εισήχθει
!</span><br />";
            else
                echo "<span class='infomsg'> Το θέμα " . $sname . " δεν
εισήχθει ! Παρακαλώ ξαναπροσπαθήστε !</span><br />";

            mysqli_close($link);
        }
    }
}

```

```

}
else // Αν δεν έχει πιστοποιηθεί εμφάνισε ένα μήνυμα ενημέρωσης
{
??
<span class='infomsg'>Η σελίδα αυτή είναι μόνο για εξουσιοδοτημένους χρήστες
!!</span>
<p>Για να εισάγετε θέμα κάντε πρώτα login από τη φόρμα στο πάνω μέρος της
σελίδας. Αν δεν έχετε κάνει εγγραφή για να αποκτήσετε λογαριασμό κάντε από το το
σύνδεσμο <b>Εγγραφή</b> στα αριστερά.</p>
<?php
}

}

/* -----
Η συνάρτηση τυπώνει τη φόρμα εισαγωγής Μηνύματος και εισάγει νέο μήνυμα εφόσον
αυτή έχει αποσταλλεί.
*/
function insertMessagePage($pageurl) {
echo "<h2>Εισαγωγή Μηνύματος</h2>\n";
echo "<p />\n";

/* Αν ο χρήστης έχει πιστοποιηθεί εμφάνισε τη φόρμα και αν έχει αποσταλλεί
προχώρα σε εισαγωγή του μηνύματος */
if (isset($_SESSION['userid']))
{
// Συνδέσου στον MySQL εξυπηρετητή
$link = connectToServer();

// Επέλεξε όλα τα θέματα και δημιούργησε δυναμικά μία λίστα στην οποία (με το
<option>) εμφάνισε
// το όνομα του θέματος αλλά σαν τιμή βάζε το κλειδί (sid) ώστε αν επιλέξει
κάποιος να σταλεί το κλειδί.
$sqlq = "select * from subjects";
$res = executeQuery($sqlq, $link);

if (!$res)
die("Cannot select from table " . mysqli_error($link));
$num = mysqli_num_rows($res);

// Η φόρμα πρέπει να σταλλεί στο $pageurl
echo "<form class='formmessage' action='$pageurl' method='post'>";
echo "<fieldset><legend>Φόρμα Εισαγωγής Μηνύματος:</legend>";
echo "Θέμα στο οποίο θα εισάγετε το μήνυμα: <select name='sid'>";
// Βάλε στη λίστα όλα τα θέματα
for ($i=1; $i <= $num; ++$i)
{
$r = mysqli_fetch_array($res);
echo "<option value=\"\" . $r['sid'] . \">\" . $r['sname'] . "</option>";
}
echo "</select><br />";
//Βάλε μία περιοχή κειμένου στην οποία θα γράψει ο χρήστης το μήνυμά του
echo "<textarea name='message' rows='10' cols='40'>";
echo "</textarea><br />";
echo "<input type='submit' value='Εισαγωγή' />";
echo "<input type='reset' value='Καθαρισμός' />";
echo "</fieldset></form>";

// Αν έστειλε τη φόρμα κάνε την εισαγωγή
if (isset($_POST['message']))
{

```

```

/* Έλεγχξε αν έδωσε όνομα θέματος. Αν δεν έχει δώσει βάζω στη μεταβλητή $err
μία τιμή διαφορετική από κενό την οποία ελέγχω παρακάτω. */
$error = "";
if (!isset($_POST['message']) || empty(trim($_POST['message'])))
    $error .= "<li>Δεν έχετε δώσει κείμενο μηνύματος !</li>\n";
else
    $msg = $_POST['message'];

if (!isset($_POST['sid']) || empty(trim($_POST['sid'])))
    $error .= "<li>Δεν έχετε επιλέξει θέμα στο οποίο θα εισαχθεί το
μηνύματος !</li>\n";
else
    $sid = $_POST['sid'];

// Αν δεν έδωσε σωστά στοιχεία βγάλε μήνυμα λάθους
if ($error != "")
    echo "<ol>" . $error . "</ol>\n";
else // αλλιώς ξεκίνα την εισαγωγή
{
    // Συνδέσου με το server και τη Β.Δ.
    $link = connectToServer();
    // Κάνε μετατροπή για να μην έχω πρόβλημα με ειδικούς χαρακτήρες
    $msg = mysqli_real_escape_string($link, $msg);
    $sid = mysqli_real_escape_string($link, $sid);

    /* Δημιούργησε το ερώτημα εισαγωγής και εκτέλεσέ το. Πρέπει να
επηρεάσει ακριβώς μία γραμμή. Δημιούργησε την Ημερομηνία-ώρα στην κατάλληλη
μορφή. */
    $today = getdate();
    $datetime = $today['year'] . "-" . $today['mon'] . "-" .
$today['mday'];
    $datetime .= " " . $today['hours'] . ":" . $today['minutes'] . ":" .
$today['seconds'];

    $sqlq = "insert into messages (userid, sid, message, postdate) ";
    $sqlq .= " values (" . $_SESSION['userid'] . ", '$sid', '$msg',
'$datetime') ";
    $res = executeQuery($sqlq, $link);

    if (!$res)
        die("Cannot insert into table " . mysqli_error($link));

    $num = mysqli_affected_rows($link);
    if ($num == 1)
        echo "<span class='infomsg'> Το μήνυμα εισήχθει !</span><br
/>";
    else
        echo "<span class='infomsg'> Το μήνυμα δεν εισήχθει. Παρακαλώ
ξαναπροσπαθήστε !</span><br />";

    mysqli_close($link);
}
}
}
else // Αν δεν έχει πιστοποιηθεί εμφάνισε ένα μήνυμα ενημέρωσης
{
    ?>
    <span class="infomsg">Η σελίδα αυτή είναι μόνο για εξουσιοδοτημένους χρήστες
!!</span>
    <p>Για να εισάγετε Μήνυμα κάντε πρώτα login από τη φόρμα στο πάνω μέρος της
σελίδας. Αν δεν έχετε κάνει εγγραφή για να αποκτήσετε λογαριασμό κάντε από το

```

```

μενού από την επιλογή Εγγραφή στα αριστερά.</p>";
<?php
}
}

/* -----
Η συνάρτηση κάνει τη σύνδεση του χρήστη, δηλαδή τον πιστοποιεί και δημιουργεί
τις απαραίτητες μεταβλητές στη σύνοδο
*/
function userLoginVerify() {
// Έλεγχξε αν έστειλε τη φόρμα αλλιώς μην κάνεις τίποτα
if (isset($_POST['login']))
{
    // Έλεγχξε αν έδωσε login και password.
    $error = "";
    if (!isset($_POST['login']) || empty(trim($_POST['login'])))
        $error .= "<li>Δεν έχετε δώσει login !</li>\n";
    else
        $login = $_POST['login'];

    if (!isset($_POST['passwd']) || empty($_POST['passwd']))
        $error .= "<li>Δεν έχετε δώσει passwd !</li>\n";
    else // το password το είχα κάνει κρυπτογράφηση με τη password_hash άρα θα το
ελέγξω με τη password_verify
        $passwd = $_POST['passwd'];

    // Αν δεν έδωσε σωστά στοιχεία επέστρεψε ένα μήνυμα λάθους
    if (!empty($error))
    {
        return "<span class='errmsg'>ΑΠΟΤΕΛΕΣΜΑ ΠΙΣΤΟΠΟΙΗΣΗΣ
ΧΡΗΣΤΗ</span>\n<ol>$error</ol>";
    }
    else // Έλεγχξε αν υπάρχει ο χρήστης αυτός στον πίνακα χρηστών
    {
        // Συνδέσου με το server και τη Β.Δ.
        $link = connectToServer();
        // Κάνε μετατροπή για να μην έχω πρόβλημα με ειδικούς χαρακτήρες
        $login = mysqli_real_escape_string($link, $login);

        $sqlq = "select * from users where login='$login'";
        $res = executeQuery($sqlq, $link);

        if (!$res)
            die("Cannot select from table " . mysqli_error($link));

        $num = mysqli_affected_rows($link);

        // Αν υπάρχει αυτός ο χρήστης
        if ($num == 1)
        {
            $r = mysqli_fetch_array($res);
            mysqli_close($link);

            $user_password = $r['passwd'];
            //Αν έδωσε σωστά το password
            if (password_verify($passwd, $user_password))
            {
                /* Δημιουργώ στη σύνοδο τα στοιχεία που χρειάζομαι για να ελέγξω
αν ο χρήστης έχει πιστοποιηθεί και να τα χρησιμοποιώ για να αναγνωρίσω ποιος
είναι (για τις εισαγωγές θέματος και μηνυμάτων). */
                $_SESSION['userid'] = $r['userid'];
            }
        }
    }
}
}

```



```

        $_SESSION['login'] = $r['login'];
        $_SESSION['image'] = $r['image'];
        return "<span class='infomsg'>Καλωσήρθες " . $r['login'] . "
!</span>\n";
    }
    else
        return "<span class='errmsg'>Δεν ήταν δυνατή η σύνδεση με τα
στοιχεία που δώσατε. Παρακαλώ ξαναπροσπαθήστε !</span>\n";
    }
    else
    {
        mysqli_close($link);
        return "<span class='errmsg'>Δεν ήταν δυνατή η σύνδεση με τα στοιχεία
που δώσατε. Παρακαλώ ξαναπροσπαθήστε !</span>\n";
    }
}
}

/* -----
Η συνάρτηση κάνει τη σύνδεση του χρήστη, δηλαδή τον πιστοποιεί και δημιουργεί
τις απαραίτητες μεταβλητές στη σύνοδο
*/
function userLogout() {
/* Αν ο χρήστης έχει πιστοποιηθεί (έχει κάνει login) κάνει αποσύνδεση
καταστρέφοντας τα στοιχεία του πίνακα και όλο το session. Αν δεν είναι
πιστοποιημένος μην κάνεις τίποτα. */
if (isset($_SESSION['userid']))
{
    $login = $_SESSION['login'];
    unset($_SESSION['login']);
    unset($_SESSION['userid']);
    unset($_SESSION['image']);
    session_destroy();
    return "<span class='infomsg'>Ευχαριστούμε για τη επικοινωνία, ελπίζουμε
να σε ξαναδούμε σύντομα $login !</span>";
}
else
    return "";
}

/* -----
Η συνάρτηση τυπώνει κείμενο ενημέρωσης ότι δεν υπάρχει η σελίδα που ζητήθηκε
*/
function unknownPage() {
?>
<h2>Ανύπαρκτη Σελίδα</h2>
<p>Παρακαλώ επιλέξτε μία από τις επιλογές του μενού.</p>
<?php
}

?>

```

7.2.3 Αρχείο layout.css

Περιέχει πληροφορίες για το layout της ιστοσελίδας (χρησιμοποιείται για λόγους επίδειξης της χρήσης CSS και όχι για εκμάθηση αφού είναι ένα πολύ απλό παράδειγμα).

```

* {
    box-sizing: border-box;
}

```

```
header {
    height:120px;
}

footer {
    height:80px;
    text-align:center;
}

div.headerleft {
    vertical-align: top;
    display: inline-block;
    width:23%;
}

div.headercenter {
    vertical-align: top;
    display: inline-block;
    width:50%;
}

div.headerright {
    vertical-align: top;
    display: inline-block;
    width:23%;
}

body {
    min-width: 1000px;
    max-width: 1000px;
    font-size: 1.0em;
}
```

7.2.4 Αρχείο menu.css

Περιέχει πληροφορίες για τη μορφοποίηση του μενού της ιστοσελίδας (χρησιμοποιείται για λόγους επίδειξης της χρήσης CSS και όχι για εκμάθηση αφού είναι ένα πολύ απλό παράδειγμα).

```
ul.menu {
    width:100%;
    height:40px;
    background-color: #5b5bd7;
}

ul li {
    display: inline-block;
    width:160px;
}

ul li a {
    display: inline-block;
    width:100%;
    text-decoration:none;
    color: #fff;
    background-color: #5b5bd7;
    border:1px outset #2d2db9;
    text-align: center;
    line-height:38px;
    vertical-align: middle;
}

ul li a:hover {
    background-color: #6f6fdc;
    border:1px inset #5b5bd7;
    font-weight: bold;
}
```

```
}
```

7.2.5 Αρχείο format.css

Περιέχει πληροφορίες για τη μορφή (χρησιμοποιείται για λόγους επίδειξης της χρήσης CSS και όχι για εκμάθηση αφού είναι ένα πολύ απλό παράδειγμα) ορισμένων αντικειμένων.

```
#imglogo {  
    width:200px;  
    height:120px;  
}  
  
img.userphoto {  
    width: 80px;  
    height: 80px;  
}  
  
span.infomsg {  
    color: #2222aa;  
    font-weight:bold;  
}  
  
span.errormsg {  
    color: #aa2222;  
    font-weight:bold;  
}  
  
h1 {  
    font-size: 1.8em;  
}  
h2 {  
    font-size: 1.6em;  
}  
h3 {  
    font-size: 1.4em;  
}  
  
form.formlogin {  
    width:90%;  
    margin:auto;  
}  
  
form.formregister,  
form.formsearch,  
form.formsubject,  
form.formmessage {  
    width:70%;  
    margin:auto;  
}  
  
form.formlogin fieldset,  
form.formregister fieldset,  
form.formsearch fieldset,  
form.formsubject fieldset,  
form.formmessage fieldset {  
    background-color: #f4f7f8;  
}  
  
form.formlogin legend,  
form.formregister legend,  
form.formsearch legend,
```

```
form.formsubject legend,
form.formmessage legend {
    color: #777;
}

form.formlogin input,
form.formregister input,
form.formsearch input,
form.formsubject input,
form.formmessage input {
    display:inline-block;
    width:50%;
    padding:2px 2px;
    margin:2px 2px;
    border:1px solid #777;
}
form.formlogin input[type=submit],
form.formlogin input[type=reset],
form.formregister input[type=submit],
form.formregister input[type=reset],
form.formsearch input[type=submit],
form.formsearch input[type=reset],
form.formsubject input[type=submit],
form.formsubject input[type=reset],
form.formmessage input[type=submit],
form.formmessage input[type=reset] {
    width:120px;
    color: #fff;
    background-color: #5b5bd7;
}

form.formlogin input[type=submit]:hover,
form.formlogin input[type=reset]:hover,
form.formregister input[type=submit]:hover,
form.formregister input[type=reset]:hover,
form.formsearch input[type=submit]:hover,
form.formsearch input[type=reset]:hover,
form.formsubject input[type=submit]:hover,
form.formsubject input[type=reset]:hover,
form.formmessage input[type=submit]:hover,
form.formmessage input[type=reset]:hover {
    background-color: #6f6fdc;
}

form.formlogin label,
form.formregister label,
form.formsearch label,
form.formsubject label,
form.formmessage label {
    display:inline-block;
    width:40%;
    font-weight:bold;
}

table.tableresults {
    width:90%;
    margin:auto;
    border:1px solid #2222bb;
    border-collapse:collapse;
    background-color: #cccccc;
```

```

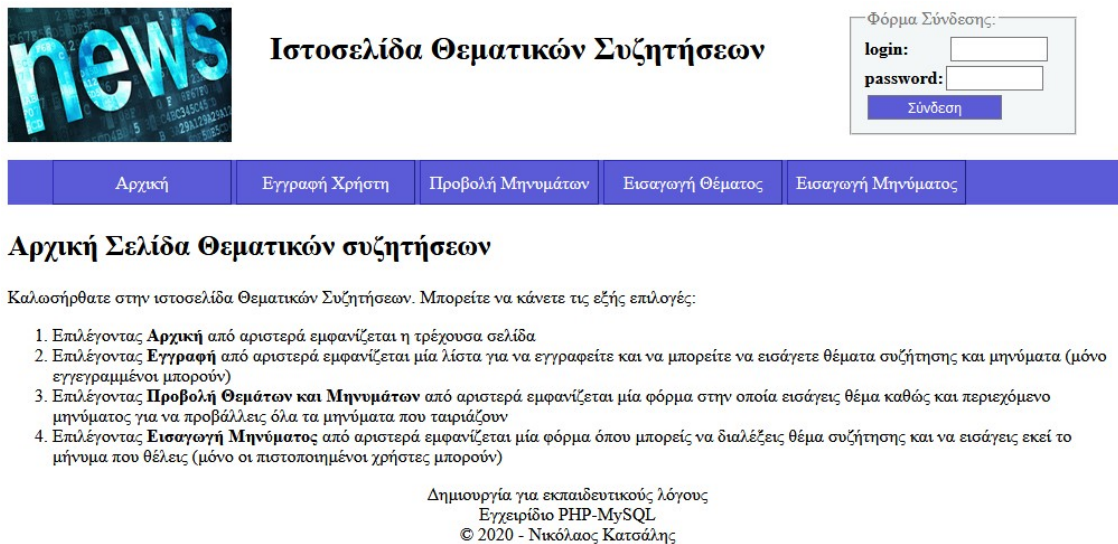
}

table.tableresults th,
table.tableresults td {
    border:1px solid #2222bb;
}

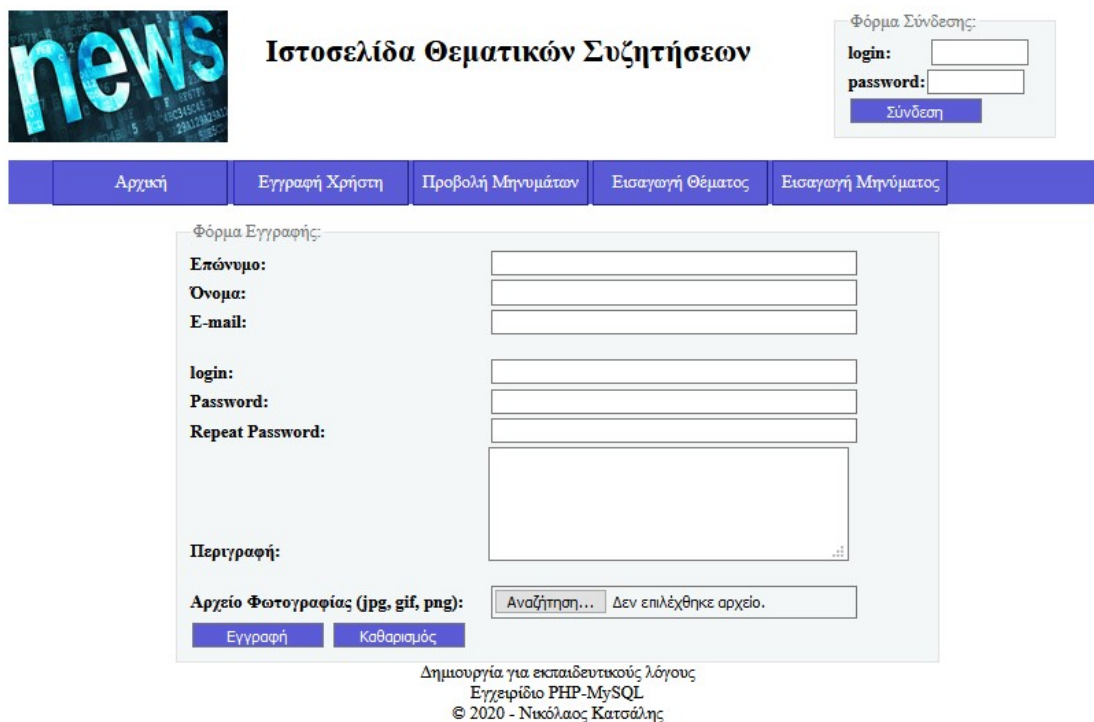
```

7.2.6 Εικόνες Εφαρμογής

Ακολουθούν εικόνες της εφαρμογής κατά τη διάρκεια της λειτουργίας.



Εικόνα 19: Αρχική Σελίδα



Εικόνα 20: Σελίδα εγγραφής χρήστη



Ιστοσελίδα Θεματικών Συζητήσεων

Φόρμα Σύνδεσης:

login: password: [Αρχική](#)[Εγγραφή Χρήστη](#)[Προβολή Μηνυμάτων](#)[Εισαγωγή Θέματος](#)[Εισαγωγή Μηνύματος](#)

Αναζήτηση-Προβολή Μηνυμάτων

Στην ιστοσελίδα αυτή μπορείτε να αναζητήσετε/προβάλετε τα μηνύματα που θέλετε. Η διαδικασία είναι να επιλέξετε το θέμα για το οποίο θέλετε να προβάλετε τα μηνύματα του και να συμπληρώσετε το κείμενο που περιέχουν τα μηνύματα που θέλετε να προβάλετε. Τότε θα αναζητηθούν και θα προβληθούν όλα τα μηνύματα, του συγκεκριμένου μόνο θέματος, στα οποία βρίσκεται το συγκεκριμένο κείμενο που δώσατε.

Αν θέλετε να προβάλετε όλα τα μηνύματα ενός θέματος απλώς επιλέξετε το θέμα και μην γράψετε τίποτα στο κείμενο μηνύματος.

Αν θέλετε να προβάλετε όλα τα μηνύματα που έχουν κάποιο κείμενο ανεξαρτήτου θέματος, τότε μην επιλέξετε κανένα θέμα και συμπληρώστε το κείμενο που θέλετε στο κείμενο μηνύματος.

Αν θέλετε να προβάλετε όλα τα μηνύματα όλων των θεμάτων, τότε αφήστε και τα δύο πεδία κενά.

Φόρμα Αναζήτησης-Προβολής Μηνυμάτων:

Επιλογή Θέματος: Κείμενο Μηνύματος:

Βρέθηκε 1 αποτέλεσμα !

Θέμα	Χρήστης	Μήνυμα
Αυτοκίνητα posted at 2020-03-19 22:00:11	katsalis 	Το πρώτο μήνυμα στην ενότητα με θέμα αυτοκίνητα. Για δοκιμή

Εικόνα 21: Σελίδα Αναζήτησης-Προβολής Μηνυμάτων (δεν χρειάζεται πιστοποίηση)



Ιστοσελίδα Θεματικών Συζητήσεων

Φόρμα Σύνδεσης:

login: password: [Αρχική](#)[Εγγραφή Χρήστη](#)[Προβολή Μηνυμάτων](#)[Εισαγωγή Θέματος](#)[Εισαγωγή Μηνύματος](#)

Εισαγωγή Θέματος

Η σελίδα αυτή είναι μόνο για εξουσιοδοτημένους χρήστες !!

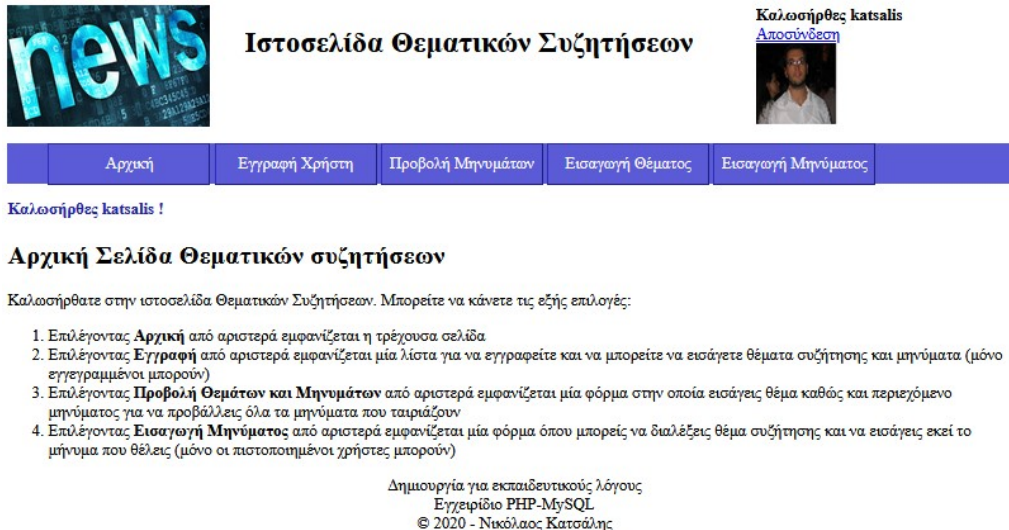
Για να εισάγετε Θέμα κάντε πρώτα login από τη φόρμα στο πάνω μέρος της σελίδας. Αν δεν έχετε κάνει εγγραφή για να αποκτήσετε λογαριασμό κάντε από το το σύνδεσμο **Εγγραφή** στα αριστερά.

Δημιουργία για εκπαιδευτικούς λόγους

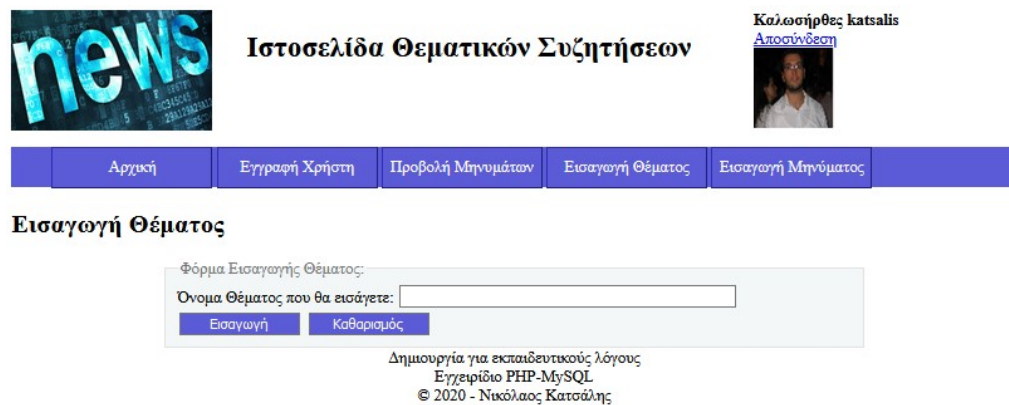
Εγχειρίδιο PHP-MySQL

© 2020 - Νικόλαος Κατσάλης

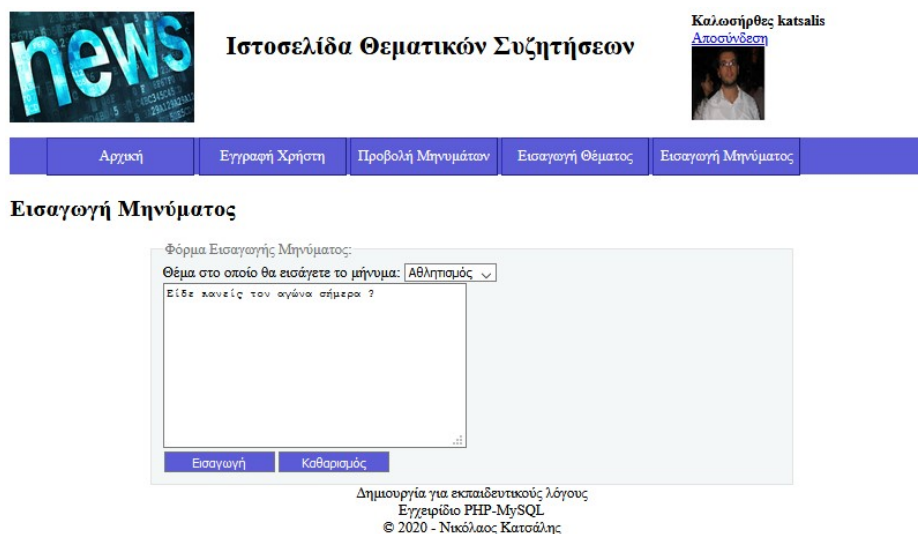
Εικόνα 22: Σελίδα Εισαγωγής Θέματος (δεν έχει γίνει σύνδεση και βγάζει μήνυμα)



Εικόνα 23: Αρχική Σελίδα (αμέσως αφού κάναμε σύνδεση στο λογαριασμό μας)



Εικόνα 24: Σελίδα Εισαγωγής Θέματος (είμαστε συνδεδεμένοι με το λογαριασμό μας)



Εικόνα 25: Σελίδα Εισαγωγής Μηνύματος (είμαστε συνδεδεμένοι με το λογαριασμό μας)

8 Αναφορές

- *PHP manual from PHP Documentation Group* (<https://www.php.net/docs.php>)
- *MySQL Documentation* (<https://dev.mysql.com/doc/>)
- *Ελληνικό Εγχειρίδιο MySQL Query Browser* (<http://dev.mysql.com/doc/query-browser/el/index.html>)
- *MySQL PHP API* (<https://dev.mysql.com/doc/apis-php/en/>)
- *Apache HTTP Server Documentation* (<https://httpd.apache.org/docs/>)
- *HTTP/1.1 Specification* (<https://www.w3.org/Protocols/>)
- *MIME Media Types* (<https://www.iana.org/assignments/media-types/media-types.xhtml>)